

ASP .Net Core Tutorial for Beginners - Part 2

Playlist: ASP NET Core Tutorial - Kudvenkat

Съдържание

ASP .Net Core Tutorial for Beginners - Part 2	1
63 – Logging to file using NLog.....	3
64 - LogLevel configuration.....	8
65 - Identity tutorial from scratch	19
66 - Register new user	25
67 - Identity UserManager and SignInManager.....	30
68 - Identity password complexity	35
69 - Show or hide login and logout links based on login status	38
70 - Implementing login functionality	43
71 – Authorization	51
72 - Redirect user to original url after login.....	56
73 - Open redirect vulnerability	58
74 - Client side validation	61
75 - Remote validation	65
76 - Custom validation attribute	69
77 - Extend IdentityUser	73
78 - Creating roles	83
79 - Get list of roles	89
80 - Edit role.....	94
81 - Add or remove users from role	105
82 - Role based authorization	118
83 - Show or hide navigation menu based on user role	122
84 - List all users from asp net core identity database.....	125
85 - Edit identity user	134
86 - Delete identity user	142
87 - Delete confirmation.....	144
88 - Delete identity role.....	149
89 - Enforce ON DELETE NO ACTION	153

90 - Custom error page	158
91 - Manage user roles in asp net core identity.....	169
92 - Model binding not working on submitting razor view with foreach loop	176
93 - Manage user claims	184
94 - Claims based authorization.....	192
95 - Role based vs claims based authorization	196
96 - Authorization in views	200
97 - Change AccessDenied route.....	206
98 - Claim type and claim value in claims policy based authorization	207
99 - Create custom authorization policy using func.....	211
100 - Custom authorization requirements and handlers	215
101 - Custom authorization requirement and handler example	217
102 - Multiple custom authorization handlers	225
103 - Custom authorization handler success vs failure	228
104 - External identity providers.....	233
105 - Create google oauth credentials Client Id and Client Secret	234
106 - ASP NET Core google authentication setting up the UI	244
107 - ExternalLoginCallback action.....	252
108 - Register application with facebook	261
109 - Facebook authentication	267
110 - Secret manager	273
111 - Why email confirmation is important.....	279
112 - Block login if email is not confirmed.....	280
113 - Email confirmation	291
114 - External login email confirmation	300
115 - Forgot password.....	314
116 - Reset password	320
117 - How tokens are generated and validated.....	328
118 - Password reset token lifetime.....	335
119 - Custom token provider	338
120 - Encryption and Decryption Example	342
121 - Change Password.....	350
122 - Add password to local account linked to external login.....	358

123 - Account logout.....	367
124 – Wrap up.....	374

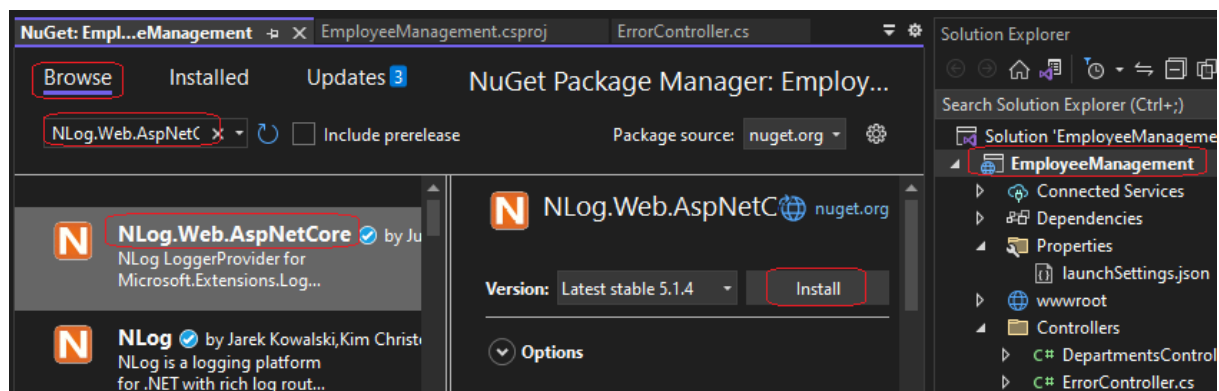
Забележка: Учила съм се самостоятелно, като тези записки не са предвиждани за споделяне, а само и единствено за преговор. Проектът ми е с картинки от любимата ми художничка Josephine Wall - крайно неподходящи за бизнес-портфолио и в самия проект ще ги сменя със стандартни, но на скрийншотите все ще се виждат първоначалните.

63 – Logging to file using NLog

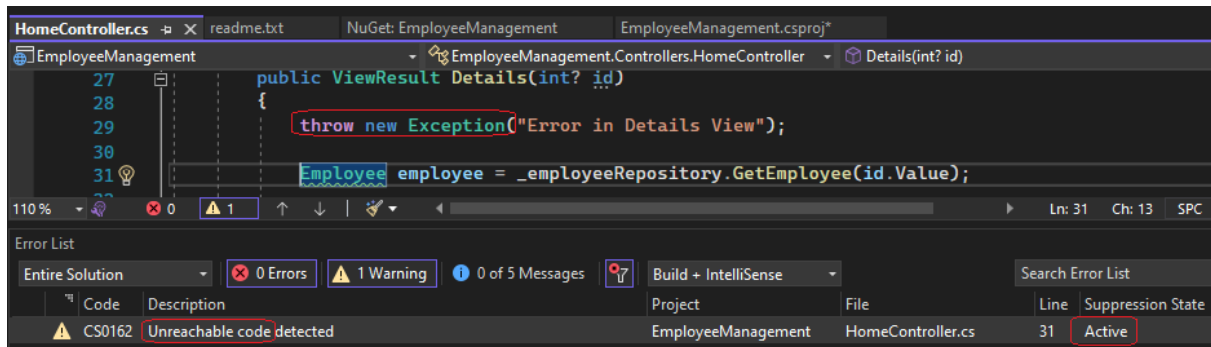
Third Party Logging Providers



Д.б. върху проекта във VS -> Manage NuGet Packages -> **NLog.Web.AspNetCore**



Нямах видими грешки и предупреждения, докато не отворих HomeController:



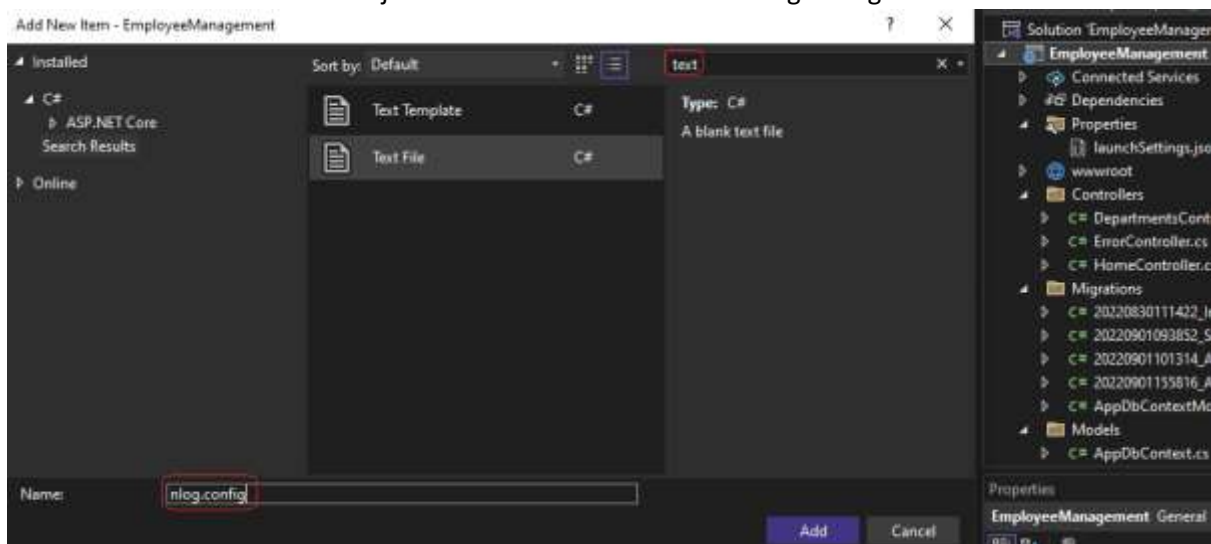
Тогава се вижда предупреждение, че има недостъпен код, поради ред код за изключение, който дори е блед на бял фон. Това е пренебрежимо съобщение, което може да игнорираме, така или иначе пакетът е инсталиран.

```
public IActionResult Details(int? id)
{
    throw new Exception("Error in Details View");
    Employee employee = _employeeRepository.GetEmployee(id.Value);

    if (employee == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id.Value);
    }
}
```

Това е просто безопасна демонстрация на Logging exceptions. За конфигурационна информация, NLog търси файл `NLog.config` в root project directory.

Project->Add->New item-> text -> nlog.config



Поставяме базов код:


```
controller.cs  nlog.config  readme.txt  NuGet: EmployeeManagement  Emplo

<!-- the targets to write to -->
<targets>
  <!-- write logs to file -->
  <target name="allfile" xsi:type="File"
    fileName="c:\DemoLogs\nlog-all-${shortdate}.log" />
</targets>

<!-- rules to map from logger name to target -->
<rules>
  <!--All logs, including from Microsoft-->
  <logger name="*" minlevel="Trace" writeTo="allfile" />
</rules>
</nlog>
```

```
<?xml version="1.0" encoding="utf-8" ?>
```

```
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
```

```
<!-- the targets to write to -->
```

```
<targets>
```

```
<!-- write logs to file -->
```

```
<target name="allfile" xsi:type="File"
```

```
  fileName="c:\DemoLogs\nlog-all-${shortdate}.log" />
```

```
</targets>
```

```
<!-- rules to map from logger name to target -->
```

```
<rules>
```

```
<!--All logs, including from Microsoft-->
```

```
<logger name="*" minlevel="Trace" writeTo="allfile" />
```

```
</rules>
```

```
</nlog>
```

С този код казваме NLog да записва основната log information `c:\DemoLogs\` във файлове с имена `nlog-all-${shortdate}.log`, където на мястото на `-${shortdate}` стоят новите данни, които, ако не съществуват, ще бъдат създадени. Изискваме минимален код:

```
<!-- rules to map from logger name to target -->
<rules>
  <!--All logs, including from Microsoft-->
  <logger name="*" minlevel="Trace" writeTo="allfile" />
</rules>
```

nlog.config

Properties

nlog.config File Properties

Advanced

Build Action	Content
Copy to Output Directory	Copy if newer
Custom Tool	Do not copy
Custom Tool Namespace	Copy always
Misc	Copy if newer
File Name	resourcemanager
Full Name	Full Name

Copy to Output Directory

Specifies the source file will be copied to the output directory.

Повече информация: <https://github.com/NLog/NLog/wiki/Configuration-file>

В runtime, този файл е желателно да се копира в output directory:

Д. б. върху nlog.config -> properties -> Copy to output directory-> copy if newer,

Кое в новата версия е заложено по подразбиране.

Последна стъпка е да добавим NLog като един от providers. В

<https://github.com/dotnet/aspnetcore/blob/c4efbfb1f9a3b6241140150ae9abf219bde16b2/src/DefaultBuilder/src/WebHost.cs> търсим logging:

```
193 .ConfigureLogging((hostingContext, loggingBuilder) =>
194 {
195     loggingBuilder.Configure(options =>
196     {
197         options.ActivityTrackingOptions = ActivityTrackingOptions.SpanId
198                                     | ActivityTrackingOptions.TraceId
199                                     | ActivityTrackingOptions.ParentId;
200     });
201     loggingBuilder.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
202     loggingBuilder.AddConsole();
203     loggingBuilder.AddDebug();
204     loggingBuilder.AddEventSourceLogger();
205 }
```

Копираме тези няколко реда код:

```
.ConfigureLogging((hostin
gContext, loggingBuilder)
=>
{
    loggingBuilder.Configure(options =>
    {
        options.ActivityTrackingOptions =
ActivityTrackingOptions.SpanId
|
ActivityTrackingOptions.TraceId
|
ActivityTrackingOptions.ParentId;
    });

    loggingBuilder.AddConfiguration(hostingContext.Configuration.Ge
tSection("Logging"));
    loggingBuilder.AddConsole();
    loggingBuilder.AddDebug();
}
```

```
loggingBuilder.AddEventSourceLogger();
})
```

И ги поставяме в program.cs

```

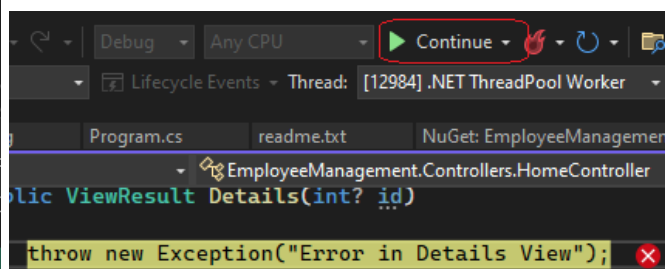
nlog.config Program.cs* readme.txt NuGet: EmployeeManagement EmployeeManagement.csproj*
EmployeeManagement.Program CreateWebHostBuilder(string[] args)
public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
{
    Host.CreateDefaultBuilder(args)
        .ConfigureLogging((hostingContext, loggingBuilder) =>
        {
            loggingBuilder.Configure(options =>
            {
                options.ActivityTrackingOptions = ActivityTrackingOptions.SpanId
                    | ActivityTrackingOptions.TraceId
                    | ActivityTrackingOptions.ParentId;
            });
            loggingBuilder.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
            loggingBuilder.AddConsole();
            loggingBuilder.AddDebug();
            loggingBuilder.AddEventSourceLogger();
        })
        .ConfigureWebHostDefaults(webBuilder =>

```

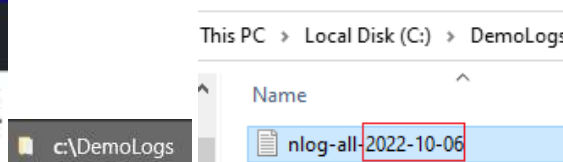
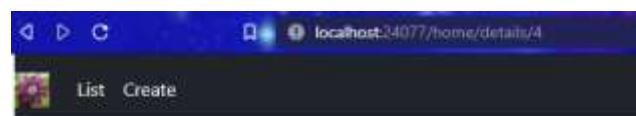
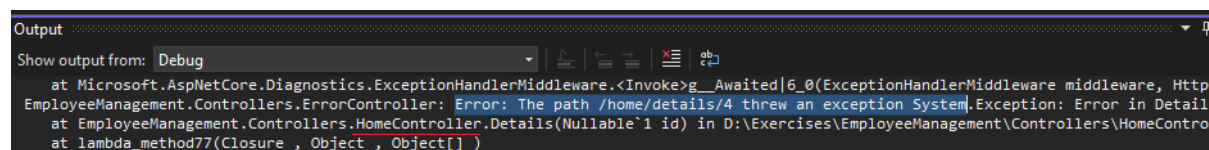
```

loggingBuilder.AddConfiguration(hostingContext
loggingBuilder.AddConsole();
loggingBuilder.AddDebug();
loggingBuilder.AddEventSourceLogger();
loggingBuilder.AddNLog();
}
}
using Microsoft.Extensions.Logging;
using NLog.Extensions.Logging;

```



Отново като стартираме проекта с Debug mode -> View -> грешка -> Continue ->



В търсачката виждаме, че директория C:\DemoLogs е автоматично добавена.

Файлт при създаване в туториъла е едва 3 Kb, а при мен са в пъти по-големи, като после се увеличават при всяко действие на апликацията, записвайки всичко, дори ако самият документ в момента е отворен, пише върху него в реално време. Ако приложението се стартира в друг ден, се създава нов файл с актуалната дата.

Имаме 4 стъпки за инсталиране на NLog:

Logging to File in asp.net core using nlog

- Install **NLog.Web.AspNetCore** nuget package
- Create **nlog.config** file

```
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <targets>
    <target name="allfile" xsi:type="File"
            fileName="c:\logs\nlog-all-${shortdate}.log"/>
  </targets>

  <rules>
    <logger name="*" minlevel="Trace" writeTo="allfile" />
  </rules>
</nlog>
```

- Enable copy to **bin** folder
- Add **NLog** as one of the Logging Providers

```
public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
    WebHost.CreateDefaultBuilder(args)
        .ConfigureLogging((hostingContext, logging) =>
        {
            logging.AddConfiguration(
                hostingContext.Configuration.GetSection("Logging"));
            logging.AddConsole();
            logging.AddDebug();
            logging.AddEventSourceLogger();
            // Enable NLog as one of the Logging Provider
            logging.AddNLog(); ←
        })
        .UseStartup<Startup>();
```

64 - LogLevel configuration

Имаме тип Enum данни с различен приоритет, със съответни методи Trace method, Debug method, които се извикват с **представка Log**.

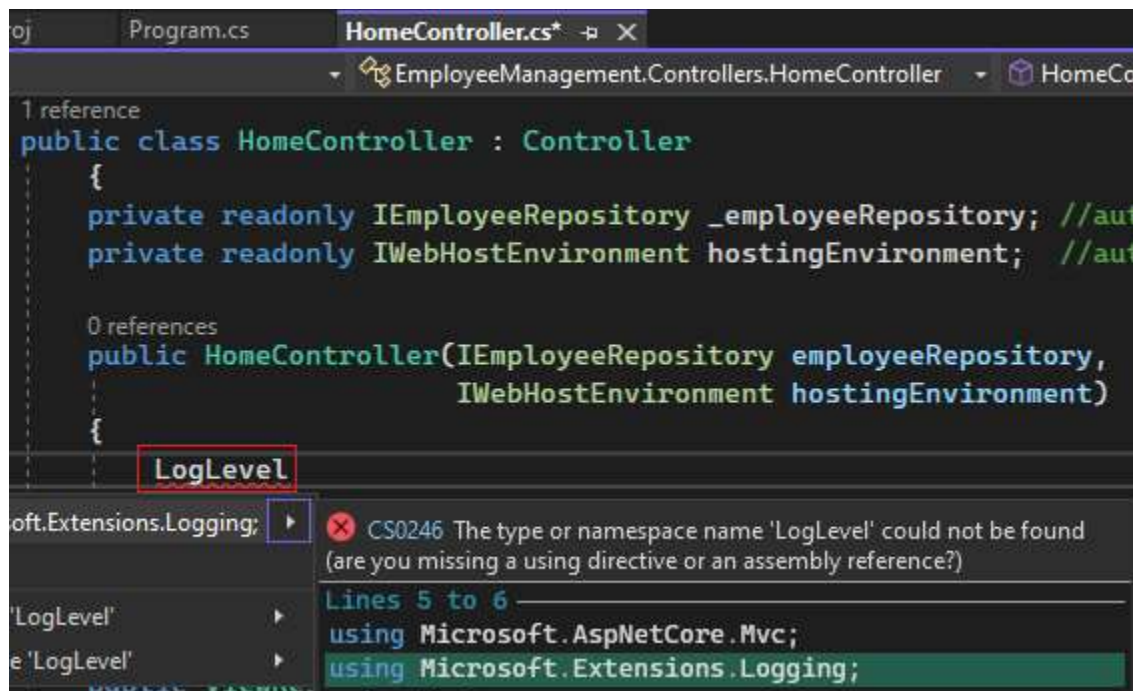
ASP.NET Core LogLevel Configuration

LogLevel indicates the severity of the logged message

	methods
Trace = 0	LogTrace()
Debug = 1	LogDebug()
Information = 2	LogInformation()
Warning = 3	LogWarning()
Error = 4	LogError()
Critical = 5	LogCritical()
None = 6	N/A

Trace = lowest level; None = highest level;

Когато зададем в appsettings.json -> Trace, виждаме абсолютно всички съобщения от Trace до/включително Critical.



```
1 reference
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //aut
    private readonly IWebHostEnvironment hostingEnvironment; //aut

    0 references
    public HomeController(IEmployeeRepository employeeRepository,
        IWebHostEnvironment hostingEnvironment)
    {
        LogLevel
    }
}

Microsoft.Extensions.Logging;
CS0246 The type or namespace name 'LogLevel' could not be found
(are you missing a using directive or an assembly reference?)
Lines 5 to 6
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
```

Добавяме LogLevel демонстративно, но ще го изтрием -> Go to definition:

```
LogLevel [from metadata]
namespace Microsoft.Extensions.Logging.Abstractions
{
    public enum LogLevel
    {
        Trace = 0,
        Debug = 1,
        Information = 2,
        Warning = 3,
        Error = 4,
        Critical = 5,
        None = 6
    }
}
```

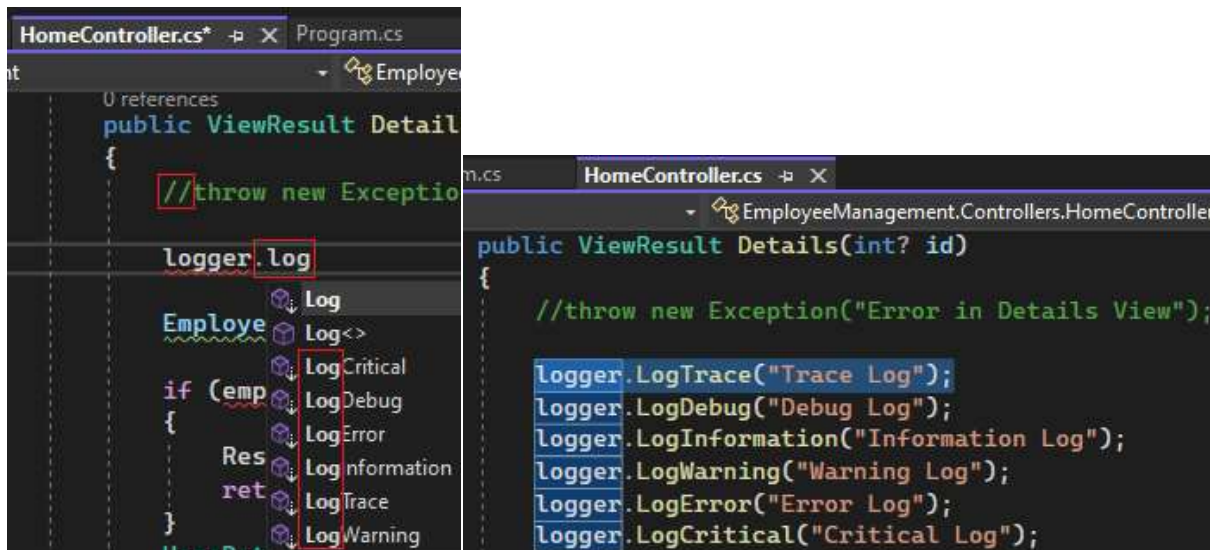
```
appsettings.json*
{
  "Logging": {
    "LogLevel": {
      "Default": "Trace",
      "Microsoft": "Warning"
    }
  }
}
```

Изтриваме LogLevel в HomeController, защото не ни трябва, но добавяме променлива logger тип ILogger и автоматично с втората опция на Ctrl. -> необходимото private field

```
HomeController.cs*
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto added with Ctrl. -> II option
    private readonly IWebHostEnvironment hostingEnvironment; //auto added with Ctrl. -> II option
    private readonly ILogger logger; //auto added with Ctrl. -> II option

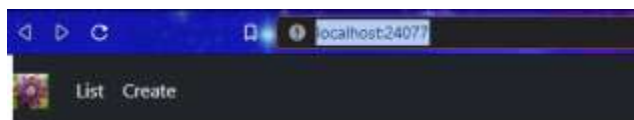
    public HomeController(IEmployeeRepository employeeRepository,
        IWebHostEnvironment hostingEnvironment,
        ILogger logger)
    {
        _employeeRepository = employeeRepository;
        this.hostingEnvironment = hostingEnvironment;
        this.logger = logger;
    }
}
```

Закоментираме реда с изключението:



```
logger.LogTrace("Trace Log");
logger.LogDebug("Debug Log");
logger.LogInformation("Information Log");
logger.LogWarning("Warning Log");
logger.LogError("Error Log");
logger.LogCritical("Critical Log");
```

Save -> Run in Debug mode -> Custom error view instead of list of employees, което означава, че имаме някъде грешка в кода.



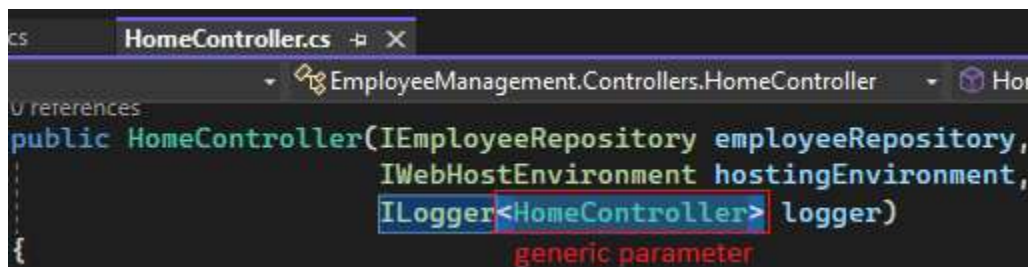
Output

Show output from: Debug

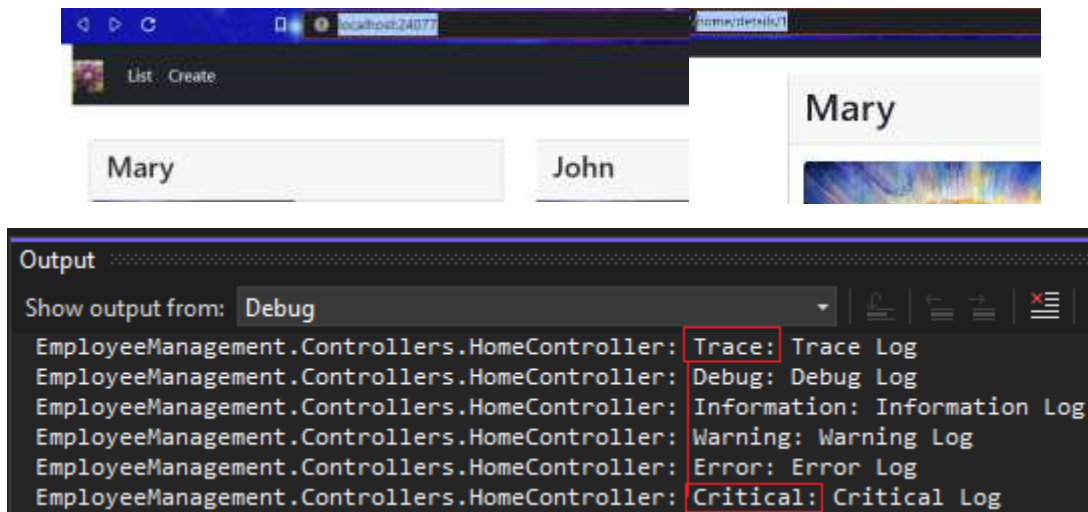
Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware: Error: An unhandled exception has occurred while

System.InvalidOperationException: Unable to resolve service for type 'Microsoft.Extensions.Logging.ILogger' while attempting to activate 'EmployeeManagement.Controllers.HomeController'.

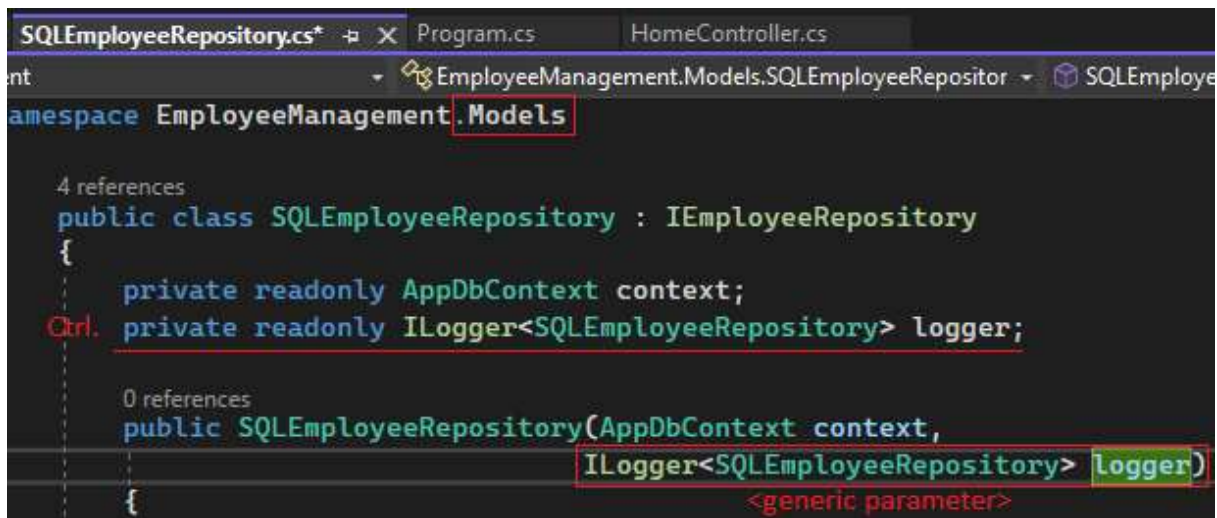
Това е, защото сме забравили да включим <HomeController> като generic parameter:



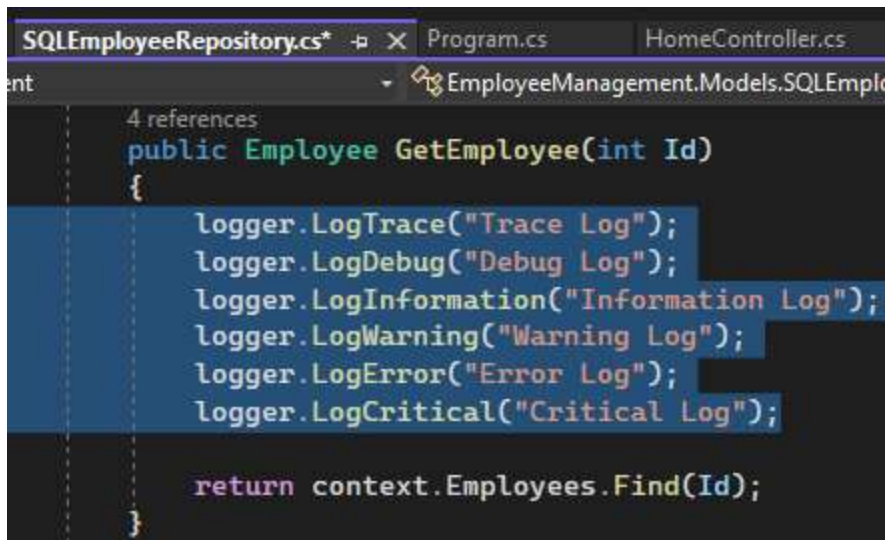
С което грешката е поправена и имаме списъка, както и бутон View вече работи.



Виждаме всички съобщения от Trace до Critical, защото в appsettings.json сме задали Trace, а в същото време нито едно съобщение от Microsoft, т.к. там сме задали само Warning предупреждения (т.е. от категория Warning и нагоре ->Error -> Critical). В момента обаче, такива съобщения няма. Ако искаме да напълним конзолата със съобщения за цялата информация, ще зададем Information.



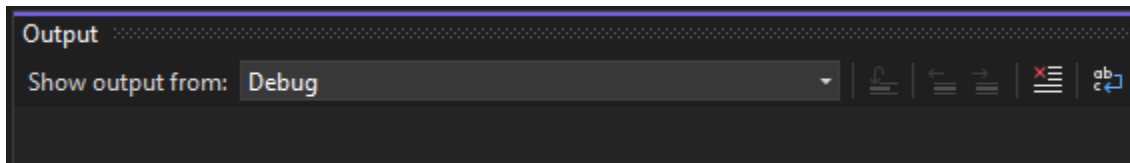
Вмъкваме логери в метод, който се извиква всеки път при преглед на детайлите на служител:



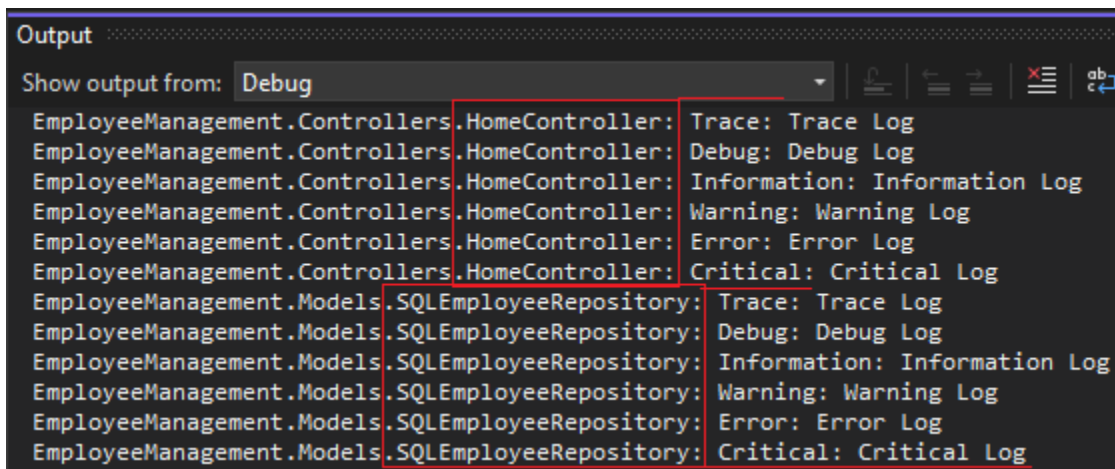
```
SQLEmployeeRepository.cs*  Program.cs  HomeController.cs
EmployeeManagement.Models.SQLEmplo
4 references
public Employee GetEmployee(int Id)
{
    logger.LogTrace("Trace Log");
    logger.LogDebug("Debug Log");
    logger.LogInformation("Information Log");
    logger.LogWarning("Warning Log");
    logger.LogError("Error Log");
    logger.LogCritical("Critical Log");

    return context.Employees.Find(Id);
}
```

Debug -> Зарежда се в брауъра списъка със служители, но Output е празен.



Когато дадем View за детайли на конкретен служител и Stop Debugging, Output вече се напълва със съобщения от двете места, в които сме извикали логовете:



Копираме името на категорията:

```
EmployeeManagement.Controllers.HomeController: Trace: Trace Log
```

И го поставяме в appsettings, като още с paste дописва автокод:

```
ny.cs appsettings.json* X Program.cs HomeController.cs
schemastore.org/appsettings.json
"Logging": {
  "LogLevel": {
    "Default": "Trace",
    "EmployeeManagement.Controllers.HomeController": null,
    "Microsoft": "Warning"
  }
}
```

⚠ Value must be one of the following types: string
⚠ Value must be one of the following values: "Trace", "Debug", "Information", "Warning", "Error", "Critical", "None"

Експериментално задаваме тези стойности:

```
"Default": "Trace",
"EmployeeManagement.Controllers.HomeController": "Warning",
"EmployeeManagement.Models.SQLEmployeeRepository": "Trace",
"Microsoft": "Warning"
```

Отново Debug->View->Output:

```
Output
Show output from: Debug
EmployeeManagement.Controllers.HomeController: Warning: Warning Log
EmployeeManagement.Controllers.HomeController: Error: Error Log
EmployeeManagement.Controllers.HomeController: Critical: Critical Log
EmployeeManagement.Models.SQLEmployeeRepository: Trace: Trace Log
EmployeeManagement.Models.SQLEmployeeRepository: Debug: Debug Log
EmployeeManagement.Models.SQLEmployeeRepository: Information: Information Log
EmployeeManagement.Models.SQLEmployeeRepository: Warning: Warning Log
EmployeeManagement.Models.SQLEmployeeRepository: Error: Error Log
EmployeeManagement.Models.SQLEmployeeRepository: Critical: Critical Log
```

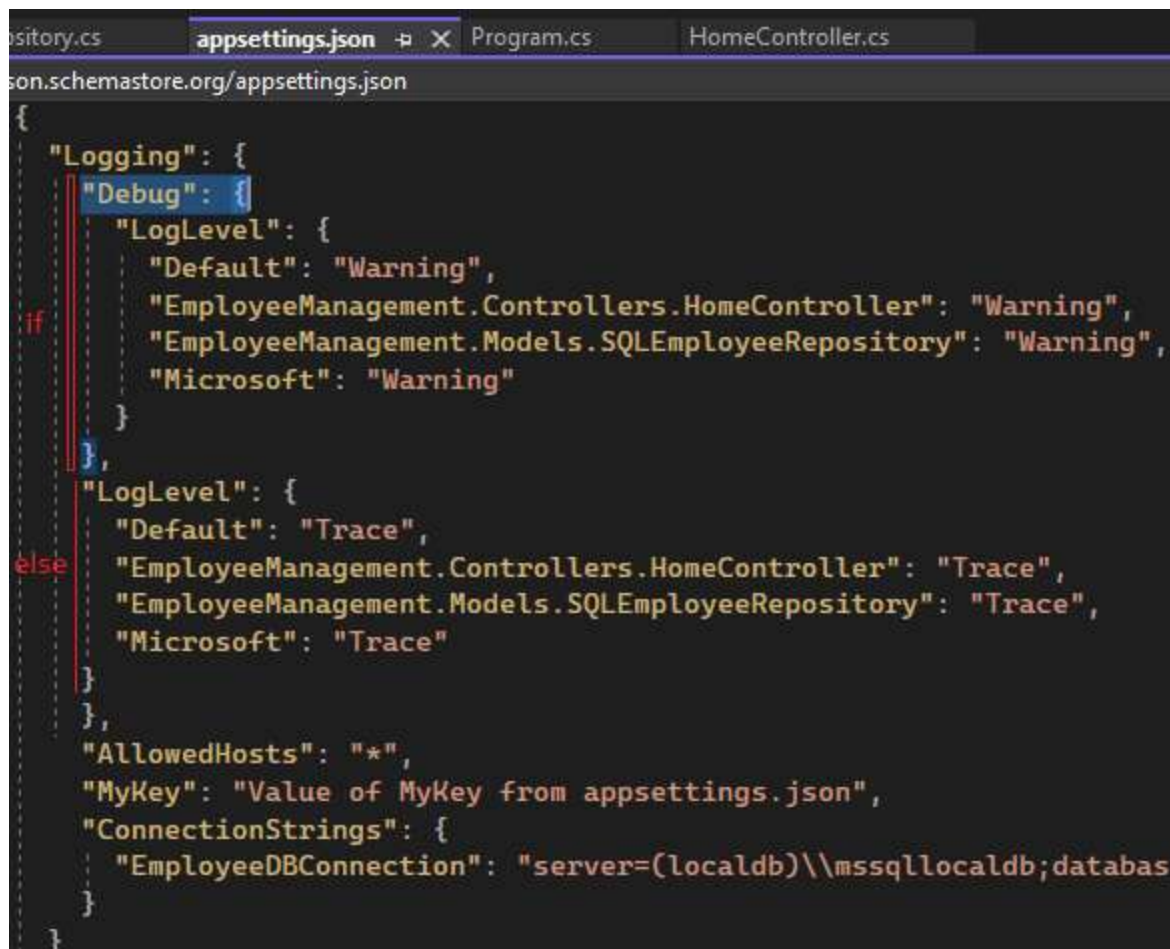
Към момента регистрираме данни само от тип категория, но може и от Provider, например

- Console logging provider -> записва физически и извежда съобщения в конзолата,
- Debug logging provider -> в Output window in Debug mode;

Имаме възможност да вмъкваме блок код при определени условия, например:

```
"Logging": {
  "Debug": {
    "LogLevel": "Debug"
  }
}
```

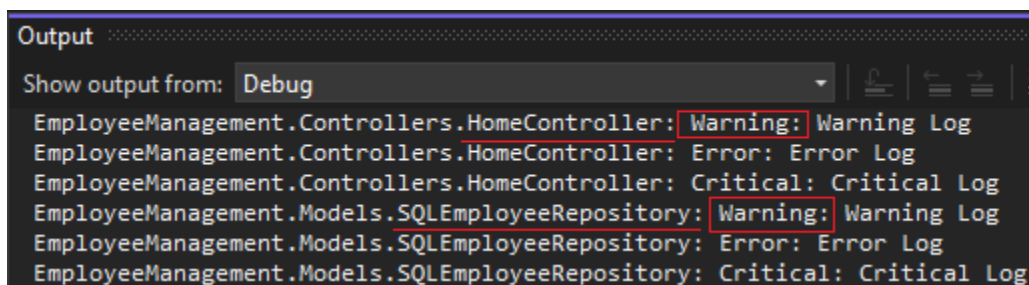
, т.е. в Debug mode да се изпълнява блоковия код за LogLevel:



```
{
  "Logging": {
    "Debug": {
      "LogLevel": {
        "Default": "Warning",
        "EmployeeManagement.Controllers.HomeController": "Warning",
        "EmployeeManagement.Models.SQLEmployeeRepository": "Warning",
        "Microsoft": "Warning"
      }
    },
    "LogLevel": {
      "Default": "Trace",
      "EmployeeManagement.Controllers.HomeController": "Trace",
      "EmployeeManagement.Models.SQLEmployeeRepository": "Trace",
      "Microsoft": "Trace"
    }
  },
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json",
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;databas
  }
}
```

Във всички останали случаи, когато се стартира Console logging provider, even source, even log и т.н. да се използва другия лог. В случая за пример нагласяме на Debug, всички съобщения за Warning, докато в останалите случаи Trace.

В случая даваме Debug -> в браузера при зареждане на Home, имаме запис на съобщенията от него, а при View, от детайлите на модела с хранилището:



```
Output
Show output from: Debug
EmployeeManagement.Controllers.HomeController: Warning: Warning Log
EmployeeManagement.Controllers.HomeController: Error: Error Log
EmployeeManagement.Controllers.HomeController: Critical: Critical Log
EmployeeManagement.Models.SQLEmployeeRepository: Warning: Warning Log
EmployeeManagement.Models.SQLEmployeeRepository: Error: Error Log
EmployeeManagement.Models.SQLEmployeeRepository: Critical: Critical Log
```

Сега да видим в останалите случаи, например при стартиране от конзолата. За да намалим броя на съобщенията, задаваме в Microsoft по-високо ниво warning:

```
"LogLevel": {
  "Default": "Trace",
  "EmployeeManagement.Controllers.HomeController": "Trace",
  "EmployeeManagement.Models.SQLEmployeeRepository": "Trace",
  "Microsoft": "Warning"
}
```

Въпреки това обаче, се появява съобщение от Microsoft.

```

C:\> Select Command Prompt - dotnet run

Microsoft Windows [Version 10.0.19042.631]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\hp>cd /d d:\Exercises\EmployeeManagement

d:\Exercises\EmployeeManagement>dotnet run
Building...
info: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[63]
      User profile is available. Using 'C:\Users\hp\AppData\Local\ASP.NET
      dows DPAPI to encrypt keys at rest.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: d:\Exercises\EmployeeManagement

```

Когато заредим 5000 ->

```

C:\> Command Prompt - dotnet run

info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[2]
      Executed action EmployeeManagement.Controllers.HomeController.Index (EmployeeManagement) in 9.6353ms
info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
      Request finished HTTP/1.1 GET http://localhost:5000/ - - 200 - text/html; charset=utf-8 11.1325ms

```

View->

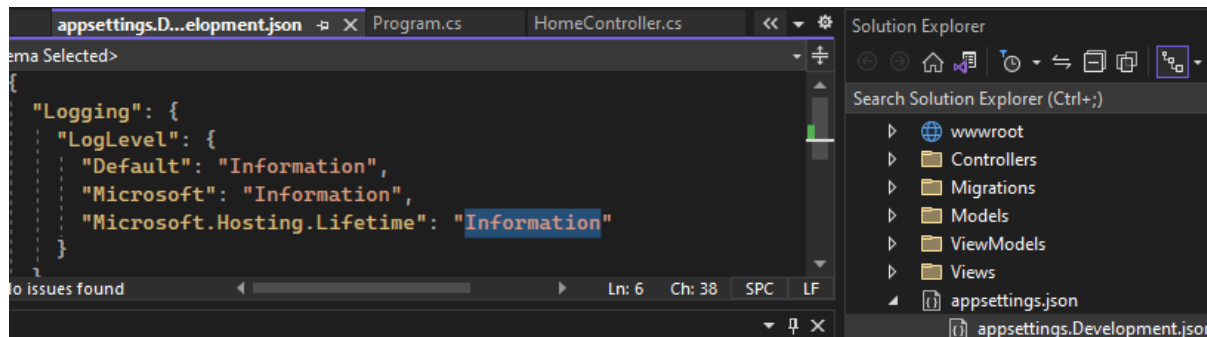
```

C:\> Command Prompt - dotnet run

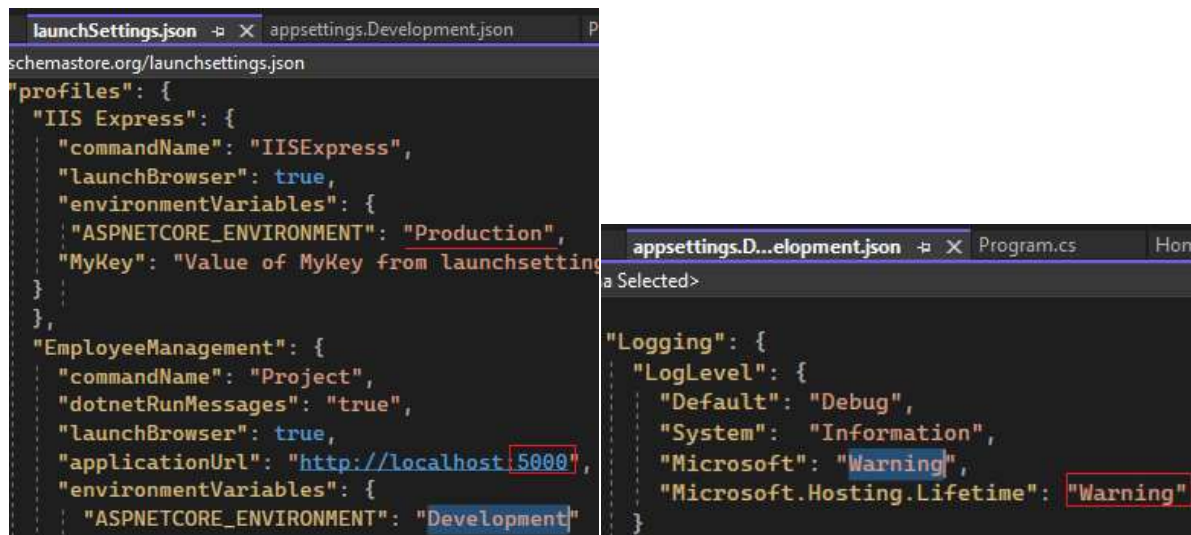
trce: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Trace Log
dbug: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Debug Log
info: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Information Log
warn: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Warning Log
fail: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Error Log
crit: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Critical Log
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [e].[Id], [e].[Department], [e].[Email], [e].[Name], [e].[PhotoPath]
      FROM [Employees] AS [e]
      WHERE [e].[Id] = @__p_0

```

Защо обаче от Microsoft подават толкова съобщения, при положение, че сме задали в appsettings.json само Warning? Причината е, че в локалния appsettings.Development.json сме задали за Microsoft = information и той презаписва глобалния, а в момента сме в среда Development.



Така че сменяме в локалния json -> Microsoft: Warning



Макар, че рестартирах, при мен CLI заби и накрая заредих 5000 от брауъра, без да изчакам, тогава изписа и логерите:

Administrator: Command Prompt - dotnet run

```
C:\Windows\system32>cd /d d:\Exercises\EmployeeManagement
d:\Exercises\EmployeeManagement>dotnet run
Building...
trce: EmployeeManagement.Controllers.HomeController[0]
      Trace Log
dbug: EmployeeManagement.Controllers.HomeController[0]
      Debug Log
info: EmployeeManagement.Controllers.HomeController[0]
      Information Log
warn: EmployeeManagement.Controllers.HomeController[0]
      Warning Log
fail: EmployeeManagement.Controllers.HomeController[0]
      Error Log
crit: EmployeeManagement.Controllers.HomeController[0]
      Critical Log
trce: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Trace Log
dbug: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Debug Log
info: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Information Log
warn: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Warning Log
fail: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Error Log
crit: EmployeeManagement.Models.SQLEmployeeRepository[0]
      Critical Log
```

Logs can be filtered by

- Log Category
- Logging Provider
- Even both



65 - Identity tutorial from scratch

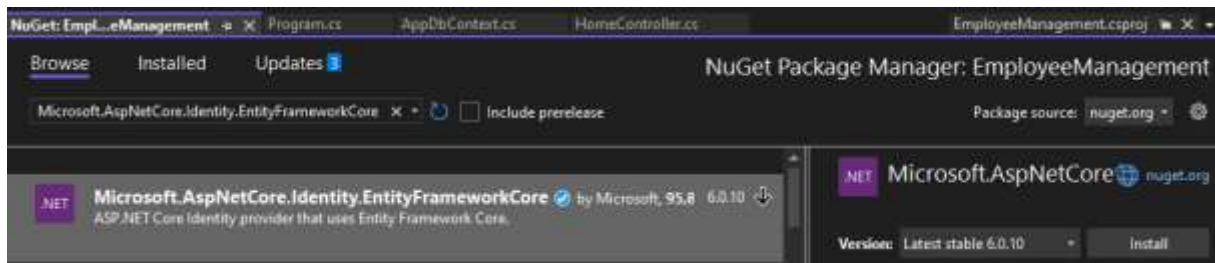
ASP.NET Core Identity Tutorial

ASP.NET Core Identity is a membership system

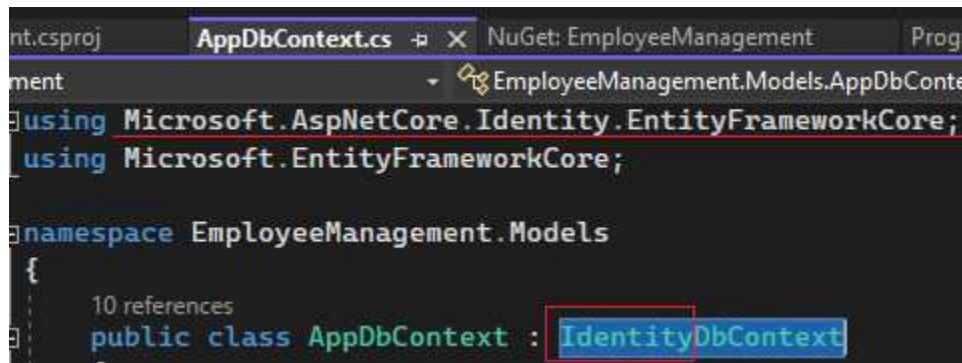
- Create, Read, Update and Delete user accounts
- Account confirmation
- Authentication & Authorization
- Password Recovery
- Two-factor authentication with SMS
- Supports external login providers like Microsoft, Facebook, Google etc
- and much more

За целта първо унаследяването трябва да е IdentityDbContext, вместо само DbContext

Налага се да инсталирам, за да извикам необходимия namespace
Microsoft.AspNetCore.Identity.EntityFrameworkCore:



Identity притежава комплект от всички свойства за управление на вградената таблица БД, в случая SQL server.



Когато направим няколко последователни проследявания с Go to Definition -> IdentityDbContext, ще видим, че всъщност той е разширение и се унаследява от по-ограничения DbContext, като го включва в себе си, затова не е необходимо да дефинираме допълнително още веднъж DbContext. IdentityDbContext включва DbContext + повече Identity функционалност.

Step 1 : Your application DbContext class must inherit from IdentityDbContext

```
public class AppDbContext : IdentityDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Seed();
    }
}
```

Следваща стъпка е да добавим неговата Identity services в Startup.cs -> Configure Services


```

system class, system class
services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();
custom class as generic parameter

```

```

services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();

```

Step 2 : Add ASP.NET Core Identity Services

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(options =>
        options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>();

    services.AddMvc().AddXmlSerializerFormatters();
    services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
}

```

Step 3 : Add Authentication middleware

```

public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseExceptionHandler("/Error");
        app.UseStatusCodePagesWithReExecute("/Error/{0}");
    }

    app.UseStaticFiles();

    app.UseAuthentication();

    app.UseMvc(routes =>
    {
        routes.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
    });
}

```

Добавяме Authentication middleware. Редът отново е много важен - трябва да е преди MVC:

```

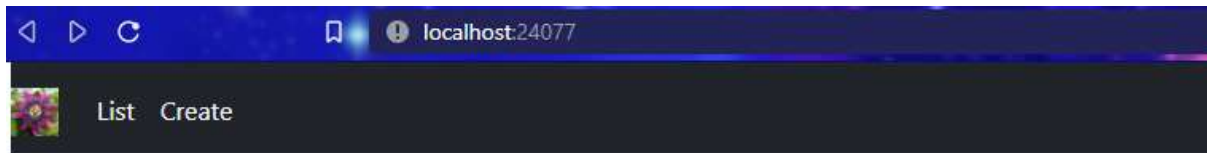
app.UseStaticFiles();
app.UseAuthentication();
app.UseMvc(routes => //app.

```


Компилираме, за да се уверим, че нямаме грешки:

```
Output
Show output from: Build
1>EmployeeManagement -> D:\Exercises\EmployeeManagement\bin\Debug\net6.0\EmployeeManagement.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

Появява се страницата с грешка:



An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

```
<h3>
  An error occurred while processing your request.
  <br />
  The support team is notified and we are working on the fix.
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
```

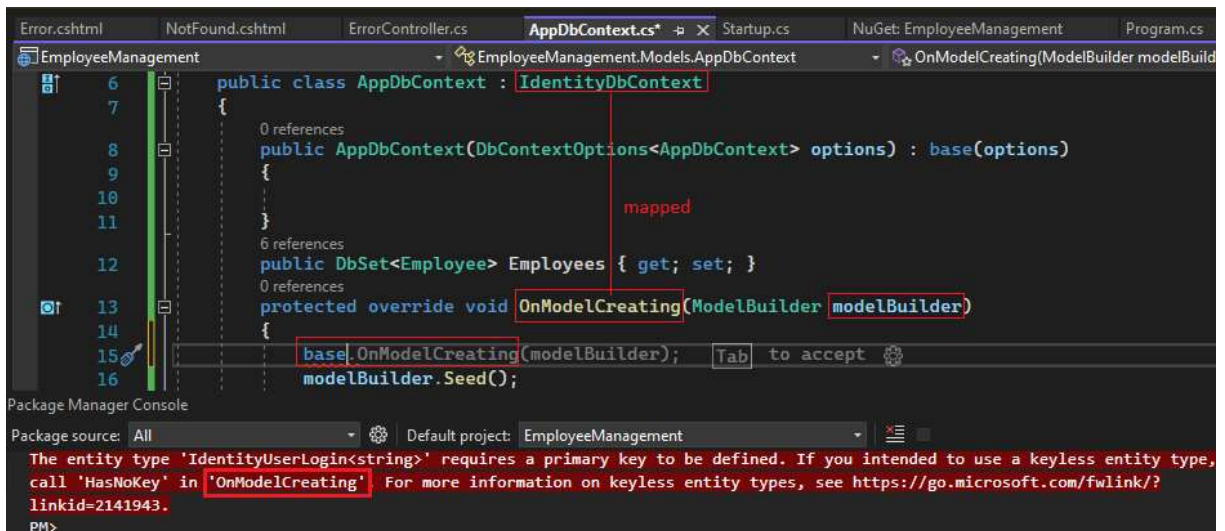
Step 4 : Generate ASP.NET Core Identity Tables

Add-Migration

View-> Other Windows -> Package Manager:

Add-Migration AddingIdentity

Keys of IdentityTables are mapped in OnModelCreatingMethod of this IdentityDbContext system class.

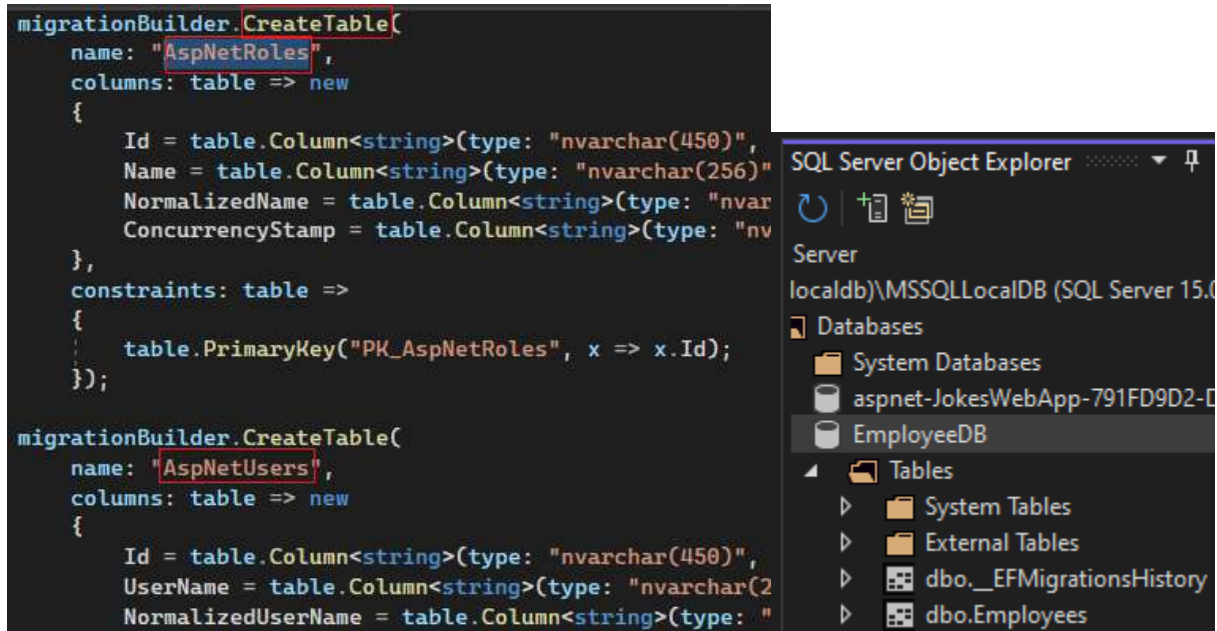


```

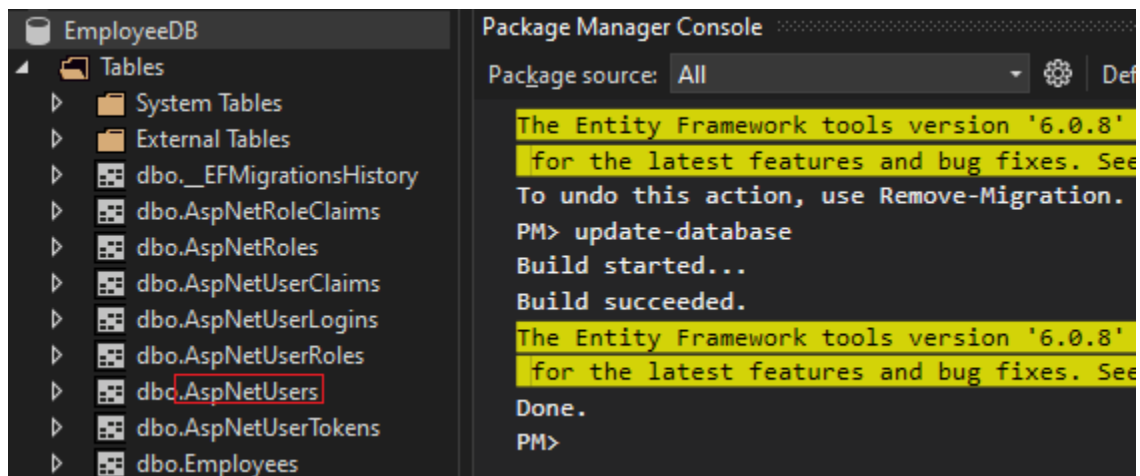
PM> add-migration AddingIdentity
Build started...
Build succeeded.
The Entity Framework tools version '6.0.8' is older than that of the runtime '6.0.10'.
bug fixes. See https://aka.ms/AAC1fbw for more information.
To undo this action, use Remove-Migration.
PM>

```

Миграцията създава няколко таблици:



Към момента имаме само 2 БД, но когато обновим миграцията update-database, веднага се заредят няколко нови.



->View Data

id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencySt...	PhoneNumber	PhoneNumber...	TwoFactorDna...	Lockout
1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Нейните колони съответстват в системно заложените в клас IdentityUser, аналогично за IdentityRole. Към момента в тях няма допълнителна информация като град и пол, това предстои да се направи като разширение.

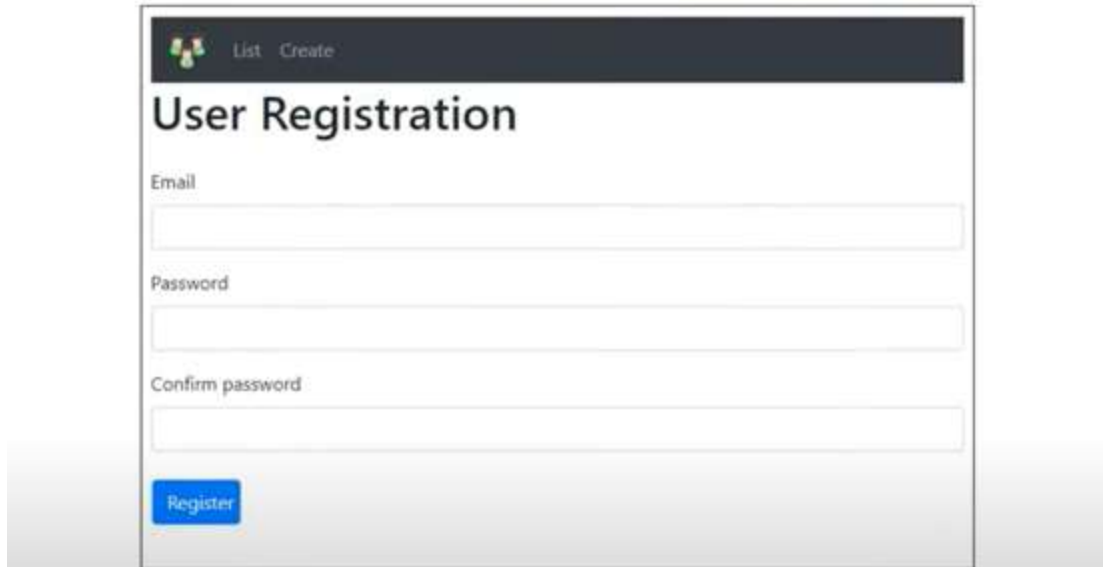
Step 4 : Generate ASP.NET Core Identity Tables

Add-Migration

Update-Database

66 - Register new user

ASP.NET Core Identity - User Registration



Първо ни е необходим ViewModel class, който да достави данните от

UserRegistration view -> Controller

ViewModels folder -> Add -> New Item -> Class -> RegisterViewModel.cs

```
namespace EmployeeManagement.ViewModels
{
    0 references
    public class RegisterViewModel
    {
        [Required] validation attribute
        [EmailAddress]
        0 references
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)] to mask the characters
        0 references
        public string Password { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm password")] to separate 2 words Confirm_password
        [Compare("Password", compare conform with password and if they don't match:
        ErrorMessage = "Password and confirmation password do not match.")
        label is single word without space interval
        0 references
        public string ConfirmPassword { get; set; }
    }
}
```

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations;
```

```

namespace EmployeeManagement.ViewModels
{
    public class RegisterViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

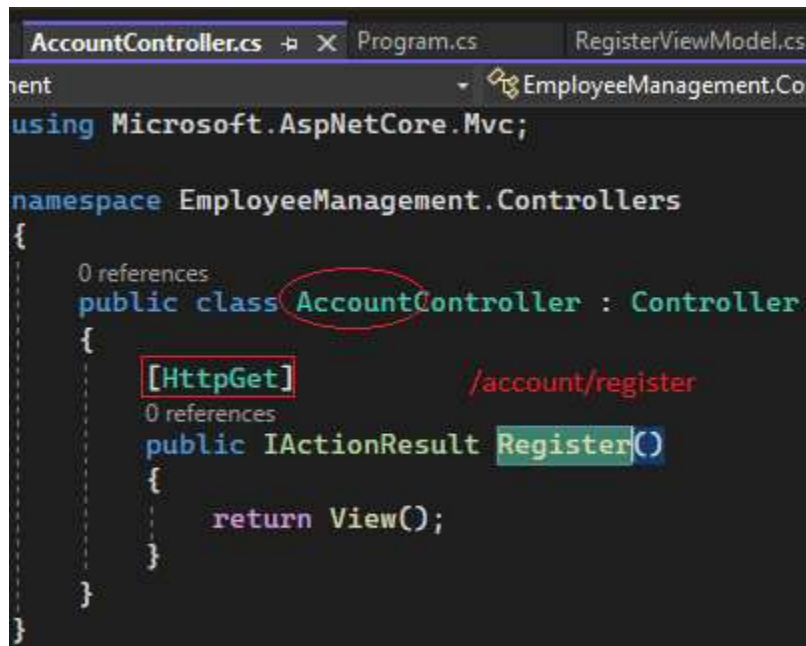
        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm password")]
        [Compare("Password",
            ErrorMessage = "Password and confirmation password do not match.")]
        public string ConfirmPassword { get; set; }
    }
}

```

Папка Controllers -> Add -> New item -> Controller class -> AccountController.cs

Или Add-> Controller -> MVC Controller



```

using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    public class AccountController : Controller
    {
        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }
    }
}

```



```
}
```

В папка Views -> нова папка Account -> razor view Register.cshtml

Направихме ViewModel, Controller, сега View:

Най-важно е да извикаме модела с клас RegisterViewModel, за да може да работим с дефинираните в него email, password, ConfirmPassword:

```
.cs Register.cshtml Program.cs RegisterViewModel.cs HomeController.cs
@model RegisterViewModel

@{
    ViewBag.Title = "User Registration";
}

<h1>User Registration</h1>

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="ConfirmPassword"></label>
                <input asp-for="ConfirmPassword" class="form-control" />
                <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Register</button>
        </form>
    </div>
</div>
```

```
@model RegisterViewModel
```

```
@{
    ViewBag.Title = "User Registration";
}
```

```
<h1>User Registration</h1>
```

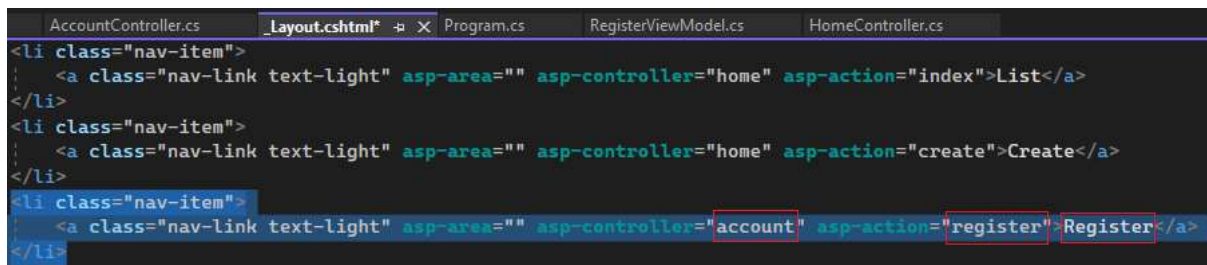
```
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
        </form>
    </div>
</div>
```

```

<div class="form-group">
    <label asp-for="Password"></label>
    <input asp-for="Password" class="form-control" />
    <span asp-validation-for="Password" class="text-danger"></span>
</div>
<div class="form-group">
    <label asp-for="ConfirmPassword"></label>
    <input asp-for="ConfirmPassword" class="form-control" />
    <span asp-validation-for="ConfirmPassword" class="text-
danger"></span>
</div>
<button type="submit" class="btn btn-primary">Register</button>
</form>
</div>
</div>

```

Остава да добавим линк Register в заглавното меню:



```

<li class="nav-item">
    <a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="index">List</a>
</li>
<li class="nav-item">
    <a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="create">Create</a>
</li>
<li class="nav-item">
    <a class="nav-link text-light" asp-area="" asp-controller="account" asp-action="register">Register</a>
</li>

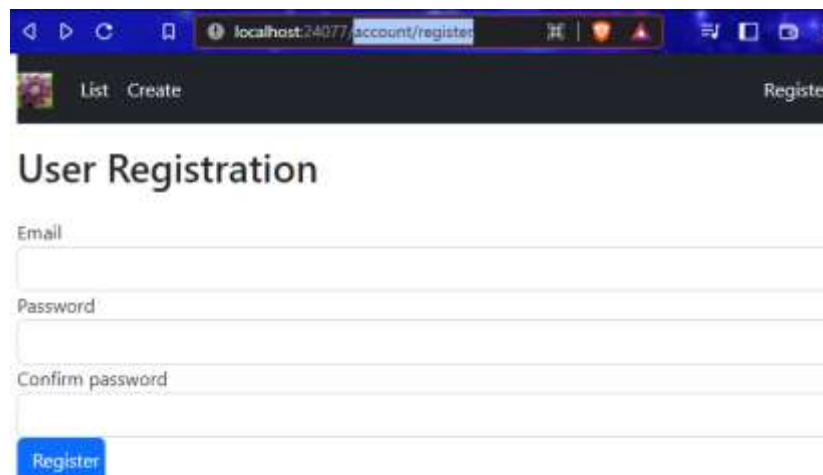
```

За да преместим елемент Register вдясно, използваме `ml-auto`

```

<ul class="navbar-nav ml-auto">
    <li class="nav-item">
        <a class="nav-link text-light" asp-area="" asp-controller="account" asp-
action="register">Register</a>
    </li>
</ul>

```



localhost:24077/account/register

List Create Register

User Registration

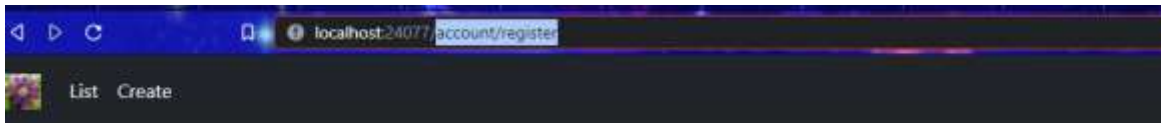
Email

Password

Confirm password

Register

Когато натиснем бутон Register, отваря Not found, защото сме декорирали само с [HttpGet], но все още нямаме post. Ако нямаше дори Get, нямаше да се случва нищо при натискане на бутона. Следва обработка на post в следващото видео.



Sorry, the resource you requested could not be found

[Click here to navigate to the home page](#)

67 - Identity UserManager and SignInManager

За да въведем частни параметри като град и пол, използваме наш custom IdentityUser class, който враждаме като generic parameter:

UserManager & SignInManager

services

UserManager<IdentityUser>

- CreateAsync
- DeleteAsync
- UpdateAsync
- Etc...

methods

SignInManager<IdentityUser>

- SignInAsync
- SignOutAsync
- IsSignedIn
- Etc...

```
AccountController.cs | RegisterViewModel.cs | HomeController.cs
EmployeeManagement.Controllers.AccountContro
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    0 references
    public class AccountController : Controller
    {
        [HttpGet]
        0 references
        public IActionResult Register()
        {
            return View();
        }

        [HttpPost]
        0 references
        public IActionResult Register(RegisterViewModel model)
        {
            return View();
        }
    }
}
```

Създаваме нов ctor, съответно извикваме необходимото namespace + private fields:

```
1 using EmployeeManagement.ViewModels;
2 using Microsoft.AspNetCore.Identity;
3 using Microsoft.AspNetCore.Mvc;
4
5 namespace EmployeeManagement.Controllers
6 {
7     1 reference
8     public class AccountController : Controller
9     {
10         private readonly UserManager<IdentityUser> userManager;
11
12         0 references
13         public AccountController(UserManager<IdentityUser> userManager,
14                                 SignInManager<IdentityUser> signInManager)
15         {
16             this.userManager = userManager;
17         }
18     }
19 }
```

В част 101 – Async and await in C# example се обяснява по-подробно за Async:

Async = asynchronous (Несинхронен) метод, който трябва да се избягва. Той изисква променливата да бъде тип await, а типът на върнатия резултат Task<>

Използваме имейл за име на потребителя и отделно неговия имейл.

```
[HttpPost]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid) Tag helper in view asp-validation-for let's user name = email email = email
    {
        var user = new IdentityUser { UserName = model.Email, Email = model.Email };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded) bool Cookie: session = false permanent = true
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors) else display all errors
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }
}
```

Използваме временни бисквитки за сесията, които се унищожават при затваряне на браузъра.

```
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;

namespace EmployeeManagement.Controllers
{
    public class AccountController : Controller
    {
        private readonly UserManager<IdentityUser> userManager;
        private readonly SignInManager<IdentityUser> signInManager;

        public AccountController(UserManager<IdentityUser> userManager,
            SignInManager<IdentityUser> signInManager)
        {
            this.userManager = userManager;
            this.signInManager = signInManager;
        }

        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Register(RegisterViewModel model)
        {
            if (ModelState.IsValid)
            {
                var user = new IdentityUser { UserName = model.Email, Email =
model.Email };
                var result = await userManager.CreateAsync(user, model.Password);
            }
        }
    }
}
```



```

        if (result.Succeeded)
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }

        return View(model);
    }
}
}

```

При изпращане на празни полета или елементарна парола като abc извежда грешки:

User Registration - The Email field is required. - The Password field is required. Email The Email field is required. Password The Password field is required. Confirm password Register	User Registration - Passwords must be at least 6 characters. - Passwords must have at least one digit ('0'-'9'). - Passwords must have at least one lowercase ('a'-'z'). - Passwords must have at least one uppercase ('A'-'Z'). Email david@abv.bg Password Confirm password Register
User Registration Password and confirmation password do not match. Email david@abv.bg Password Confirm password Register	User Registration Passwords must have at least one non alphanumeric character. Email david@abv.bg Password Confirm password Register

Save password?

Username

david@abv.bg

Password

!@abAB12

Save

Never

Интересното е, че в БД, паролата е в криптиран вид.

dbo.AspNetUsers [Data]									
dbo.AspNetUsers [Design]									
Register.cshtml									
AccountController.cs									
RegisterViewModel.cs									
Max Rows: 1000									
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	Concu
▶	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False	AQAAAAEAACcQAAAA...	BI3N2KVBNLNV...	c6058d
◊	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Register POST Action

```
[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser
        {
            UserName = model.Email,
            Email = model.Email
        };

        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }

    return View(model);
}
```

Направих опит вместо string.Empty, да бъде просто "" – работи и по двата начина

Save password?

Username

Password

Save

Never

```
foreach (var error in result.Errors)
{
    ModelState.AddModelError("", error.Description);
}
```

68 - Identity password complexity

Default Settings in PasswordOptions

```
public class PasswordOptions
{
    public int RequiredLength { get; set; } = 6;
    public int RequiredUniqueChars { get; set; } = 1;
    public bool RequireNonAlphanumeric { get; set; } = true;
    public bool RequireLowercase { get; set; } = true;
    public bool RequireUppercase { get; set; } = true;
    public bool RequireDigit { get; set; } = true;
}
```

<https://github.com/aspnet/aspnetcore>

<https://github.com/dotnet/aspnetcore> - Open Source:



[src/Identity/Extensions.Core/src/PasswordOptions.cs](#)

```
6  /// <summary>
7  /// Specifies options for password requirements.
8  /// </summary>
9  public class PasswordOptions
```

C# Showing the top match: Last indexed on Nov 6, 2021

```

4 namespace Microsoft.AspNetCore.Identity;
5
6 /// <summary>
7 /// Specifies options for password requirements.
8 /// </summary>
9 public class PasswordOptions
10 {
11     /// <summary>
12     /// Gets or sets the minimum length a password must be. Defaults to 6.
13     /// </summary>
14     public int RequiredLength { get; set; } = 6;
15
16     /// <summary>
17     /// Gets or sets the minimum number of unique characters which a password must contain. Defaults to 1.
18     /// </summary>
19     public int RequiredUniqueChars { get; set; } = 1;
20
21     /// <summary>
22     /// Gets or sets a flag indicating if passwords must contain a non-alphanumeric character. Defaults to true.
23     /// </summary>
24     /// <value>True if passwords must contain a non-alphanumeric character, otherwise false.</value>
25     public bool RequireNonAlphanumeric { get; set; } = true;

```

Може да ги персонализираме в Startup -> ConfigureServices

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString(

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>());

    services.Configure<IdentityOptions>

    services.AddMvc(option => option.EnableEndpointRouting = false)

```

Go to Definition:

```

namespace Microsoft.AspNetCore.Identity
{
    ... public class IdentityOptions
    {
        public IdentityOptions();

        ... public ClaimsIdentityOptions ClaimsIdentity { get; set; }
        ... public UserOptions User { get; set; }
        ... public PasswordOptions Password { get; set; }
        ... public LockoutOptions Lockout { get; set; }
        ... public SignInOptions SignIn { get; set; }
        ... public TokenOptions Tokens { get; set; }
        ... public StoreOptions Stores { get; set; }
    }

```

Можем да персонализираме изискванията по един от двата начина:

Override Password Default Settings

```
services.Configure<IdentityOptions>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
    options.Password.RequireNonAlphanumeric = false;
});
```

OR

```
services.AddIdentity<IdentityUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
    options.Password.RequireNonAlphanumeric = false;
})
.AddEntityFrameworkStores<AppDbContext>();
```

```
services.Configure<IdentityOptions>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
});
```

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString(

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>();

    services.Configure<IdentityOptions>(options =>
    {
        options.Password.RequiredLength = 10;
        options.Password.RequiredUniqueChars = 3;
    });

    services.AddMvc(option => option.EnableEndpointRouting = false)
```

Има възможност да направим същото директно с AddIdentity:


```

public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString('

services.AddIdentity<IdentityUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
}).AddEntityFrameworkStores<AppDbContext>());

services.AddMvc(option => option.EnableEndpointRouting = false)

```

```

services.AddIdentity<IdentityUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;
})

```

Когато се опитаме да регистрираме нов потребител, вече имаме обновени изискванията:

User Registration

- Passwords must be at least 10 characters.
- Passwords must have at least one digit ('0'-'9').
- Passwords must have at least one lowercase ('a'-'z').
- Passwords must have at least one uppercase ('A'-'Z').
- Passwords must use at least 3 different characters.

69 - Show or hide login and logout links based on login status

Show or Hide Login and Logout Links

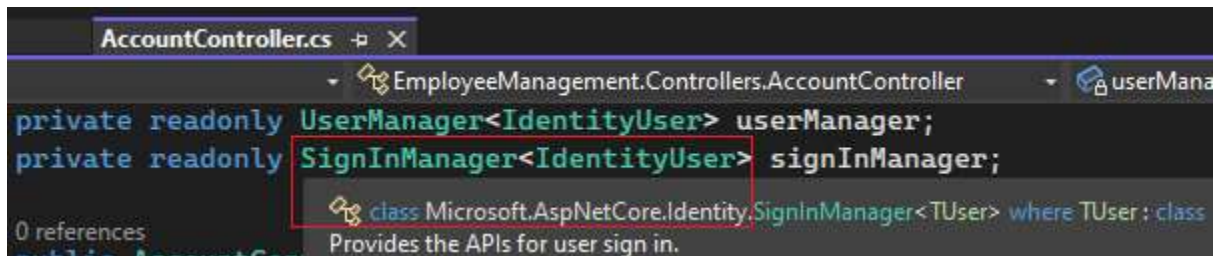
If NOT Logged In



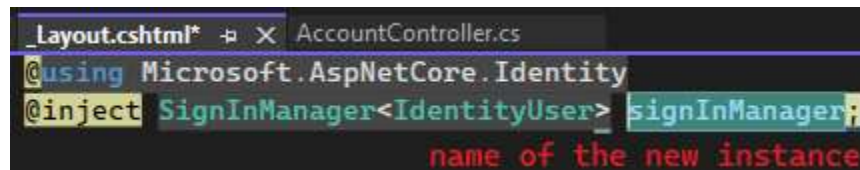
If Logged In



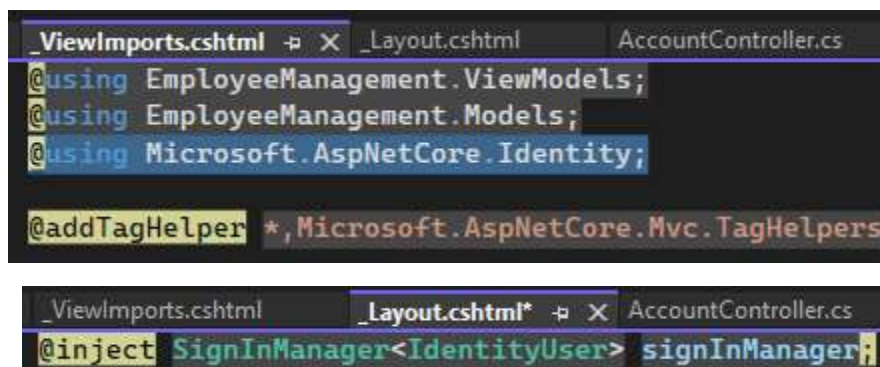
Промените за вписване и отписване трябва да се отразяват в Layout view чрез инжектиране на инстанция на SignInManager class. Първо забелязваме, че той се намира в библиотека Microsoft.AspNetCore.Identity



```
@using Microsoft.AspNetCore.Identity
@inject SignInManager<IdentityUser> signInManager;
```



Понеже тази Identity библиотека ще се налага да се използва няколко пъти, за да се намали повторението на код, ще я преместим във _ViewImports



Claims principle = loggedIn user

Не се препоръчва употреба на Get за LogOut, защото може да се злоупотреби, например при кликане върху картинка, потребителят да бъде отписан. За отписване се използва само Post:

```
swImports.cshtml | _Layout.cshtml* | AccountController.cs
<ul class="navbar-nav ml-auto">
  <li class="nav-item">
    <a class="nav-link text-light" asp-area="" asp-controller="account" asp-action="register">Register</a>
  </li>
</ul>
```

Нанасяме необходимите промени:

- * Ако е вписан – да се вижда имейла
- * ако не е – Login + Register

```
ml | _Layout.cshtml* | AccountController.cs
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
  <ul class="navbar-nav flex-grow-1"></ul>
  <ul class="navbar-nav ml-auto">
    <@if (signInManager.IsSignedIn(User))
      {
        instance method property
        <li class="nav-item"> security reasons
          <form method="post" asp-controller="account" asp-action="logout">
            <button type="submit" style="width:auto" override .css width=75px
              class="nav-link btn btn-link py-0">
              Logout @User.Identity.Name
            </button>
          </form>
        </li>
      }
    else
    {
      <li class="nav-item">
        <a class="nav-link" asp-controller="account" asp-action="register">
          Register
        </a>
      </li>
      <li class="nav-item">
        <a class="nav-link" asp-controller="account" asp-action="login">
          Login
        </a>
      </li>
    }
  </ul>
```

```
<ul class="navbar-nav ml-auto">
  @if (signInManager.IsSignedIn(User))
  {
    <li class="nav-item">
      <form method="post" asp-controller="account" asp-
action="logout">
        <button type="submit" style="width:auto"
          class="nav-link btn btn-link py-0">
```

```

                                Logout @User.Identity.Name
                                </button>
                                </form>
                            </li>
                        }
                    else
                    {
                        <li class="nav-item">
                            <a class="nav-link" asp-controller="account" asp-
action="register">
                                Register
                            </a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" asp-controller="account" asp-
action="login">
                                Login
                            </a>
                        </li>
                    }
                </ul>

```

Остава да се имплементира Login action -> AccountController.cs, за който не се изисква нищо особено

```

1 reference
public class AccountController : Controller
{
    private readonly UserManager<IdentityUser> userManager;
    private readonly SignInManager<IdentityUser> signInManager;

    0 references
    public AccountController(UserManager<IdentityUser> userManager,
        SignInManager<IdentityUser> signInManager)
    {
        this.userManager = userManager;
        this.signInManager = signInManager;
    }

    [HttpPost]
    0 references
    public async Task<IActionResult> Logout()
    {
        await signInManager.SignOutAsync();
        return RedirectToAction("index", "home");
    }
}

```

```

[HttpPost]
public async Task<IActionResult> Logout()
{
    await signInManager.SignOutAsync();
    return RedirectToAction("index", "home");
}

```

localhost:24077/account/login

[Click here to navigate to the home page](#)

Username

Password

[Logout aurora@abv.bg](#)

[illegible]

Show or Hide Login and Logout Links

```
@using Microsoft.AspNetCore.Identity
@Inject SignInManager<IdentityUser> SignInManager
<div class="collapse navbar-collapse" id="collapsibleNavbar">
  <ul class="navbar-nav ml-auto">
    @if (SignInManager.IsSignedIn(User))
    {
      <li class="nav-item">
        <form method="post" asp-controller="account" asp-action="logout">
          <button type="submit" class="nav-link btn btn-link py-0" style="width:auto">
            Logout @User.Identity.Name
          </button>
        </form>
      </li>
    }
    else
    {
      <li class="nav-item">
        <a class="nav-link" asp-controller="account" asp-action="register">Register</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" asp-controller="account" asp-action="login">Login</a>
      </li>
    }
  </ul>
</div>
```

Logout Action

```
[HttpPost]
public async Task<IActionResult> Logout()
{
    await signInManager.SignOutAsync();
    return RedirectToAction("index", "home");
}
```

70 - Implementing login functionality

Login Functionality in ASP.NET Core

LoginViewModel

Login View

Login Actions - Account Controller

Където Login Actions = за Get + Post

LoginViewModel

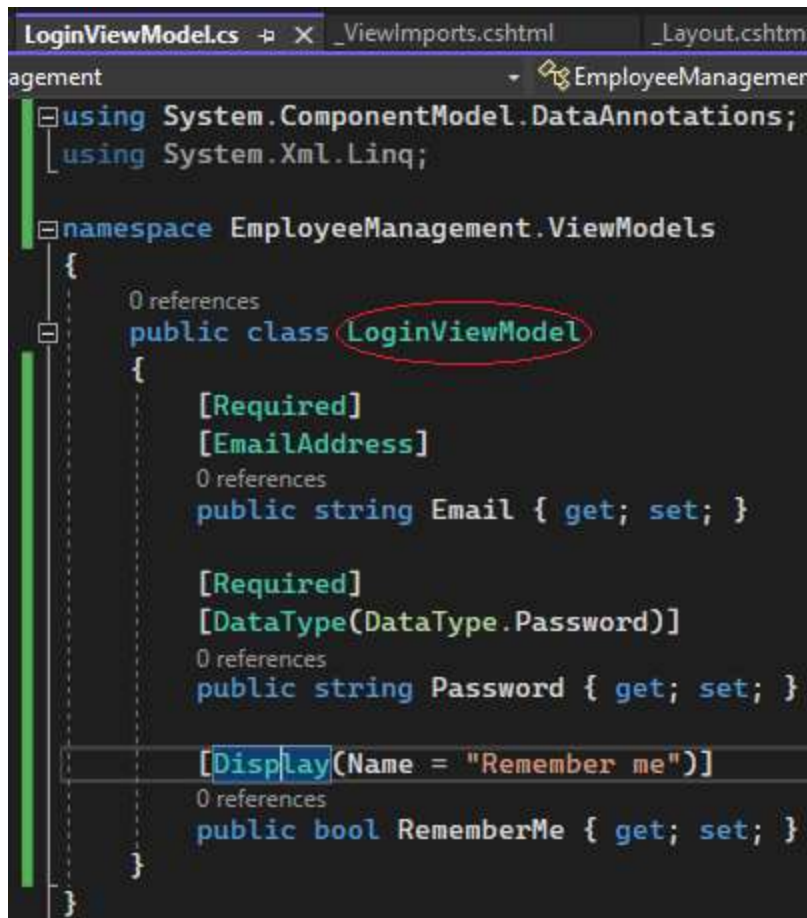
```
public class LoginViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    public bool RememberMe { get; set; }
}
```

ViewModels folder -> add -> New item -> class -> LoginViewModel.cs

Необходими са ни 3 свойства: Email, Password, Remember we:



```
using System.ComponentModel.DataAnnotations;
using System.Xml.Linq;
```

```
namespace EmployeeManagement.ViewModels
{
    public class LoginViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Display(Name = "Remember me")]
        public bool RememberMe { get; set; }
    }
}
```

След това създаваме Login view, в който клас `LoginViewModel` ще бъде модел.
Views/Account-> Add -> New item -> Razor view -> Login.cshtml

```
Login.cshtml* X LoginViewModel.cs _ViewImports.cshtml _Layout.cshtml AccountController
@model LoginViewModel

@{
    ViewBag.Title = "User Login";
}

<h1>User Login</h1>

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group">
                <div class="checkbox">
                    <label asp-for="RememberMe">
                        <input asp-for="RememberMe" />
                        @Html.DisplayNameFor(m => m.RememberMe)
                    </label>
                </div>
            </div>
        </form>
    </div>
</div>
```

+ Login bnt -> Post request

@model LoginViewModel

```
@{
    ViewBag.Title = "User Login";
}
```

<h1>User Login</h1>

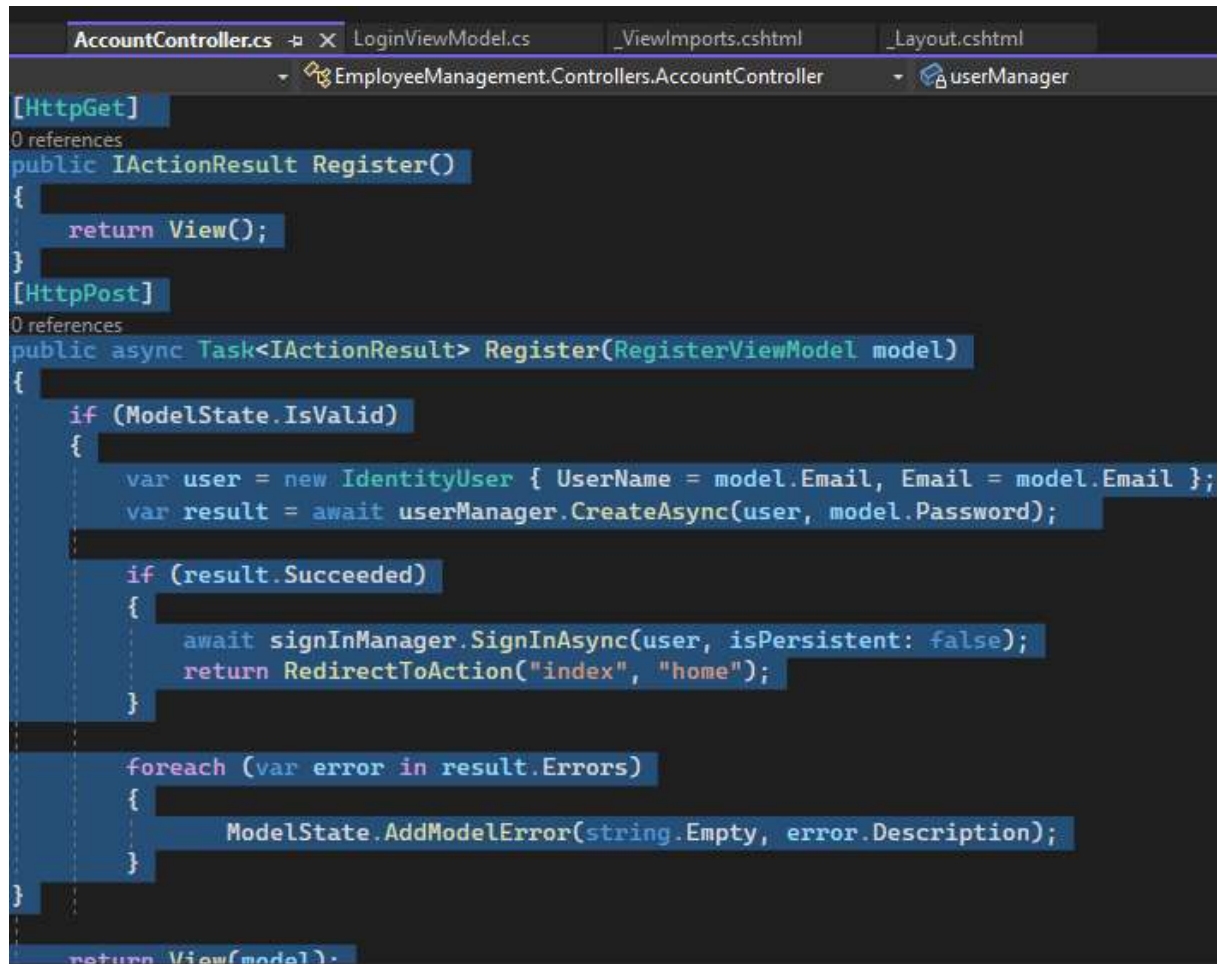
```
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group">
                <div class="checkbox">
                    <label asp-for="RememberMe">
                        <input asp-for="RememberMe" />
                        @Html.DisplayNameFor(m => m.RememberMe)
                    </label>
                </div>
            </div>
        </form>
    </div>
</div>
```

```

<div class="form-group">
    <div class="checkbox">
        <label asp-for="RememberMe">
            <input asp-for="RememberMe" />
            @Html.DisplayNameFor(m => m.RememberMe)
        </label>
    </div>
</div>
<button type="submit" class="btn btn-primary">Login</button>
</form>
</div>
</div>

```

Понеже Login много наподобява Register, може да го копираме и изменим:



```

[HttpPost]
0 references
public IActionResult Register()
{
    return View();
}

[HttpPost]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser { UserName = model.Email, Email = model.Email };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }

        return View(model);
    }
}

```


Вместо userManager -> signInManager.

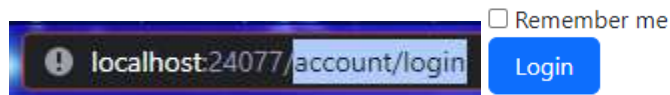
Не създаваме нов потребител!

```
AccountController.cs* X LoginViewModel.cs _ViewImports.cshtml _Layout.cshtml
EmployeeManagement.Controllers.AccountController Login(LoginViewMod

[HttpGet]
0 references
public IActionResult Login()
{
    return View();
}
[HttpPost]
0 references
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync();
        if
        ▲ 1 of 2 ▼ (awaitable) Task<Microsoft.AspNetCore.Identity.SignInResult> SignInManager<IdentityUser>
        .PasswordSignInAsync(IdentityUser user, string password, bool isPersistent, bool lockoutOnFailure)
```

Проверката за вписване изисква подаване на 4 properties: име, в случая имейл, парола, бисквитки (IsPersistent) в булев формат, които в случая идват от RememberMe, булева true/false за действие при грешка при вписване. Ако вписването е успешно, препращаме потребителя в начална страница, ако не – извежда съобщение за грешка и презарежда страницата с данните.

```
[HttpPost]
0 references
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
        model.RememberMe, false);
        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }
    }
    else ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    return View(model);
}
```



```
[HttpGet]
public IActionResult Login()
{
    return View();
}
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
        model.RememberMe, false);
        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }
    }
    else ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    return View(model);
}
```

```

        if (ModelState.IsValid)
        {
            var result = await signInManager.PasswordSignInAsync(model.Email,
model.Password,
                                model.RememberMe,
false);
            if (result.Succeeded)
            {
                return RedirectToAction("index", "home");
            }

            ModelState.AddModelError(string.Empty, "Invalid Login attempt");
        }
        return View(model);
    }
}

```

User Login

- Invalid Login attempt

Email

aurora@abv.bg

Password

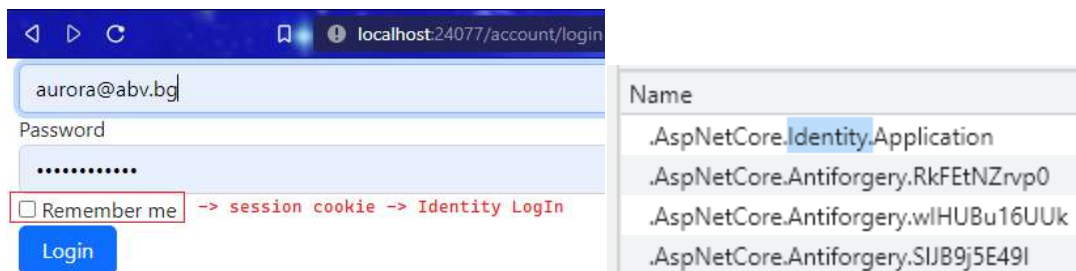
☐ Remember me

Login

Нека разберем разликите между 2-та вида бисквитки:

Name	Value	Domain	Path	Expires /..	Size
.AspNetCore.Antiforgery.RkFetNZvp0	CfDI8OSfvHT1aOVFsqe1stiqPO-dfiunO9QrTajFW5F...	localhost	/	Session	190
.AspNetCore.Antiforgery.wIHU8u16Uuk	CfDI8OSfvHT1aOVFsqe1stiqPM_p7tjzbJ3clv_1aFPw...	localhost	/	Session	190
.AspNetCore.Antiforgery.SUB9j5E49l	CfDI8OSfvHT1aOVFsqe1stiqPNzb31tqzg_Sq8bTVB...	localhost	/	Session	190

Developer Tools -> Application -> Cookies -> имаме само antiforgery (срещу фалшификации).



С тази бисквитка всеки път се подават данните от браузъра до сървъра за вписване. Понеже не чекнахме `Remember me -> session cookie`, която незабавно се губи когато се затвори подпрозореца на браузъра. Нека го проверим:

Save password?

Username

Password 

Не запомням паролата в браузъра, при Logout, бисквитка Identity се унищожава. Вписвам отново, създава се, но при мен бисквитката за Identity стои независимо дали затварям подпрозорец, целия браузър, дали стартирам приложението наново. В туториъла обаче бисквитките изчезват и `register-login` се занулява. Запазва се само при маркиране на `Remember me`. Ако искаме да занулим постоянна `persistent cookie`, просто даваме Logout, така че при зареждане на браузъра наново, ще трябва да се впишем.

Login View

```
@model LoginViewModel
<form method="post">
  <div asp-validation-summary="All" class="text-danger"></div>
  <div class="form-group">
    <label asp-for="Email"></label>
    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
  </div>
  <div class="form-group">
    <label asp-for="Password"></label>
    <input asp-for="Password" class="form-control" />
    <span asp-validation-for="Password" class="text-danger"></span>
  </div>
  <div class="form-group">
    <div class="checkbox">
      <label asp-for="RememberMe">
        <input asp-for="RememberMe" />
        @Html.DisplayNameFor(m => m.RememberMe)
      </label>
    </div>
  </div>
  <button type="submit" class="btn btn-primary">Login</button>
</form>
```

Login Actions

```
[HttpGet]
public IActionResult Login()
{
    return View();
}
```

```
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email,
            model.Password, model.RememberMe, false);

        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }

        ModelState.AddModelError(string.Empty, "Invalid Login Attempt");
    }

    return View(model);
}
```

71 – Authorization

След идентификация, имаме достъп до информация и възможности какво можем и не можем да правим. Това е авторизация.

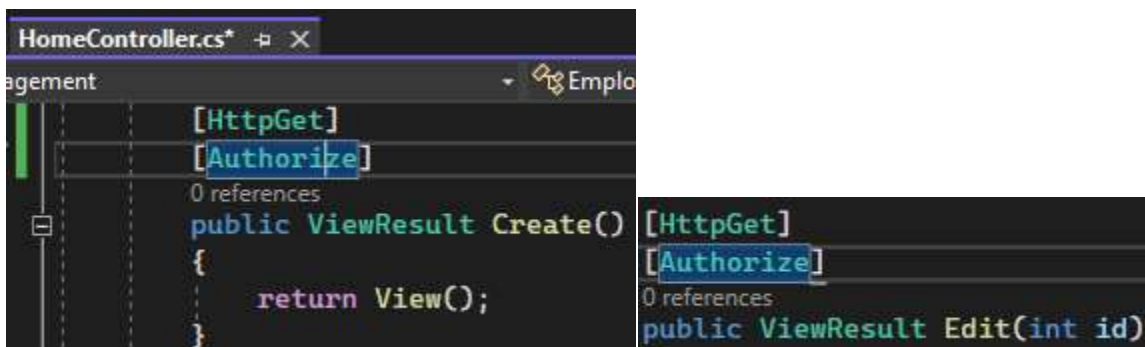
Authorization in ASP.NET Core

Authentication is identifying who the user is

Authorization is identifying what the user can and cannot do

[Authorize] attribute controls Authorization

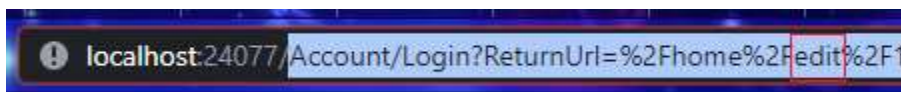
HomeController -> Index + Details actions нямат проблеми за достъп от анонимен потребител, но Create + Edit имат нужда от защита, затова ги декорираме с [Authorize] атрибут. Това е необходимо за Get + Post.



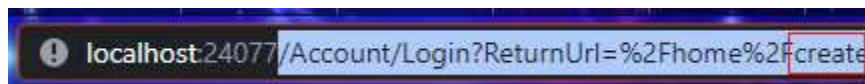
```
HomeController.cs* X
Management
[HttpGet]
[Authorize]
0 references
public ActionResult Create()
{
    return View();
}

[HttpGet]
[Authorize]
0 references
public ActionResult Edit(int id)
```

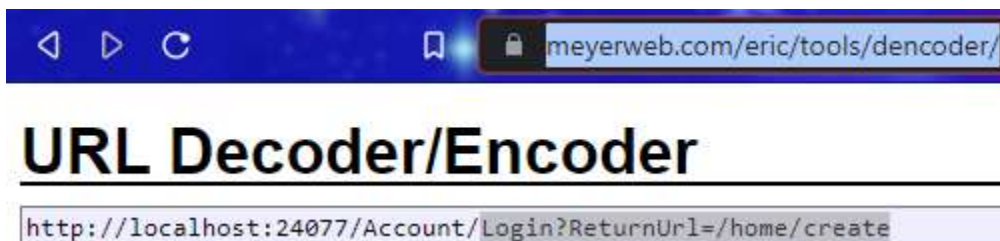
При Logout, вече имаме възможност да виждаме списъка и детайлите на служителите, но в момента, в който поискаме с бутоните да създадем или редактираме служител, ни пренасочва към линк за Login.



Account/Login?ReturnUrl=%2Fhome%2Fcreate



<https://meyerweb.com/eric/tools/dencoder/>



Откъдето забелязваме, че %2F съответства на знака /

Зададен в този вид адреса с ReturnUrl= ще препрати потребителя към /home/create веднага, когато се впише.

Вместо индивидуално навсякъде да поставяме [Authorize], може да го използваме глобално в контролера, като по този начин без вписване, няма да имаме достъп до нищо от него, включително Index + Details.


```
HomeController.cs  X
EmployeeManagement
namespace EmployeeManagement.Controllers
{
    [Authorize]
    2 references
    public class HomeController : Controller
```

Ако искаме специално Index + Details да бъдат освободени за неоторизиран достъп, ги украсяваме с [AllowAnonymous]:

```
HomeController.cs*  X
EmployeeManagement.Controllers.HomeContro
[AllowAnonymous]
0 references
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}
[AllowAnonymous]
0 references
public IActionResult Details(int? id)
```

На този етап сме приложили

[Authorize] – глобално в контролера

[AllowAnonymous] – индивидуално в actions.

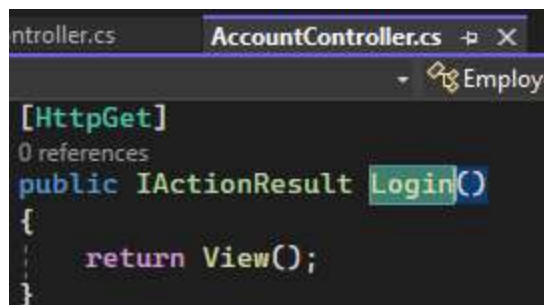
Това няма да има същият ефект, ако ги разменим според необходимостта и оставим [AllowAnonymous] в контролера и [Authorize] в индивидуалните actions, защото тогава главния [AllowAnonymous] в контролера ще засенчи и пренебрегне вградените [Authorize].

В Startup.cs може да се използва [Authorize] глобално, вместо във всеки контролер отделно чрез вградените възможности на MVC.Authorize. Трябва да създадем нов филтър:

```
services.AddMvc(option => option.EnableEndpointRouting = false)
    .AddXmlSerializerFormatters();
```

Добавяме необходимия код с двата необходими namespace:

```
using Microsoft.AspNetCore.Mvc.Authorization;
using Microsoft.AspNetCore.Authorization;
```

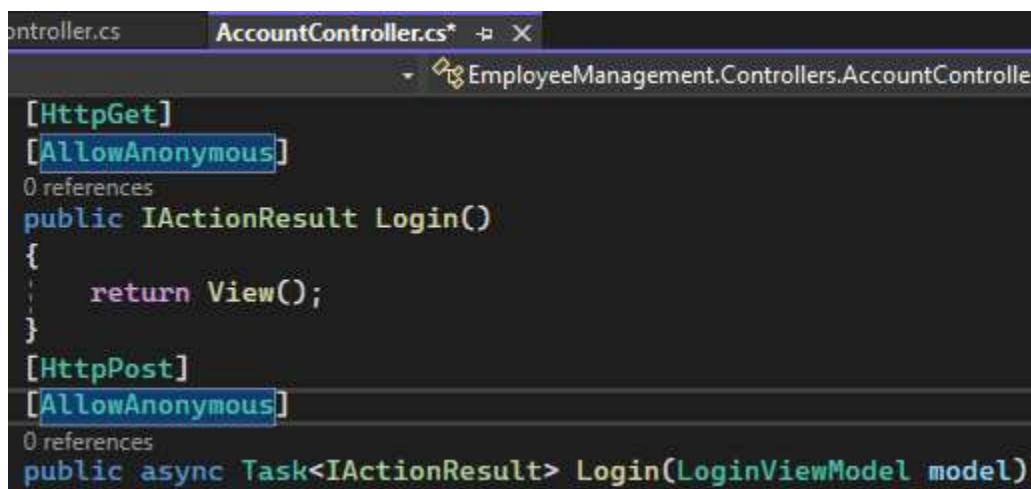



```

AccountController.cs
[HttpGet]
0 references
public IActionResult Login()
{
    return View();
}

```

Потребителят хем трябва да се впише, хем се изисква ауторизация (да бъде вече вписан за това) за това. Затова постоянно прави Redirect to URL -> Login, а когато стигне Login, се изисква пак препращане към същия адрес, докато не стане прекалено дълъг и сървърът даде грешка при обработката. За да се поправи тази грешка, просто добавяме [AllowAnonymous] -> Login:

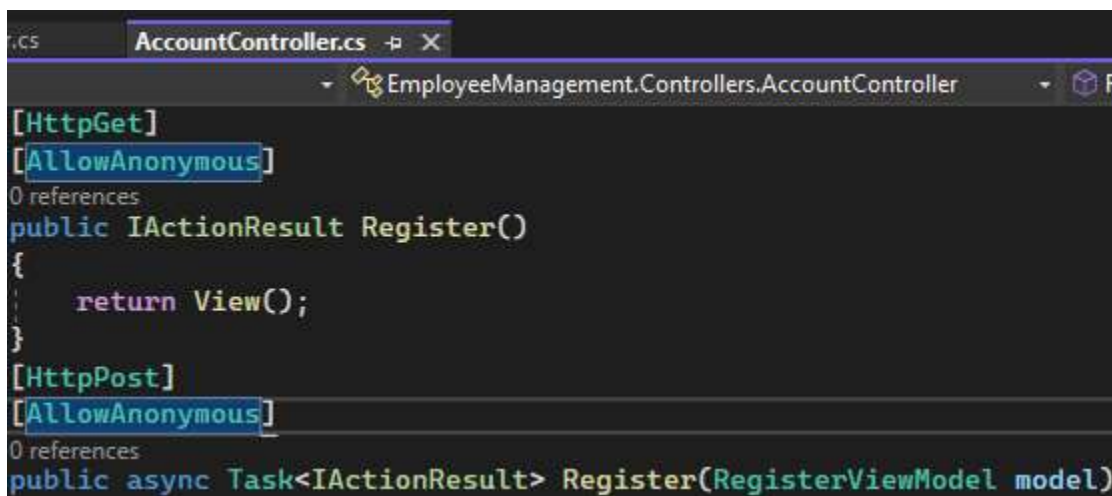


```

AccountController.cs*
EmployeeManagement.Controllers.AccountController
[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Login()
{
    return View();
}
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model)

```

Както и пред Register, за да може новите потребители да имат този достъп.



```

AccountController.cs
EmployeeManagement.Controllers.AccountController
[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Register()
{
    return View();
}
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)

```

Това е кодът, с който създадохме филтър и политика за ауторизация:

Apply Authorize Attribute Globally

```
public void ConfigureServices(IServiceCollection services)
{
    // Other Code

    services.AddMvc(config => {
        var policy = new AuthorizationPolicyBuilder()
            .RequireAuthenticatedUser()
            .Build();
        config.Filters.Add(new AuthorizeFilter(policy));
    });

    // Other Code
}
```

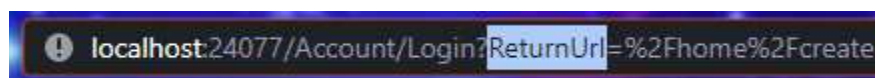
На този етап създаваме аутентификация без допълнителни специфики, но има възможности за настройки, които ще разгледаме по-нататък.

Authorization in ASP.NET Core



72 - Redirect user to original url after login

На този етап, при вписване, автоматично се връщаме в /home/index, но



ReturnUrl ни дава възможност след вписване да останем на адреса след него, в случая /home/create.

```
ntroller.cs AccountController.cs [X]
  EmployeeManagement.Controllers.AccountController Login(LoginViewModel model)

[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                                model.RememberMe, false);

        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }

        ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    }

    return View(model);
}
```

Това може да се случи чрез `modelBinding`, където подаваме променлива `returnUrl` като стринг, за да може автоматично да я прихване:

```
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                                model.RememberMe, false);

        if (result.Succeeded)
        {
            ! if is not null
            if(!string.IsNullOrEmpty(returnUrl))
            {
                return Redirect(returnUrl);
            }
            else
            {
                return RedirectToAction("index", "home");
            }
        }
    }
}
```

```
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email,
model.Password,
model.RememberMe,
false);
        if (result.Succeeded)
```



```

{
    if(!string.IsNullOrEmpty(returnUrl))
    {
        return Redirect(returnUrl);
    }
    else
    {
        return RedirectToAction("index", "home");
    }
}

```

Redirect to Original URL after Login

<http://localhost/Account/Login?ReturnUrl=%2Fhome%2Fcreate>

Model Binding

```

public IActionResult Login(LoginViewModel model, string returnUrl)
{
    if (isLoginSuccess)
    {
        if (!string.IsNullOrEmpty(returnUrl))
        {
            return Redirect(returnUrl);
        }
        else
        {
            return RedirectToAction("index", "home");
        }
    }
    return View(model);
}

```

По този начин, от /home/create при създаване на нов акаунт, оставаме на същата страница, което обаче създава сериозен проблем в сигурността.

73 - Open redirect vulnerability

Application Vulnerable to Open Redirect Attacks

- Your application redirects to a URL that's specified via the request such as the **querystring** or **form data**
- The redirection is performed without checking if the URL is a **local URL**

Към момента, и двете условия за уязвимост са изпълнени заради променливата return URL, без да имаме проверка дали URL е локален.

```
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                            model.RememberMe, false);
        if (result.Succeeded)
        {
            if(!string.IsNullOrEmpty(returnUrl))
            {
                LocalRedirect - in our application
                return Redirect(returnUrl);
            }
            else
            {
                external, trusted URL
                return RedirectToAction("index", "home");
            }
        }
    }

    ModelState.AddModelError(string.Empty, "Invalid Login attempt");
}
```

LocalRedirect – only to local URL If we provide an external URL – exception.

In Developer environment:

An unhandled exception occurred while processing the request.

InvalidOperationException: The supplied URL is not local. A URL with an absolute path is considered local if it does not have a host/authority part. URLs using virtual paths ("~/") are also local.

Microsoft.AspNetCore.Mvc.Infrastructure.LocalRedirectResultExecutor.ExecuteAsync(ActionContext context, LocalRedirectResult result)

Open Redirect Vulnerability Example

- The user is tricked into clicking a link in an email where the **returnUrl** is set to the attackers website

I -> I

<http://example.com/account/login?returnUrl=http://example.com/account/login>

- The user logs in successfully on the authentic site and he is then redirected to the attackers website
- The user logs in again on the attackers website, thinking that the first login attempt was unsuccessful
- The user is then redirected back to the authentic site
- During this entire process, the user does not know his credentials are stolen

```
if(!string.IsNullOrEmpty(returnUrl)) // && URL.IsLocalURL(returnURL));  
{  
    return LocalRedirect(returnUrl);  
}
```

Prevent Open Redirect Attacks

```
public IActionResult Login(string returnUrl)  
{  
    return LocalRedirect(returnUrl);  
}
```

OR

```
public IActionResult Login(string returnUrl)  
{  
    if (Url.IsLocalUrl(returnUrl) && Url.IsLocalUrl(returnUrl))  
    {  
        return Redirect(returnUrl);  
    }  
    else  
    {  
        return RedirectToAction("index", "home");  
    }  
}
```

Link: <http://localhost:24077/account/login> -> index.

74 - Client side validation

В части 42-43 разглеждахме server-side validation, при които имаме атрибути като [Required], [StringLength]... забавящи обработката и връщането на данните към клиента с милисекунди. Това може да бъде спестено с проверка в локалната машина на потребителя.

ASP.NET Core Server Side Validation

The screenshot shows a 'User Login' form with two input fields: 'Email' and 'Password'. Both fields have red error messages below them: 'The Email field is required.' and 'The Password field is required.' respectively. A 'Remember me' checkbox and a 'Login' button are at the bottom. To the right, a code snippet shows the `LoginViewModel` class with `[Required]` attributes on the `Email` and `Password` properties.

```
public class LoginViewModel
{
    [Required]
    public string Email { get; set; }

    [Required]
    public string Password { get; set; }
}
```

`if (ModelState.IsValid)` - Breakpoint

ASP.NET Core Client Side Validation

jquery.js

jquery.validate.js

jquery.validate.unobtrusive.js

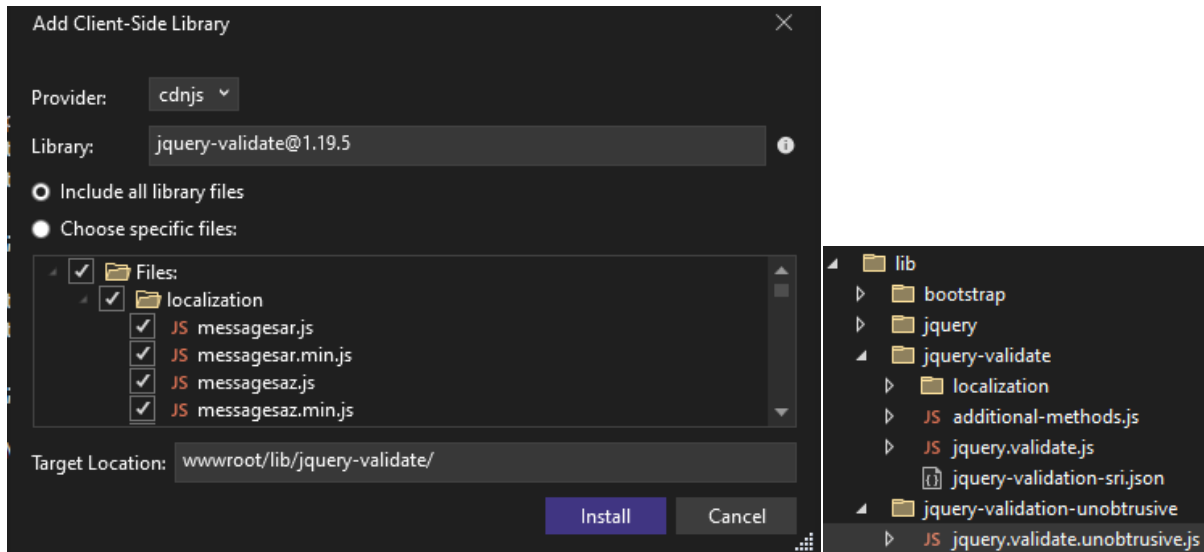
- You may use **Library Manager** to install client side libraries
- **Library Manager : Part 34 of ASP.NET Core tutorial**

Към момента имаме jquery, остава да инсталираме и останалите.

oller.cs _Layout.cshtml AccountController.cs*

```
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
<script src="~/lib/jquery/jquery.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>
```

Project folder -> Add -> Client-side library -> jquery-validate



D&D

oller.cs _Layout.cshtml AccountController.cs* jquery.validate.unobtrusive.js

```
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
<script src="~/lib/jquery/jquery.js"></script>
<script src="~/lib/jquery-validate/jquery.validate.js"></script>
<script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>
```



```

ntroller.cs | _Layout.cshtml | AccountController.cs* | jquery.validate.unobtrusive.js
EmployeeManagement.Controllers.AccountControl | Login(LoginViewModel model, string returnUrl)

[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    if (ModelState.IsValid) BreakPoint
    {
        var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                                model.RememberMe, false);

        if (result.Succeeded)
        {
            if(!string.IsNullOrEmpty(returnUrl)) // && URL.LocalURL(returnURL));
            {
                return LocalRedirect(returnUrl);
            }
            else
            {
                return RedirectToAction("index", "home");
            }
        }

        ModelState.AddModelError(string.Empty, "Invalid Login attempt");
    }
    return View(model); on the local machine check
}

```

localhost:24077/account/login

List Create

User Login

- The Email field is required.
- The Password field is required.

Email

The Email field is required.

Password

The Password field is required.

☐ Remember me

Login

User Login

- The Email field is required.
- The Password field is required.

Email

The Email field is not a valid e-mail address.

Password

The Password field is required.

☐ Remember me

Login

В този урок, още след въвеждане на първия знак в полето за парола, проверката тече автоматично на локалната машина и съобщението за грешка изчезва, съответно ако този знак се изтрие и полето отново стане празно, съобщението се появява. За подобни промени в реално време е важна подредбата:

```
<script src="~/lib/jquery/jquery.js"></script>
```

```

<script src="~/lib/jquery-validate/jquery.validate.js"></script>
<script src="~/lib/jquery-validation-
unobtrusive/jquery.validate.unobtrusive.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>

```

```

public class LoginViewModel
{
    [Required]
    [EmailAddress]
    4 references
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    4 references
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    4 references
    public bool RememberMe { get; set; }
}

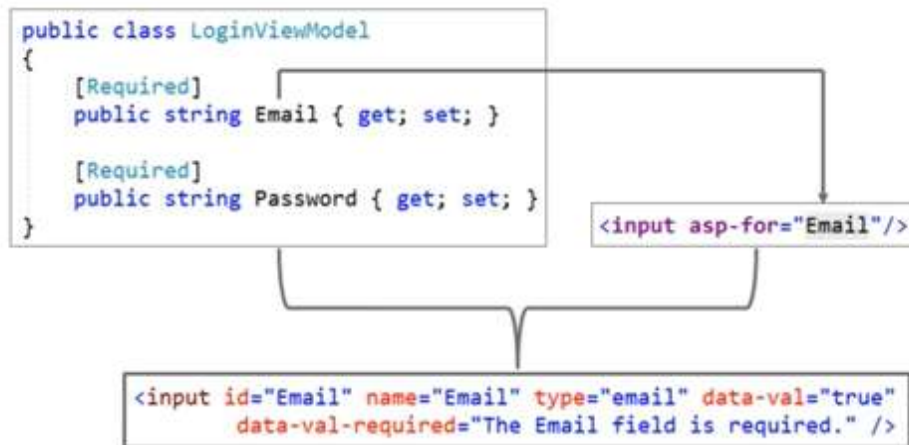
```

```

<div class="form-group">
    <label for="Email">Email</label>
    <input class="form-control" type="email" data-val="true" data-val-email="The Email field is not a valid e-mail address." data-val-required="The Email
    <span class="text-danger field-validation-valid" data-valmsg-for="Email" data-valmsg-replace="true"></span>
</div>

```

data-val attributes



➤ **jquery.validate.unobtrusive.js** reads **data-val** attributes and performs the client side validation.

```
Startup.cs  HomeController.cs  Layout.cshtml  AccountController.cs  jquery.validate.unobtrusive.js
<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
  <script src="~/lib/jquery/jquery.js"></script>
  <script src="~/lib/jquery-validate/jquery.validate.js"></script>
  <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.js"></script>
</environment>
<environment exclude="Development"> Staging, Production...
  <!-- CSS only -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-gH2yIjQKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNL/vI1Bx"
    crossorigin="anonymous"
    asp-fallback-href="~/lib/bootstrap/css/bootstrap.min.css"
    asp-fallback-test-class="sr-only"
    asp-fallback-test-property="position"
    asp-fallback-test-value="absolute"
    asp-suppress-fallback-integrity="true">
  <script src="~/lib/jquery/jquery.slim.min.js"></script> No validation
  <script src="~/lib/bootstrap/js/bootstrap.esm.min.js"></script>
</environment>
```

Client Side Validation - Not Working

- Make sure browser support for JavaScript is **NOT** disabled
- Make sure the client-side validation libraries are **loaded in the order specified**

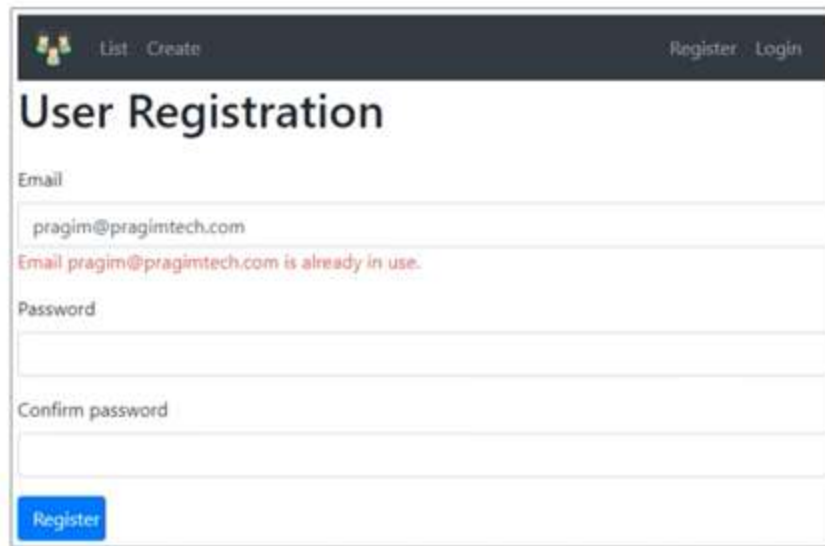
```
<script src="jquery.js"></script>
<script src="jquery.validate.js"></script>
<script src="jquery.validate.unobtrusive.js"></script>
```

- Make sure the validation libraries are loaded **for the environment you are testing against**. Development, Staging, Production etc.

75 - Remote validation

Съобщение за грешка като зает имейл при нова регистрация, от гледна точка на производителност, е по-добре да се провери на сървъра, отколкото на локалната машина. За да се извърши на локалния компютър, трябва всички заети имейли да му бъдат изпратени от сървъра, където да се осъществи тази проверка. При 1 мил. заети адреса, това е ужасна идея. Това е един от случаите, в които валидация с отдалечен достъп е екстремно полезна.

ASP.NET Core Remote Validation



Първото, което трябва да направим, е server-side function check дали съществува този имейл. Ако не е, да позволи създаването на нов.

MVC uses jQuery validate method to call IsEmailInUse server-side function.

jQuery validate uses AJAX call to this IsEmailInUse method.

jQuery validate method expects a JSON response.

`Json($"Email{email} is already in use")` – C# 6 string interpolation.

```
up.cs  HomeController.cs  AccountController.cs  X
- EmployeeManagement.Controllers.AccountControl  IsEmailInUse
[AcceptVerbs("Get", "Post")] // [HttpGet][HttpPost]
[AllowAnonymous] even non-registered
0 references
public async Task<IActionResult> IsEmailInUse(string email)
{
    var user = await userManager.FindByEmailAsync(email); // returns the name
    if (user == null) email is not taken
    {
        return Json(true);
    }
    else
    {
        return Json($"Email{email} is already in use");
    }
}
```

```
[AcceptVerbs("Get", "Post")] // [HttpGet][HttpPost]
```



```

[AllowAnonymous]
public async Task<IActionResult> IsEmailInUse(string email)
{
    var user = await userManager.FindByEmailAsync(email);
    if (user == null)
    {
        return Json(true);
    }
    else
    {
        return Json($"Email{email} is already in use");
    }
}

```

Към момента, грешка при валидацията ще се изпише едва след натискане на бутона за регистрация,

User Registration

- Username 'aurora@abv.bg' is already taken.

Email

Password

Confirm password

Register

[Remote(action:"IsEmailInUse", controller:"Account")]

```

LoginViewModel.cs
AccountController.cs
EmployeeManagement.ViewModels.LoginViewMod

4 references
public class LoginViewModel
{
    [Required]
    [EmailAddress]
    [Remote(action:"IsEmailInUse", controller:"Account")]
    4 references
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    4 references
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    4 references
    public bool RememberMe { get; set; }
}

```

localhost:15410/account/register

List Create

User Registration

Email

Email pragim@pragimtech.com is already in use

Password

Confirm password

Register

В този урок, веднага след изписването на имейла и смяната в полето под него, се изписва съобщението за грешка. Ако се върнем и сменим дори с 1 знак имейла, грешката незабавно изчезва, но при мен не се появява подобна корекция в реално време. Не го считам за проблем, т.к. проверих в други няколко пъти и по същият начин не прави проверка в реално време.

ASP.NET Core Remote Validation

```
public class RegisterViewModel
{
    [Remote(action: "IsEmailInUse", controller: "Account")]
    public string Email { get; set; }

    // Other Properties
}

[AcceptVerbs("Get", "Post")]
public async Task<IActionResult> IsEmailInUse(string email)
{
    var user = await userManager.FindByEmailAsync(email);

    if (user == null)
    {
        return Json(true);
    }
    else
    {
        return Json($"Email {email} is already in use.");
    }
}
```



Client-Side Scripts Required

```
<script src="jquery.js"></script>
<script src="jquery.validate.js"></script>
<script src="jquery.validate.unobtrusive.js"></script>
```

76 - Custom validation attribute

Built-in Validation Attributes

- Required
- Range
- StringLength
- Compare
- Regular Expression
- etc.....

Parts 42 and 43 of ASP.NET Core tutorial

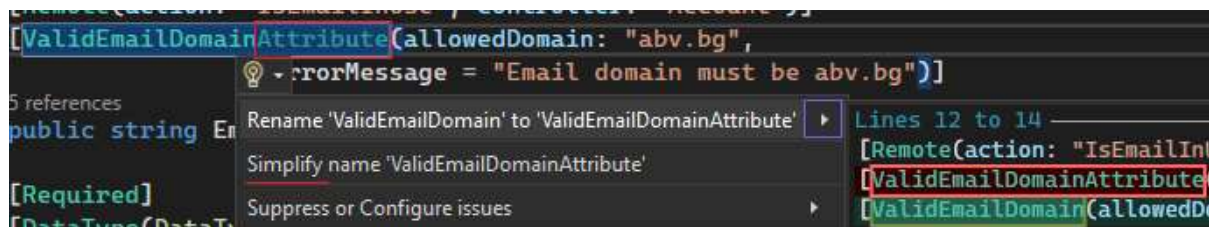
Можем да създадем свои атрибути, които да използваме в рамките на проекта или много проекти, стига да ги отделим в отделен class library project.

Custom Validation Attribute

The screenshot shows a web application interface for user registration. At the top, there is a dark navigation bar with links for 'List', 'Create', 'Register', and 'Login'. The main heading is 'User Registration'. Below the heading, a red error message states: 'Email domain must be pragimtech.com'. The form contains three input fields: 'Email' (with the value 'test@test.com'), 'Password', and 'Confirm password'. A red error message 'Email domain must be pragimtech.com' is displayed below the email field. At the bottom of the form is a blue 'Register' button.

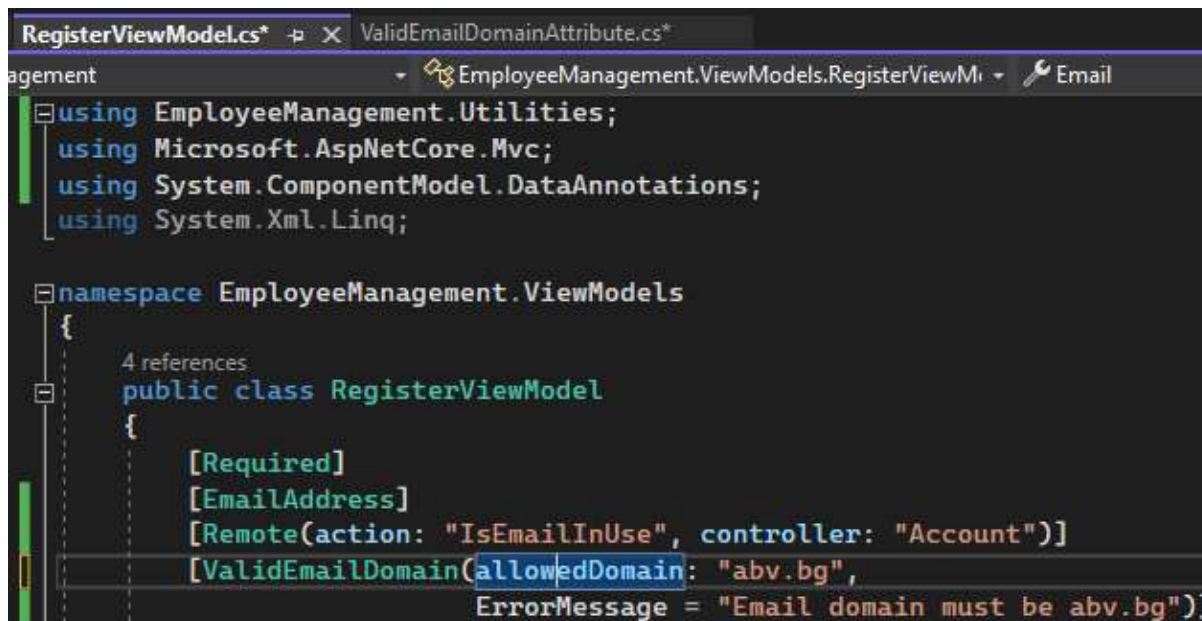
В RegisterViewModel.cs добавяме няколко необходими атрибута, чрез които разрешаваме само домейн abv.bg без да е hard-code, а останалите не се допускат.

[ValidEmailDomainAttribute] може да се прибави Attribute като пояснение за името на атрибута, но това не е задължително и може да се пропусне. Търсим опростяване.



То изисква namespace EmployeeManagement.Utilities;

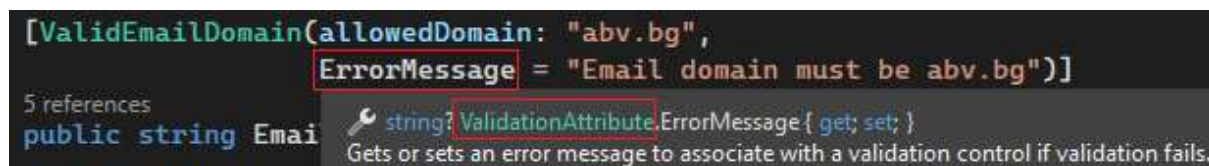
Създаваме нова променлива, която наричаме allowedDomain, която ще съдържа за по-голяма гъвкавост допустимото име за домейн, вместо да е хардкодното. Тя ще бъде с червено подчертаване до момента, в който се създаде constructor, който я използва с необходимото й private field в другия клас ValidEmailDomainAttribute.cs.



[Remote(action: \"IsEmailInUse\", controller: \"Account\")]

Първо проверяваме дали имейлът е вече зает, а след това дали домейнът е правилен.

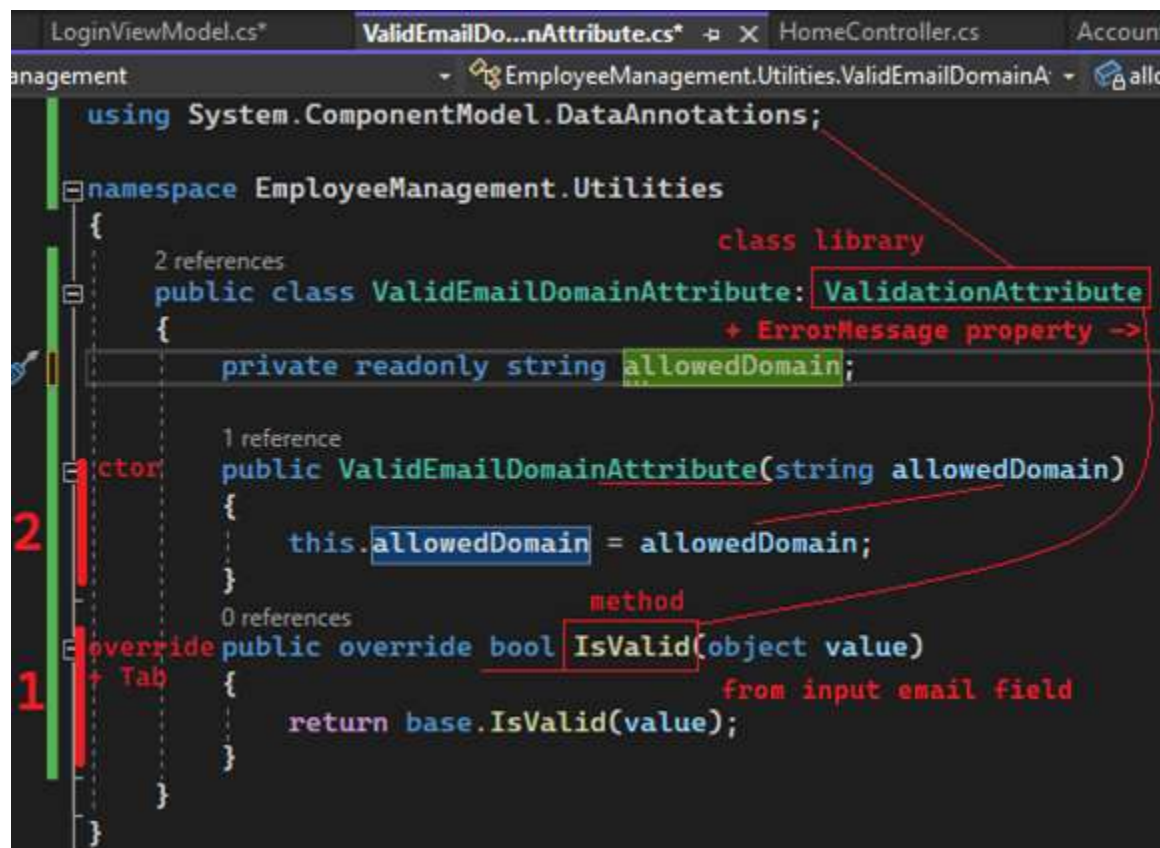
[ValidEmailDomain(allowedDomain: \"abv.bg\",
ErrorMessage = \"Email domain must be abv.bg\")]



Ако искаме да допуснем регистрация на нови потребители само с конкретен домейн имейл адрес като abv.bg / pragimtech.com. Може да използваме regular expression validator, но нека създадем custom:

Project folder -> New folder -> Utilities -> Add -> New class -> ValidEmailDomainAttribute.cs

Използваме вградена библиотека ValidationAttribute, която ни дава възможност за вграден метод IsValid, после създаваме конструктор с променливата allowedDomain, която ще се използва в RegisterViewModel.cs. Когато я създадем, автоматично червеното подчертаване в съседния файл изчезва. В същото време, тя дава възможност и за променливата в RegisterViewModel за съобщение за грешка ErrorMessage.



Value идва като променлива от input полето за регистрация на имейл на потребителя. То е тип object, а за да работим с него, трябва да го конвертираме в string, който да се раздели на 2 части: [0] преди @ и [1] след нея, което е името на домейна. На нас ни трябва втората част, която за да не бъде чувствителна на големи и малки букви, ще конвертираме само в главни букви с ToUpper. По същият начин, променливата allowedDomain конвертираме в главни букви и сравняваме двете с == за съвпадение. Включваме return, като знаем, че ще получим булева стойност за отговор при подобно сравнение.

```

public override bool IsValid(object value) object -> string
{
    convert
    string[] strings = value.ToString().Split('@');
    return strings[1].ToUpper() == allowedDomain.ToUpper();
}
value is splited in 2 strings 0-before @ and 1-after
1 and allowedDomain are converted to be keyunsensitive
1 is compared to allowedDomain

```

using System.ComponentModel.DataAnnotations;

```

namespace EmployeeManagement.Utilities
{
    public class ValidEmailDomainAttribute: ValidationAttribute
    {
        private readonly string allowedDomain;

        public ValidEmailDomainAttribute(string allowedDomain)
        {
            this.allowedDomain = allowedDomain;
        }
        public override bool IsValid(object value)
        {
            string[] strings = value.ToString().Split('@');
            return strings[1].ToUpper() == allowedDomain.ToUpper();
        }
    }
}

```

След натискане на бутона за регистрация:

User Registration

- Email domain must be abv.bg

Email

Email domain must be abv.bg

Password

Save password?

Confirm password

Username

Password



Register

Custom Validation Attribute Example

```
public class RegisterViewModel
{
    [ValidEmailDomain(allowedDomain: "pragimtech.com",
        ErrorMessage = "Email domain must be pragimtech.com")]
    public string Email { get; set; }
}
```

```
public class ValidEmailDomainAttribute : ValidationAttribute
{
    private readonly string allowedDomain;

    public ValidEmailDomainAttribute(string allowedDomain)
    {
        this.allowedDomain = allowedDomain;
    }

    public override bool IsValid(object value)
    {
        string[] strings = value.ToString().Split('@');
        return strings[1].ToUpper() == allowedDomain.ToUpper();
    }
}
```

77 - Extend IdentityUser

Why should we extend IdentityUser class

```
...public class IdentityUser<TKey> where TKey : IEquatable<TKey>
{
    ...public IdentityUser();
    ...public IdentityUser(string userName);

    ...public virtual DateTimeOffset? LockoutEnd { get; set; }
    ...public virtual bool TwoFactorEnabled { get; set; }
    ...public virtual bool PhoneNumberConfirmed { get; set; }
    ...public virtual string PhoneNumber { get; set; }
    ...public virtual string ConcurrencyStamp { get; set; }
    ...public virtual string SecurityStamp { get; set; }
    ...public virtual string PasswordHash { get; set; }
    ...public virtual bool EmailConfirmed { get; set; }
    ...public virtual string NormalizedEmail { get; set; }
    ...public virtual string Email { get; set; }
    ...public virtual string NormalizedUserName { get; set; }
    ...public virtual string UserName { get; set; }
    ...public virtual TKey Id { get; set; }
    ...public virtual bool LockoutEnabled { get; set; }
    ...public virtual int AccessFailedCount { get; set; }

    ...public override string ToString();
}
```

What if we want to store additional User data like Gender, City, Country etc.

Extend IdentityUser Class

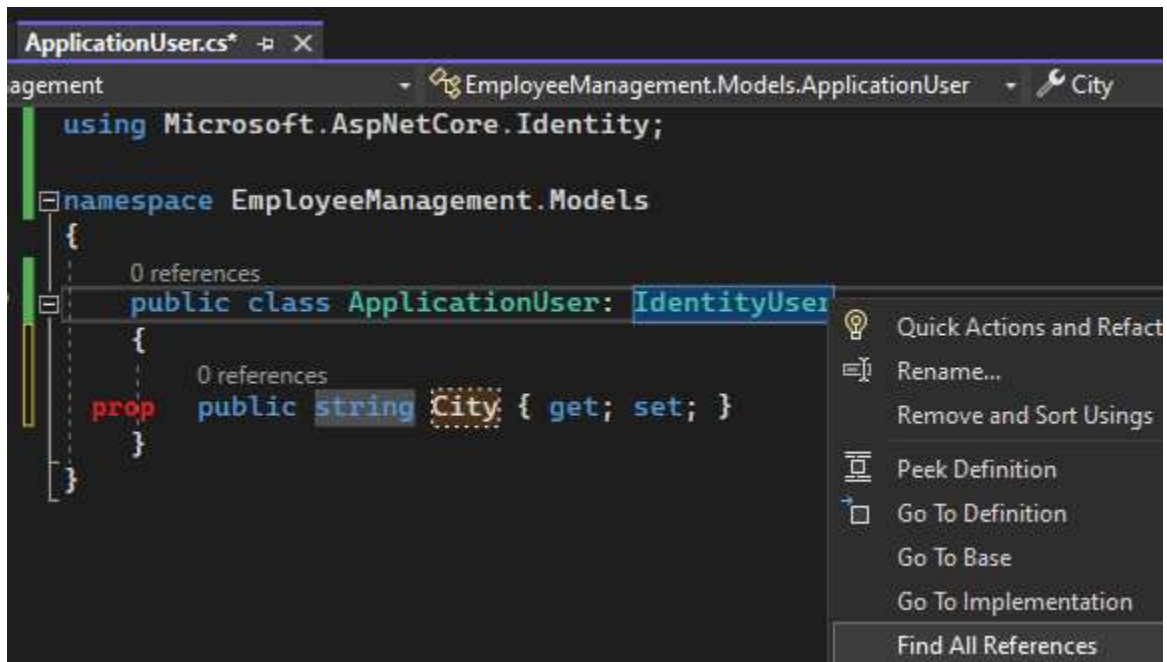
customary name

```
public class ApplicationUser : IdentityUser
{
    public string City { get; set; }
}
```

Replace all IdentityUser references with ApplicationUser

Models folder -> Add -> Class -> ApplicationUser.cs

Създаваме го с всички необходими properties, като в случая е само City, но може да са повече. После намираме всички references и най-лесният начин е с д.б. -> Find All References от контекстното меню, като в туториъла са 7, а при мен 8.



```
EmployeeManagement (7)
var user = new IdentityUser
public class ApplicationUser : IdentityUser
services.AddIdentity<IdentityUser, IdentityRole>(options =>
private readonly UserManager<IdentityUser> userManager;
private readonly SignInManager<IdentityUser> signInManager;
public AccountController(UserManager<IdentityUser> userManager,
SignInManager<IdentityUser> signInManager)
```

Имам едно споменаване в _Layout повече. Кликваме вдясно директно върху файл AccountController.cs, за да извика всички 4 наведнъж, които съдържа:

'IdentityUser' references - Entire solution

Entire solution | | Group by: Project only | Keep Results |

Code	File
EmployeeManagement (8)	
private readonly UserManager<IdentityUser> userManager;	AccountController.cs
private readonly SignInManager<IdentityUser> signInManager;	AccountController.cs
public AccountController(UserManager<IdentityUser> userManager,	AccountController.cs
SignInManager<IdentityUser> signInManager)	AccountController.cs
var user = new IdentityUser { UserName = model.Email, Email = model.Email };	AccountController.cs
public class ApplicationUser : IdentityUser	ApplicationUser.cs
services.AddIdentity<IdentityUser, IdentityRole>(options =>	Startup.cs
@inject SignInManager<IdentityUser> signInManager;	_Layout.cshtml

Навсякъде заменяме вградения IdentityUser -> ApplicationUser, който си персонализирахме.

AccountController.cs | ApplicationUserController*

EmployeeManagement.Controllers.AccountContr | userManager

```
public class AccountController : Controller
{
    ApplicationUser
    private readonly UserManager<IdentityUser> userManager;
    private readonly SignInManager<IdentityUser> signInManager;

    0 references
    public AccountController(UserManager<IdentityUser> userManager,
        SignInManager<IdentityUser> signInManager)
```

```
private readonly UserManager<ApplicationUser> userManager;
using EmployeeManagement.Models;
Models.ApplicationUser
Generate type 'ApplicationUser'
```

CS0246 The type or namespace name 'Application' found (are you missing a using directive or an assembly reference?)

```
using EmployeeManagement.Models;
```

Внимаваме да не коригираме току-що създадения. Ние искаме IdentityUser да остане вграден, извикан в ApplicationUser.

```
public class ApplicationUser: IdentityUser
{
    public string { get; set; }
}

User' references - Entire solution
solution | [icon] | [icon] | [icon] | Group by: Project only | [icon] Keep Results
Code | File
EmployeeManagement (8)
private readonly UserManager<IdentityUser> userManager; AccountController.cs
private readonly SignInManager<IdentityUser> signInManager; AccountController.cs
public AccountController(UserManager<IdentityUser> userManager, AccountController.cs
    SignInManager<IdentityUser> signInManager) AccountController.cs
var user = new IdentityUser { UserName = model.Email, Email = model.Email }; AccountController.cs
public class ApplicationUser: IdentityUser ApplicationUser.cs
```

Не съм убедена дали да правя промяна там, но не ми даде грешка при корекция:

```
_Layout.cshtml | AccountController.cs* | ApplicationUser.cs*
@Inject SignInManager<IdentityUser> signInManager
```

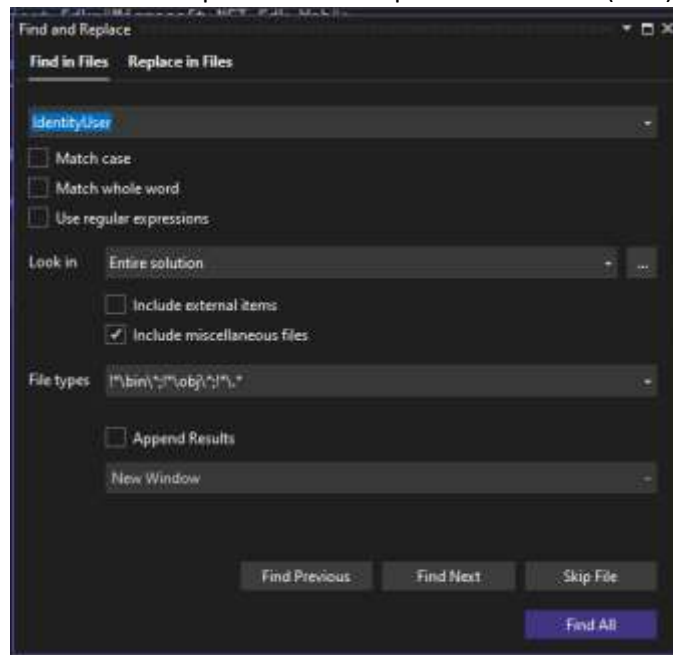
Проверяваме отново всички референции и трябва да е останала само 1 – на декларирането:

В старите версии просто не го е показвало като Reference, затова се извежда error в браузъра

An unhandled exception occurred while processing the request.

InvalidOperationException: No service for type 'Microsoft.AspNetCore.Identity.SignInManager'[(Microsoft.AspNetCore.Identity.IdentityUser)] has been registered.
Microsoft.Extensions.DependencyInjection.ServiceProviderServiceExtensions.GetRequiredService(IServiceProvider provider, Type serviceType)

Затова селектира папката на проекта -> Ctrl + F (Find):



Където показва нашия `_Layout`. След корекцията, страницата се зарежда правилно.

Следващата стъпка е да въведем колона за този нов property City.

Когато разлистим докрай таблицата, в нея няма колона за град:

	St...	PhoneNumber	PhoneNumber...	TwoFactorEna...	LockoutEnd	LockoutEnabled	AccessFailedC...
d...		NULL	False	False	NULL	True	0
f...		NULL	False	False	NULL	True	0
6...		NULL	False	False	NULL	True	0
a...		NULL	False	False	NULL	True	0
3...		NULL	False	False	NULL	True	0
		NULL	NULL	NULL	NULL	NULL	NULL

```
PM> add-migration Extend_IdentityUser
```

Забелязваме обаче, че кодът на новата миграция не изпълнява нищо – няма добавяне на City:

The screenshot shows a Visual Studio editor with a C# file named `Extend_IdentityUser : Migration`. The code includes a `protected override void Up(MigrationBuilder migrationBuilder)` method, which is currently empty except for a red box containing the text "no code for City". Below the code, the Package Manager Console shows the command `PM> add-migration Extend_IdentityUser` and its output: "Build started...", "Build succeeded.", and a warning: "The Entity Framework tools version '6.0.8' is older than that of the runtime '6.0.10'. Update the tools for the latest features and bug fixes. See https://aka.ms/AAC1fbw for more information." The console also shows "To undo this action, use Remove-Migration."

Затова я премахваме. Причината е, че класът, който отговаря за DB, не знае за нашето разширение, затова ние го влягаме като `<generic parameter>`

The screenshot shows the `AppDbContext.cs` file in Visual Studio. The class `AppDbContext` is defined as `public class AppDbContext : IdentityDbContext<ApplicationUser>`. The `ApplicationUser` generic parameter is highlighted with a red box.

С тази промяна вече, миграцията съдържа код за нова колона City в таблица AspNetUsers:

The screenshot shows the `Up` method in the migration class. The code is `migrationBuilder.AddColumn<string>(name: "City", table: "AspNetUsers", type: "nvarchar(max)", nullable: true);`. The `Up` method signature is circled in red, and the entire `AddColumn` call is enclosed in a red box.

Потвърждаваме миграцията и приложението ѝ в БД с `update-database`. Само `Refresh` не е достатъчно за опресняване на таблицата – затваряме и отваряме наново:

Startup.cs dbo.AspNetUsers [Data] 2022101908192...identityUser.cs AppDbContext.cs

Това ни дава възможност за ново поле City, което добавяме където е необходимо:

```

RegisterViewModel.cs* | 2022101908192...identityUser.cs
EmployeeManagement | EmployeeManagement.ViewMod | Ci
0 references
public string City { get; set; }

Register.cshtml* | RegisterViewModel.cs
<div class="form-group">
  <label asp-for="City"></label>
  <input asp-for="City" class="form-control" />
</div>

```

```
Employee.cs | ApplicationUser.cs | AccountController.cs | Register.cshtml* | Create.cshtml*
EmployeeManagement.Controllers.AccountCont | Register(RegisterViewMo

[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City
        };
        var result = await userManager.CreateAsync(user, model.Password);
    }
}
```

User Registration

Email	
Password	
Confirm password	
City	Username julia@abv.bg
	Password 321ewqEwQ#@!
Register	

Откривам проблем обаче: имам няколко регистрирани потребителя само с register, които обаче не са влезли видимо като данни в DB, не позволява create на техните имена и имейли, а в същото време не се показват в list.

Create -> Employees

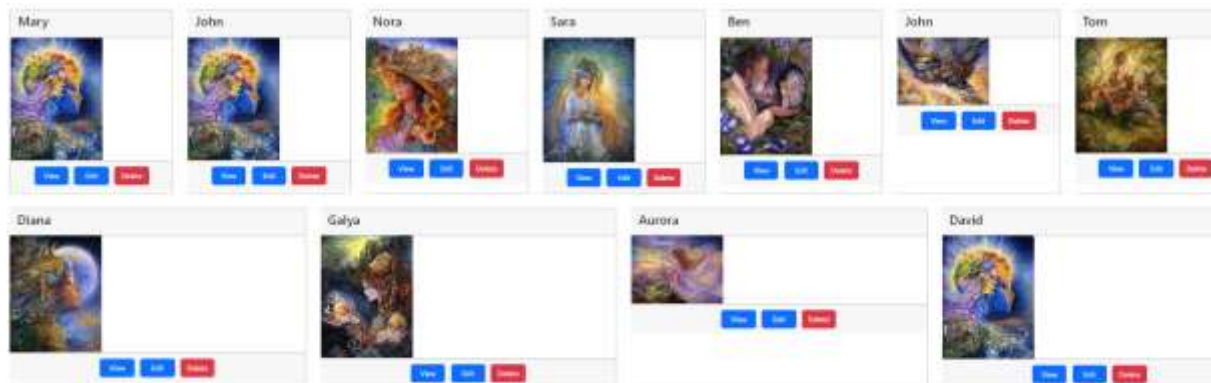
	Id	Name	Email	Department	PhotoPath
▶	1	Mary	mary@abv.bg	2	NULL
	2	John	john@abv.bg	1	NULL
	4	Nora	nora@abv.bg	1	953a612b-1894-...
	5	Sara	sara@abv.bg	1	1cfd88aa-b69d-...
	6	Ben	ben@abv.bg	2	694b5a23-c4c6-...
	7	John	john@abv.bg	2	d186b857-6296...
	8	Tom	tom@abv.bg	3	23e5040f-c96c-...
	9	Diana	diana@abv.bg	1	ae2a4c14-1306-...
	10	Galya	galya@abv.bg	0	92d15452-3d6f-...
	11	Aurora	aurora@abv.bg	1	5191edaf-b0d6-...
	12	David	david@abv.bg	2	NULL
⚙	NULL	NULL	NULL	NULL	NULL

t.e. Register -> AspNetUsers -> He ce показват в List

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	Secu
▶	7d3-cba6c7e5935e	julia@abv.bg	JULIA@ABV.BG	julia@abv.bg	JULIA@ABV.BG	False	AQAAAAEAAC...	MME
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAAEAAC...	FBRV
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False	AQAAAAEAAC...	73VP
	b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False	AQAAAAEAAC...	EK3E
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False	AQAAAAEAAC...	BI3N
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False	AQAAAAEAAC...	YMA
⚙	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

User Registration

- Username 'Tim@abv.bg' is already taken.



Например, регистриран е bob@abv.bg и той създава нов потребител със същите данни.

[Logout bob@abv.bg](#)

Bob



Employee ID: 13

Employee Email: bob@abv.bg

Employee Department: IT

Register POST Action

```
[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser
        {
            UserName = model.Email,
            Email = model.Email
        };

        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }

    return View(model);
}
```

78 - Creating roles

RoleManager in ASP.NET Core

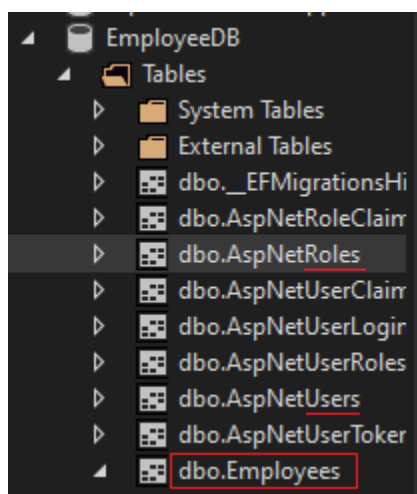
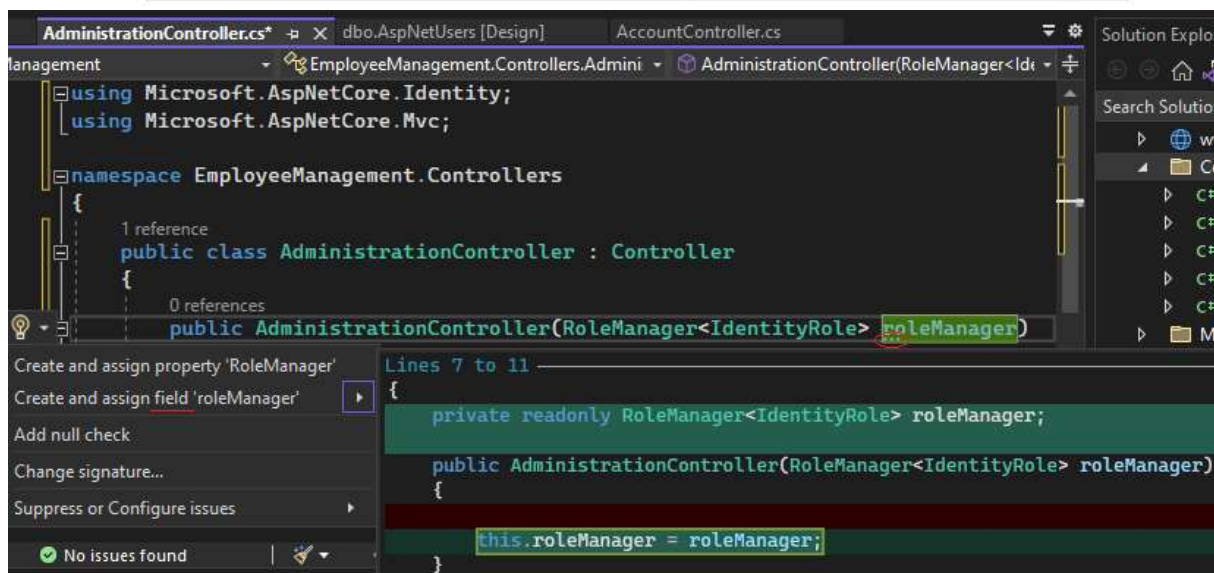
UserManager<IdentityUser>

RoleManager<IdentityRole>

```
public class AdministrationController : Controller
{
    private readonly RoleManager<IdentityRole> roleManager;

    public AdministrationController(RoleManager<IdentityRole> roleManager)
    {
        this.roleManager = roleManager;
    }

    // Rest of the code
}
```



The screenshot shows the 'dbo.AspNetRoles' table in the 'Data' view. The table has five columns: `Id`, `Name`, `NormalizedNa...`, and `ConcurrencySt...`. The first row shows all values as `NULL`.

Id	Name	NormalizedNa...	ConcurrencySt...
NULL	NULL	NULL	NULL

Като изключим таблица Employees, останалите са генерирани от Identity API (app programming interface). Към момента нямаме попълнени Roles. За да го създадем, ни трябва само име, а ID ще бъде автоматично попълнено, като за целта трябва да създадем ViewModel.

```

CreateRoleViewModel.cs
using System.ComponentModel.DataAnnotations;

namespace EmployeeManagement.ViewModels
{
    3 references
    public class CreateRoleViewModel
    {
        [Required]
        3 references
        public string RoleName { get; set; }
    }
}

```

```

AdministrationController.cs*  dbo.AspNetUsers [Design]
EmployeeManagement.Controllers.Admini CreateRole()

using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    1 reference
    public class AdministrationController : Controller
    {
        private readonly RoleManager<IdentityRole> roleManager;

        0 references
        public AdministrationController(RoleManager<IdentityRole> roleManager)
        {
            this.roleManager = roleManager;
        }

        [HttpGet]
        0 references
        public IActionResult CreateRole()
        {
            return View();
        }
    }
}

```

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
```

```

public class AdministrationController : Controller
{
    private readonly RoleManager<IdentityRole> roleManager;

    public AdministrationController(RoleManager<IdentityRole> roleManager)
    {
        this.roleManager = roleManager;
    }
    [HttpGet]
    public IActionResult CreateRole()
    {
        return View();
    }
}
}

```

Views->Account->CreateRole.cshtml, copy->paste from Register.cshtml + modify:

```

@model CreateRoleViewModel
@{
    ViewBag.Title = "Create New Role";
}

<h1>Create New Role</h1>

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="RoleName"></label>
                <input asp-for="RoleName" class="form-control" />
                <span asp-validation-for="RoleName" class="text-danger"></span>
            </div>
            <button style="width:auto" type="submit"
                class="btn btn-primary">Create Role</button>
        </form>
    </div>
</div>

```

```

@model CreateRoleViewModel
@{
    ViewBag.Title = "Create New Role";
}

<h1>Create New Role</h1>

<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="RoleName"></label>
                <input asp-for="RoleName" class="form-control" />
                <span asp-validation-for="RoleName" class="text-danger"></span>
            </div>
        </form>
    </div>
</div>

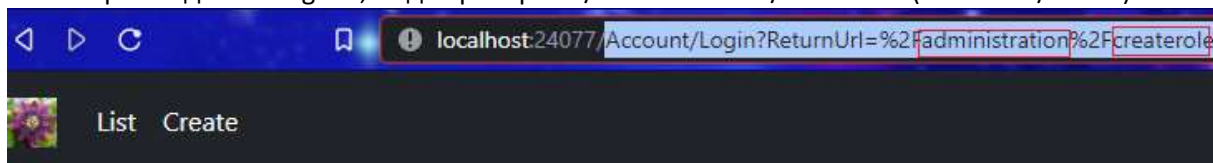
```

```

</div>
<button style="width:auto" type="submit"
        class="btn btn-primary">Create Role</button>
</form>
</div>
</div>

```

Трябва да сме LogOut, за да проверим: /administration/createrole (controller/action)



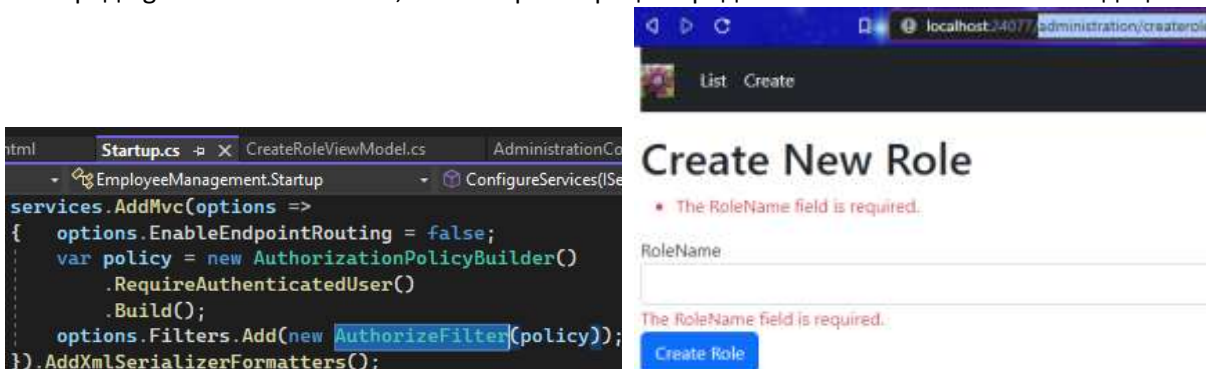
User Login

Email

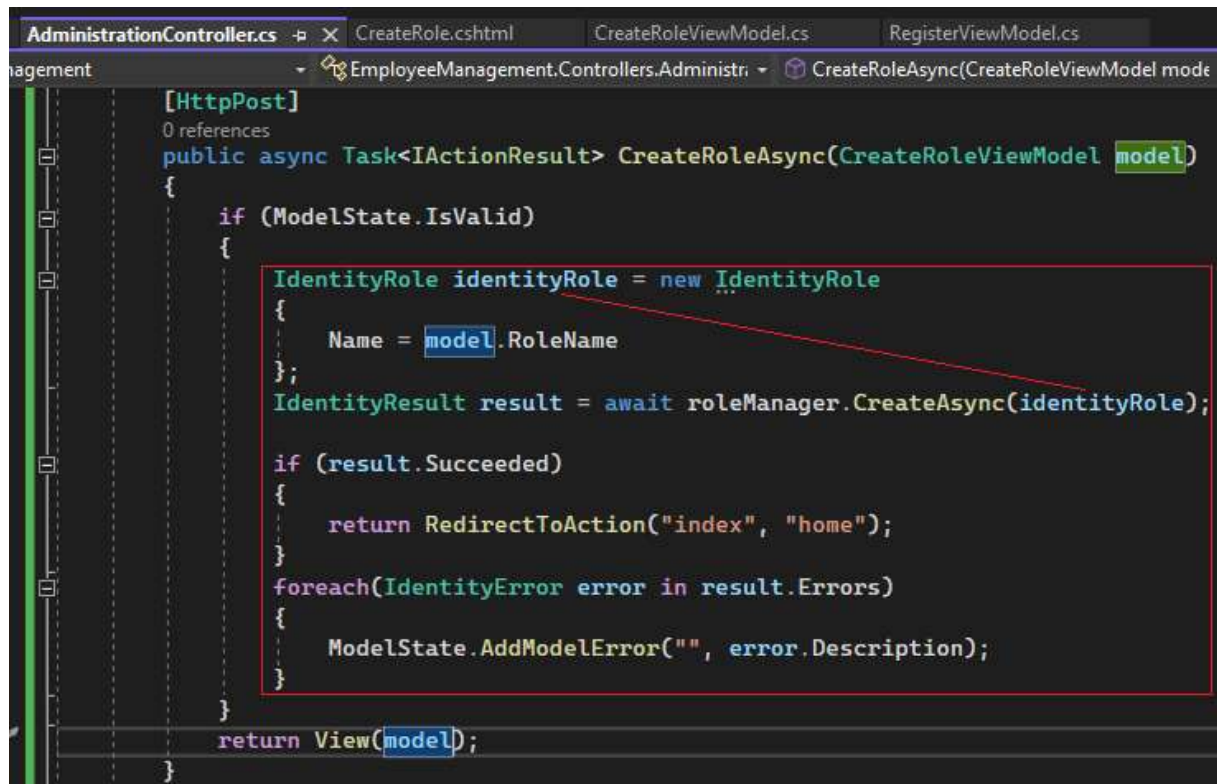
Password

☐ Remember me

Заради global authorizeFilter, изисква регистрация преди CreateNewRole. Има и валидация



Ако се опитаме да въведем role – Admin, понеже все още нямаме Post -> 404.



```
[HttpPost]
0 references
public async Task<IActionResult> CreateRoleAsync(CreateRoleViewModel model)
{
    if (ModelState.IsValid)
    {
        IdentityRole identityRole = new IdentityRole
        {
            Name = model.RoleName
        };
        IdentityResult result = await roleManager.CreateAsync(identityRole);

        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }
        foreach (IdentityError error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }
    }

    return View(model);
}
```

```
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;

namespace EmployeeManagement.Controllers
{
    public class AdministrationController : Controller
    {
        private readonly RoleManager<IdentityRole> roleManager;

        public AdministrationController(RoleManager<IdentityRole> roleManager)
        {
            this.roleManager = roleManager;
        }

        [HttpGet]
        public IActionResult CreateRole()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> CreateRoleAsync(CreateRoleViewModel model)
        {
            if (ModelState.IsValid)
            {
                IdentityRole identityRole = new IdentityRole
                {
                    Name = model.RoleName
                };
            }
        }
    }
}
```

```

        IdentityResult result = await roleManager.CreateAsync(identityRole);

        if (result.Succeeded)
        {
            return RedirectToAction("index", "home");
        }
        foreach (IdentityError error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }
    }
    return View(model);
}
}
}

```

Създаваме Role -> Admin, прехвърля ни в index, проверяваме в DB:

Startup.cs dbo.AspNetRoles [Data] AdministrationController.cs				
	Id	Name	NormalizedNa...	ConcurrencySt...
▶	139-ebbb0d1244ae	Admin	ADMIN	05fa09d0-0f8c-...
⬇	NULL	NULL	NULL	NULL

Creating Roles in ASP.NET Core

```

[HttpPost]
public async Task

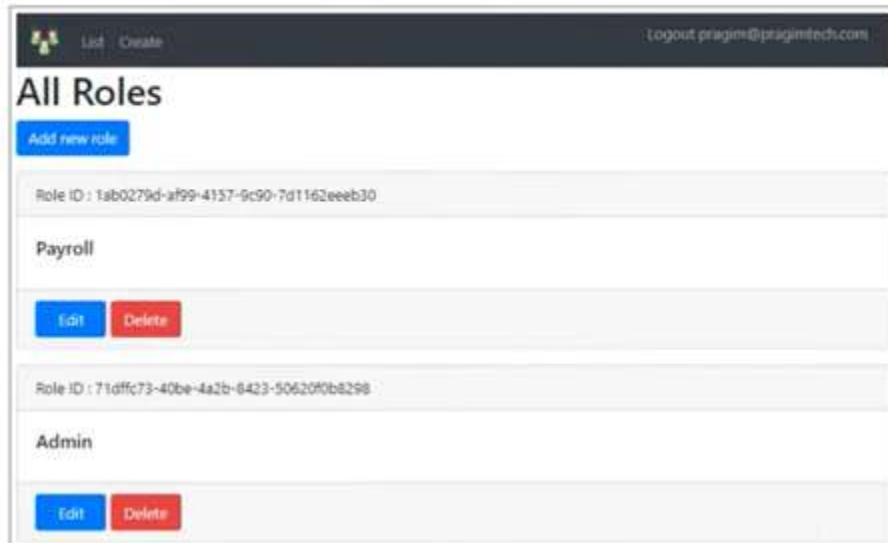
```

79 - Get list of roles

We want to display

- Role ID
- Role Name and
- A pair of buttons to Edit and Delete a role

Get list of Roles in ASP.NET Core

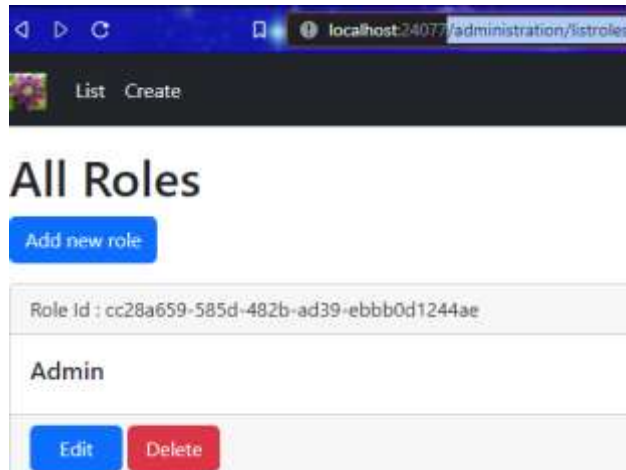


Как да извлечем от БД всички roles -> да ги изобразим във View с възможност за редакция?

```
[HttpGet]
0 references
public IActionResult ListRoles()
{
    var roles = roleManager.Roles;
    return View(roles);
}
```

System.Linq.IQueryable<IdentityRole>
Gets an IQueryable collection of Roles if the

The IQueryable interface inherits the IEnumerable interface so that if it represents a query, the results of that query can be enumerated.



```
[HttpGet]  
public IActionResult ListRoles()  
{  
    var roles = roleManager.Roles;  
    return View(roles);  
}
```

Views->Administration->ListRoles.cshtml

```
CreateRoleViewModel.cs | ListRoles.cshtml | AdministrationController.cs | RegisterViewModel.cs
@model IEnumerable<IdentityRole>
: IQueryable

@{
    ViewBag.Title = "All Roles";
}

<h1>All Roles</h1>

@if (Model.Any())
{
    <a class="btn btn-primary mb-3" style="width:auto" asp-action="CreateRole"
      asp-controller="administration">Add new role</a>

    foreach (var role in Model)
    {
        <div class="card mb-3">
            <div class="card-header">
                Role Id : @role.Id
            </div>
            <div class="card-body">
                <h5 class="card-title">@role.Name</h5>
            </div>
            <div class="card-footer">
                <a href="#" class="btn btn-primary">Edit</a>
                <a href="#" class="btn btn-danger">Delete</a>
            </div>
        </div>
    }
}
```

```
@model IEnumerable<IdentityRole>

@{
    ViewBag.Title = "All Roles";
}

<h1>All Roles</h1>

@if (Model.Any())
{
    <a class="btn btn-primary mb-3" style="width:auto" asp-action="CreateRole"
      asp-controller="administration">Add new role</a>

    foreach (var role in Model)
    {
        <div class="card mb-3">
            <div class="card-header">
                Role Id : @role.Id
            </div>
            <div class="card-body">
                <h5 class="card-title">@role.Name</h5>
            </div>
            <div class="card-footer">
```

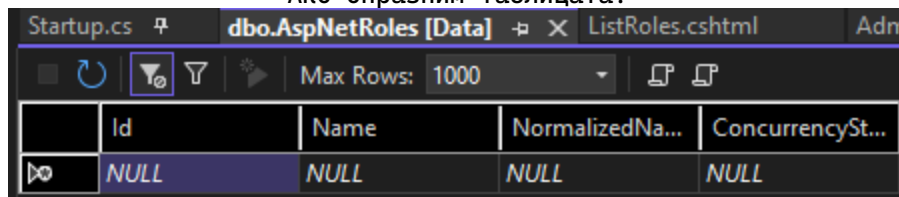


```

        <a href="#" class="btn btn-primary">Edit</a>
        <a href="#" class="btn btn-danger">Delete</a>
    </div>
</div>
}
else
{
    <div class="card">
        <div class="card-header">
            No roles created yet
        </div>
        <div class="card-body">
            <h5 class="card-title">
                Use the button below to create a role
            </h5>
            <a class="btn btn-primary" style="width:auto"
                asp-controller="administration" asp-action="CreateRole">
                Create Role
            </a>
        </div>
    </div>
}

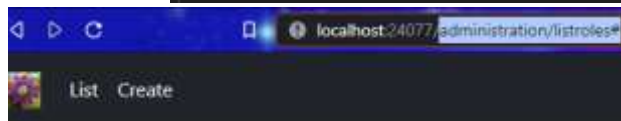
```

Ако опразним таблицата:



The screenshot shows a SQL Server Enterprise Manager window with the 'dbo.AspNetRoles [Data]' table selected. The table is empty, with columns: Id, Name, NormalizedName, and ConcurrencyStamp. The 'Max Rows' is set to 1000.

	Id	Name	NormalizedNa...	ConcurrencySt...
	NULL	NULL	NULL	NULL

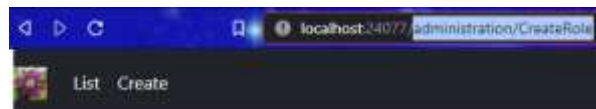


All Roles

No roles created yet

Use the button below to create a role

Create Role



Create New Role

RoleName

Admin

Create Role

localhost:24077/administration/CreateRole

List Create

Create New Role

• Role name 'Admin' is already taken.

RoleName

Admin

Create Role

localhost:24077/administration/ksroles

List Create

All Roles

Add new role

Role Id : 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

User

Edit Delete

Role Id : 7a53c902-d9f9-4026-b521-26b8485f1611

Admin

Edit Delete

Get list of Roles in ASP.NET Core

Controller

```
[HttpGet]
public IActionResult ListRoles()
{
    var roles = roleManager.Roles;
    return View(roles);
}
```

View

```
@model IEnumerable<IdentityRole>

@foreach (var role in Model)
{
    <div>
        Role ID : @role.Id
    </div>
    <div class="card-body">
        Role Name : @role.Name
    </div>
}
```

80 - Edit role

Edit Role in ASP.NET Core

Logout pragin@pragintech.com

Edit Role

Id: 7e348bb4-06b1-45d2-9e0f-88053521fac8

RoleName: Admin

Update Cancel

Users in this role

None at the moment

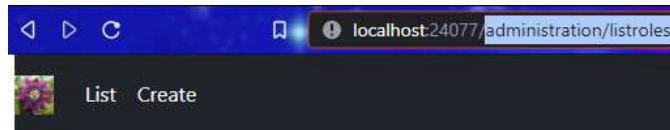
Add Users Remove Users

Потребителят няма да има възможност да коригира видимото ID, а само позициите. Тип string, защото не се разглежда като число.

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace EmployeeManagement.ViewModels
{
    public class EditRoleViewModel
    {
        public string Id { get; set; }
        [Required(ErrorMessage = "Role name is required")]
        public string RoleName { get; set; }
        public List<string> Users { get; set; }
    }
}
```

Искаме при натискане на Edit, да се прихване ID, да ни пренасочи към EditRole view и да се изпише в адрес бара, за да знаем кого редактираме.

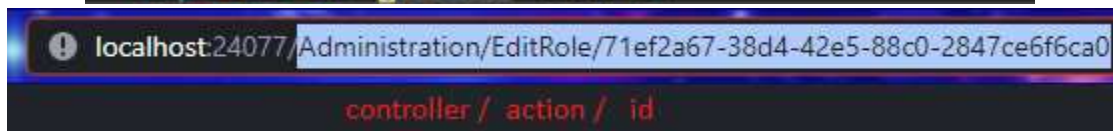


All Roles

Add new role

Role Id : 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
User
Edit Delete
Role Id : 7a53c902-d9f9-4026-b521-26b8485f1611
Admin
Edit Delete

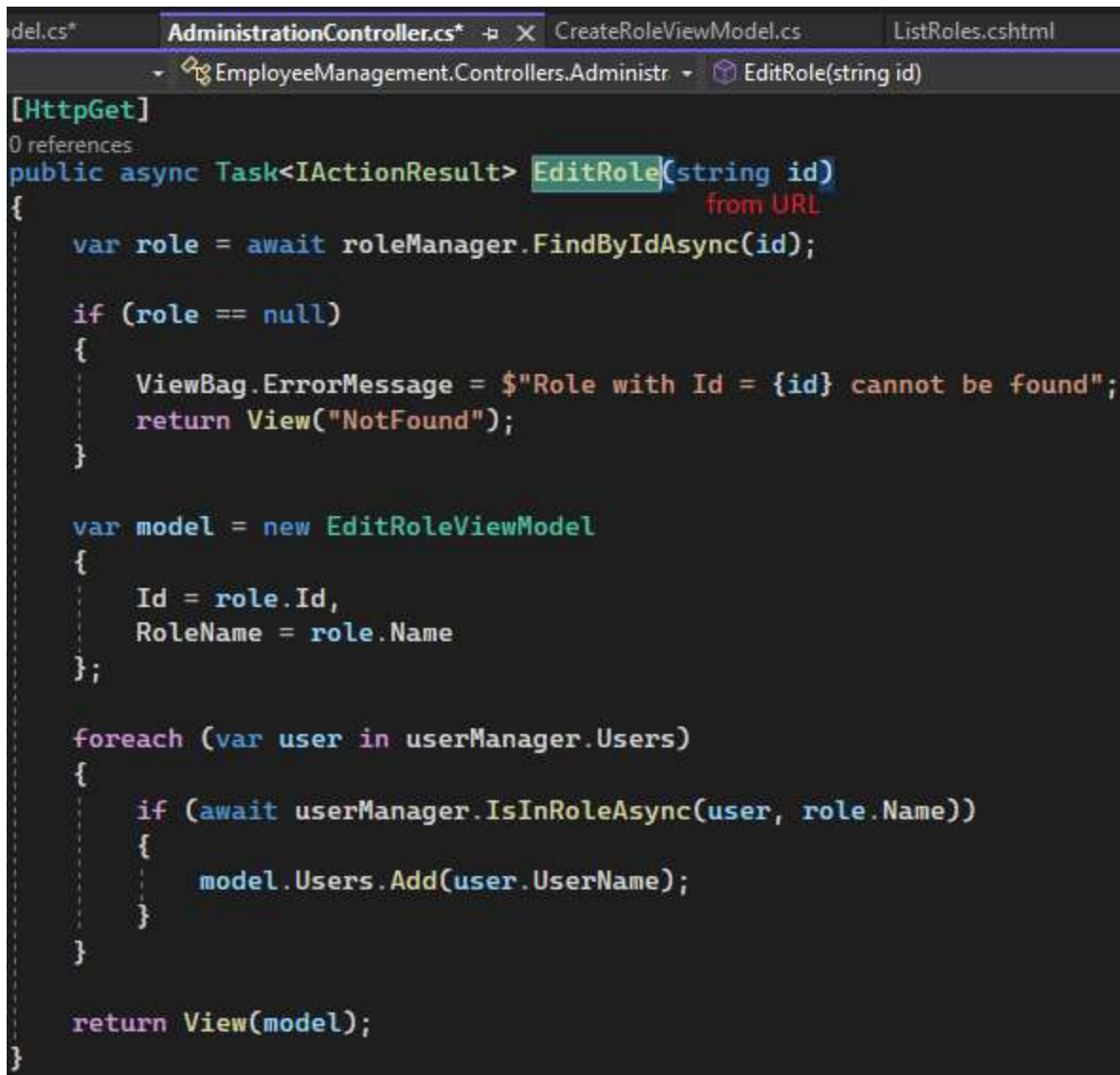
```
CreateRoleViewModel.cs  ListRoles.cshtml  AdministrationControll
<a class="btn btn-primary"
  asp-controller="Administration" asp-action="EditRole"
  asp-route-id="@role.Id">Edit</a>
```



```
AdministrationController.cs
using EmployeeManagement.Models;
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;

namespace EmployeeManagement.Controllers
{
    1 reference
    public class AdministrationController : Controller
    {
        private readonly RoleManager<IdentityRole> roleManager;
        private readonly UserManager<ApplicationUser> userManager;

        0 references
        public AdministrationController(RoleManager<IdentityRole> roleManager,
            UserManager<ApplicationUser> userManager)
        {
            this.roleManager = roleManager;
            this.userManager = userManager;
        }
    }
}
```

```
del.cs* AdministrationController.cs* CreateRoleViewModel.cs ListRoles.cshtml
EmployeeManagement.Controllers.Administr EditRole(string id)

[HttpGet]
0 references
public async Task<IActionResult> EditRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }

    var model = new EditRoleViewModel
    {
        Id = role.Id,
        RoleName = role.Name
    };

    foreach (var user in userManager.Users)
    {
        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            model.Users.Add(user.UserName);
        }
    }

    return View(model);
}
```

```
[HttpGet]
public async Task<IActionResult> EditRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }

    var model = new EditRoleViewModel
    {
        Id = role.Id,
        RoleName = role.Name
    };

    foreach (var user in userManager.Users)
    {
```

```

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            model.Users.Add(user.UserName);
        }
    }

    return View(model);
}

```

```

EditRole.cshtml* EditRoleViewModel.cs* AdministrationController.cs* CreateRoleViewModel.cs*
@model EditRoleViewModel

@{
    ViewBag.Title = "Edit Role";
}

<h1>Edit Role</h1>

<form method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Id" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="Id" disabled class="form-control">
            user is not allowed to edit
        </div>
    </div>
    <div class="form-group row">
        <label asp-for="RoleName" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="RoleName" class="form-control">
            <span asp-validation-for="RoleName" class="text-danger"></span>
        </div>
    </div>

    <div asp-validation-summary="All" class="text-danger"></div>

```

```

<div class="form-group row">
  <div class="col-sm-10">
    <button type="submit" class="btn btn-primary">Update</button>
    <a asp-action="ListRoles" class="btn btn-primary">Cancel</a>
    Back
  </div>
</div>

<div class="card">
  <div class="card-header">
    <h3>Users in this role</h3>
  </div>
  <div class="card-body">
    @if (Model.Users.Any()) is there any users from the list
    {
      foreach (var user in Model.Users)
      {
        <h5 class="card-title">@user</h5>
      }
    }
    else
    {
      <h5 class="card-title">None at the moment</h5>
    }
  </div>
  <div class="card-footer">
    <a href="#" class="btn btn-primary" style="width:auto">Add Users</a>
    <a href="#" class="btn btn-primary" style="width:auto">Remove Users</a>
  </div>
</div>
</form>

```

An unhandled exception occurred while processing the request.

ArgumentNullException: Value cannot be null. (Parameter 'source')

System.Linq.ThrowHelper.ThrowArgumentNullException(ExceptionArgument argument)

Stack Query Cookies Headers Routing

ArgumentNullException: Value cannot be null. (Parameter 'source')

System.Linq.ThrowHelper.ThrowArgumentNullException(ExceptionArgument argument)

System.Linq.Enumerable.Any<TSource>(IEnumerable<TSource> source)

AspNetCoreGeneratedDocument.Views_Administration_EditRole.<ExecuteAsync>b__20_0() in **EditRole.cshtml**

+ 38. @if (Model.Users.Any())

Microsoft.AspNetCore.Razor.Runtime.TagHelpers.TagHelperExecutionContext.GetChildContentAsync(bool useCache)

Microsoft.AspNetCore.Mvc.TagHelpers.RenderAtEndOfFormTagHelper.ProcessAsync(TagHelperContext context, TagHelperExecutionContext)

Microsoft.AspNetCore.Razor.Runtime.TagHelpers.TagHelperRunner.<RunAsync>g__Awaited|0_0(Task task, TagHelperExecutionContext)

AspNetCoreGeneratedDocument.Views_Administration_EditRole.ExecuteAsync() in **EditRole.cshtml**

+ 4. ViewBag.Title = "Edit Role";

Microsoft.AspNetCore.Mvc.Razor.RazorView.RenderPageCoreAsync(IRazorPage page, ViewContext context)

Това е, защото Users is a collection property so can't be null but at the moment it is not initialized.

```
public List<string> Users { get; set; }
```

class System.Collections.Generic.List<T>
Represents a strongly typed list of objects that can be accessed by index. Provides methods to search, sort, and manipulate lists.

За да я коригираме, вмъкваме сторг и я инициализираме.

```

namespace EmployeeManagement.ViewModels
{
    6 references
    public class EditRoleViewModel
    {
        1 reference
        public EditRoleViewModel()
        {
            ctor      Users = new List<string>();      initialize
        }
        5 references
        public string Id { get; set; }
        [Required(ErrorMessage = "Role name is required")]
        5 references
        public string RoleName { get; set; }
        4 references
        prop      public List<string> Users { get; set; }
    }
}

```

```

using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace EmployeeManagement.ViewModels
{
    public class EditRoleViewModel
    {
        public EditRoleViewModel()
        {
            Users = new List<string>();
        }
        public string Id { get; set; }
        [Required(ErrorMessage = "Role name is required")]
        public string RoleName { get; set; }
        public List<string> Users { get; set; }
    }
}

```


Edit Role

Id

71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

Can't be edited

RoleName

User

Update

Cancel

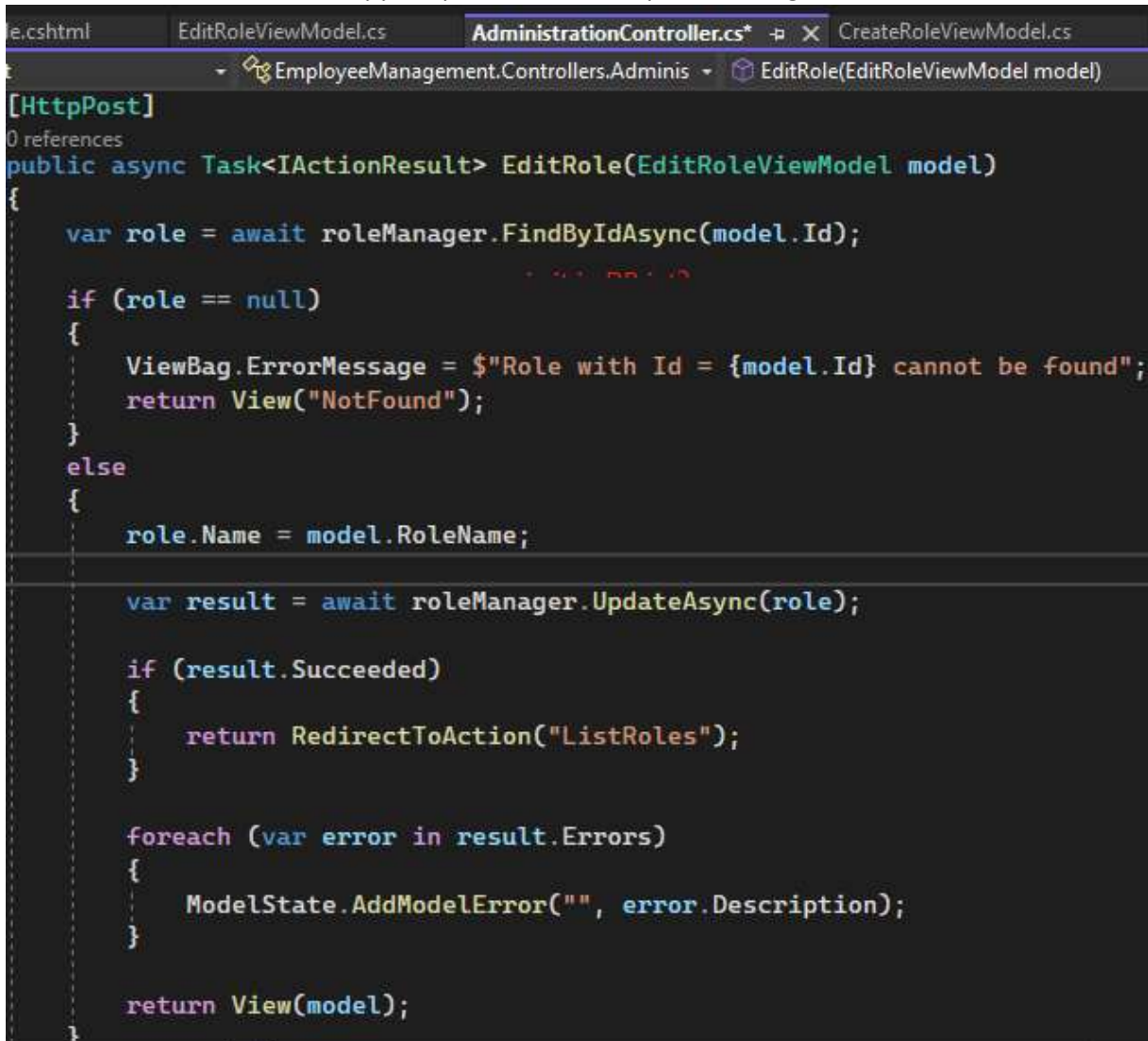
Users in this role

None at the moment

Add Users

Remove Users

Copy [HttpGet] -> Paste [HttpPost]+changes



The screenshot shows a Visual Studio editor window with the file 'AdministrationController.cs' open. The method 'EditRole' is highlighted, showing its implementation. The code is as follows:

```
[HttpPost]
0 references
public async Task<IActionResult> EditRole(EditRoleViewModel model)
{
    var role = await roleManager.FindByIdAsync(model.Id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {model.Id} cannot be found";
        return View("NotFound");
    }
    else
    {
        role.Name = model.RoleName;

        var result = await roleManager.UpdateAsync(role);

        if (result.Succeeded)
        {
            return RedirectToAction("ListRoles");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }

        return View(model);
    }
}
```

```
[HttpPost]
public async Task<IActionResult> EditRole(EditRoleViewModel model)
{
    var role = await roleManager.FindByIdAsync(model.Id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {model.Id} cannot be found";
        return View("NotFound");
    }
    else
    {
        role.Name = model.RoleName;

        var result = await roleManager.UpdateAsync(role);
```

```

    if (result.Succeeded)
    {
        return RedirectToAction("ListRoles");
    }

    foreach (var error in result.Errors)
    {
        ModelState.AddModelError("", error.Description);
    }

    return View(model);
}
}

```

All Roles

Add new role

Role Id : 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
User1
Edit Delete

Role Id : 7a53c902-d9f9-4026-b521-26b8485f1611
Admin
Edit Delete

Edit Role

Id

71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName

Admin

• Role name 'Admin' is already taken.

Update

Cancel

Users in this role

None at the moment

Add Users

Remove Users

81 - Add or remove users from role

Към момента все още нямаме users in role, искаме да дадем възможност за редакция и обединим двата бутона:

Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

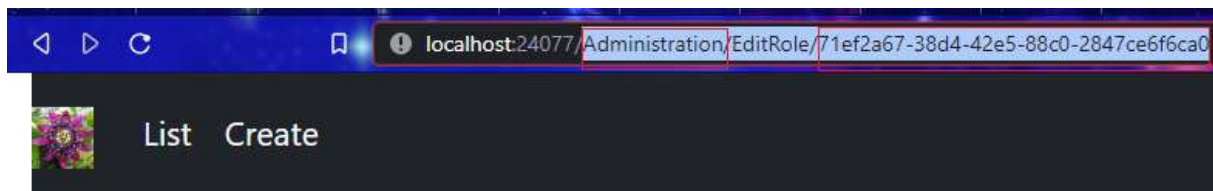
[Update](#) [Cancel](#)

Users in this role

None at the moment

[Add Users](#) [Remove Users](#)

```
<div class="card-footer">  
  <a asp-action="EditUsersInRole" asp-controller="Administration"  
    asp-route-roleId="@Model.Id" class="btn btn-primary"  
    style="width:auto">Add or remove users</a>  
</div>
```



Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

[Update](#) [Cancel](#)

Users in this role

None at the moment

[Add or remove users](#)

localhost:24077/Administration/EditUsersInRole?roleId=71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

Add or Remove Users from Role

 List · Create Logout `pragim@pragimtech.com`

Add or remove users from this role

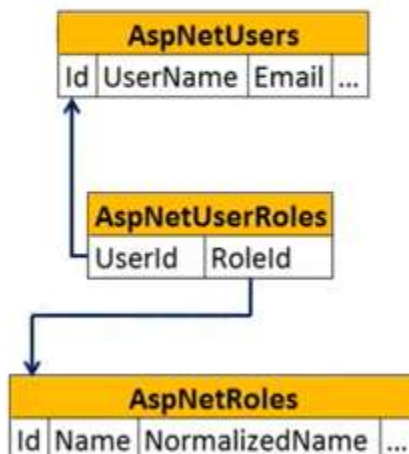
☒ xyz1@pragimtech.com

☒ pragim@pragimtech.com

Update

Cancel

AspNetUserRoles Identity Table



- Users are stored in **AspNetUsers** table
- Roles are stored in **AspNetRoles** table
- Many-to-Many relationship between **AspNetUsers** and **AspNetRoles** tables
- User and Role mapping data is stored in **AspNetUserRoles** table
- This table has just 2 columns : **UserId** & **RoleId**
- Both are **foreign keys**

Add or Remove Users from Role

Logout pragim@pragimtech.com

List Create

Add or remove users from this role

- ☒ xyz1@pragimtech.com
- ☒ pragim@pragimtech.com

AspNetUserRoles	
UserId	RoleId

Role 1 -> n Users
RoleId -> UserId

not to duplicate
RoleId for every
user

```
public class UserRoleViewModel
{
    public string UserId { get; set; }
    public string RoleId { get; set; }
    public string UserName { get; set; }
    public bool IsSelected { get; set; }
}
```

move RoleId
out of
UserRoleViewModel
class

ViewBag.RoleId -> to pass it from the controller to the view.

UserRoleViewModel.cs EditRole.cshtml EditRoleViewMo

namespace EmployeeManagement.Models

```
{
    0 references
    public class UserRoleViewModel
    {
        0 references
        public string UserId { get; set; }
        0 references
        public string UserName { get; set; }
        0 references
        public bool IsSelected { get; set; }
    }
}
```

```
odel.cs AdministrationController.cs EditRole.cshtml EditRoleViewModel.cs CreateRo
EmployeeManagement.Controllers.AdministrationController EditUsersInF

[HttpGet]
0 references
public async Task<IActionResult> EditUsersInRole(string roleId)
{
    ViewBag.roleId = roleId; store it to call it later

    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
        return View("NotFound");
    }

    var model = new List<UserRoleViewModel>();

    foreach (var user in userManager.Users)
    {
        var userRoleViewModel = new UserRoleViewModel
        {
            UserId = user.Id,
            UserName = user.UserName
        };
    }
}
```

```
        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRoleViewModel.IsSelected = true;
        }
        else
        {
            userRoleViewModel.IsSelected = false;
        }

        model.Add(userRoleViewModel);
    }

    return View(model);
}
```

```
[HttpGet]
public async Task<IActionResult> EditUsersInRole(string roleId)
{
    ViewBag.roleId = roleId;

    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
```

```

    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
        return View("NotFound");
    }

    var model = new List<UserRoleViewModel>();

    foreach (var user in userManager.Users)
    {
        var userRoleViewModel = new UserRoleViewModel
        {
            UserId = user.Id,
            UserName = user.UserName
        };

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRoleViewModel.IsSelected = true;
        }
        else
        {
            userRoleViewModel.IsSelected = false;
        }

        model.Add(userRoleViewModel);
    }

    return View(model);
}

```

Administration folder -> EditUsersInRole.cshtml

```

EditUsersInRole.cshtml  UserRoleViewModel.cs  AdministrationController.cs  EditRole.cshtml  EditRoleV
@model List<UserRoleViewModel>

@{
    var roleId = ViewBag.roleId;
}

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Add or remove users from this role</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Count; i++)
            {
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model[i].UserId" />
                    <input type="hidden" asp-for="@Model[i].UserName" />
                    <input asp-for="@Model[i].IsSelected" class="form-check-input" />
                    <label class="form-check-label" asp-for="@Model[i].IsSelected">
                        @Model[i].UserName
                    </label>
                </div>
            }
        </div>
        <div class="card-footer">
            <input type="submit" value="Update" class="btn btn-primary"
                style="width:auto" />

            <a asp-action="EditRole" asp-route-id="@roleId"
                class="btn btn-primary" style="width:auto">Cancel</a>
        </div>
    </div>
</form>

```

```

@model List<UserRoleViewModel>

@{
    var roleId = ViewBag.roleId;
}

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Add or remove users from this role</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Count; i++)
            {
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model[i].UserId" />
                    <input type="hidden" asp-for="@Model[i].UserName" />
                    <input asp-for="@Model[i].IsSelected" class="form-check-input"

```

```

        <label class="form-check-label" asp-for="@Model[i].IsSelected">
            @Model[i].UserName
        </label>
    </div>
}
</div>
<div class="card-footer">
    <input type="submit" value="Update" class="btn btn-primary"
        style="width:auto" />
    <a asp-action="EditRole" asp-route-id="@roleId"
        class="btn btn-primary" style="width:auto">Cancel</a>
</div>
</div>
</form>

```

localhost:24077/Administration/EditUsersInRole?roleId=71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

List Create

Add or remove users from this role

- ☐ julia@abv.bg
- ☐ aurora@abv.bg
- ☐ sara@abv.bg
- ☐ bob@abv.bg
- ☐ david@abv.bg
- ☐ tim@abv.bg

Update Cancel

Към момента, никой от потребителите не е в тази role, затова никой check-box не е маркиран. Ако маркираме и дадем Update, ще трябва да се запази в БД. Освен при маркиране на тоглера, искаме при кликуване върху самият надпис на потребителя да се селектира и асоциирания към него тоглер. Това се изпълнява с

```

<input asp-for="@Model[i].IsSelected" class="form-check-input" />
<label class="form-check-label" asp-for="@Model[i].IsSelected">
    @Model[i].UserName
</label>

```

При Update все още нямаме [HttpPost] EditUsersInRole action in AdministrationController -> 404

localhost:24077/Administration/EditUsersInRole?roleId=71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

Добавяме го, според променливите, които сме получили: лист + roleId от URL.


```
151 [HttpPost]
152 0 references
153 public async Task<IActionResult> EditUsersInRole(List<UserRoleViewModel> model,string roleId)
154
155     return View(model);
```

Добавяме Breakpoint -> Debug mode, селектирам 1,2,3 -> Update:

- ☒ julia@abv.bg
- ☒ aurora@abv.bg
- ☒ sara@abv.bg
- ☐ bob@abv.bg
- ☐ david@abv.bg
- ☐ tim@abv.bg

Index	IsSelected	UserName
[0]	true	"bob@abv.bg"
[1]	false	
[2]	false	
[3]	false	
[4]	false	
[5]	false	

Breakpoint is hit: - виждаме, че model съдържа 6 стойности, като за първата IsSelected = true, (маркирана е), по същият начин като 2,3 докато 4,5,6 не са избрани, съответно false.

Index	IsSelected	UserId	UserName
[0]	true	"1030daee-596d-414e-87d3-cba5c7e3935e"	"julia@abv.bg"
[1]	false		
[2]	false		
[3]	false		
[4]	false		
[5]	false		

Другите данни, които се пренасят, са за UserID, UserName, които са `<input type="hidden">`

Спираме дебъгването.

```

AdministrationController.cs
[HttpPost]
0 references
public async Task<IActionResult> EditUsersInRole(List<UserRoleViewModel> model, string roleId)
{
    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null) if role is empty
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
        return View("NotFound");
    }

    for (int i = 0; i < model.Count; i++) find all users in this role
    {
        var user = await userManager.FindByIdAsync(model[i].UserId);

        IdentityResult result = null;

        if (model[i].IsSelected && !(await userManager.IsInRoleAsync(user, role.Name)))
        {
            result = await userManager.AddToRoleAsync(user, role.Name);
        }
        else if (!model[i].IsSelected && await userManager.IsInRoleAsync(user, role.Name))
        {
            result = await userManager.RemoveFromRoleAsync(user, role.Name);
        }
        else
        {
            continue;
        }
    }
}

```

```

        if (result.Succeeded)
        {
            if (i < (model.Count - 1)) there are more model objects to proceed
                continue;
            else end of the loop
                return RedirectToAction("EditRole", new { Id = roleId });
        }

        if model parameter is empty, there are any userRole model objects
        return RedirectToAction("EditRole", new { Id = roleId });
    }
}

```

```

using EmployeeManagement.Models;
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Threading.Tasks;

```

```

namespace EmployeeManagement.Controllers
{

```

```

public class AdministrationController : Controller
{
    private readonly RoleManager<IdentityRole> roleManager;
    private readonly UserManager<ApplicationUser> userManager;

    public AdministrationController(RoleManager<IdentityRole> roleManager,
                                   UserManager<ApplicationUser> userManager)
    {
        this.roleManager = roleManager;
        this.userManager = userManager;
    }

    [HttpGet]
    public IActionResult CreateRole()
    {
        return View();
    }

    [HttpPost]
    public async Task<IActionResult> CreateRoleAsync(CreateRoleViewModel model)
    {
        if (ModelState.IsValid)
        {
            IdentityRole identityRole = new IdentityRole
            {
                Name = model.RoleName
            };
            IdentityResult result = await roleManager.CreateAsync(identityRole);

            if (result.Succeeded)
            {
                return RedirectToAction("listroles", "administration");
            }
            foreach (IdentityError error in result.Errors)
            {
                ModelState.AddModelError("", error.Description);
            }
        }
        return View(model);
    }

    [HttpGet]
    public IActionResult ListRoles()
    {
        var roles = roleManager.Roles;
        return View(roles);
    }

    // Role ID is passed from the URL to the action
    [HttpGet]
    public async Task<IActionResult> EditRole(string id)
    {
        var role = await roleManager.FindByIdAsync(id);

        if (role == null)
        {
            ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
            return View("NotFound");
        }

        var model = new EditRoleViewModel

```

```

        {
            Id = role.Id,
            RoleName = role.Name
        };

        foreach (var user in userManager.Users)
        {
            if (await userManager.IsInRoleAsync(user, role.Name))
            {
                model.Users.Add(user.UserName);
            }
        }

        return View(model);
    }

    [HttpPost]
    public async Task<IActionResult> EditRole(EditRoleViewModel model)
    {
        var role = await roleManager.FindByIdAsync(model.Id);

        if (role == null)
        {
            ViewBag.ErrorMessage = $"Role with Id = {model.Id} cannot be found";
            return View("NotFound");
        }
        else
        {
            role.Name = model.RoleName;

            var result = await roleManager.UpdateAsync(role);

            if (result.Succeeded)
            {
                return RedirectToAction("ListRoles");
            }

            foreach (var error in result.Errors)
            {
                ModelState.AddModelError("", error.Description);
            }

            return View(model);
        }
    }

    [HttpGet]
    public async Task<IActionResult> EditUsersInRole(string roleId)
    {
        ViewBag.roleId = roleId;

        var role = await roleManager.FindByIdAsync(roleId);

        if (role == null)
        {
            ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
            return View("NotFound");
        }
    }

```

```

var model = new List<UserRoleViewModel>();

foreach (var user in userManager.Users)
{
    var userRoleViewModel = new UserRoleViewModel
    {
        UserId = user.Id,
        UserName = user.UserName
    };

    if (await userManager.IsInRoleAsync(user, role.Name))
    {
        userRoleViewModel.IsSelected = true;
    }
    else
    {
        userRoleViewModel.IsSelected = false;
    }

    model.Add(userRoleViewModel);
}

return View(model);
}
[HttpPost]
public async Task<IActionResult> EditUsersInRole(List<UserRoleViewModel>
model, string roleId)
{
    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
        return View("NotFound");
    }

    for (int i = 0; i < model.Count; i++)
    {
        var user = await userManager.FindByIdAsync(model[i].UserId);

        IdentityResult result = null;

        if (model[i].IsSelected && !(await userManager.IsInRoleAsync(user,
role.Name)))
        {
            result = await userManager.AddToRoleAsync(user, role.Name);
        }
        else if (!model[i].IsSelected && await
userManager.IsInRoleAsync(user, role.Name))
        {
            result = await userManager.RemoveFromRoleAsync(user, role.Name);
        }
        else
        {
            continue;
        }

        if (result.Succeeded)

```



```

        {
            if (i < (model.Count - 1))
                continue;
            else
                return RedirectToAction("EditRole", new { Id = roleId });
        }
    }

    return RedirectToAction("EditRole", new { Id = roleId });
}
}
}

```

Правим проверки:

☐ julia@abv.bg
☐ aurora@abv.bg
☒ sara@abv.bg
☒ bob@abv.bg
☐ david@abv.bg
☐ tim@abv.bg

User

Admin

Admin

-

-

UserId	RoleId
1030daee-596d-414e-87d3-cba6c7e5935e	71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
1f9ea9da-40d8-4413-a222-9fd26303740c	71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
87cabcf7-6319-4672-a7c8-db5a048fa5f7	7a53c902-d9f9-4026-b521-26b8485f1611
b2056455-c2b4-42d5-883d-27475d32038b	7a53c902-d9f9-4026-b521-26b8485f1611
NULL	NULL

Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

Users in this role

julia@abv.bg
aurora@abv.bg

Edit Role

Id: 7a53c902-d9f9-4026-b521-26b8485f1611

RoleName: Admin

Users in this role

sara@abv.bg
bob@abv.bg

82 - Role based authorization

- аутентификация – кой е потребителят;
- авторизация – какво може
 - [Authorize] – проста, за всички изисква просто да са вписани
 - [Authorize(Roles = "Admin")] – за конкретен потребител

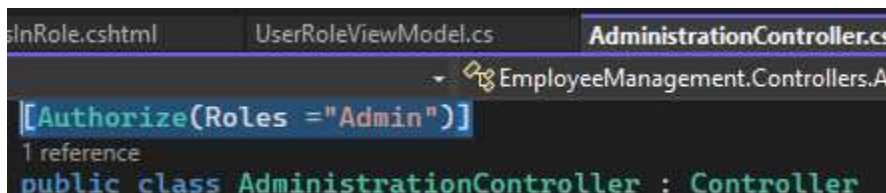
ASP.NET Core Role Based Authorization

Simple Authorization - Part 71 of ASP.NET Core Tutorial

```
[Authorize]
public class SomeController : Controller
{
}
```

Only members of the Administrator role have access

```
[Authorize(Roles = "Administrator")]
public class AdministrationController : Controller
{
}
```



```
AdministrationController.cs
[Authorize(Roles = "Admin")]
public class AdministrationController : Controller
```

☐ julia@abv.bg	User
☐ aurora@abv.bg	User
☒ sara@abv.bg	Admin
☒ bob@abv.bg	Admin
☐ david@abv.bg	-
☐ tim@abv.bg	-

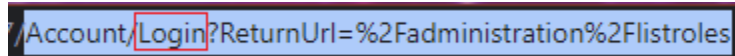
При това положение, въведен в контролера, изисква вписване преди да даде достъп до administration/listroles ->

При login като обикновен user:



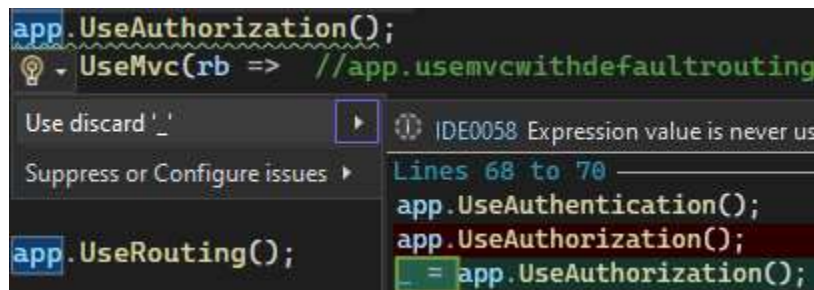
localhost:24077/Account/AccessDenied?ReturnUrl=%2Fadministration%2Flistroles

При logout прехвърля към формата за вписване:



/Account/Login?ReturnUrl=%2Fadministration%2Flistroles

При мен, обаче, след вписване пренасочва към 404. Без [Authorize] няма проблеми. Четох, че след .Net 3.0 изисква добавяне на **app.UseAuthorization();** след app.UseAuthentication();, но при мен изкарва Warning, който пак стои със зелено подчертаване.



```
app.UseAuthorization();
UseMvc(rb => //app.usemvcwithdefaultrouting
Use discard '_'
Suppress or Configure issues
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.UseAuthorization();
```

Тест да добавя

```
services.AddAuthorization(options =>
options.AddPolicy("Role",
policy => policy.RequireClaim(ClaimTypes.Role, "Admin")));
```

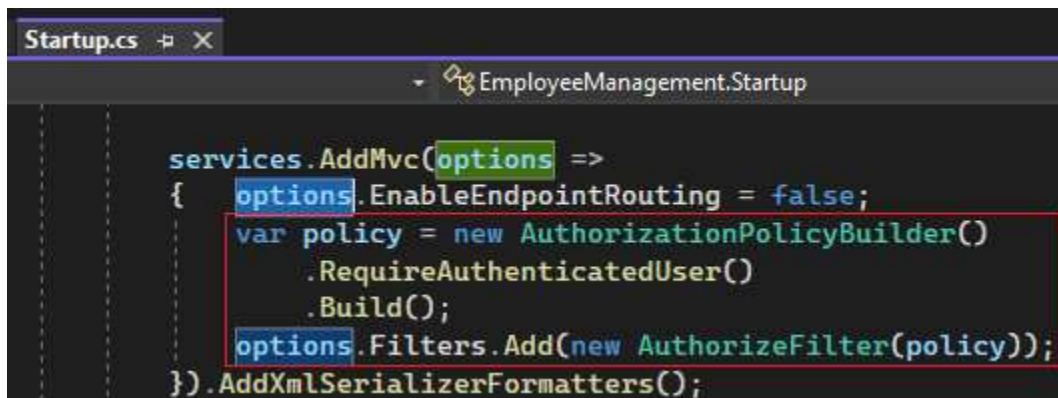
Не става, независимо, че давам Logout всеки път и вписвам админски потребител, при самото пренасочване връща 404.

По същият начин не работи при [Authorize(Roles = "User")]

Само [Authorize] разрешава.

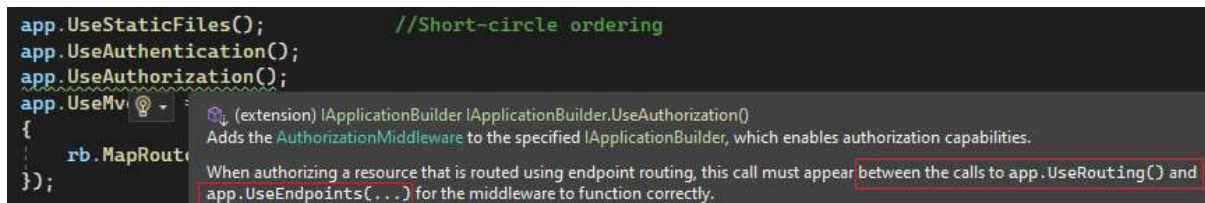
```
services.AddAuthorization(options =>
options.AddPolicy("Role",
policy => policy.RequireRole("Admin")));
```

Имам:



```
Startup.cs  X
EmployeeManagement.Startup

services.AddMvc(options =>
{
options.EnableEndpointRouting = false;
var policy = new AuthorizationPolicyBuilder()
.RequireAuthenticatedUser()
.Build();
options.Filters.Add(new AuthorizeFilter(policy));
}).AddXmlSerializerFormatters();
```



```
app.UseStaticFiles(); //Short-circle ordering
app.UseAuthentication();
app.UseAuthorization();
app.UseMvc();
{
rb.MapRoute();
});
```

(extension) IApplicationBuilder IApplicationBuilder.UseAuthorization()
Adds the `AuthorizationMiddleware` to the specified `IApplicationBuilder`, which enables authorization capabilities.

When authorizing a resource that is routed using endpoint routing, this call must appear between the calls to `app.UseRouting()` and `app.UseEndpoints(...)` for the middleware to function correctly.

```

app.UseStaticFiles();
app.UseAuthentication();

app.UseMvc(rb => //app.usemvcw
{
    rb.MapRoute("default", "{co
});

app.UseRouting();
app.UseAuthorization();
app.UseEndpoints(endpoints =>

```

Edit Role

Id: 7a53c902-d9f9-4026-b521-26b8485f1611

RoleName: Admin

Update
Cancel

Users in this role

sara@abv.bg
 bob@abv.bg

Add or remove users

С това преместване, зеленото подчертаване изчезна.

`UseAuthorization();` трябва да бъде след `UseAuthentication();` и между `app.UseRouting();` и `app.UseEndpoints();`

Разбрах откъде е грешката: един интервал, който не се вижда след Admin_ когато съм задавала името. Открих го с Admin1, при User също нямаше проблеми. Това показва уязвимост. Трябва да има абсолютно съвпадение! По-резонно щеше да бъде, ако бях объркала A с кирилица.

ASP.NET Core Role Based Authorization

Members of either Administrator or User role have access

```

[Authorize(Roles = "Administrator,User")]
public class AdministrationController : Controller
{
}

```

To access users have to be members of both
Administrator role and User role

```

[Authorize(Roles = "Administrator")]
[Authorize(Roles = "User")]
public class AdministrationController : Controller
{
}

```

Добавяме още един потребител User, така че или Admin и User имат право на достъп. За да влязат в сила промените, винаги трябва да дадем Logout-> Login.

Правим тестови проверки.

`[Authorize(Roles = "Admin,User")]`

не е еквивалентно като действие с
[Authorize(Roles = "Admin,User")]
[Authorize(Roles = "User")]

Във втория случай, трябва потребителят да бъде едновременно Admin + User

ASP.NET Core Role Based Authorization

pragim@pragimtech.com	Admin
abc@gmail.com	User
test@pragimtech.com	Admin, User

ASP.NET Core Role Based Authorization

<pre>[Authorize(Roles = "Administrator, User")] public class AdministrationController : Controller { public ActionResult ABC() { } [Authorize(Roles = "Administrator")] public ActionResult XYZ() { } [AllowAnonymous] public ActionResult Anyone() { } }</pre>	Members of Administrator or User role have access
	Only members of Administrator role have access
	Anyone including anonymous users have access

Оставям aurora@abv.bg да бъде Admin+User.

83 - Show or hide navigation menu based on user role

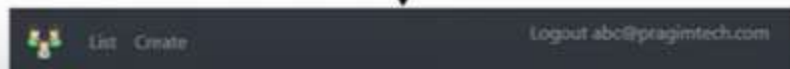
Искаме Admin да има достъп до Manage Roles в менюто, докато обикновен User да няма дори видимост към него.

Show or Hide Menu based on User Role

If the logged-in user is in Admin Role



If the logged-in user is NOT IN Admin Role



В _Layout view искаме да Create динамично управление: ако е SignedIn като Admin, Manage roles да се вижда, в противен случай да е скрит. Много е важно SignInManager да бъде инжектиран в началото.

```
Layout.cshtml | X | db.AspNetRoles [Data] | db.AspNetUserRoles [Data] | AdministrationController.cs | HomeController.cs
Inject SignInManager<ApplicationUser> signInManager;

<!DOCTYPE html>
<html>
<head>...</head>
<body>
<div class="container-fluid">
<nav class="navbar navbar-expand-sm navbar-toggleable-sm bg-dark navbar-dark border-bottom box-shadow mb-3">
<a class="navbar-brand">...</a>
<button class="navbar-toggler">...</button>
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
<ul class="navbar-nav flex-grow-1">
<li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="index">List</a>
</li>
<li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="create">Create</a>
</li>
<li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="administration" asp-action="listroles">Manage Roles</a>
</li>
</ul>
</div>
</div>
</div>
</body>
</html>
```

```
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item">
        <a class="nav-link text-light" asp-area="" asp-controller="administration"
asp-action="listroles">Manage Roles</a>
    </li>
}
```

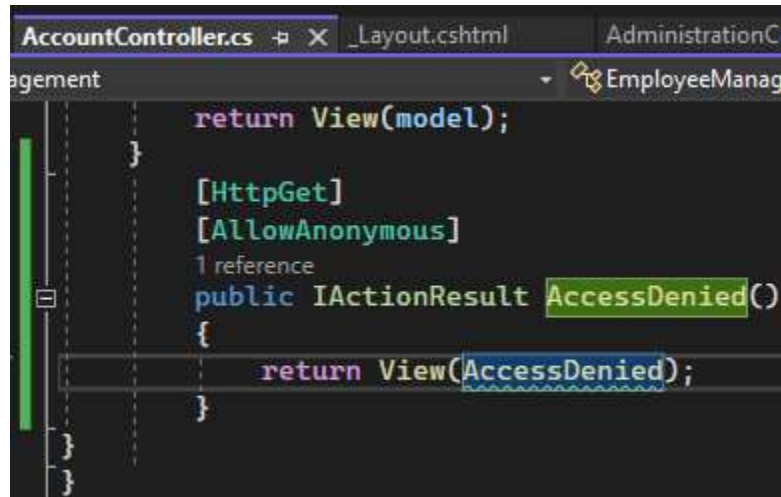
От предходния урок оставяме само `[Authorize(Roles = "Admin")]`

Тестваме:

* Админ има достъп през менюто до Manage Roles

- * невисан потребител не вижда Manage Roles,
/administration/listroles иска login
- * вписан User не вижда Manage Roles,
/administration/listroles дава access denied + 404, защото все още нямаме View.

Искаме страницата, която изписва AccessDenied да бъде видима от всички потребители



```

[HttpGet]
[AllowAnonymous]
public IActionResult AccessDenied()
{
    return View(AccessDenied);
}

```

Account folder + AccessDenied.cshtml

Show or Hide Menu based on User Role

```

[HttpGet]
[AllowAnonymous]
public IActionResult AccessDenied()
{
    return View();
}

```

```

<div class="text-center">
  <h1 class="text-danger">Access Denied</h1>
  <h6 class="text-danger">You do not have permission to view this resource</h6>
  
</div>

```

Поради изнасянето на html скелето в _Layout, View не се нуждае от него и съдържа само:

```
AccessDenied.cshtml  AccountController.cs  _Layout.cshtml  AdministrationController.cs
<div class="text-center">
  <h1 class="text-danger">Access Denied</h1>
  <h6 class="text-danger">You do not have permission to view this resource</h6>
  
</div>
```

```
<div class="text-center">
  <h1 class="text-danger">Access Denied</h1>
  <h6 class="text-danger">You do not have permission to view this resource</h6>
  
</div>
```

```
localhost:24077/Account/AccessDenied?ReturnUrl=%2Fadministration%2Flistroles
```

Access Denied

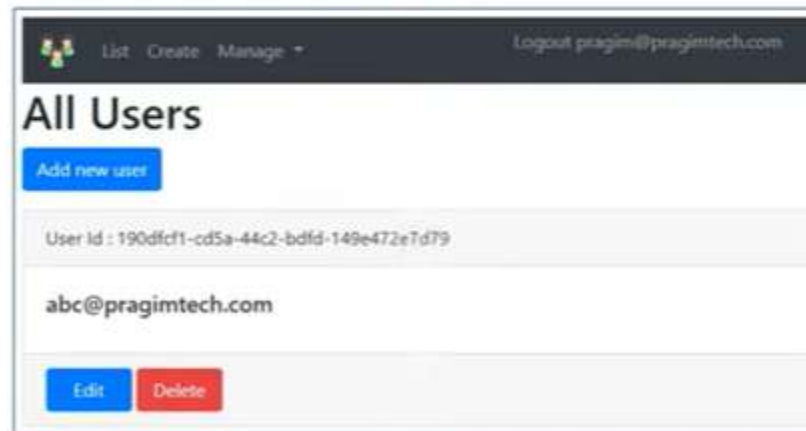
You do not have permission to view this resource



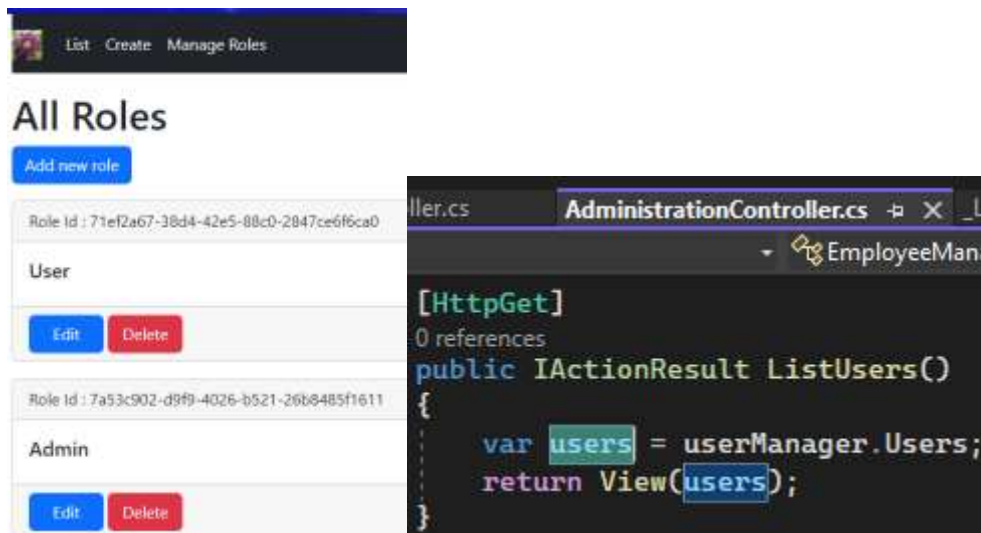
84 - List all users from asp net core identity

database

List all Users from Identity Database



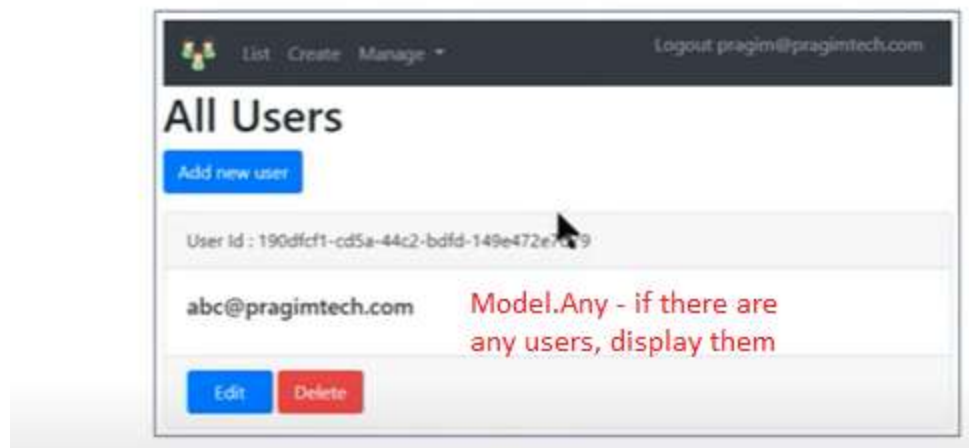
Към момента можехме да изобразим само ролите /listroles:



Искаме по сходен начин /listusers -> AdministrationController.cs:

```
[HttpGet]
public IActionResult ListUsers()
{
    var users = userManager.Users;
    return View(users);
}
```

List all Users from Identity Database



Administration folder -> ListUsers.cshtml:

```
ListUsers.cshtml | AccountController.cs | AdministrationController.cs
@model IEnumerable<ApplicationUser>

@{
    ViewBag.Title = "All Users";
}

<h1>All Users</h1>

@if (Model.Any())
{
    <a asp-action="Register" asp-controller="Account"
      class="btn btn-primary mb-3" style="width:auto">
        Add new user
    </a>
}
```



```

foreach (var user in Model)
{
    <div class="card mb-3">
        <div class="card-header">
            User Id : @user.Id
        </div>
        <div class="card-body">
            <h5 class="card-title">@user.UserName</h5>
        </div>
        <div class="card-footer">
            <a href="#" class="btn btn-danger">Edit</a>
            <a href="#" class="btn btn-danger">Delete</a>
        </div>
    </div>
}

```

```

else
{
    <div class="card">
        <div class="card-header">
            No users created yet
        </div>
        <div class="card-body">
            <h5 class="card-title">
                Use the button below to create a user
            </h5>
            <a class="btn btn-primary" style="width:auto"
                asp-controller="Account" asp-action="Register">
                Add new user
            </a>
        </div>
    </div>
}

```

@model IEnumerable<ApplicationUser>

```

@{
    ViewBag.Title = "All Users";
}

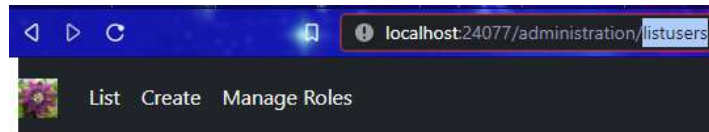
```

<h1>All Users</h1>

```

@if (Model.Any())
{
    <a asp-action="Register" asp-controller="Account"
        class="btn btn-primary mb-3" style="width:auto">
        Add new user
    </a>
}

```

All Users

Add new user

User Id : 1030daee-596d-414e-87d3-cba6c7e5935e

julia@abv.bg

Edit

Delete

User Id : 1f9ea9da-40d8-4413-a222-9fd26303740c

aurora@abv.bg

Edit

Delete

User Id : 87cabcf7-6319-4672-a7c8-db5a048fa5f7

sara@abv.bg

Искаме drop-down меню, ако потребителят е вписан като Admin:

List all Users from Identity Database



```
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdownMenuLink"
            data-toggle="dropdown" aria-haspopup="true" aria-expanded="false">
            Manage
        </a>
        <div class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <a asp-action="ListUsers" asp-controller="Administration"
                class="dropdown-item">Users</a>
            <a asp-action="ListRoles" asp-controller="Administration"
                class="dropdown-item">Roles</a>
        </div>
    </li>
}
```

Сменяме блоковия код:

```
ListUsers.cshtml | _Layout.cshtml | AdministrationController.cs
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item">
        <a class="nav-link text-light" asp-area="" asp-controller="administration"
            asp-action="listroles">Manage Roles</a>
    </li>
}
```

С drop-down:

```
ListUsers.cshtml | _Layout.cshtml | AdministrationController.cs
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdownMenuLink"
            data-toggle="dropdown" aria-haspopup="true" aria-expanded="false">
            Manage
        </a>
        <div class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <a class="dropdown-item" asp-controller="Administration"
                asp-action="ListUsers">Users</a>
            <a class="dropdown-item" asp-controller="Administration"
                asp-action="ListRoles">Roles</a>
        </div>
    </li>
}
```

```
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdownMenuLink"
            data-hover="dropdown" aria-haspopup="true" aria-expanded="false">
            Manage
        </a>
        <div class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <a class="dropdown-item" asp-controller="Administration"
                asp-action="ListUsers">Users</a>
            <a class="dropdown-item" asp-controller="Administration"
                asp-action="ListRoles">Roles</a>
        </div>
    </li>
}
```

ARIA (Accessible Rich Internet Applications)

Оставям засега в този вид, макар, че не се разливта:

```

<li class="nav-item dropdown">
  <a class="nav-link dropdown-toggle text-light"
    href="#"
    id="navbarDropdownMenuLink"
    data-bs-toggle="dropdown"
    data-bs-hover="dropdown"
    aria-haspopup="true"
    aria-expanded="false">
    Manage
  </a>
  <div class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
    <a class="dropdown-item" asp-controller="Administration"
      asp-action="ListUsers">Users</a>
    <a class="dropdown-item" asp-controller="Administration"
      asp-action="ListRoles">Roles</a>
  </div>
</li>

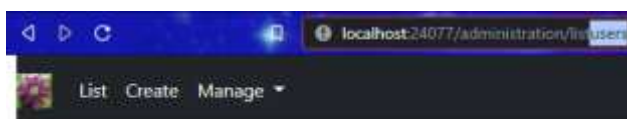
```

```

<li class="nav-item dropdown">
  <a class="nav-link dropdown-toggle text-light"
    href="#"
    id="navbarDropdownMenuLink"
    data-bs-toggle="dropdown"
    data-bs-hover="dropdown"
    aria-haspopup="true"
    aria-expanded="false">
    Manage
  </a>

```

На този етап, докато сме вписани като Admin, имаме възможност да регистрираме нов потребител. Когато го направим, автоматично ни изхвърля и вписва в профила на току-що създадения нов потребител. Искаме обаче, когато админът създава друг user, да си остава в своя профил.



All Users

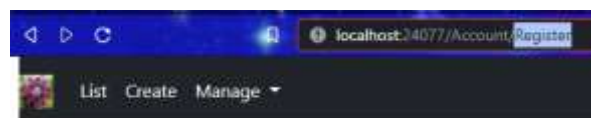
Add new user

User id : 1030daee-596d-414e-87d3-cba6c7e5935e

julia@abv.bg

Edit

Delete



User Registration

Logout jason@abv.bg

Id	UserName
1030daee-596d-...	julia@abv.bg
1f9ea9da-40d8-...	aurora@abv.bg
87cabcf7-6319-...	sara@abv.bg
b2056455-c2b4-...	bob@abv.bg
c4120a9-5bca-...	david@abv.bg
d2f456bd-46fc-...	tim@abv.bg
f86605f9-0791-...	jason@abv.bg
NULL	NULL

Това става, като при проверката дали регистрацията е успешна, добавим още една проверка, дали потребителят е Admin:

```
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City
        };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
            {
                return RedirectToAction("ListUsers", "Administration");
            }
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("index", "home");
        }
    }
}
```

```
if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    return RedirectToAction("ListUsers", "Administration");
}
```

По този начин, ако не сме вписани и направим нова регистрация, при проверката за Admin, if {} блока се прескача и ни вписва в новия профил. Ако сме вписани като Admin, този

блок ще върне true и ще ни препрати към administrationController, така че админът да види потребителя, който току-що е създаден.

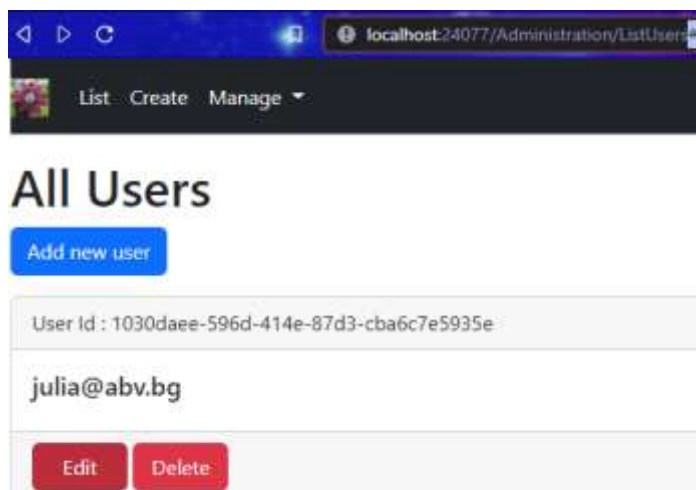
List all Users from Identity Database



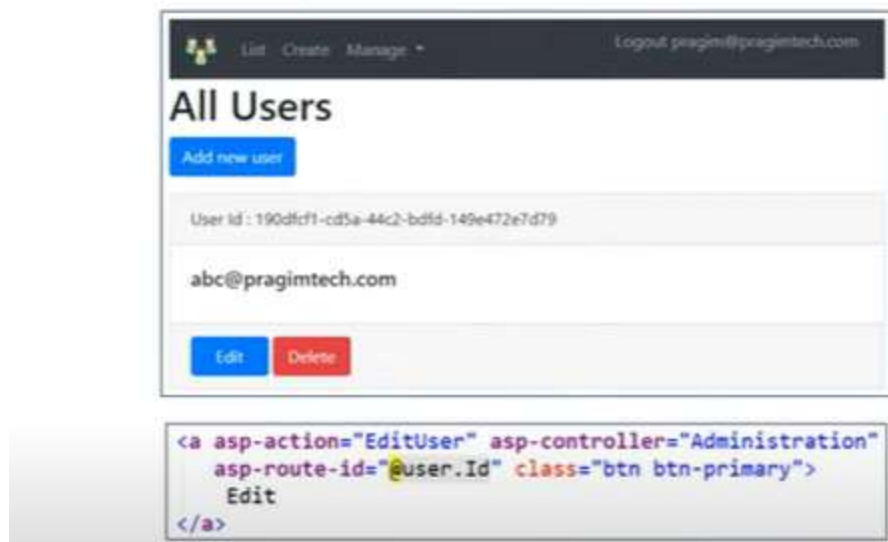
```
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdownMenuLink"
            data-toggle="dropdown" aria-haspopup="true" aria-expanded="false">
            Manage
        </a>
        <div class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <a asp-action="ListUsers" asp-controller="Administration"
                class="dropdown-item">Users</a>
            <a asp-action="ListRoles" asp-controller="Administration"
                class="dropdown-item">Roles</a>
        </div>
    </li>
}
```

85 - Edit identity user

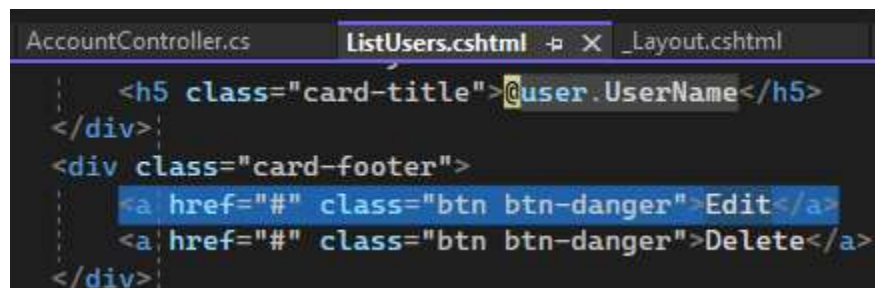
Към момента не сме имплементирали бутони Edit, Delete. Когато натиснем Edit, в адрес бара просто се добавя #



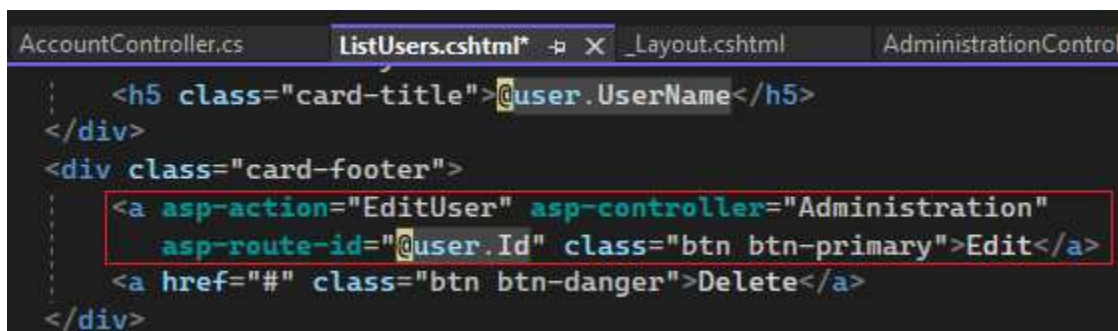
Edit Identity User in ASP.NET Core



Просто сменяме кода:

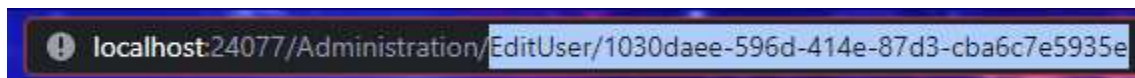


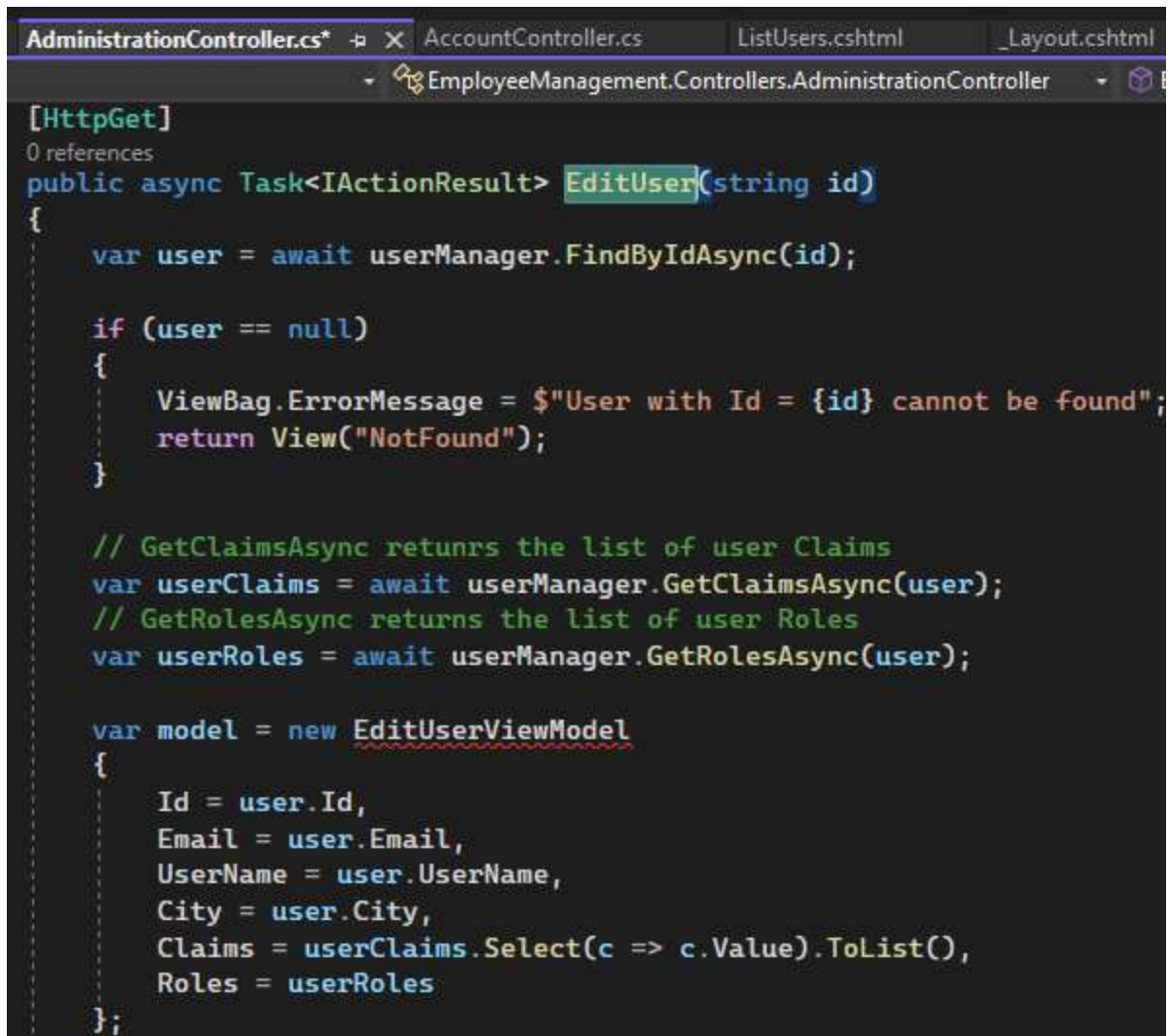
За да пренасочим потребителя към адрес, подавайки id



```
<a asp-action="EditUser" asp-controller="Administration"
  asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
```

Сега, при Edit, адресът автоматично прихваща id, но връща 404, защото все още не сме имплементирали EditUser в AdministrationController:





```
[HttpGet]
0 references
public async Task<IActionResult> EditUser(string id)
{
    var user = await userManager.FindByIdAsync(id);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
        return View("NotFound");
    }

    // GetClaimsAsync returns the list of user Claims
    var userClaims = await userManager.GetClaimsAsync(user);
    // GetRolesAsync returns the list of user Roles
    var userRoles = await userManager.GetRolesAsync(user);

    var model = new EditUserViewModel
    {
        Id = user.Id,
        Email = user.Email,
        UserName = user.UserName,
        City = user.City,
        Claims = userClaims.Select(c => c.Value).ToList(),
        Roles = userRoles
    };
}
```

```
[HttpGet]
public async Task<IActionResult> EditUser(string id)
{
    var user = await userManager.FindByIdAsync(id);

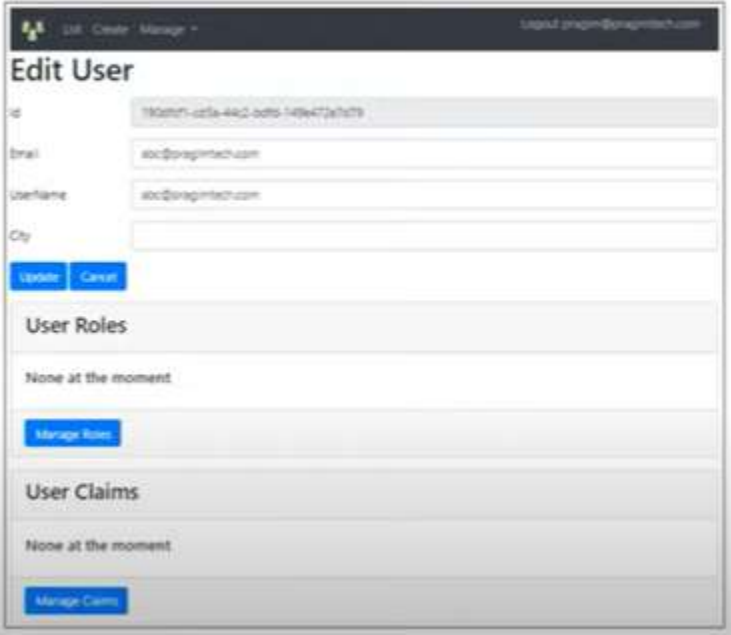
    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
        return View("NotFound");
    }

    // GetClaimsAsync returns the list of user Claims
    var userClaims = await userManager.GetClaimsAsync(user);
    // GetRolesAsync returns the list of user Roles
    var userRoles = await userManager.GetRolesAsync(user);

    var model = new EditUserViewModel
    {
```

```
Id = user.Id,  
Email = user.Email,  
UserName = user.UserName,  
City = user.City,  
Claims = user.Claims.Select(c => c.Value).ToList(),  
Roles = userRoles  
};  
  
return View(model);  
}
```

Edit Identity User in ASP.NET Core



The screenshot shows the 'Edit User' form in an ASP.NET Core application. The form is titled 'Edit User' and has a navigation bar at the top with 'List', 'Create', and 'Manage' links. The user's email 'logsd@pragimtech.com' is displayed in the top right corner. The form contains the following fields and sections:

- id:** A text field containing the GUID '1f02d7f1-225a-44d2-bd5b-149e472e7c79'.
- email:** A text field containing 'logsd@pragimtech.com'.
- username:** A text field containing 'logsd@pragimtech.com'.
- city:** An empty text field.
- Buttons:** 'Update' and 'Cancel' buttons are located below the 'city' field.
- User Roles:** A section titled 'User Roles' showing 'None at the moment' and a 'Manage Roles' button.
- User Claims:** A section titled 'User Claims' showing 'None at the moment' and a 'Manage Claims' button.

Properties:

EditUserViewModel

```
public class EditUserViewModel
{
    public EditUserViewModel()
    {
        initialize Claims = new List<string>(); Roles = new List<string>();
    }

    properties: public string Id { get; set; }

    [Required]
    public string UserName { get; set; }

    validate [Required][EmailAddress]
    public string Email { get; set; }

    public string City { get; set; }

    public List<string> Claims { get; set; }

    interface public IList<string> Roles { get; set; }
}
```

```
public class EditUserViewModel
{
    public EditUserViewModel()
    {
        Claims = new List<string>();
        Roles = new List<string>();
    }

    public string Id { get; set; }

    [Required]
    public string UserName { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    public string City { get; set; }

    public List<string> Claims { get; set; }

    public IList<string> Roles { get; set; }
}
```

Administration + EditUser.cshtml

@model EditUserViewModel

@{
 ViewBag.Title = "Edit User";

```
}
```

```
<h1>Edit User</h1>
```

```
<form method="post" class="mt-3">
  <div class="form-group row">
    <label asp-for="Id" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="Id" disabled class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="Email" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="Email" class="form-control">
      <span asp-validation-for="Email" class="text-danger"></span>
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="UserName" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="UserName" class="form-control">
      <span asp-validation-for="UserName" class="text-danger"></span>
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="City" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="City" class="form-control">
    </div>
  </div>

  <div asp-validation-summary="All" class="text-danger"></div>

  <div class="form-group row">
    <div class="col-sm-10">
      <button type="submit" class="btn btn-primary">Update</button>
      <a asp-action="ListUsers" class="btn btn-primary">Cancel</a>
    </div>
  </div>

  <div class="card">
    <div class="card-header">
      <h3>User Roles</h3>
    </div>
    <div class="card-body">
      @if (Model.Roles.Any())
      {
        foreach (var role in Model.Roles)
        {
          <h5 class="card-title">@role</h5>
        }
      }
    </div>
  </div>
```

```

    }
  }
  else
  {
    <h5 class="card-title">None at the moment</h5>
  }
</div>
<div class="card-footer">
  <a href="#" style="width:auto" class="btn btn-primary">
    Manage Roles
  </a>
</div>
</div>

<div class="card mt-3">
  <div class="card-header">
    <h3>User Claims</h3>
  </div>
  <div class="card-body">
    @if (Model.Claims.Any())
    {
      foreach (var claim in Model.Claims)
      {
        <h5 class="card-title">@claim</h5>
      }
    }
    else
    {
      <h5 class="card-title">None at the moment</h5>
    }
  </div>
  <div class="card-footer">
    <a href="#" style="width:auto" class="btn btn-primary">
      Manage Claims
    </a>
  </div>
</div>
</form>

```

В случая важното е, че <input> за Id не бива да се коригира, затова е оставено с disabled атрибут. Също не сме указали името на контролера, защото EditUser и ListUsers се управляват от него и се намират в папка Administration, така че това не е необходимо, а по желание. Имаме Model-object.Roles-property.Any-method.

```

[HttpPost]
public async Task<ActionResult> EditUser(EditUserViewModel model)
{
  var user = await userManager.FindByIdAsync(model.Id);

  if (user == null)

```

```

{
    ViewBag.ErrorMessage = $"User with Id = {model.Id} cannot be found";
    return View("NotFound");
}
else
{
    user.Email = model.Email;
    user.UserName = model.UserName;
    user.City = model.City;

    var result = await userManager.UpdateAsync(user);

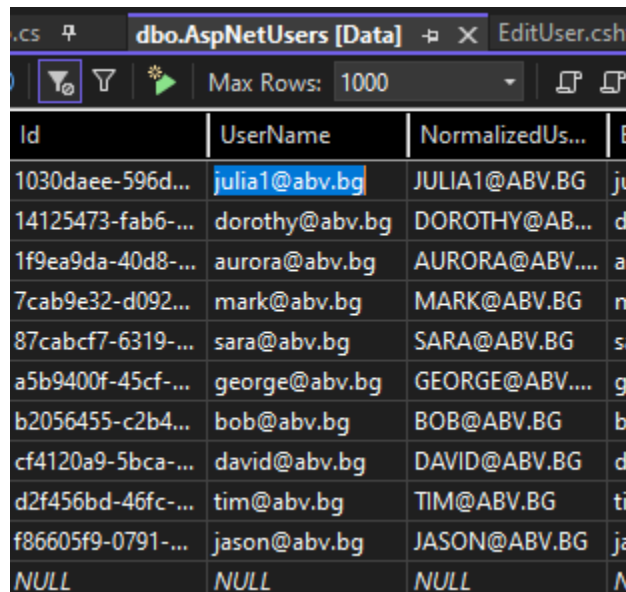
    if (result.Succeeded)
    {
        return RedirectToAction("ListUsers");
    }

    foreach (var error in result.Errors)
    {
        ModelState.AddModelError("", error.Description);
    }

    return View(model);
}
}

```

Проверяваме, че update работи:



Id	UserName	NormalizedUs...	Email
1030daee-596d-4b3a-b86f-000000000000	julia1@abv.bg	JULIA1@ABV.BG	julia1@abv.bg
14125473-fab6-4b3a-b86f-000000000000	dorothy@abv.bg	DOROTHY@ABV.BG	dorothy@abv.bg
1f9ea9da-40d8-4b3a-b86f-000000000000	aurora@abv.bg	AURORA@ABV.BG	aurora@abv.bg
7cab9e32-d092-4b3a-b86f-000000000000	mark@abv.bg	MARK@ABV.BG	mark@abv.bg
87cabcf7-6319-4b3a-b86f-000000000000	sara@abv.bg	SARA@ABV.BG	sara@abv.bg
a5b9400f-45cf-4b3a-b86f-000000000000	george@abv.bg	GEORGE@ABV.BG	george@abv.bg
b2056455-c2b4-4b3a-b86f-000000000000	bob@abv.bg	BOB@ABV.BG	bob@abv.bg
cf4120a9-5bca-4b3a-b86f-000000000000	david@abv.bg	DAVID@ABV.BG	david@abv.bg
d2f456bd-46fc-4b3a-b86f-000000000000	tim@abv.bg	TIM@ABV.BG	tim@abv.bg
f86605f9-0791-4b3a-b86f-000000000000	jason@abv.bg	JASON@ABV.BG	jason@abv.bg
NULL	NULL	NULL	NULL

The main difference between List and IList in C# is that **List is a class** that represents a list of objects which can be accessed by index while **IList is an interface** that represents a collection of objects which can be accessed by index.

Edit User

Id	1030daee-596d-414e-87d3-cba6c7e5935e
Email	julia1@abv.bg
UserName	dorothy@abv.bg
City	London
• Username 'dorothy@abv.bg' is already taken.	
Update Cancel	

Имаме валидация при заето име по време на редакция.

86 - Delete identity user

Delete Identity User in ASP.NET Core

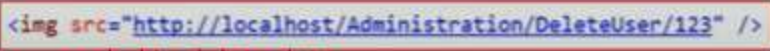
 [List](#) [Create](#) [Manage](#) [Logout](#) [pragim@pragimtech.com](#)

All Users

User Id : 190dfcf1-cd5a-44c2-bd1d-149e472e7d79

abc@pragimtech.com

```
<form method="post" asp-action="DeleteUser" asp-route-id="@user.Id">
  <a asp-controller="Administration" asp-action="Edituser"
    asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
  <button type="submit" class="btn btn-danger">Delete</button>
</form>
```

GET  is not recommended to delete data


```

<a asp-action="EditUser" asp-controller="Administration"
  asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
<a href="#" class="btn btn-danger">Delete</a>
</div>

"card">
class="card-header">
users created yet

```

```

Surround With:
<a href="#"
</div>
content
dd
figure
form
html
Markup snippet for a HTML form
Shortcut: form

```

```

<form action="/" method="post">
  <a asp-action="EditUser" asp-controller="Administration"
    asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
  <a href="#" class="btn btn-danger">Delete</a>
</form>

```

```

<form asp-action="DeleteUser" asp-route-id="@user.Id" method="post">
  <a asp-action="EditUser" asp-controller="Administration"
    asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
  <button type="submit" class="btn btn-danger">Delete</button>
</form>

```

Принатискане на submit, формата се изпраща с Post. Може да изтрием потребителя директно с Get, но това не е препоръчително, т.к. прави пробиви в сигурността: може при клик върху картинка 123 да се изтрие потребител 123 без неговото знание, а търсачките използват GET за прихващане индексите на страниците. По този начин може да изтривят данни също, което не е желателно. Затова винаги Delete се изпълнява чрез POST.

AdministrationController:

```

[HttpPost]
public async Task<IActionResult> DeleteUser(string id)
{
    var user = await userManager.FindByIdAsync(id);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
        return View("NotFound");
    }
}

```

```

    }
    else
    {
        var result = await userManager.DeleteAsync(user);

        if (result.Succeeded)
        {
            return RedirectToAction("ListUsers");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }

        return View("ListUsers");
    }
}

```

Проверяваме в <http://localhost:24077/administration/ListUsers> + `dbo.AspNetUsers.sql`, че работи.

Delete Identity User in ASP.NET Core

```

[HttpPost]
public async Task<IActionResult> DeleteUser(string id)
{
    var user = await userManager.FindByIdAsync(id);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
        return View("NotFound");
    }
    else
    {
        var result = await userManager.DeleteAsync(user);

        if (result.Succeeded)
            return RedirectToAction("ListUsers");

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }

        return View("ListUsers");
    }
}

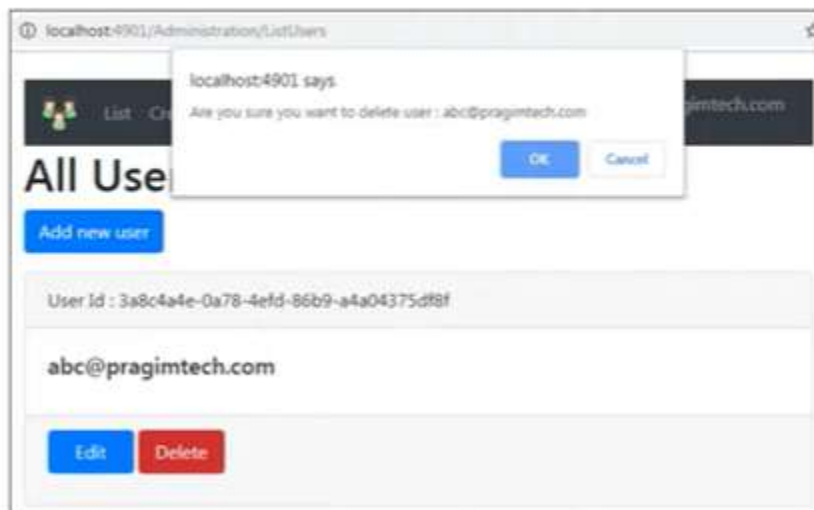
```

87 - Delete confirmation

На този етап, изтриването е без каквото и да е предупредително съобщение, а понеже е необратима операция, е желателно да го има.

Първи начин: подпрозорец на брауъра:

ASP.NET Core Delete Confirmation



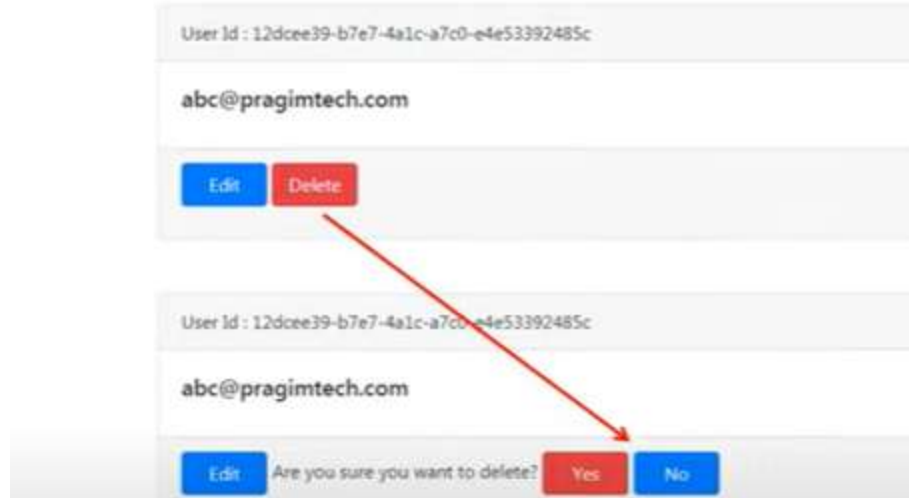
```
<button type="submit" class="btn btn-danger"
    onclick="return confirm('Are you sure you want to delete user : @user.UserName')">
    Delete
    JavaScript: true = OK, false = CANCEL
</button>
```

```
<button type="submit" class="btn btn-danger"
    onclick="return confirm('Are you sure you want to delete user : @user.UserName')">
    Delete
</button>
```

Атрибут onclick ни трябва за натискане върху бутона Delete, като JavaScript функция confirm връща true при ОК върху диалоговия прозорец, с което потвърждава изпращането на формата, а false за cancel.

Повечето хора не харесват подобни диалогови прозорци, а предпочитат съобщението да бъде част от страницата:

ASP.NET Core Delete Confirmation



Това предупредително съобщение да се появява само при избор на Delete, като при Yes, с Post се обработи заявката и изтрие потребителя, докато с No, съобщението се скрие и отново бутон Delete стане видим.

```
asp-route-id="@user.Id" class="btn btn-primary">Edit</a>

<span style="display:none">
  <span>Are you sure you want to delete?</span>
  <button type="submit" class="btn btn-danger">Yes</button>
  <a href="#" class="btn btn-primary">No</a>
</span>

<span>
  <a href="#" class="btn btn-danger">Delete</a>
</span>
```

Това става лесно, като дадем Wrap with <div> и го сменим със , като основната разлика между двата тага е, че <div> се използва за блок, а за ред/линия. И двата тага лесно се обработват с JavaScript/css класове, атрибути, id.

```
<span id="confirmDeleteSpan" style="display:none">
  <span>Are you sure you want to delete?</span>
  <button type="submit" class="btn btn-danger">Yes</button>
  <a href="#" class="btn btn-primary">No</a>
</span>

<span id="deleteSpan">
  <a href="#" class="btn btn-danger">Delete</a>
</span>
```

Макар, че в брауъра не са видими, View source ги показва. Не е необходимо id да съвпадат като имена, както е при тоглерите и dropdown менюто.

```

<span id="confirmDeleteSpan" style="display:none">
  <span>Are you sure you want to delete?</span>
  <button type="submit" class="btn btn-danger">Yes</button>
  <a href="#" class="btn btn-primary">No</a>
</span>

<span id="deleteSpan">
  <a href="#" class="btn btn-danger">Delete</a>
</span>

```

При добавяне на `@user.Id` съответно ще се види присвоен уникалния стринг.

```

AdministrationController.cs  ListUsers.cshtml*  _Layout.cshtml
<form asp-action="DeleteUser" asp-route-id="@user.Id" method="post">
  <a asp-action="EditUser" asp-controller="Administration"
    asp-route-id="@user.Id" class="btn btn-primary">Edit</a>

  <span id="confirmDeleteSpan_@user.Id" style="display:none">
    <span>Are you sure you want to delete?</span>
    <button type="submit" class="btn btn-danger">Yes</button>
    <a href="#" class="btn btn-primary"
      onclick="confirmDelete('@user.Id', false)">No</a>
  </span>
  <span id="deleteSpan_@user.Id">
    <a href="#" class="btn btn-danger"
      onclick="confirmDelete('@user.Id', true)">Delete</a>
  </span>
</form>

```

```

<span id="confirmDeleteSpan_@user.Id" style="display:none">
  <span>Are you sure you want to delete?</span>
  <button type="submit" class="btn btn-danger">Yes</button>
  <a href="#" class="btn btn-primary"
    onclick="confirmDelete('@user.Id', false)">No</a>
</span>

```

```

<span id="deleteSpan_@user.Id">
  <a href="#" class="btn btn-danger"
    onclick="confirmDelete('@user.Id', true)">Delete</a>
</span>

```

```
</form>
```

```
</div>
```

За извеждане на съобщението ни трябва JavaScript функция, която ще запишем във вградения CustomScript.js, като целта ѝ е чрез `.show()` и `.hide()` да скрием бутон Delete и изведем предупредителното съобщение или обратното. В случая `uniqueId` присвоява подадената `user.Id`, а на `isDeleteClicked` съответно `true/false`:


```

function confirmDelete(uniqueId, isDeleteClicked) {
    var deleteSpan = 'deleteSpan_' + uniqueId;
    var confirmDeleteSpan = 'confirmDeleteSpan_' + uniqueId;

    if (isDeleteClicked) {
        $('#' + deleteSpan).hide();
        $('#' + confirmDeleteSpan).show();
    } else {
        $('#' + deleteSpan).show();
        $('#' + confirmDeleteSpan).hide();
    }
}

```

За да направим достъпен JavaScript кода, трябва да се съобразим с извикването му при определени обстоятелства от предните уроци:

```

<div>
    @RenderBody()
</div>
@if (IsSectionDefined("Scripts"))
{
    @RenderSection("Scripts", required: true)
}
</div>
</body>
</html>

```

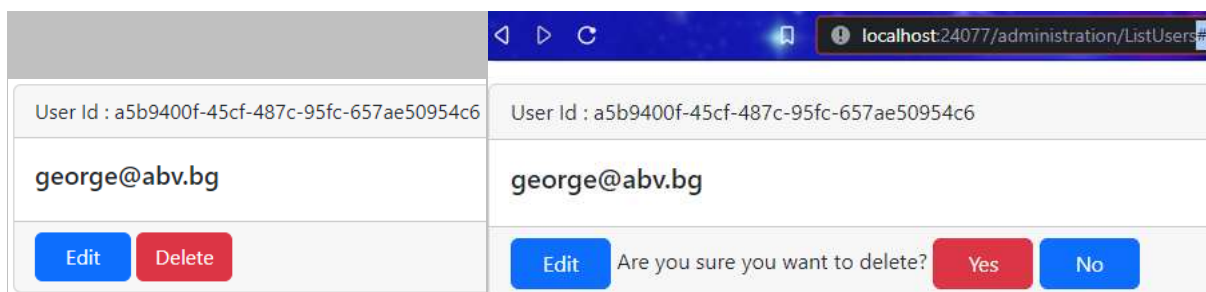
Така че, остава само да извикаме този Scripts на дъното на нашата страница ListUsers:

```

@section Scripts
{
    <script src="~/js/CustomScript.js"></script>
}

```

Тестваме скриването и показването на бутоните:



ASP.NET Core Delete Confirmation

```
<div class="card-footer">
  <form method="post" asp-action="DeleteUser" asp-route-id="@user.Id">
    <a asp-controller="Administration" asp-action="Edituser"
      asp-route-id="@user.Id" class="btn btn-primary">Edit</a>

    <span id="confirmDeleteSpan_@user.Id" style="display:none">
      <span>Are you sure you want to delete?</span>
      <button type="submit" class="btn btn-danger">Yes</button>
      <a href="#" class="btn btn-primary"
        onclick="confirmDelete('@user.Id', false)">No</a>
    </span>

    <span id="deleteSpan_@user.Id">
      <a href="#" class="btn btn-danger"
        onclick="confirmDelete('@user.Id', true)">Delete</a>
    </span>
  </form>
</div>
```

ASP.NET Core Delete Confirmation

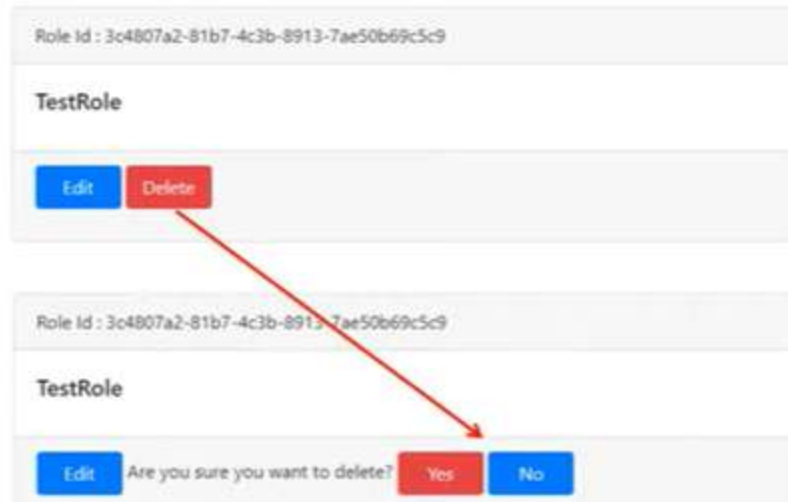
```
function confirmDelete(uniqueId, isTrue) {

  var deleteSpan = 'deleteSpan_' + uniqueId;
  var confirmDeleteSpan = 'confirmDeleteSpan_' + uniqueId;

  if (isTrue) {
    $('# ' + deleteSpan).hide();
    $('# ' + confirmDeleteSpan).show();
  } else {
    $('# ' + deleteSpan).show();
    $('# ' + confirmDeleteSpan).hide();
  }
}
```

88 - Delete identity role

Delete Identity Role in ASP.NET Core



Аналогично като в предходния урок, но ще използваме roleManager, вместо userManager и сменяме всички users->roles:

```
AdministrationController.cs
EmployeeManagement.Controllers.AdministrationController

{
    this.roleManager = roleManager;
    this.userManager = userManager;
}

[HttpGet]
0 references
public IActionResult CreateRole()...

[HttpGet]
0 references
public IActionResult ListUsers()...

[HttpGet]
0 references
public async Task<IActionResult> EditUser(string id)...

[HttpPost]
0 references
public async Task<IActionResult> EditUser(EditUserViewModel model)...

[HttpPost]
0 references
public async Task<IActionResult> DeleteUser(string id)
```

```

[HttpPost]
public async Task<IActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }
    else
    {
        var result = await roleManager.DeleteAsync(role);

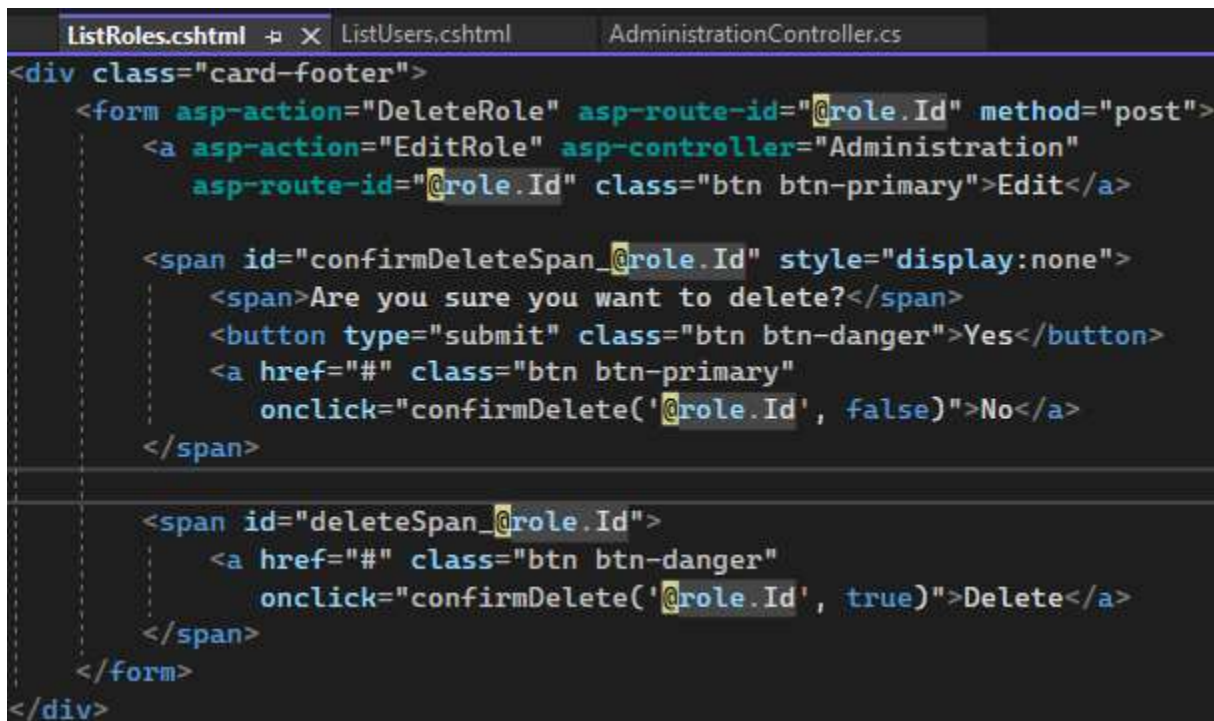
        if (result.Succeeded)
        {
            return RedirectToAction("ListRoles");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }

        return View("ListRoles");
    }
}

```

По същия начин сменяме ListUser->ListRole, като променяме формата и добавяме секцията за скрипт на дъното на страницата:



```

<div class="card-footer">
    <form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">
        <a asp-action="EditRole" asp-controller="Administration"
            asp-route-id="@role.Id" class="btn btn-primary">Edit</a>

        <span id="confirmDeleteSpan_@role.Id" style="display:none">
            <span>Are you sure you want to delete?</span>
            <button type="submit" class="btn btn-danger">Yes</button>
            <a href="#" class="btn btn-primary"
                onclick="confirmDelete('@role.Id', false)">No</a>
        </span>

        <span id="deleteSpan_@role.Id">
            <a href="#" class="btn btn-danger"
                onclick="confirmDelete('@role.Id', true)">Delete</a>
        </span>
    </form>
</div>

```

```

<form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">
    <a asp-action="EditRole" asp-controller="Administration"

```



```

        asp-route-id="@role.Id" class="btn btn-primary">Edit</a>

        <span id="confirmDeleteSpan_@role.Id" style="display:none">
            <span>Are you sure you want to delete?</span>
            <button type="submit" class="btn btn-danger">Yes</button>
            <a href="#" class="btn btn-primary"
                onclick="confirmDelete('@role.Id', false)">No</a>
        </span>

        <span id="deleteSpan_@role.Id">
            <a href="#" class="btn btn-danger"
                onclick="confirmDelete('@role.Id', true)">Delete</a>
        </span>
    </form>

```

...

@section Scripts

```

{
    <script src="~/js/CustomScript.js"></script>
}

```

Delete Identity Role in ASP.NET Core

```

<form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">
    <a class="btn btn-primary" asp-action="EditRole"
        asp-controller="Administration" asp-route-id="@role.Id">
        Edit
    </a>

    <span id="confirmDeleteSpan_@role.Id" style="display:none">
        <span>Are you sure you want to delete?</span>
        <button type="submit" class="btn btn-danger">Yes</button>
        <a href="#" class="btn btn-primary"
            onclick="confirmDelete('@role.Id', false)">No</a>
    </span>

    <span id="deleteSpan_@role.Id">
        <a href="#" class="btn btn-danger"
            onclick="confirmDelete('@role.Id', true)">Delete</a>
    </span>
</form>

```

Delete Identity Role in ASP.NET Core

```

function confirmDelete(uniqueId, isTrue) {

    var deleteSpan = 'deleteSpan_' + uniqueId;
    var confirmDeleteSpan = 'confirmDeleteSpan_' + uniqueId;

    if (isTrue) {
        $(' #' + deleteSpan).hide();
        $(' #' + confirmDeleteSpan).show();
    } else {
        $(' #' + deleteSpan).show();
        $(' #' + confirmDeleteSpan).hide();
    }
}

```

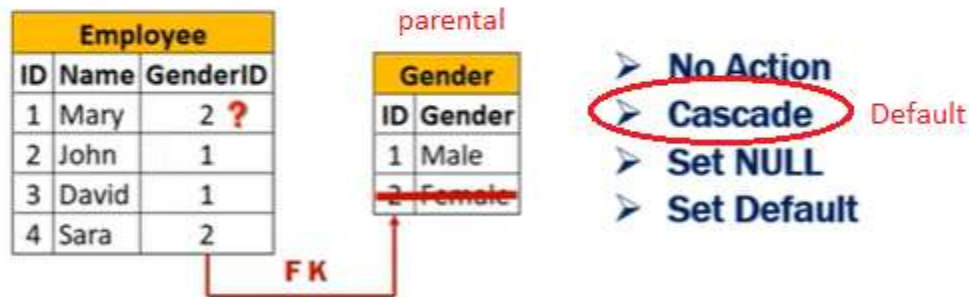

89 - Enforce ON DELETE NO ACTION

enforce foreign key constraint referential integrity

налагане на референтна цялост на ограничението на външния ключ

Cascading referential integrity constraint

Cascading referential integrity constraint allows to define the actions Microsoft SQL Server should take when a user attempts to delete or update a key to which an existing **foreign keys** points

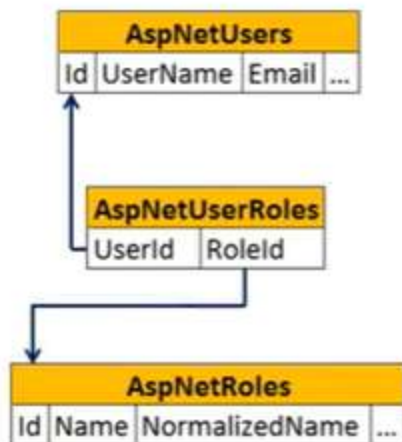


Part 5 of SQL Server Tutorial

<https://www.youtube.com/watch?v=ETepOV7Xk8&list=PL08903FB7ACA1C2FB&index=6&t=0s>

Ако изтрием данни от някоя родителска таблица, данните в наследяващата я автоматично се изтриват също.

Enforce ON DELETE NO ACTION

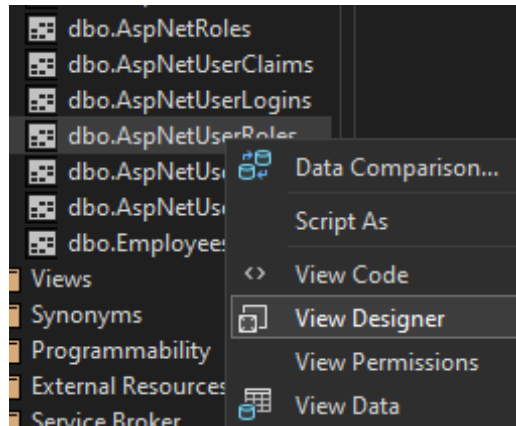


- Users are stored in **AspNetUsers** table
- Roles are stored in **AspNetRoles** table
- User and Role mapping data is stored in **AspNetUserRoles** table
- This table has just 2 columns : **UserId** & **RoleId**
- Both are **foreign keys**

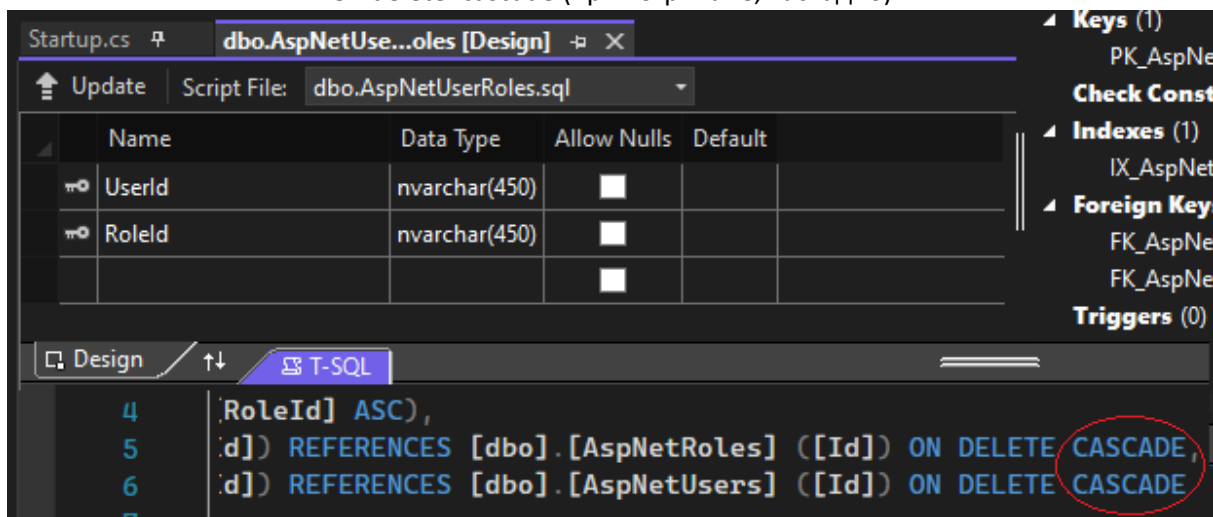
Startup.cs	dbo.AspNetRoles [Data]	dbo.AspNetUsers [Data]
	Max Rows: 1000	Max Rows: 1000
	Id	UserName
	222-9fd26303740c	aurora@abv.bg
	7cab9e32-d092-49e7-bf31-93b92a34dc49	mark@abv.bg
	87cabcf7-6319-4672-a7c8-db5a048fa5f7	sara@abv.bg
	a5b9400f-45cf-487c-95fc-657ae50954c6	george@abv.bg
	b2056455-c2b4-42d5-883d-27475d32038b	bob@abv.bg
	cf4120a9-5bca-4cbc-8e3e-8c9a636d4a83	david@abv.bg
	d2f456bd-46fc-4bea-872b-5e24a6004d85	tim@abv.bg
	f86605f9-0791-4b02-97b9-dcc28904fb73	jason@abv.bg
	NULL	NULL

Startup.cs	dbo.AspNetUserRoles [Data]	dbo.AspNetRoles [Data]
	Max Rows: 1000	Max Rows: 1000
	UserId	RoleId
	aurora	user + admin
	1f9ea9da-40d8-4413-a222-9fd26303740c	71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
	1f9ea9da-40d8-4413-a222-9fd26303740c	7a53c902-d9f9-4026-b521-26b8485f1611
	b2056455-c2b4-42d5-883d-27475d32038b	7a53c902-d9f9-4026-b521-26b8485f1611
	NULL	NULL

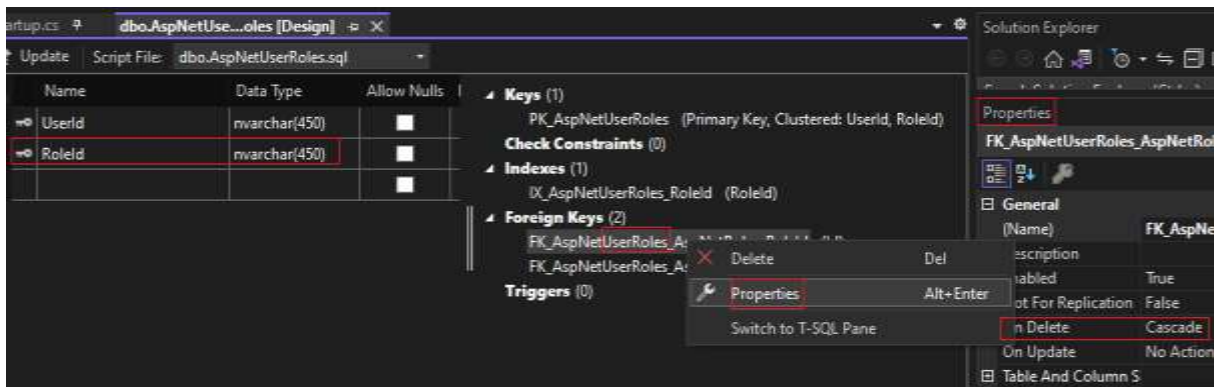
Ако изтрием данни от Roles, например изтрием потребител User, в свързващата дъщерна таблица всички записи на членове на User ще бъдат изтрети, защото те са били в тази роля. Няма да се изтрие само ролята, а потребителите да останат. По същия начин, ако изтрием от родителска таблица пол жена, от дъщерната таблица, всички потребители, които са с този пол, автоматично ще бъдат изтрети също, а не те да останат, а колоната да се нулира. Това е каскадност по подразбиране. Може да го докажем, като разгледаме дефинициите -> View Designer:



On delete: cascade (при изтриване, каскадно)



Другият начин да се види е през Properties:



Като защита, искаме да забраним Role да бъде изтривана, ако има потребители в нея. Ако искаме да изтрием Role, първо трябва да премахнем потребителите от нея и тогава да я изтрием. За целта, по подразбиране, On Delete: No action. Тогава, ако все още има потребители в тази role, по подразбиране ще върне изключение.

Enforce ON DELETE NO ACTION

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    foreach (var foreignKey in modelBuilder.Model.GetEntityTypes()
        .SelectMany(e => e.GetForeignKeys()))
    {
        foreignKey.DeleteBehavior = DeleteBehavior.Restrict;
    }
}
```

```
public enum DeleteBehavior
{
    ClientSetNull = 0,
    Restrict = 1,
    SetNull = 2,
    Cascade = 3
}
```

Properties	
FK_AspNetUserRoles_AspNetUser	
(Name)	FK_AspNetUserRoles_AspNetUser
Description	
Enabled	True
Not For Replication	False
On Delete	Cascade
On Update	No Action

AppDbContext.cs | dbo.AspNetUserRoles [Design]

0 references

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Seed();

    foreach (var foreignKey in modelBuilder.Model.GetEntityTypes()
        .SelectMany(e => e.GetForeignKeys()))
    {
        foreignKey.DeleteBehavior = DeleteBehavior.Restrict;
    }
}
```

`foreach (var foreignKey in modelBuilder.Model.GetEntityTypes().SelectMany(e => e.GetForeignKeys()))`

```
{
    foreignKey.DeleteBehavior = DeleteBehavior.Restrict;
}
```

За да влязат промените в БД, трябва да създадем миграция за тях.

The screenshot shows a Visual Studio editor with the following C# code for a migration:

```
31 migrationBuilder.DropForeignKey(
32     name: "FK_AspNetUserTokens_AspNetUsers_UserId",
33     table: "AspNetUserTokens");
34
35 migrationBuilder.AddForeignKey(
36     name: "FK_AspNetRoleClaims_AspNetRoles_RoleId",
37     table: "AspNetRoleClaims",
38     column: "RoleId",
39     principalTable: "AspNetRoles",
40     principalColumn: "Id",
41     onDelete: ReferentialAction.Restrict);
```

Below the code, the Package Manager Console shows the command:

```
PM> add-migration Test
```

Когато разгледаме създадената миграция, първо, тя изтрива всички външни ключове, след което ги връща със сменена стойност при изтриване на Restrict. Прилагаме миграцията с update-database. За да видим промените в БД, затваряме дизайнера и го отваряме наново. Виждаме, че On delete не е зададено, т.е. при това положение, по подразбиране е No action:

```
FOREIGN KEY ([RoleId]) REFERENCES [dbo].[AspNetRoles] ([Id])
FOREIGN KEY ([UserId]) REFERENCES [dbo].[AspNetUsers] ([Id])
```

On Delete	No Action
On Update	No Action

За проба създаваме TestRole, в която няма потребители и съответно имаме право да я изтрием без проблеми. При втория опит създаваме TestRole1, добавяме потребители и когато се опитаме да я изтрием, връща грешка:



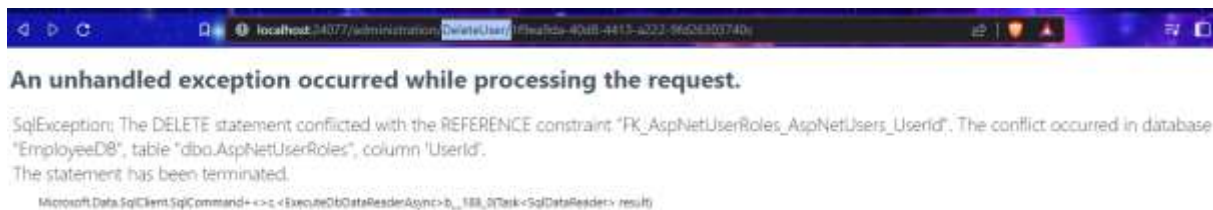
SqlException: The DELETE statement conflicted with the REFERENCE constraint "FK_AspNetUserRoles_AspNetRoles_RoleId". The conflict occurred in database "EmployeeDB", table "dbo.AspNetUserRoles", column 'RoleId'.

DbUpdateException: An error occurred while saving the entity changes. See the inner exception for details.

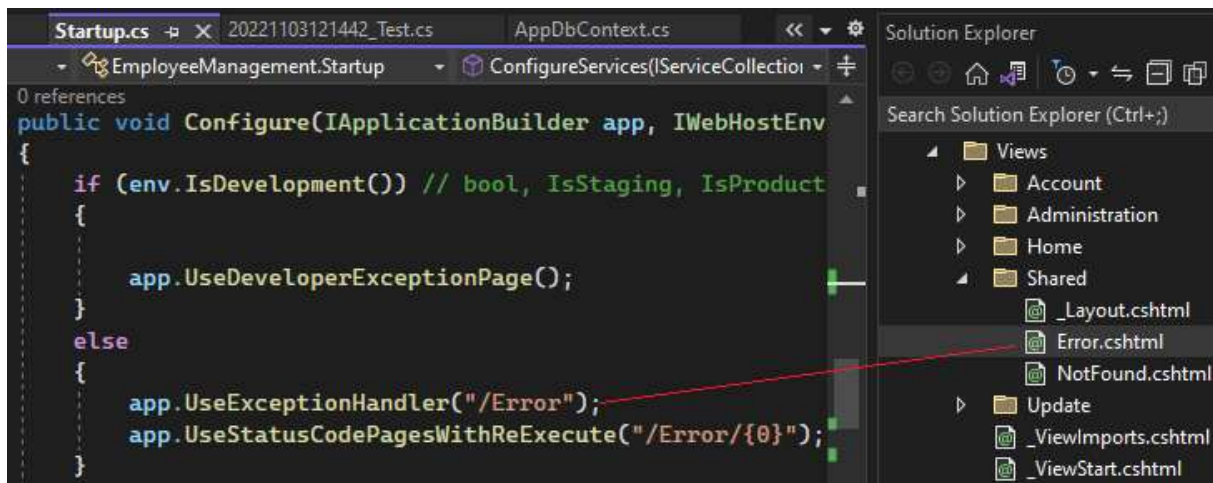
В следващия урок ще направим user-friendly exception page.

90 - Custom error page

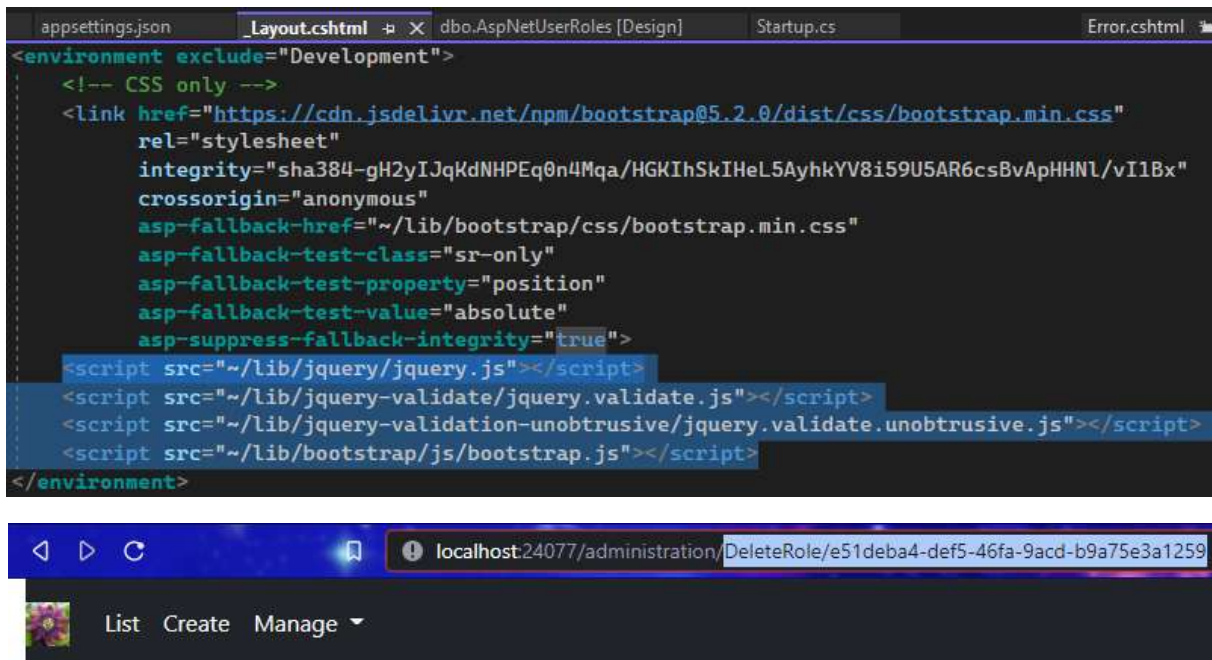
Това е продължение от предишния урок, при който при опит да изтрием Role, в който има потребители (не е празна), като защита на самата БД, On delete: No action, връща error page, която в DevelopmentEnvironment е ок, но за потребителите не, защото за обикновения потребител няма никакъв смисъл, а и е рисково за сигурността.



В Production искаме custom error page, която имаме налична, но трябва да минем в нейната среда, за да я видим.



Също, за да работи dropdown менюто, трябва да имаме възможност скриптовете да работят в тази среда:

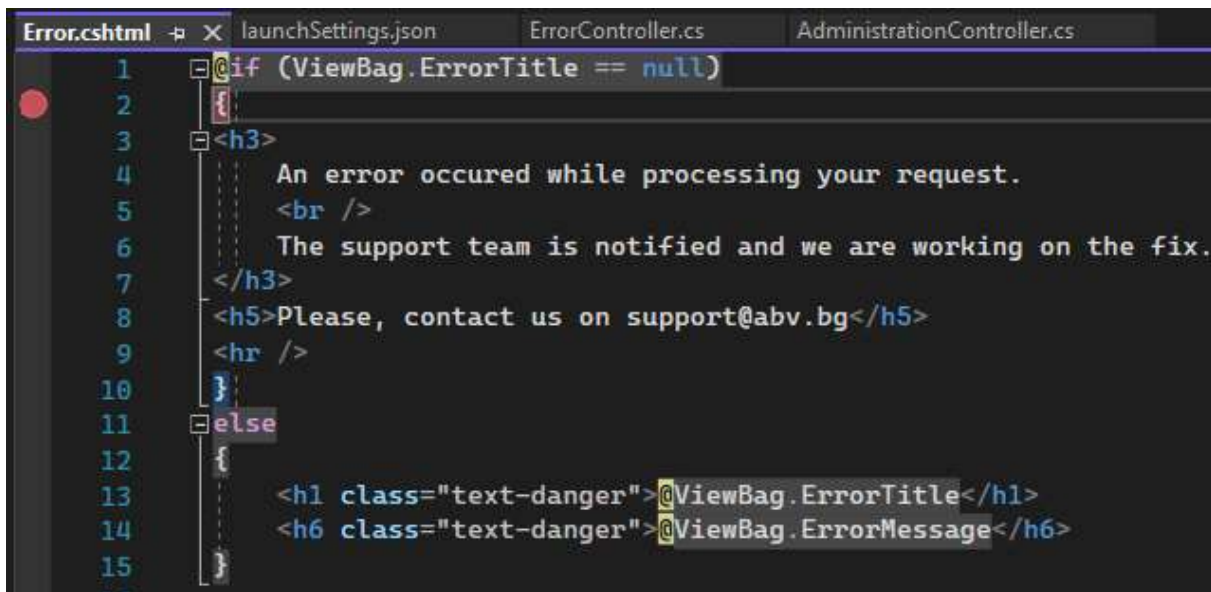


An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

Това е прекалено генерална страница за грешка, а по-добре би било да имаме повече конкретна информация към потребителя:

Custom Error Page in ASP.NET Core



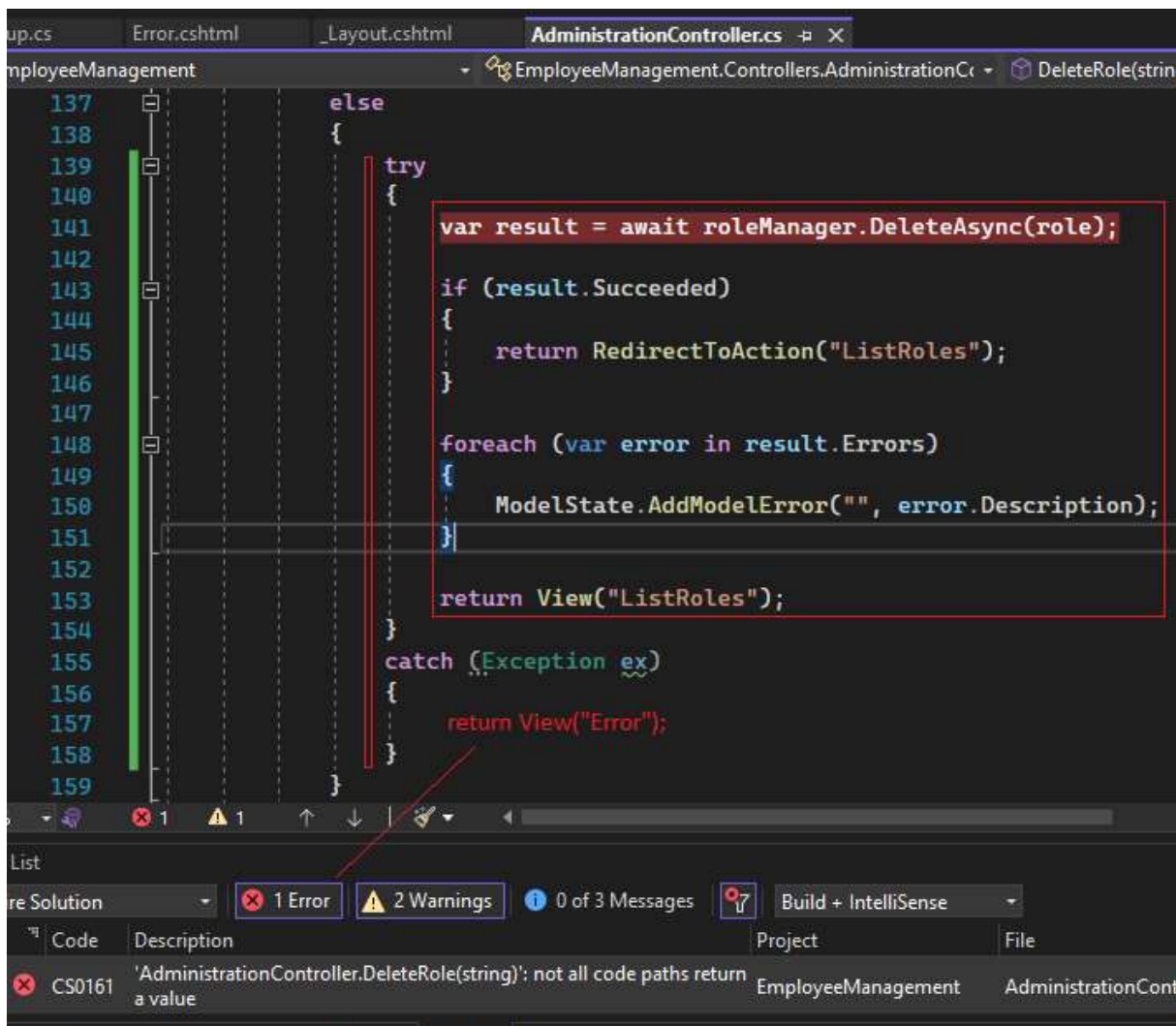


```
1  @if (ViewBag.ErrorTitle == null)
2  {
3      <h3>
4          An error occurred while processing your request.
5          <br />
6          The support team is notified and we are working on the fix.
7      </h3>
8      <h5>Please, contact us on support@abv.bg</h5>
9      <hr />
10 }
11 else
12 {
13     <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
14     <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
15 }
```

метод DeleteAsync(role) отговаря за изтриването, като редът, който отговаря за това дали ще се върне грешка или не, е

```
var result = await roleManager.DeleteAsync(role);
```

така че заграждаме следващите няколко реда в блок с try{} + catch {} за прихващане на изключение + breakpoint на този ред -> тестваме в debug mode.



Задържа се грешка, която се коригира с извикване на страницата Error.cshtml.

Сега остава въпросът да се прихванат стойности за

```
ViewBag.ErrorTitle = $"{role.Name} role is in use";
ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted as there are users in
this role. If you want to delete this role, please remove the users from the role and then try to
delete";
```

Където \$+ са знаците за нов ред

```

AdministrationController.cs* X
EmployeeManagement.Controllers.AdministrationC DeleteRole(string id)
0 references
public async Task<IActionResult> DeleteRole(string id) URL
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null) ...
    else
    {
        try
        {
            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded) ...

            foreach (var error in result.Errors) ...

            return View("ListRoles");
        }
        catch (Exception ex)
        {
            ViewBag.ErrorTitle = $"{role.Name} role is in use";
            ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted " +
                $"as there are users in this role. " +
                $"If you want to delete this role, please remove the users from the role " +
                $"and then try to delete";
            return View("Error");
        }
    }
}

```

The try-catch statement consists of a try block followed by one or more catch clauses, which specify handlers for different exceptions.

```

@if (ViewBag.ErrorTitle == null)
{
    <h3>
        An error occurred while processing your request.
        <br />
        The support team is notified and we are working on the fix.
    </h3>
    <h5>Please, contact us on support@abv.bg</h5>
    <hr />
}
else
{
    <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
    <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
}

```

BreakPoint: Поставяме на подозрителния ред в try:

```

launchSettings.json AdministrationController.cs X
EmployeeManagement EmployeeManagement.Controllers.AdministrationConti Delete
147
148 var result = await roleManager.DeleteAsync(role);
149

```

Debug:

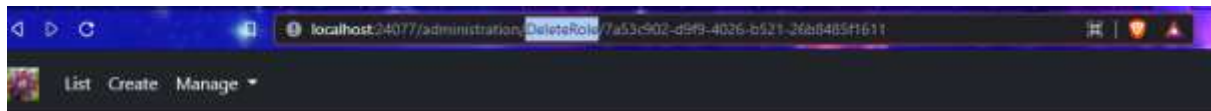
Опитваме да изтрием админ и потвърждаваме, при което командата ни изхвърля във VS:

```
147  
148 var result = await roleManager.DeleteAsync(role);  
149
```

Поставяме нов BreakPoint, за да прескочи в catch:

```
202 catch (Exception ex)  
203 {  
204     logger.LogError($"Error deleting role : {ex}");  
205     ViewBag.ErrorTitle = $"{role.Name} role is in use";  
206 }
```

Continue Debugging



Admin role is in use

Admin role cannot be deleted as there are users in this role. If you want to delete this role, please remove the users from the role and then try to delete.

Променливата ex е със зелено подчертаване, понеже не се използва, като за да се коригира това, добавяме логър, който да проследи грешките в отделен файл. В реалните приложения, грешките се логват, за да може девелъпърът да ги коригира по-късно.

```
public AdministrationController(RoleManager<IdentityRole> roleManager,  
                                UserManager<ApplicationUser> userManager,  
                                ILogger<AdministrationController> logger)  
{  
    this.roleManager = roleManager;  
    this.userManager = userManager;  
    this.logger = logger;  
}
```

```
private readonly UserManager<ApplicationUser> userManager;  
private readonly ILogger<AdministrationController> logger;
```

Сега вече може да използваме logger за прихващане на изключението.

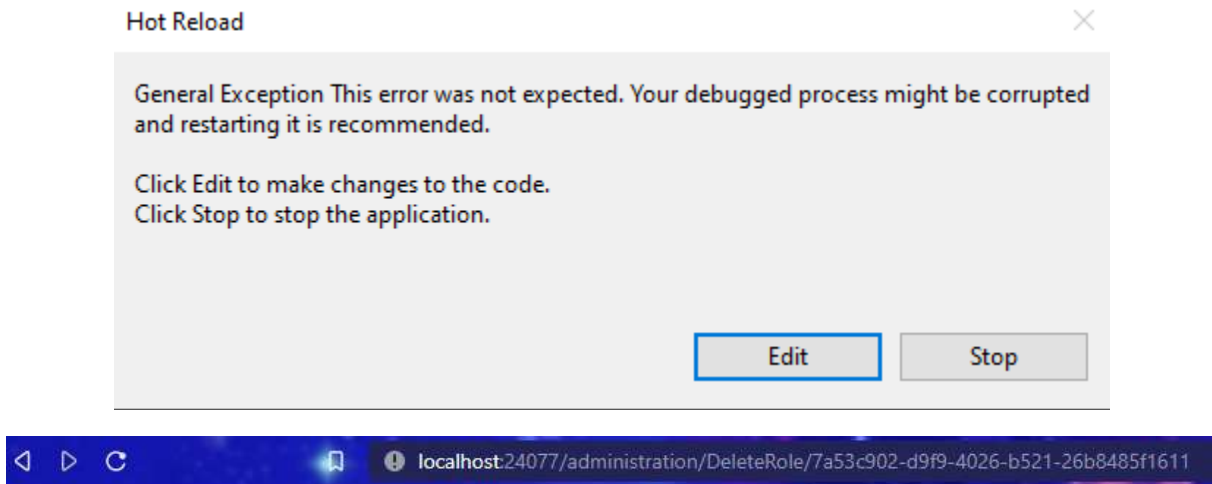
За демонстрация, поставяме ред

```
throw new Exception("Test exception");
```

непосредствено след try {, с BreakPoint на него -> Debug:

```
144 try  
145 {  
146     throw new Exception("Test exception");  
147 }
```

В този случай, Exception е general - Continue ->



В новите версии се появява това съобщение за редакция на кода, а в старите просто се зарежда старата страница със старото съобщение за грешка.

По-добрият вариант, вместо генералния `catch (Exception ex)` -> специфичния `catch (DbUpdateException ex)`, защото иначе всеки път ще прихваща всички видове грешки и връща червената страница. Искаме в общия случай черната, а при определени обстоятелства само червената. Направим ли го, вече `catch` ще пресее грешката, ще види, че не може да я обработи и ще я препрати към `ErrorController`, който ще я запише в `log` файла и препрати към `Error view`, без да присвои стойност на `ErrorTittle`, който ще остане `null`, при което `if == true` и ще се изведе черното съобщение. Всичко това е, за да връща черната страница в общия случай, а само при опит за изтриване на заета `role` – червената с информативното съобщение за корекция.

Нека сменим:

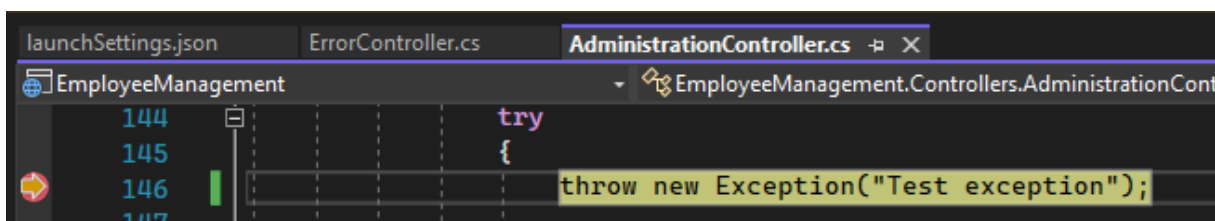
```
catch (DbUpdateException ex)
```

По този начин ще бъдат прихванати с `catch` само специфичните за БД грешки, а не всички `general`.

Стартираме по същият начин проекта в `Debug mode` и стартира страницата с желаното съобщение.

Правим 3-та демонстрация: `Debug`, после нов `BreakPoint` в `ErrorController`,

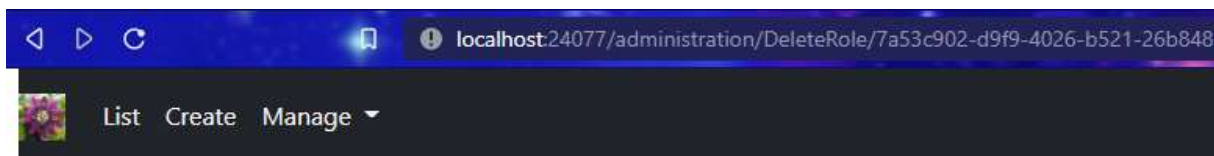
Потвърждаваме изтриване на Админ, за да видим какво става. Изхвърля ни към



В моята версия отново извежда съобщение за редакция.

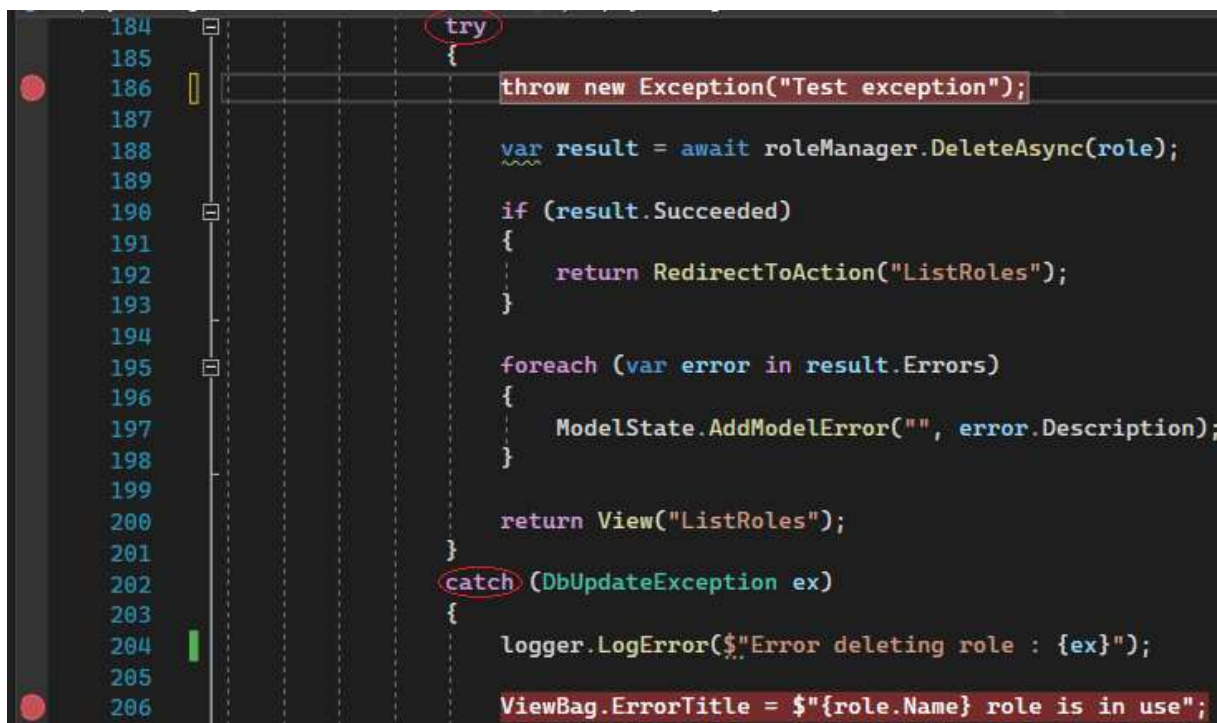
В този случай, нямаме подадена стойност за ViewBag.ErrorTitle – както се вижда при посочване, тя е null, така че се изпълнява кодът от if блока с обикновеното съобщение за грешка.

При активация на BreakPoint, изпълнението е с жълта стрелка. Поставяме нова BreakPoint -> Continue:



An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

Последно оставяме няколко Breakpoint:



```

Error.cshtml launchSettings.json ErrorController.cs AdministrationController.cs
EmployeeManagement EmployeeManagement.Controllers.ErrorController Error
0 references
37 public IActionResult Error()
38 {
39     var exceptionDetails = HttpContext.Features.Get<IExceptionHandler>();
40
41     logger.LogError($"The path {exceptionDetails.Path} " +
42         $"threw an exception {exceptionDetails.Error}");
43
44     ViewBag.ExceptionPath = exceptionDetails.Path;
45     ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
46     ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;
47
48     return View("Error");
49 }

```

```

Error.cshtml launchSettings.json ErrorController.cs AdministrationController.cs
1 @if (ViewBag.ErrorTitle == null)
2 {
3     <h3>
4         An error occurred while processing your request.
5         <br />
6         The support team is notified and we are working on the fix.
7     </h3>
8     <h5>Please, contact us on support@abv.bg</h5>
9     <hr />
10 }
11 else
12 {
13     <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
14     <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
15 }

```

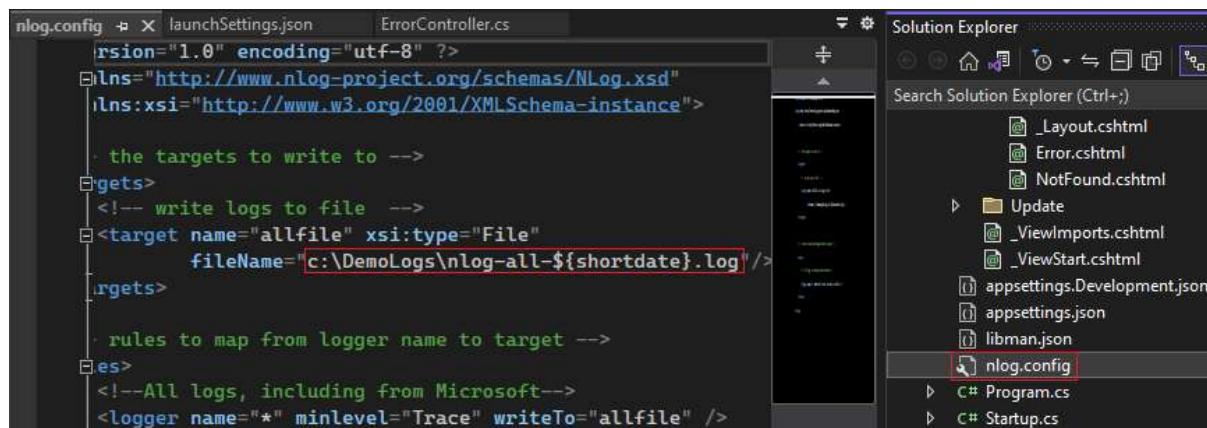
Като всеки Continue ни прехвърля от един BreakPoint към следващ и накрая, понеже ErrorTitle == null, виждаме обикновения NotFound View, за general Errors.



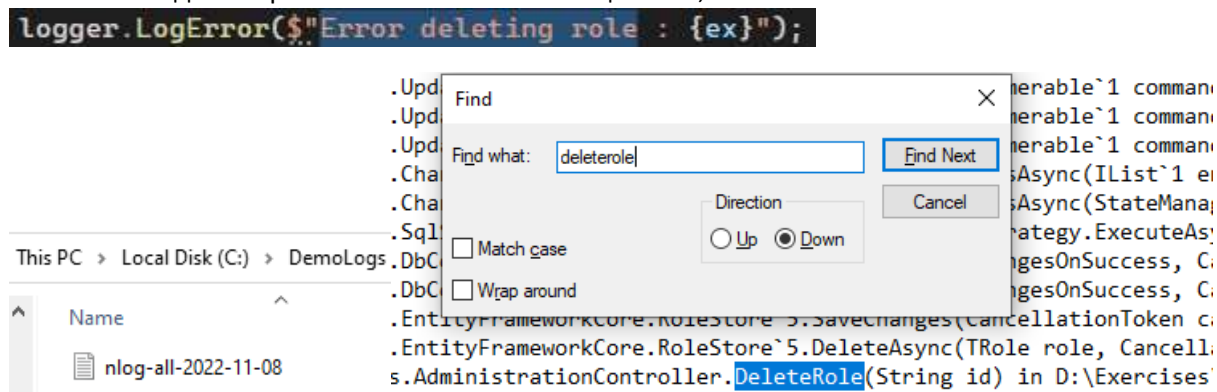
An error occurred while processing your request.
 The support team is notified and we are working on the fix.
 Please, contact us on support@abv.bg

Закоментираме вече излишния ред `//throw new Exception("Test exception");`

При тези обстоятелства, ако се опитаме в Debug mode да изтрием админ, се отваря персонализираната страница **Admin role cannot be deleted as there are users in this role** и се отчита в nlog.config, чието местоположение може да намерим:



Може в него да потърсим DeleteRole или съобщението, което сме изписали:



[Microsoft.EntityFrameworkCore.DbUpdateException]: An error occurred while saving the entity changes.

```
[HttpPost]
public async Task<IActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }
    else
    {
        try
        {
            //throw new Exception("Test exception");

            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded)
            {
                return RedirectToAction("ListRoles");
            }
        }
    }
}
```



```

    }

    foreach (var error in result.Errors)
    {
        ModelState.AddModelError("", error.Description);
    }

    return View("ListRoles");
}
catch (DbUpdateException ex)
{
    logger.LogError($"Error deleting role : {ex}");

    ViewBag.ErrorTitle = $"{role.Name} role is in use";
    ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted " +
        $"as there are users in this role. " +
        $"If you want to delete this role, please remove the users
from the role " +
        $"and then try to delete";
    return View("Error");
}
}
}

```

Custom Error Page in ASP.NET Core



```

@if (ViewBag.ErrorTitle == null)
{
    <h3>
        An occurred while processing your request.
        The support team is notified and we are working on the fix
    </h3>
    <h5>Please contact us on pragim@pragimtech.com</h5>
}
else
{
    <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
    <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
}

```

```

[HttpPost]
public async Task<ActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null) {...}
    else
    {
        try
        {
            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded) {...}

            foreach (var error in result.Errors) {...}

            return View("ListRoles");
        }
        catch (DbUpdateException ex)
        {
            logger.LogError($"Exception Occured : {ex}");
            ViewBag.ErrorTitle = $"{role.Name} role is in use";
            ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted";
            return View("Error");
        }
    }
}

```

Controller Action

91 - Manage user roles in asp net core identity

Към момента ListUsers->ManageRoles само показва ролите, които заема потребителят, а бутонът за управление не работи. За да се редактират ролите, това ставаше само от ListRoles.

The screenshot shows a web browser window with the URL `localhost:24977/Administration/EditUser.cshtml`. The page title is "Edit User". The form contains the following fields:

- Id:** c4120a9-5bca-4cbc-8e3e-8c9a636d4a83
- Email:** david@abv.bg
- UserName:** david@abv.bg
- City:** (empty text box)

Below the form are two buttons: "Update" and "Cancel". Underneath is a section titled "User Roles" which displays "None at the moment" and a "Manage Roles" button.

Искаме кликването върху Manage Roles -> ManageUserRoles action, прихващайки id.

```
EditUser.cshtml*  X
<a asp-action="ManageUserRoles" asp-route-userId="@Model.Id"
    Manage Roles
```

Създаваме го

```
AdministrationController.cs*  X EditUser.cshtml*
EmployeeManagement.Controllers.AdministrationCt ManageUserR

[HttpGet]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)
{
}
}
```

Като искаме необходимият View да изглежда така:

UserRolesViewModel

 List Create Manage

Logout pragim@pragimtech.com

Manage User Roles

☐ Admin

☐ TestRole

☐ User

Update

Cancel

user:roles = 1:n
avoid duplicate
ViewBag

```
public class UserRolesViewModel
{
    public string RoleId { get; set; }
    public string UserId { get; set; }
    public string RoleName { get; set; }
    public bool IsSelected { get; set; }
}
```

```
UserRolesViewModel.cs AdministrationController.cs*
EmployeeManagement.ViewM
namespace EmployeeManagement.ViewModels
{
    0 references
    public class UserRolesViewModel
    {
        0 references
        public string RoleId { get; set; }
        0 references
        public string RoleName { get; set; }
        0 references
        public bool IsSelected { get; set; }
    }
}
```

AdministrationController.cs:

```
AdministrationController.cs AdministrationController.cs
EmployeeManagement.Controllers.AdministrationC ManageUserRoles(List<UserRo

[HttpGet]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)
{
    ViewBag.userId = userId;

    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }
}
```

```

var model = new List<UserRolesViewModel>();

foreach (var role in roleManager.Roles)
{
    var userRolesViewModel = new UserRolesViewModel
    {
        RoleId = role.Id,
        RoleName = role.Name
    };
    // populate properties Id, Name, IsSelected
    if (await userManager.IsInRoleAsync(user, role.Name))
    {
        userRolesViewModel.IsSelected = true;
    }
    else
    {
        userRolesViewModel.IsSelected = false;
    }

    model.Add(userRolesViewModel); // add to list
}
// -> View(model);

```

```

[HttpGet]
public async Task<IActionResult> ManageUserRoles(string userId)
{
    ViewBag.userId = userId;

    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    var model = new List<UserRolesViewModel>();

    foreach (var role in roleManager.Roles)
    {
        var userRolesViewModel = new UserRolesViewModel
        {
            RoleId = role.Id,
            RoleName = role.Name
        };

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRolesViewModel.IsSelected = true;
        }
        else
        {
            userRolesViewModel.IsSelected = false;
        }
    }
}

```



```

    }

    model.Add(userRolesViewModel);
}

return View(model);
}

```

//userManager.IsInRoleAsync(user, role.Name) е много важен ред, т.к. обединява user + role, като използва userManager, а не roleManager.

Views/Administration/ManageUserRoles.cshtml:

```

@model List<UserRolesViewModel>
@{
    var userId = ViewBag.userId;
}

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Manage User Roles</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Count; i++)
            {
                for each object in this list Razor generate this set of input elements
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model[i].RoleId" />
                    <input type="hidden" asp-for="@Model[i].RoleName" />
                    <input asp-for="@Model[i].IsSelected" class="form-check-input" />
                    <label class="form-check-label" asp-for="@Model[i].IsSelected">
                        @Model[i].RoleName
                    </label>
                </div>
            }
            <div asp-validation-summary="All" class="text-danger"></div>
        </div>
        <div class="card-footer">
            <input type="submit" value="Update" class="btn btn-primary" style="width:auto" />
            <a asp-action="EditUser" asp-route-id="@userId" class="btn btn-primary" style="width:auto">Cancel</a>
        </div>
    </div>
</form>

```

```

@model List<UserRolesViewModel>

@{
    var userId = ViewBag.userId;
}

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Manage User Roles</h2>

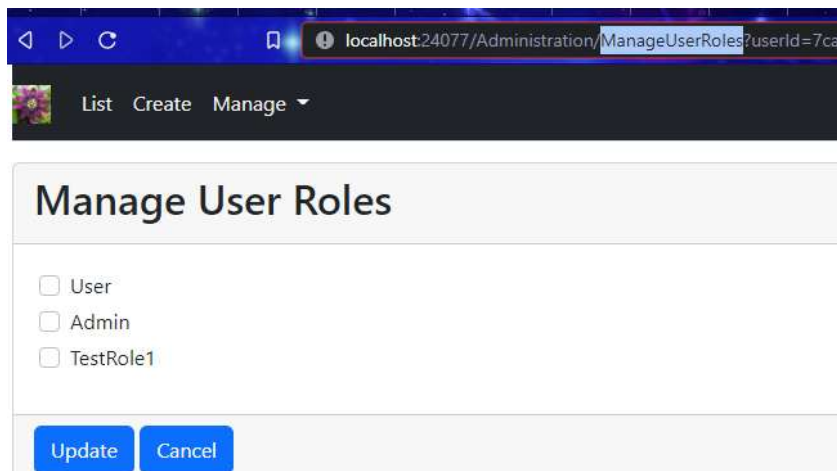
```

```

</div>
<div class="card-body">
    @for (int i = 0; i < Model.Count; i++)
    {
        <div class="form-check m-1">
            <input type="hidden" asp-for="@Model[i].RoleId" />
            <input type="hidden" asp-for="@Model[i].RoleName" />
            <input asp-for="@Model[i].IsSelected" class="form-check-input"

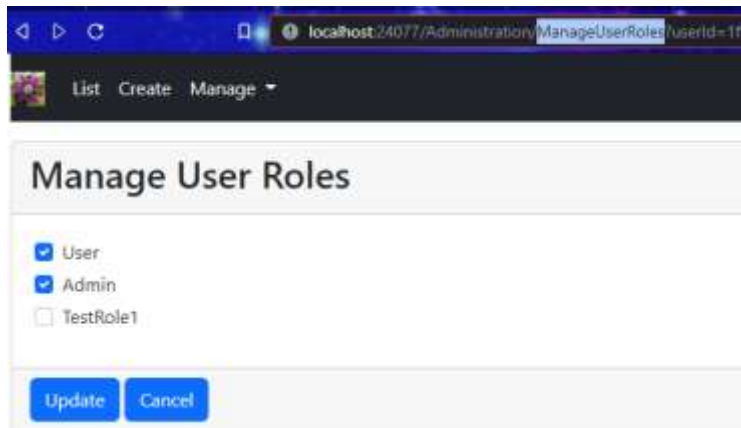
            <label class="form-check-label" asp-for="@Model[i].IsSelected">
                @Model[i].RoleName
            </label>
        </div>
    }
    <div asp-validation-summary="All" class="text-danger"></div>
</div>
<div class="card-footer">
    <input type="submit" value="Update" class="btn btn-primary"
        style="width:auto" />
    <a asp-action="EditUser" asp-route-id="@userId"
        class="btn btn-primary" style="width:auto">Cancel</a>
</div>
</div>
</form>

```



Cancel работи, но Update бутон submit form все още не, защото нямаме post метод. Имаме и 2 скрити елемента за съхранение на RoleId, RoleName. `asp-validation-summary="All"` за валидация на грешките.

Environment: Development, за да редактираме грешки при необходимост.



Update -> 404.

Update: Първо проверяваме дали може да намерим такъв потребител. Ако не – NotFoundView. Ако той съществува, искаме да се изведе списък с всички роли, които му принадлежат чрез `userManager.GetRolesAsync(user)`. После премахваме всички тези съществуващи роли с `userManager.RemoveFromRolesAsync(user, roles)`. Ако изтриването им е успешно, се добавят селектираните роли към потребителя.

```
AdministrationController.cs  X
EmployeeManagement.Controllers.AdministrationC  ManageUserRoles(List<UserRolesView

[HttpPost]
public async Task<IActionResult>
ManageUserRoles(List<UserRolesViewModel> model, string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    var roles = await userManager.GetRolesAsync(user);
    var result = await userManager.RemoveFromRolesAsync(user, roles);

    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Cannot remove user existing roles");
        return View(model);
    }

    result = await userManager.AddToRolesAsync(user,
        model.Where(x => x.IsSelected).Select(y => y.RoleName));
}
```

```

    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Cannot add selected roles to user");
        return View(model);
    }

    return RedirectToAction("EditUser", new { Id = userId });
}

```

```

[HttpPost]
public async Task<IActionResult>
ManageUserRoles(List<UserRolesViewModel> model, string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    var roles = await userManager.GetRolesAsync(user);
    var result = await userManager.RemoveFromRolesAsync(user, roles);

    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Cannot remove user existing roles");
        return View(model);
    }

    result = await userManager.AddToRolesAsync(user,
        model.Where(x => x.IsSelected).Select(y => y.RoleName));

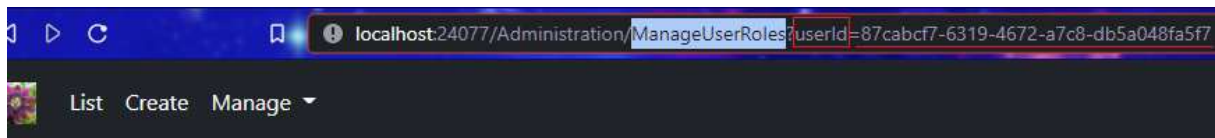
    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Cannot add selected roles to user");
        return View(model);
    }
}

```

Промените се отразяват в БД UserRoles след refresh.

92 - Model binding not working on submitting razor view with foreach loop

Защо присвояването на данни чрез URL не работи адекватно с foreach цикъл:



Manage User Roles

- ☐ User
- ☐ Admin
- ☐ TestRole1

[Update](#) [Cancel](#)

```
AdministrationController.cs  X
EmployeeManagement.Controllers.AdministrationC ManageUserRoles(List<UserRo

[HttpGet]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)
{
    ViewBag.userId = userId;

    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }
}
```

Накратко: Get приема от URL userId -> създава променлива model, която е list и попълвайки всички нейни свойства, всеки път създава нов обект в инстанция, който после добавя към list и предава на View.


```

var model = new List<UserRolesViewModel>();

foreach (var role in roleManager.Roles)
{
    var userRolesViewModel = new UserRolesViewModel
    {
        RoleId = role.Id,
        RoleName = role.Name
    };
    // populate properties Id, Name, IsSelected
    if (await userManager.IsInRoleAsync(user, role.Name))
    {
        userRolesViewModel.IsSelected = true;
    }
    else
    {
        userRolesViewModel.IsSelected = false;
    }

    model.Add(userRolesViewModel); // add to list
}
// -> View(model);

```

Очаквано, моделът, който подаваме на View е list, като най-важното е, че цикъл for за всеки обект от списъка кара razor да генерира тези групи елементи (2 скрити, един чек-бокс и 1 лейбъл за името), като submit btn Update потвърждава и изпраща формата до post.

```
controller.cs ManageUserRoles.cshtml X
@model List<UserRolesViewModel>
@{
    var userId = ViewBag.userId;

    <form method="post">
        <div class="card">
            <div class="card-header">
                <h2>Manage User Roles</h2>
            </div>
            <div class="card-body">
                @for (int i = 0; i < Model.Count; i++)
                { for each object in this list Razor generate this set of input elements
                    <div class="form-check m-1">
                        <input type="hidden" asp-for="@Model[i].RoleId" />
                        <input type="hidden" asp-for="@Model[i].RoleName" />
                        <input asp-for="@Model[i].IsSelected" class="form-check-input" />
                        <label class="form-check-label" asp-for="@Model[i].IsSelected">
                            @Model[i].RoleName
                        </label>
                    </div>
                }
                <div asp-validation-summary="All" class="text-danger"></div>
            </div>
            <div class="card-footer">
                <input type="submit" value="Update" class="btn btn-primary" style="width:auto" />
                <a asp-action="EditUser" asp-route-id="@userId" class="btn btn-primary" style="width:auto">Cancel</a>
            </div>
        </div>
    </form>
}
```

Post методът автоматично ще присвои на model тип list 3-те input елемента (2 hidden + 1 check-box isSelected)

```
AdministrationController.cs
EmployeeManagement.Controllers.AdministrationC
ManageUserRoles(List<UserRole

[HttpPost]
public async Task<IActionResult>
ManageUserRoles(List<UserRolesViewModel> model, string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    var roles = await userManager.GetRolesAsync(user);
    var result = await userManager.RemoveFromRolesAsync(user, roles);

    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Cannot remove user existing roles");
        return View(model);
    }

    result = await userManager.AddToRolesAsync(user,
        model.Where(x => x.IsSelected).Select(y => y.RoleName));
}
```

```
if (!result.Succeeded)
{
    ModelState.AddModelError("", "Cannot add selected roles to user");
    return View(model);
}

return RedirectToAction("EditUser", new { Id = userId });
}
```

За да разберем това автоматично присвояване на 3-те input -> Post, поставяме Breakpoint:

```
ManageUserRoles.cshtml*
AdministrationController.cs*
EmployeeManagement
171 [HttpPost]
172 public async Task<IActionResult>
173 ManageUserRoles(List<UserRolesViewModel> model, string userId)
174 {
175     var user = await userManager.FindByIdAsync(userId);
176
```

Debug ->

localhost:24077/Administration/ManageUserRoles?userId=1f9ea9da-40d8-4413-a222-9fd26303740c

List Create Manage

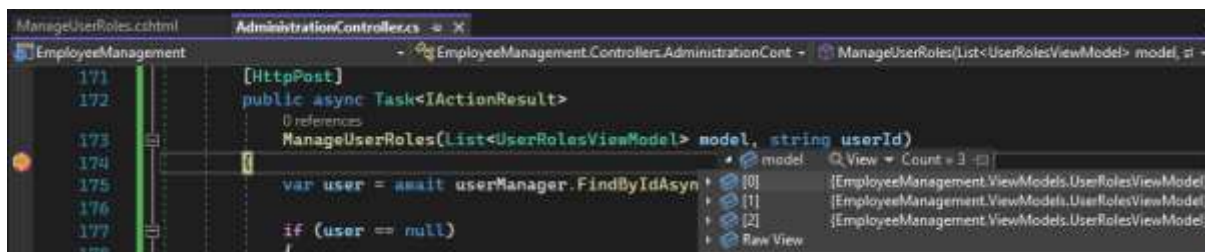
Manage User Roles

- ☒ User
- ☒ Admin
- ☒ TestRole1

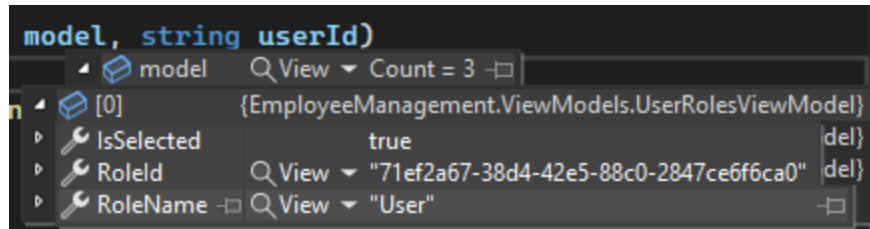
Update Cancel

```
<div class="card-header">
  <h2>Manage User Roles</h2>
</div>
<div class="card-body">
  <div class="form-check m-1">
    <input type="hidden" id="z0__RoleId" name="[0].RoleId" value="71ef2a67-38d4-42e5-88c0-2847ce6f6ca0" />
    <input type="hidden" id="z0__RoleName" name="[0].RoleName" value="User" />
    <input class="form-check-input" type="checkbox" data-val="true" data-val-required="The IsSelected field
    <label class="form-check-label" for="z0__IsSelected">
      User
    </label>
  </div>
  <div class="form-check m-1">
    <input type="hidden" id="z1__RoleId" name="[1].RoleId" value="7a53c902-d9f9-4026-b521-26b8485f1611" />
    <input type="hidden" id="z1__RoleName" name="[1].RoleName" value="Admin" />
    <input class="form-check-input" type="checkbox" checked="" data-val="true" data-val-required="The
    <label class="form-check-label" for="z1__IsSelected">
      Admin
    </label>
  </div>
  <div class="form-check m-1">
    <input type="hidden" id="z2__RoleId" name="[2].RoleId" value="e51deba4-def5-46fa-9acd-b9a75e3a1259" />
    <input type="hidden" id="z2__RoleName" name="[2].RoleName" value="TestRole1" />
    <input class="form-check-input" type="checkbox" checked="" data-val="true" data-val-required="The
    <label class="form-check-label" for="z2__IsSelected">
```

Update -> BreakPoint, която вече е hit с жълта стрелка и ако селектираме model, той вече е възприел 3 елемента:



и ако изберем всеки от тях, той съдържа попълнени параметри, автоматично прихванати от Input таговете във формата. Това е благодарение на model-binding MVC.



Continue -> само че при мен не позволява да продължа без редакция на кода, за разлика от по-старата версия. Явно трябва да видя в настройките.

Спираме Debug и вместо for, нека видим при foreach:

```
@for (int i = 0; i < Model.Count; i++)
```

```
@for (int i = 0; i < Model.Count; i++)
```

```
@foreach (var role in Model )
```

```
@foreach (var role in Model )
```

Debug -> View source -> резултатът е потресаващ: независимо кой label с име или check-box се селектира, само първият елемент се маркира (в моя браузър не е така), но при ViewSource:

Стара версия (дублиране на hidden id = name, checkbox id = for)



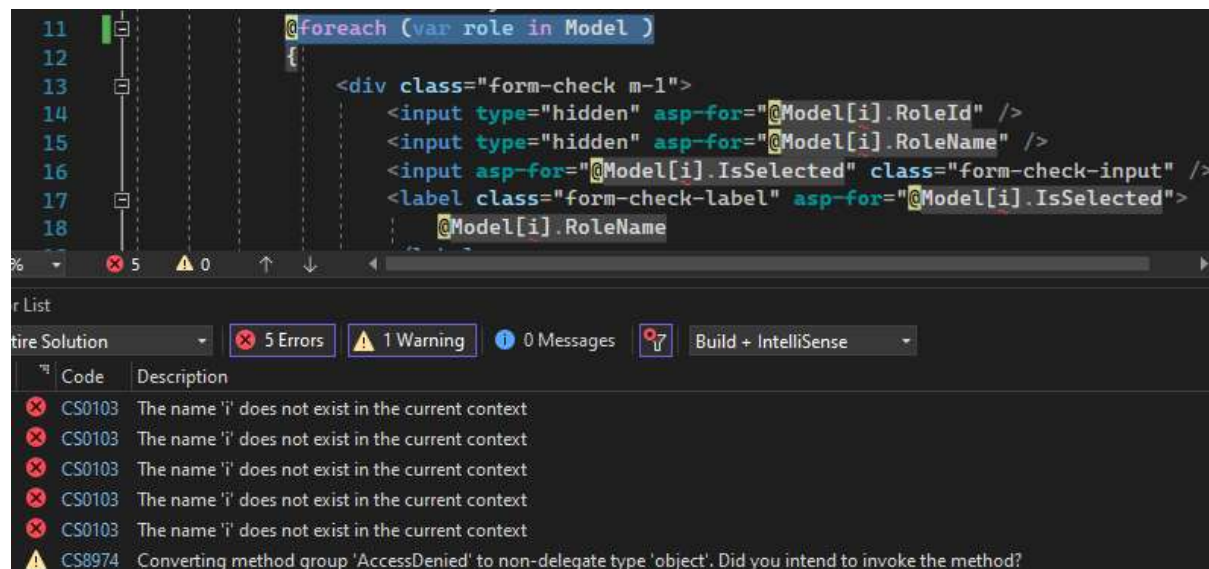
Нова (дублиране само на checkbox id = for):


```

<div class="form-check m-1">
  <input type="hidden" id="i0_RoleId" name="[0].RoleId" value="71ef2a67-36d4-42e5-b8c0-2847ce6f8ca0" />
  <input type="hidden" id="i0_RoleName" name="[0].RoleName" value="User" />
  <input class="form-check-input" type="checkbox" checked="checked" data-val="true" data-val-required="The IsSelected field is required." id="i0_IsSelected" name="i0_IsSelected" />
  <label class="form-check-label" for="i0_IsSelected">
    User
  </label>
</div>
<div class="form-check m-1">
  <input type="hidden" id="i1_RoleId" name="[1].RoleId" value="7a55c982-d9f9-4828-b521-2688485f1611" />
  <input type="hidden" id="i1_RoleName" name="[1].RoleName" value="Admin" />
  <input class="form-check-input" type="checkbox" checked="checked" data-val="true" data-val-required="The IsSelected field is required." id="i1_IsSelected" name="i1_IsSelected" />
  <label class="form-check-label" for="i1_IsSelected">
    Admin
  </label>
</div>
<div class="form-check m-1">
  <input type="hidden" id="i2_RoleId" name="[2].RoleId" value="a51deba4-def5-48fa-9acd-b9a75e3a1259" />
  <input type="hidden" id="i2_RoleName" name="[2].RoleName" value="TextRole1" />
  <input class="form-check-input" type="checkbox" data-val="true" data-val-required="The IsSelected field is required." id="i2_IsSelected" name="i2_IsSelected" />
  <label class="form-check-label" for="i2_IsSelected">
    TextRole1
  </label>
</div>

```

Виждаме, че има проблем: присвоените стойности на id = for и за трите елемента се дублират, което се отчита с множество грешки и червени подчертавания на i. По същият начин name на първия и втория hidden input съвпадат. Поради всичко това, model binder of MVC не знае кои от тези входни параметри къде да присвои.



Затова, когато направим промяна, натиснем Update, BreakPoint is hit, посочим model, в старите версии няма присвояване, а model count = 0. При мен обаче, промените са отразени, което е много добре. Явно затова ми е позволявало маркиране на всички check-box, не само първия.

Model Binding Not Working

- With a FOR loop, the generated input element names are unique and contains an integer indexer in the name
- So the model binder knows it has to map the values from these input elements to the list parameter on the controller action
- With a FOREACH loop the names are not unique and hence the model binder does not know it has to map the values to the list parameter

Възстановявам for.

93 - Manage user claims

API (**A**pplication **P**rogramming **I**nterface) is a set of commands, which interfaces the programs with the processors.

Claim – a name value pair which can be used for making access control decisions. For example, we want to allow a logged-in user to be able to edit employee details, only if he has "Edit Employee" claim. We can use claims to make authorization checks like this. Since we are using claims to make authorization checks, this is called **claims-based authorization**.

Manage User Claims

 List Create Manage * Logout pragim@pragimtech.com

Edit User

Id

7e35ba66-f855-4c61-ac1b-71c57af5c631

Email

pragim@pragimtech.com

UserName

pragim@pragimtech.com

City

London

Update

Cancel

User Claims

None at the moment

Manage Claims

```
<a asp-action="ManageUserClaims"
  asp-route-userId="@Model.Id"
  class="btn btn-primary">
  Manage Claims
</a>
```

```
EditUser.cshtml ManageUserRoles.cshtml
<div class="card-footer">
  <a asp-action="ManageUserClaims" asp-route-userId="@Model.Id"
    style="width:auto" class="btn btn-primary">
    Manage Claims
  </a>
```

Този код е достатъчен при натискане на бутон Manage Claims да бъдем пренасочени към URL

localhost:24077/Administration/ManageUserClaims?userId=87cabcf7-6319-4672-a7c8-db5a048fa5f7

пренасяйки userId чрез него, който засега връща 404, затова, нека го създадем:

```
AdministrationController.cs* X ManageUserRoles.cshtml
ent EmployeeManagement.Controllers.AdministrationCor ManageUserClaims(string L

[HttpGet]
0 references
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    return View(user);
}
```

Сложихме кода, ако не е намерен потребител с такова ID. За да продължим, трябва да извлечем списък с всички claims за нашето приложение. Те могат да дойдат от:

- * таблица БД;
- * конфигурационен файл;
- * статичен клас.

Нека опитаме в статичен клас Models folder -> Add new class -> ClaimsStore.cs

```
ClaimsStore.cs* X AdministrationController.cs* ManageUserRoles.cshtml
EmployeeManagement.Models.ClaimsStore

using System.Collections.Generic;

namespace EmployeeManagement.Models
{
    0 references
    public static class ClaimsStore
    {
        public static List<Claim>

        using System.Security.Claims;
        The name 'Claim' does not exist in the current context.
        using System.Collections.Generic;
        using System.Security.Claims;
    }
}
```

Изпуснала съм интервал на Create Role

```
property
public static List<Claim> AllClaims = new List<Claim>()
{
    new Claim("CreateRole", "CreateRole")
}

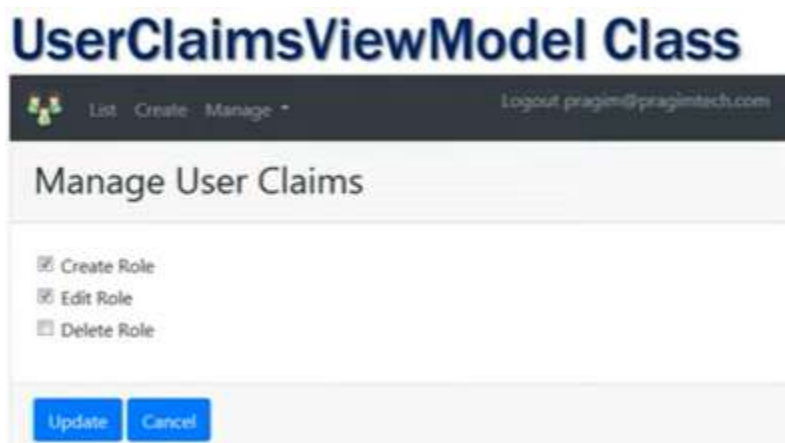
▲ 2 of 7 ▼ Claim(string type, string value)
overloaded Initializes a new instance of the Claim class with the specified claim type, and value.
version value: The claim value.
```

```

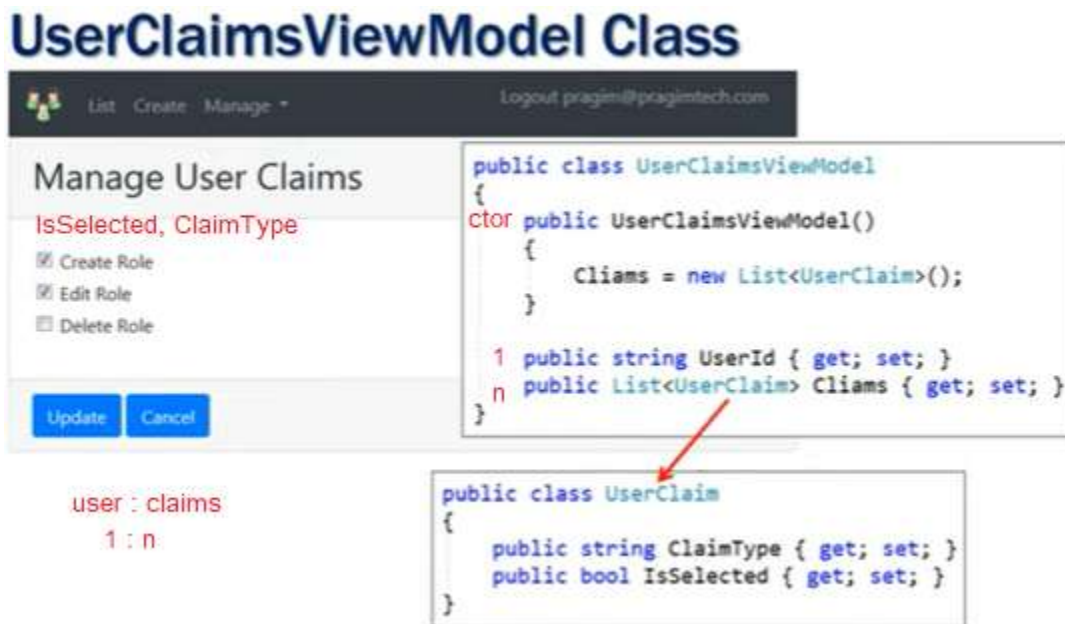
using System.Collections.Generic;
using System.Security.Claims;

namespace EmployeeManagement.Models
{
    public static class ClaimsStore
    {
        public static List<Claim> AllClaims = new List<Claim>()
        {
            new Claim("Create Role", "Create Role"),
            new Claim("Edit Role", "Edit Role"),
            new Claim("Delete Role", "Delete Role")
        };
    }
}

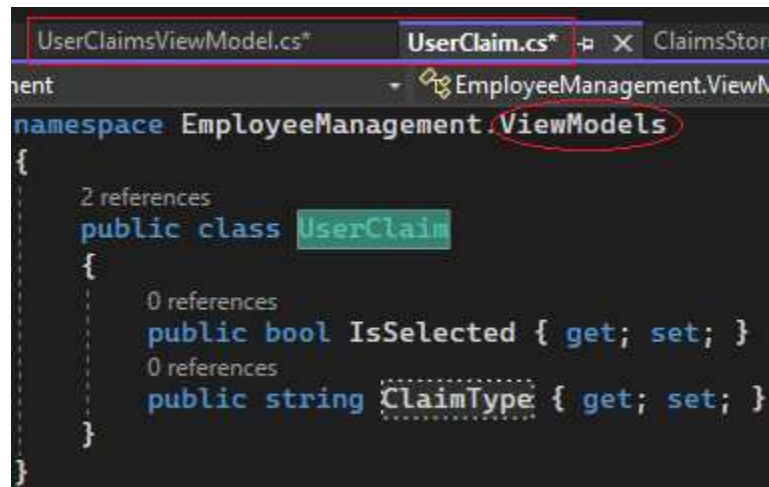
```



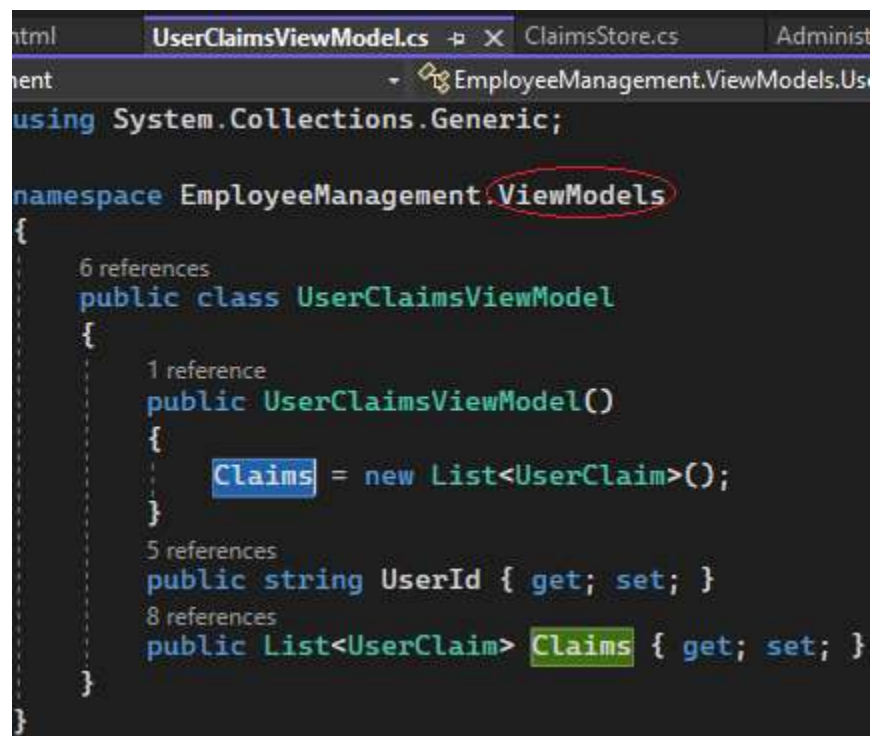
Извеждаме списъка с всички Claims, като използвайки foreach loop минаваме през всички и ако дадено право се повтаря, маркираме check-box-a му, ако не, оставяме го празно.



Използваме допълнителния конструктор UserClaimsViewModel, въпреки, че може без него (директно с UserId + Claims), за да инициализираме първоначални данни за Claims, избягвайки null reference exceptions.



```
namespace EmployeeManagement.ViewModels
{
    public class UserClaim
    {
        public bool IsSelected { get; set; }
        public string ClaimType { get; set; }
    }
}
```



```
using System.Collections.Generic;
```

```

namespace EmployeeManagement.ViewModels
{
    public class UserClaimsViewModel
    {
        public UserClaimsViewModel()
        {
            Claims = new List<UserClaim>();
        }
        public string UserId { get; set; }
        public List<UserClaim> Claims { get; set; }
    }
}

```

The screenshot shows the `AdministrationController.cs` file in Visual Studio. The `ManageUserClaims` method is highlighted, and the `GetClaimsAsync` call is circled in red. The following table summarizes the annotations in the code:

Annotation	Target Code
0 references	<code>public async Task<IActionResult> ManageUserClaims(string userId)</code>
list of all claims	<code>var existingUserClaims = await userManager.GetClaimsAsync(user);</code>
to store	<code>var existingUserClaims = await userManager.GetClaimsAsync(user);</code>
instance of ViewModel class	<code>var model = new UserClaimsViewModel</code>
populate property	<code>{</code>
method	<code>GetClaimsAsync(user);</code>

```

[HttpGet]
0 references
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }
    var existingUserClaims = await userManager.GetClaimsAsync(user);
    var model = new UserClaimsViewModel
    {
        UserId = userId
    };
}

```

```

foreach (Claim claim in ClaimsStore.AllClaims)
{
    UserClaim userClaim = new UserClaim
    {
        ClaimType = claim.Type
    };

    // If the user has the claim, set IsSelected property to true, so the checkbox
    // next to the claim is checked on the UI
    if (existingUserClaims.Any(c => c.Type == claim.Type))
    {
        // var storing all claims
        // if the type of the existing claim matches with
        // the type of the claim we are iterating over
        // => true = user has that claim
        userClaim.IsSelected = true;
    }

    model.Claims.Add(userClaim); // otherwise (default) => false
    // add userClaim object to Claims collection property
    // of view model object
}

return View(model); // display it

```

```

[HttpGet]
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }
    var existingUserClaims = await userManager.GetClaimsAsync(user);

    var model = new UserClaimsViewModel
    {
        UserId = userId
    };
    foreach (Claim claim in ClaimsStore.AllClaims)
    {
        UserClaim userClaim = new UserClaim
        {
            ClaimType = claim.Type
        };

        // If the user has the claim, set IsSelected property to true,
        // so the checkbox next to the claim is checked on the UI
        if (existingUserClaims.Any(c => c.Type == claim.Type))
        {
            userClaim.IsSelected = true;
        }

        model.Claims.Add(userClaim);
    }

    return View(model);
}

```

Administration folder -> Add -> Razor view: ManageUserClaims.cshtml

```

Model.cs*  ManageUserClaims.cshtml  ClaimsStore.cs  AdministrationController.cs*  UserClaim.cs*
@model UserClaimsViewModel

<form method="post">
  <div class="card">
    <div class="card-header">
      <h2>Manage User Claims</h2>
    </div>
    <div class="card-body">
      @for (int i = 0; i < Model.Claims.Count; i++)
      {
        <div class="form-check m-1">
          <input type="hidden" asp-for="@Model.Claims[i].ClaimType" />
          <input asp-for="@Model.Claims[i].IsSelected" class="form-check-input" />
          <label class="form-check-label" asp-for="@Model.Claims[i].IsSelected">
            @Model.Claims[i].ClaimType
          </label>
        </div>
      }
      <div asp-validation-summary="All" class="text-danger"></div>
    </div>
    <div class="card-footer">
      <input type="submit" value="Update" class="btn btn-primary"
        style="width:auto" />
      <a asp-action="EditUser" asp-route-id="@Model.UserId"
        class="btn btn-primary" style="width:auto">Cancel</a>
    </div>
  </div>
</form>

```

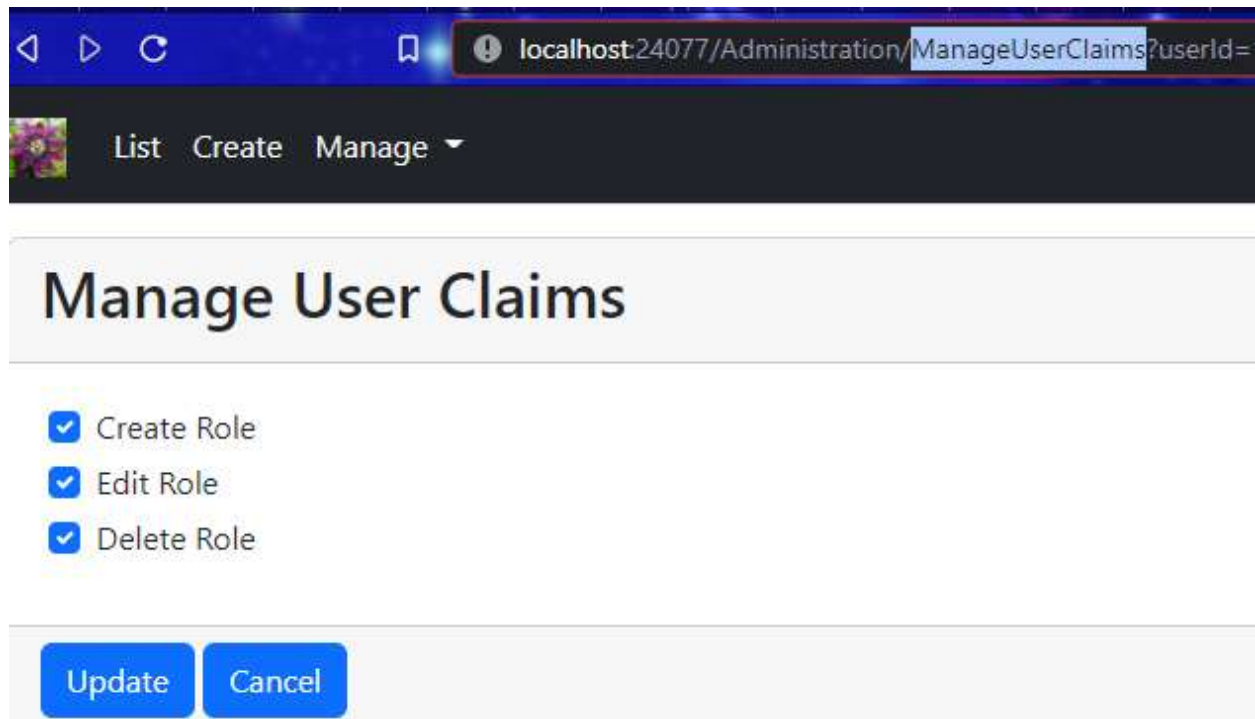
@model UserClaimsViewModel

```

<form method="post">
  <div class="card">
    <div class="card-header">
      <h2>Manage User Claims</h2>
    </div>
    <div class="card-body">
      @for (int i = 0; i < Model.Claims.Count; i++)
      {
        <div class="form-check m-1">
          <input type="hidden" asp-for="@Model.Claims[i].ClaimType" />
          <input asp-for="@Model.Claims[i].IsSelected" class="form-check-
input" />
          <label class="form-check-label" asp-
for="@Model.Claims[i].IsSelected">
            @Model.Claims[i].ClaimType
          </label>
        </div>
      }
      <div asp-validation-summary="All" class="text-danger"></div>
    </div>
    <div class="card-footer">
      <input type="submit" value="Update" class="btn btn-primary"
        style="width:auto" />
      <a asp-action="EditUser" asp-route-id="@Model.UserId"
        class="btn btn-primary" style="width:auto">Cancel</a>
    </div>
  </div>
</form>

```

</form>



The screenshot shows a web browser window with the address bar displaying `localhost:24077/Administration/ManageUserClaims?userId=`. Below the address bar is a navigation bar with a small icon and the text "List Create Manage". The main content area has a heading "Manage User Claims". Below the heading are three checkboxes, each with a blue checkmark: "Create Role", "Edit Role", and "Delete Role". At the bottom of the form are two blue buttons: "Update" and "Cancel".

94 - Claims based authorization

Искаме даден потребител да има право на определено действие, само ако изрично е указано, че притежава това право.

Claims Based Authorization

Create Role

Edit Role

Delete Role

Claims are policy based. We create a policy and include one or more claims in that policy. We then need to register the policy. Creating and registering a claims policy is typically done in one step in `ConfigureServices()` method of the `Startup` class.

Implementing Claims Based Authorization

Step 1 : Create Claims Policy

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));
});
```

Step 2 : Use the Policy for Authorization Checks

```
[HttpPost]
[Authorize(Policy = "DeleteRolePolicy")]
public async Task<ActionResult> DeleteRole(string id)
{
    // Delete Role
}
```

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));
});
```

- The `options` parameter type is `AuthorizationOptions`
- Use `AddPolicy()` method to create the policy
- The first parameter is the `name` of the policy and the second parameter is the policy itself
- To satisfy this policy requirements, the logged-in user must have `Delete Role` claim

Using Claims Policy for Authorization Checks

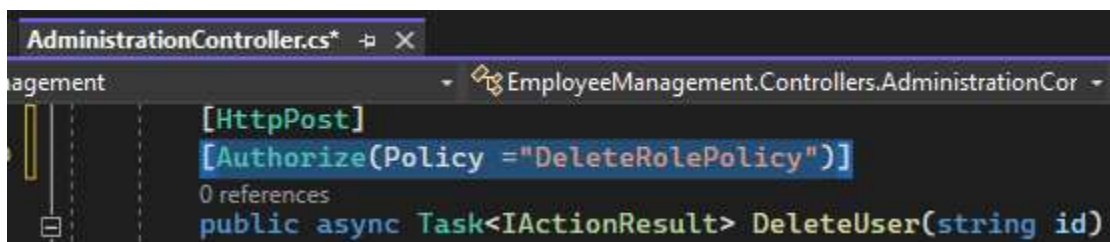
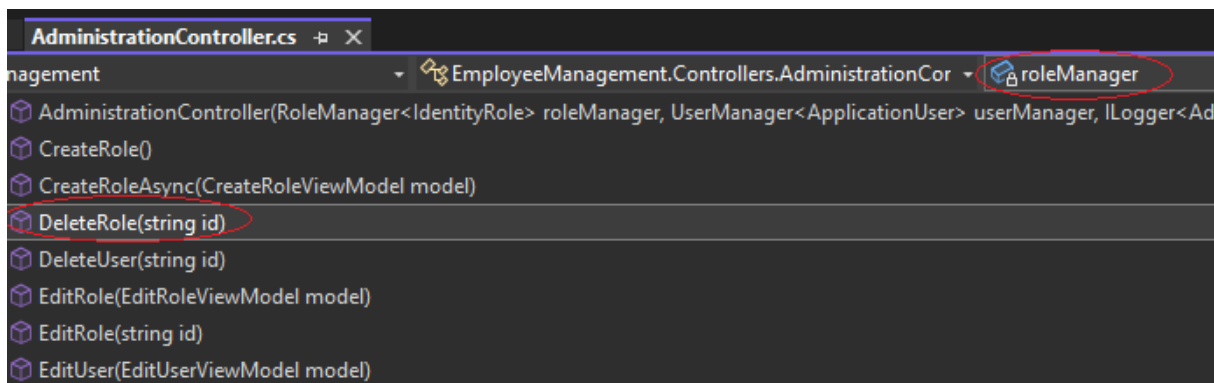
The policy can then be used on a controller or a controller action.

`[Authorize(Roles = " ")]`, аналогично

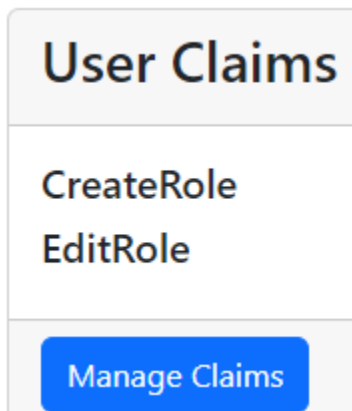
`[Authorize(Policy = " ")]`

`[Authorize(Policy = "DeleteRolePolicy")]`

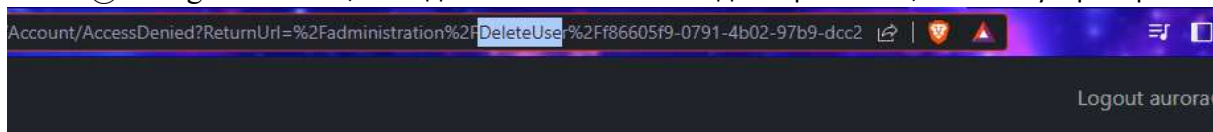
Интересното тук е падащото меню за бързо търсене на action в контролера:



Така по подразбиране нямаме DeleteRole.



Jason@abv.bg = TestRole1, ако админ Amidala се опита да изтрие Jason, тя не е авторизирана.



Access Denied

You do not have permission to view this resource



User Claims

None at the moment

Manage Claims

Manage User Claims

- ☒ CreateRole
- ☒ EditRole
- ☒ DeleteRole

Update

Cancel

TestRole1

Edit

Are you sure you want to delete?

Yes

No

Всеки подобен ъпдейт влиза в сила след отписване и повторно вписване на потребител.

Целият AdministrationController изисква Admin role, а отделно вътре в него Delete action изисква DeleteRolePolicy, което означава, че за да бъде изтрит потребител, той трябва да бъде едновременно Admin и да има изрично право да трие. Ако искаме да дадем допълнително изискване за политика, просто присъединяваме `.RequireClaim("Create Role")`; което означава, че само потребител, който има и двете права Delete + Create ще бъде допуснат до DeleteRole.

Adding Multiple Claims to Policy

```
services.AddAuthorization(options =>
{
    name
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role")
            .RequireClaim("Create Role")
    );
});
```

95 - Role based vs claims based authorization

Role v/s Claims Based Authorization

CONTROL

Role Based Authorization	RBAC
<pre>[Authorize(Roles = "Admin")] public class AdministrationController : Controller { }</pre>	Part 82

Claims Based Authorization	CBAC
<pre>[Authorize(Policy = "DeleteRolePolicy")] public async Task<IActionResult> DeleteRole(string id) { }</pre>	Part 94

User Authorization

Authorization is the process of determining what the logged-in user can and cannot do. Based on our application authorization requirements we could use Roles, Claims or a combination of both.

Role Based Access Control (RBAC)

In your organisation you may have the following roles

- Employee
- Manager

- HR

Depending on the role or roles the logged-in user belong to, you may or may not authorize access to certain resources in the application. Since we are using **Roles** to make authorization checks, this is commonly called **Role Based Access Control** (RBAC) or **Role Based Authorization**.

In ASP.NET Core to implement role based authorization we use **Authorize** attribute with **Roles** parameter.

```
[Authorize(Roles = "Admin")]
public class AdministrationController : Controller
{
}
```

We discussed role based authorization in detail in **Part 82** of **ASP.NET core tutorial**

Claims Based Access Control (CBAC)

Claims Based Authorization is also called Claims Based Access Control. Before we understand Claims Based Authorization, let's understand **what is a Claim**.

A claim is a name-value pair. It's really a piece of information about the user, not what the user can and cannot do. For example username, email, age, gender etc are all claims. How you use these claims for authorization checks in your application is up to your application business and authorization requirements.

For example, if you are building an **employee portal** you may allow the logged-in user to apply for a maternity leave if the **gender** claim value is **female**. Similarly, if you are building an **ecommerce application**, you may allow the logged-in user to submit an order if the **age** claim value is greater than or equal to **18**.

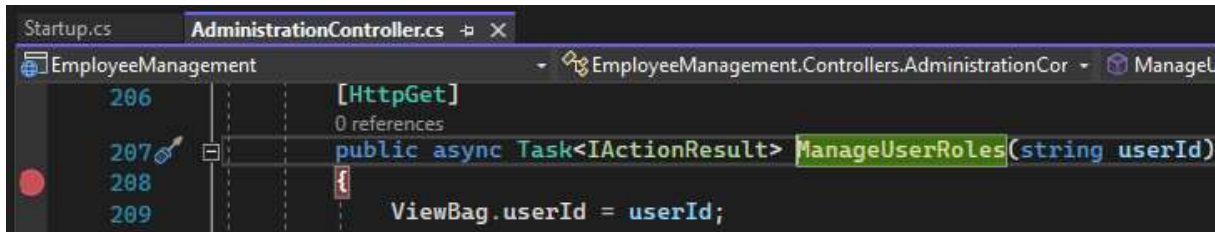
Claims are policy based. We create a policy and include one or more claims in that policy. The policy is then used along with the **policy** parameter of the **Authorize** attribute to implement claims based authorization.

```
[Authorize(Policy = "DeleteRolePolicy")]
public async Task<IActionResult> DeleteRole(string id)
{
}
```

We discussed claims based authorization in detail in **Part 94** of **ASP.NET core tutorial**

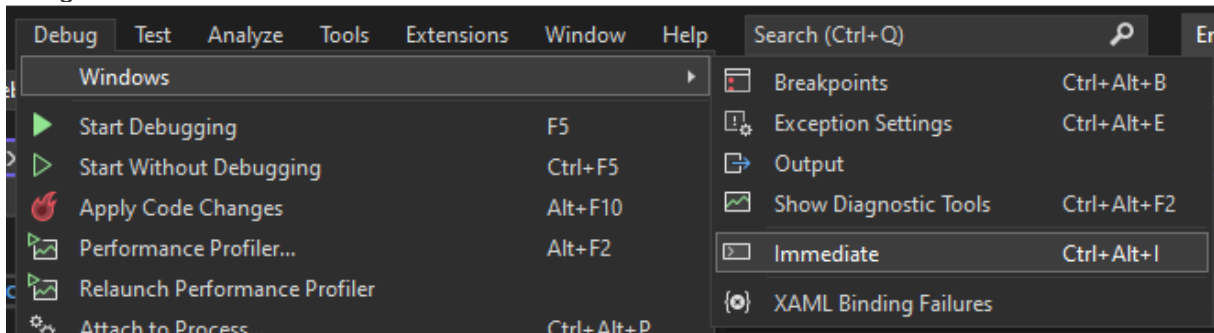
In ASP.NET Core, a role is just a claim with type Role.

In ASP.NET Core, a Role is a Claim with Type Role

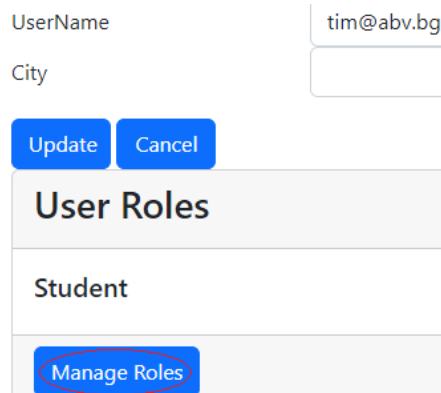


```
Startup.cs AdministrationController.cs
EmployeeManagement
[HttpGet]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)
{
    ViewBag.userId = userId;
}
```

To confirm this, execute the following expression in the **Immediate window** in Visual Studio:
Debug -> Windows -> Immediate.



Debug:



UserName tim@abv.bg

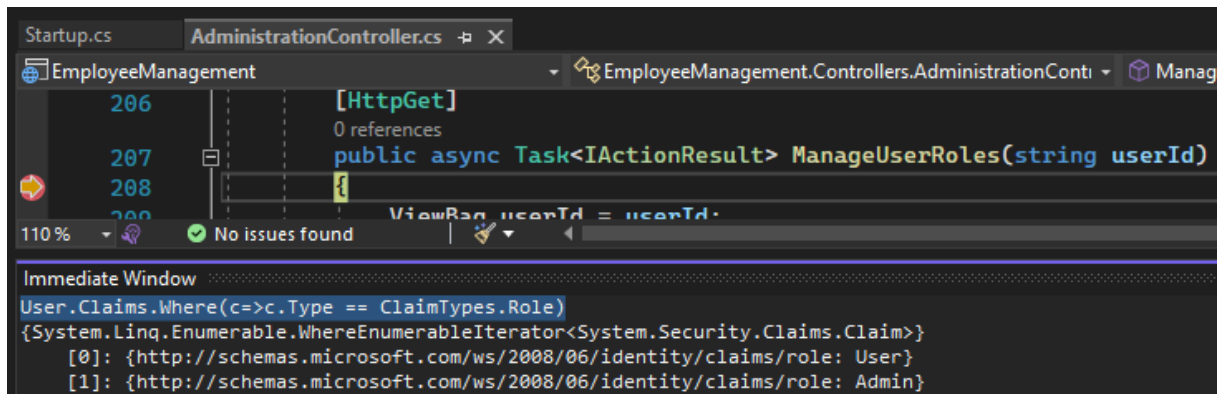
City

Update Cancel

User Roles

Student

Manage Roles



All the logged-in user roles will be listed. You will also see these roles are just [claims of type role](#).

```
User.Claims.Where(c=> c.Type == ClaimTypes.Role).ToList()
```

Има нова role Student, но не ми я извежда.

Roles Policy

We know claims are policy based. Since, a [role is also a claim of type role](#), we can also use a role with the new policy syntax.

With claims, we create a policy and include one or more claims in that policy. We can do the same thing with roles as well. Create a policy and include one or more roles in that policy.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));
    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});
```

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthorization(options =>
    {
        options.AddPolicy("AdminRolePolicy", policy => policy.RequireRole("Admin"));
    });
}
```

If you want to include multiple roles in the policy simply separate them with a comma

```
options.AddPolicy("SuperAdminPolicy", policy =>
    policy.RequireRole("Admin", "User", "Manager"));
```

The policy can then be used on a controller or a controller action.

```
[HttpPost]
[Authorize(Policy = "SuperAdminPolicy")]
public async Task<ActionResult> DeleteRole(string id)
{
    // Delete Role
}
```

Why do we have both in ASP.NET Core

In previous versions of asp.net we did not have claims based authorization, only role based authorization.

Claims based authorization is relatively new and is the recommended approach. With it we can also use claims from external identity providers like Facebook, Google, Twitter etc. We will discuss using external identity providers and the claims they provide in our upcoming videos.

Role based authorization is still supported in asp.net core for backward compatibility. While Claims based authorization is the recommended approach, depending on your application authorization requirements you may use role based authorization, claims based authorization or a combination of both.

Role v/s Claims Based Authorization

```
public void ConfigureServices(IServiceCollection services)
{
    // Claims Policy
    services.AddAuthorization(options =>
    {
        options.AddPolicy("DeleteRolePolicy",
            policy => policy.RequireClaim("Delete Role"));
    });

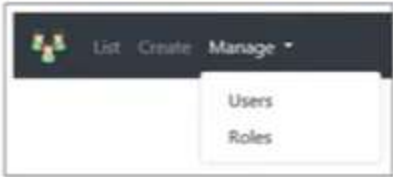
    // Roles Policy
    services.AddAuthorization(options =>
    {
        options.AddPolicy("AdminRolePolicy",
            policy => policy.RequireRole("Admin"));
    });
}
```

```
// To include multiple roles in the policy
options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin", "User", "Manager"));
```

96 - Authorization in views

Много удобна техника, когато искаме да показваме/скриваме UI елементи в зависимост от достъпа на потребителите. Например меню Manage е видимо само за админ:

Role Based Authorization Check in Views



```
@using Microsoft.AspNetCore.Identity
@inject SignInManager<ApplicationUser> SignInManager
@if (SignInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle">
            Manage
        </a>
    </li>
}
```

Part 83

Искаме бутон Edit да бъде видим и достъпен само за админи, а като самостоятелно упражнение и Delete:

All Roles

Add new role

Role Id : 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

User

Edit

Delete

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy",
        policy => policy.RequireClaim("Edit Role"));

    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});
```

За целта, първо трябва да вмъкнем необходимите

```
AdministrationController.cs  ListRoles.cshtml
@model IEnumerable<IdentityRole>

@using Microsoft.AspNetCore.Authorization
@inject IAuthorizationService authorizationService;
```

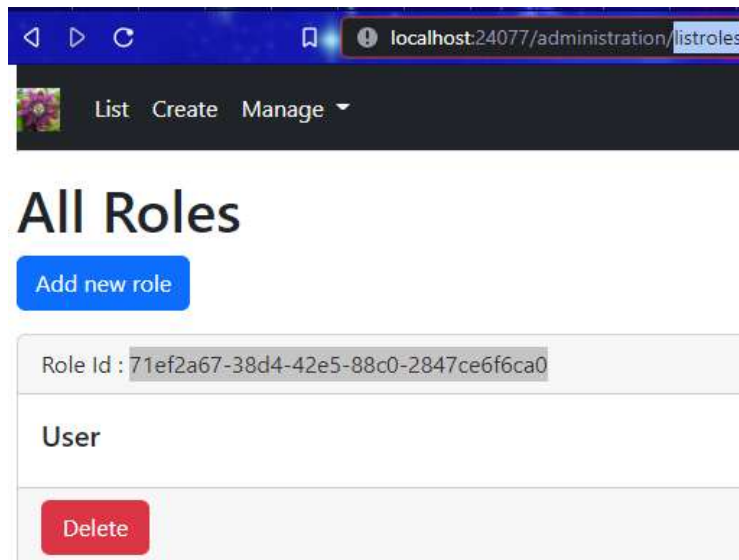
```
@using Microsoft.AspNetCore.Authorization
@inject IAuthorizationService authorizationService;
```

Ограждаме Edit бутон с if блок:

```
@if ((await authorizationService.AuthorizeAsync(User, "EditRolePolicy")).Succeeded)
{
    <a asp-controller="Administration" asp-action="EditRole"
      asp-route-id="@role.Id" class="btn btn-primary">
        Edit
    </a>
}
```

```
@if ((await authorizationService.AuthorizeAsync(User, "EditRolePolicy")).Succeeded)
{
    <a asp-controller="Administration" asp-action="EditRole"
      asp-route-id="@role.Id" class="btn btn-primary">
        Edit
    </a>
}
```

Резултат:



Потребителят няма видимост към Edit бутона, но може да получи достъп чрез адресиране:

localhost:24077/Administration/EditRole/71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

List Create Manage ▾

Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

Update Cancel

Users in this role

aurora@abv.bg
david@abv.bg

Add or remove users

Като промяната user->user1 се приема от системата, т.е. скриването на елементи не е достатъчно.

Role Id : 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

User1

Delete

Трябва да защитим и съответните Controller actions.

```
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> EditRole(string id)
```

```
[HttpPost]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> EditRole(EditRoleViewModel model)
```

При тази ситуация, ако се опитаме директно от URL да въведем редакция на потребителя, достъпът се отказва:

Account/AccessDenied?ReturnUrl=%2FAdministration%2FEditRole%2F71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

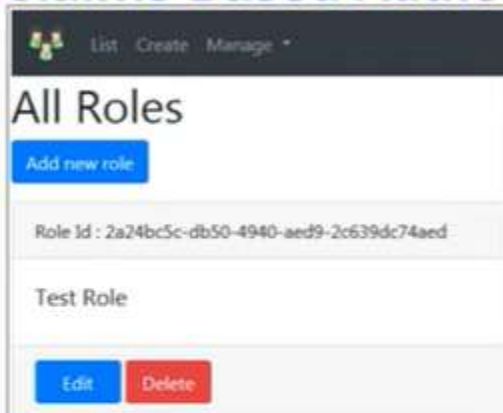
Access Denied

You do not have permission to view this resource



If you need [IAuthorizationService](#) in multiple views, consider importing in `_ViewImports.cshtml`, so you do not have to import it in every individual view.

Claims Based Authorization Check in Views



Part 31

`_ViewImports`

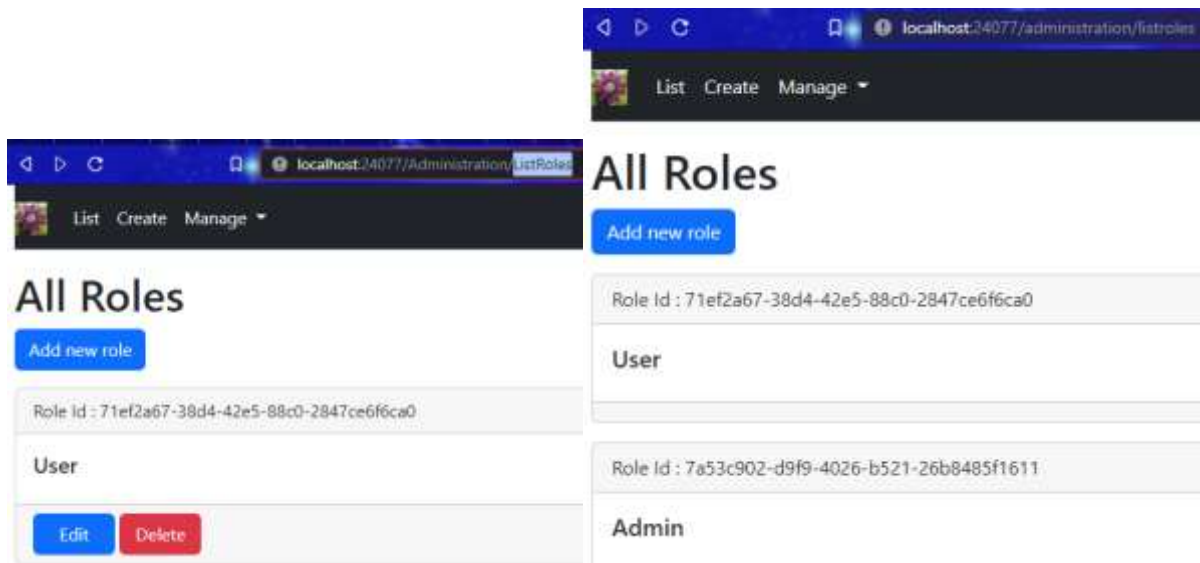
```
@model IEnumerable<IdentityRole>

@using Microsoft.AspNetCore.Authorization
@inject IAuthorizationService authorizationService;

@if ((await authorizationService.AuthorizeAsync(User, "EditRolePolicy")).Succeeded)
{
    <a asp-controller="Administration" asp-action="EditRole"
      asp-route-id="@role.Id" class="btn btn-primary">
        Edit
    </a>
}
}
```

Упражнение: оставям само админ Aurora да има 3-те Claims с достъп до edit + delete Roles.

Bob = admin, Create + Edit Claims.



Скривам бутон Delete за всеки без достъп до Delete Roles Policy:

```
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
[Authorize(Policy = "DeleteRolePolicy")]
0 references
public async Task<IActionResult> EditRole(string id)
```

```
[HttpPost]
[Authorize(Policy = "EditRolePolicy")]
[Authorize(Policy = "DeleteRolePolicy")]
```

```
ListRoles.cshtml
<div class="card-footer">
    @if ((await authorizationService.AuthorizeAsync(User, "EditRolePolicy", "DeleteRolePolicy")).Succeeded)
    {
        <form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">

            <a asp-controller="Administration" asp-action="EditRole"
              asp-route-id="@role.Id" class="btn btn-primary">
                Edit
            </a>

            <span id="confirmDeleteSpan_@role.Id" style="display:none">
                <span>Are you sure you want to delete?</span>
                <button type="submit" class="btn btn-danger">Yes</button>
                <a href="#" class="btn btn-primary"
                  onclick="confirmDelete('@role.Id', false)">No</a>
            </span>

            <span id="deleteSpan_@role.Id">
                <a href="#" class="btn btn-danger"
                  onclick="confirmDelete('@role.Id', true)">Delete</a>
            </span>
        </form>
    }
</div>
```

```
<div class="card-footer">
    @if ((await authorizationService.AuthorizeAsync(User,
"EditRolePolicy", "DeleteRolePolicy")).Succeeded)
    {
        <form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">

            <a asp-controller="Administration" asp-action="EditRole"
              asp-route-id="@role.Id" class="btn btn-primary">
                Edit
            </a>

            <span id="confirmDeleteSpan_@role.Id" style="display:none">
                <span>Are you sure you want to delete?</span>
                <button type="submit" class="btn btn-danger">Yes</button>
                <a href="#" class="btn btn-primary"
                  onclick="confirmDelete('@role.Id', false)">No</a>
            </span>

            <span id="deleteSpan_@role.Id">
                <a href="#" class="btn btn-danger"
                  onclick="confirmDelete('@role.Id', true)">Delete</a>
            </span>
        </form>
    }
</div>
```

97 - Change AccessDenied route

Change AccessDenied Route

➤ Default **AccessDenied** Route in ASP.NET Core **/Account/AccessDenied**

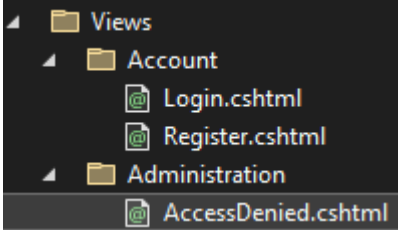
По подразбиране е /Account/AccessDenied, но ако искаме да го променим на /Administration/AccessDenied -> Startup -> ConfigureServices

```
services.ConfigureApplicationCookie(options =>
{
    options.AccessDeniedPath = new PathString("/Administration/AccessDenied");
});
```

```
services.ConfigureApplicationCookie(options =>
{
    options.AccessDeniedPath = new PathString("/Administration/AccessDenied");
});
```

След Logout->Login, промяната влиза в сила. Тогава връща 404, защото в новата директория в AdministrationController нямаме AccessDenied action. Това лесно се поправя като преместим AccessDenied action от AccountController -> AdministrationController + кореспондираща View.

```
[HttpGet]
[AllowAnonymous]
1 reference
public IActionResult AccessDenied()
{
    return View("AccessDenied");
}
```



98 - Claim type and claim value in claims policy based authorization

dbo.AspNetUserClaims table:

dbo.AspNetUserClaims [Data] AdministrationController.cs				
Max Rows: 1000				
	Id	UserId	ClaimType	ClaimValue
▶	9	1f9ea9da-40d8-...	Create Role	Create Role
	10	1f9ea9da-40d8-...	Edit Role	Edit Role
	11	1f9ea9da-40d8-...	Delete Role	Delete Role

Където в случая ClaimType съвпада с ClaimValue.

Only Claim Type in Authorization Policy

To satisfy this policy **Edit Role** claim must be present

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy",
        policy => policy.RequireClaim("Edit Role"));
});
```



Само проверяваме ClaimType, без ClaimValue. Това е достатъчно.

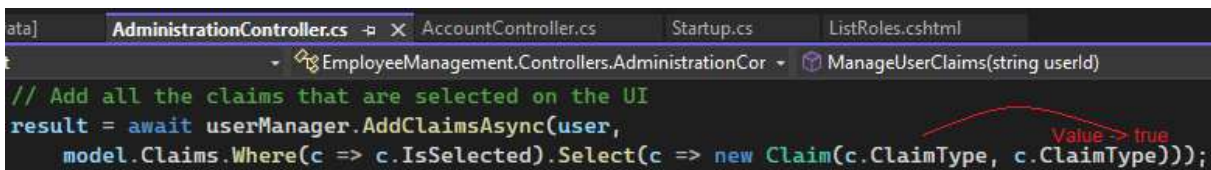
Claim Type and Claim Value

To satisfy this policy **Edit Role** claim must be present with a value of **true**

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy",
        policy => policy.RequireClaim("Edit Role", "true"));
});
```



За да се удовлетвори условието, за ClaimValue се задава стойност true.



```
AdministrationController.cs | AccountController.cs | Startup.cs | ListRoles.cshtml
EmployeeManagement.Controllers.AdministrationController | ManageUserClaims(string userId)
// Add all the claims that are selected on the UI
result = await userManager.AddClaimsAsync(user,
    model.Claims.Where(c => c.IsSelected).Select(c => new Claim(c.ClaimType, c.ClaimType)));
```

На този етап, вместо (ClaimType, ClaimValue), използваме на 2 места ClaimType. Искаме да зададем стойност bool value, така че при маркиране на check-box, да имаме true, иначе false.

```
model.Claims.Select(c => new Claim(c.ClaimType, c.IsSelected ? "true" : "false"));
```

```
model.Claims.Select(c => new Claim(c.ClaimType, c.IsSelected ? "true" : "false"));
```

Administration/EditUser/87cabcf7-6319-4672-a7c8-db5a048fa5f7

Маркираме Create, Edit, без Delete -> Update:

Manage User Claims

☒ Create Role
☒ Edit Role
☐ Delete Role

User Claims

true
 true
 false

UserId	ClaimType	ClaimValue
1f9ea9da-40d8-...	Create Role	Create Role
1f9ea9da-40d8-...	Edit Role	Edit Role
1f9ea9da-40d8-...	Delete Role	Delete Role
a5b9400f-45cf-...	Create Role	Create Role
a5b9400f-45cf-...	Edit Role	Edit Role
b2056455-c2b4-...	Delete Role	Delete Role
87cabcf7-6319-...	Create Role	true
87cabcf7-6319-...	Edit Role	true
87cabcf7-6319-...	Delete Role	false
NULL	NULL	NULL

В таблицата са се смесили стари и нови стойности. Старите ще коригираме през UI.

Трябва да коригираме в Administration/EditUser

```
[Data] AdministrationController.cs* AccountController.cs Startup.cs ListRoles.cshtml
ent EmployeeManagement.Controllers.AdministrationCor EditUser(string id)
Claims = userClaims.Select(c => c.Type + " : " + c.Value).ToList(),
```

досега е имало само c.Value, а ние добавяме c.Type + разграничител при изписването. Така виждаме и двете:

User Claims

Create Role : true
 Edit Role : true
 Delete Role : false

Manage User Claims

☒ Create Role
☒ Edit Role
☒ Delete Role

Виждаме, че има малък бър:Delete role: false, а като се отвори съответния view, чек-боксът е маркиран, което се дължи на съответния GET:

Само от проверка на ClaimType -> isSelected = true.

```
EmployeeManagement.Controllers.AdministrationCor ManageUserClaims(string userId)
if (existingUserClaims.Any(c => c.Type == claim.Type && c.Value == "true"))
{
    userClaim.IsSelected = true;
}
```

Затова добавяме изискването и Value да бъде true.

Сега автоматично чек-боксът е вече празен.

Обновяваме даните за всички потребители през UI, за да се актуализират стойностите в БД.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy", (ClaimType, ClaimValue)
        policy => policy.RequireClaim("Edit Role", "true"));
        key comparing: (unsensitive, SENSITIVE)
    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});
```

Интересното в случая е следното: когато даден потребител има права само за Delete Role, се вижда само Delete btn, но ако има (само) Edit Role, се виждат и Edit + Delete. Така е и на видеото. Явно е някаква вградено автоматично разпознаване edit да включва delete? Чудно. Нали са CRUD операции, би трябвало да са разграничени.

```
troller.cs  ListRoles.cshtml*  Startup.cs
<form asp-action="DeleteRole" asp-route-id="@role.Id" method="post">
    @if ((await authorizationService.AuthorizeAsync(User, "EditRolePolicy")).Succeeded)
    {
        <a asp-controller="Administration" asp-action="EditRole"
            asp-route-id="@role.Id" class="btn btn-primary">
            Edit
        </a>
    }

    @if ((await authorizationService.AuthorizeAsync(User, "DeleteRolePolicy")).Succeeded)
    {
        <span id="deleteSpan_@role.Id">
            <a href="#" class="btn btn-danger"
                onclick="confirmDelete('@role.Id', true)">Delete</a>
        </span>

        <span id="confirmDeleteSpan_@role.Id" style="display:none">
            <span>Are you sure you want to delete?</span>
            <button type="submit" class="btn btn-danger">Yes</button>
            <a href="#" class="btn btn-primary"
                onclick="confirmDelete('@role.Id', false)">No</a>
        </span>
    }
</form>
```

Claim Type and Claim Value

To satisfy this policy **Country** claim must be present with a value of **USA, India or UK**

```
services.AddAuthorization(options =>
{
    options.AddPolicy("AllowedCountryPolicy", ( CType, CValue1, CV2, SV3)
        policy => policy.RequireClaim("Country", "USA", "India", "UK"));
});
```

string

➤ **ClaimType** comparison is case in-sensitive

➤ **ClaimValue** comparison is case sensitive

```
options.AddPolicy("EditRolePolicy", ( CType, strings)
    policy => policy.RequireClaim("Edit Role", "true"));

options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin"));

(parameter) params string[] allowedValues
Values the claim must process one or more of for evaluation to succeed.
```

ClaimValue може да бъде лист от стрингове, като задължително трябва да се избере 1 от тях.

99 - Create custom authorization policy using func

Why Create a Custom Policy

➤ **Create a policy with multiple requirements**

- Must be a member of the **Admin** role AND
- Must have **Edit Role** claim with a value of **true** OR
- Must be a member of the **Super Admin** role


```
options.AddPolicy("EditRolePolicy",
    policy => policy.RequireClaim("Edit Role", "true"));
options.AddPolicy( (parameter) AuthorizationPolicyBuilder policy
```

Както се вижда от Intellisense, параметър policy е от тип AuthorizationPolicyBuilder. Този обект поддържа fluence index, което означава, че не поддържа множество правила и роли наведнъж.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy => policy
        .RequireRole("Admin")
        .RequireClaim("Edit Role", "true")
        .RequireRole("Super Admin")
    );
});
```



Обяснението е много просто: при подобен запис, всички 3 изисквания трябва да бъдат изпълнени. На нас това не ни върши работа, макар синтаксисът да е вярно изписан. Ние искаме първите 2 условия да бъдат изпълнение ИЛИ 3-тото.

За да го докажем, правим демонстрация:

```
options.AddPolicy("EditRolePolicy",
    policy => policy.RequireClaim("Edit Role", "true")
        .RequireRole("Admin")
        .RequireRole("Super Admin"));
```

```
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> EditRole(string id)
```

За простота коментираме изискването за Админ в началото на контролера, така че това да стане част само от EditRole action с неговия policy.

```
//[Authorize(Roles = "Admin")]
3 references
public class AdministrationController : Controller
```

Към момента дори не сме създали потребителска роля Super Admin, така че, когато впишем потребител, който е Admin и има права за 3-те: Create, Edit, Delete Role, той пак няма да види бутон Edit, а само Delete, т.к. не удовлетворява условието за Super Admin. Това демонстрира и въпросът от предходния урок: Edit е с най-висок приоритет, така че, ако има права за него, има права и върху Delete. Обратното не важи. Бутон Edit е скрит, а ако се опитаме да получим достъп директно от URL: <http://localhost:24077/Administration/EditRole/71ef2a67-38d4-42e5-88c0-2847ce6f6ca0> ни пренасочва

localhost:24077/Administration/AccessDenied?ReturnUrl=%2FAdministration%2FEditRole%2F71ef2a67-38d4-42e5-88c0-

Access Denied

You do not have permission to view this resource



User Roles

- Admin
- Super Admin

Manage Roles

User Claims

Create Role : false
Edit Role : true
Delete Role : false

Manage Claims

Създаваме Super Admin и потребител, който притежава всички права, така че да отговаря на всички изисквания Admin, Super Admin + Edit Role. Logout+Login, за да влязат в сила промените.

Сега имаме достъп, но това не е което искаме. Ние искаме потребителят да бъде Admin + Edit ИЛИ Super Admin, а не 3-те едновременно. За целта използваме персонализиран .RequireAssertion вместо .RequireClaim+.RequireRole.

```

policy.RequireAssertion();
policy("AdminRole"
policy.Require
AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireAssertion(System.Func<AuthorizationHandlerContext, bool>
handler) (+ 1 overload)

```

Вижда се, че .RequireAssertion изисква Func тип параметър, който от своя страна прихваща параметър от тип AuthorizationHandlerContext и връща параметър тип Boolean.

Нека наречем параметъра context

```

options.AddPolicy("EditRolePolicy",
    policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" &&
            param.Prop.Method claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));

```

Parameter.Property.Method()

Остава да тестваме с 3 потребителя: единият Admin + Edit, другият само Super Admin, третият само Admin без Edit.

```

services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));
});

```



Единственото интересно нещо в случая е, че Super Admin няма видимост към Manage бутона в топ менюто заради if условие в _Layout.

```

tml      _Layout.cshtml  X Startup.cs*
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle text-light"
            href="#"
            id="navbarDropdownMenuLink"
            role="button"
            data-toggle="dropdown"
            @*data-hover="dropdown"*@
            aria-haspopup="true"
            aria-expanded="false">
            Manage
        </a>

```

Нищо не пречи временно да закоментираме if условието, тогава Super Admin ще има видимост, ако не – директно от URL адреса има достъп до EditRole с видимост на Edit + Delete бутоните.

Вторият начин е да се състави функция, която връща bool дали условията са изпълнени:

Using a Func to Create a Custom Policy

```

services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.RequireAssertion(context => AuthorizeAccess(context)));
});

private bool AuthorizeAccess(AuthorizationHandlerContext context)
{
    return context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") ||
        context.User.IsInRole("Super Admin");
}

```

100 - Custom authorization requirements and handlers

Simple requirements:

```
options.AddPolicy("EditRolePolicy",  
policy => policy.RequireClaim("Edit Role"));  
  
options.AddPolicy("EditRolePolicy",  
policy => policy.RequireClaim("Edit Role", "true"));  
  
options.AddPolicy("EditRolePolicy",  
policy => policy.RequireClaim("Edit Role", "true", "yes"));
```

Multiple requirements:

Policies with multiple built-in requirements

For this policy to succeed, the user must be in the **Admin** role **AND** must have **Edit Role** claim type with a claim value of either **true** or **yes**

```
options.AddPolicy("EditRolePolicy", policy => policy  
    .RequireClaim("Edit Role", True, yes)  
    .RequireRole("Admin"));
```

The user must be in the **Admin** role and has claim type **Edit Role** with a value of **true**

OR

The user must be in the **Super Admin** role

```
options.AddPolicy("EditRolePolicy", policy => policy.RequireAssertion(context =>  
    context.User.IsInRole("Admin") &&  
    context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true")  
    context.User.IsInRole("Super Admin")));
```

От Intellisense се вижда, че Build-in Requirements са съответно:

.RequireClaim - .ClaimsAuthorizationRequirement

```
RequireClaim("Delete Role");
```

AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireClaim(string claimType) (+ 2 overloads)
Adds a Microsoft.AspNetCore.Authorization.Infrastructure.ClaimsAuthorizationRequirement to the current instance.

.RequireRole - .RolesAuthorizationRequirement

```
.RequireRole("Admin");
```

AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireRole(params string[] roles) (+ 1 overload)
Adds a Microsoft.AspNetCore.Authorization.Infrastructure.RolesAuthorizationRequirement to the current instance.

.RequireAssertion - .AssertionRequirement

```
RequireAssertion(context =>
```

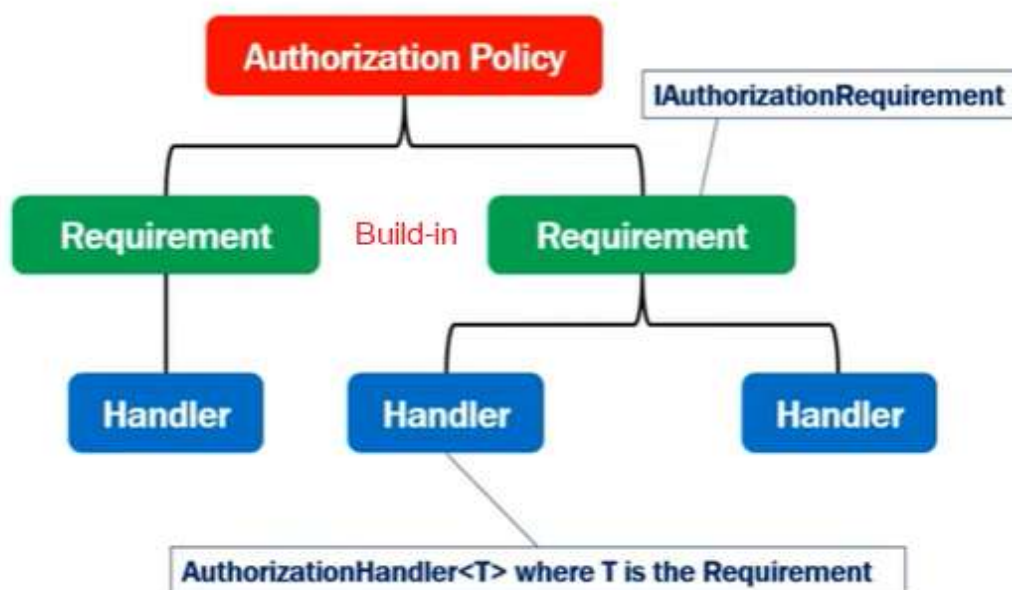
AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireAssertion(System.Func<AuthorizationHandlerContext, bool> handler) (+ 1 overload)
Adds an Microsoft.AspNetCore.Authorization.Infrastructure.AssertionRequirement to the current instance.

Built-in asp.net core authorization requirements

- `RequireClaim()` method adds `ClaimsAuthorizationRequirement`.
- `RequireRole()` method adds `RolesAuthorizationRequirement`.
- `RequireAssertion()` method adds `AssertionRequirement`.

За прости приложения, тези Requirements ще свършат работа, но за по-сложни, трябва да въведем персонализирани:

Custom authorization requirement



An authorization policy has one or more requirements. Each authorization requirement has one or more handlers.

All the built-in authorization requirements implement `IAuthorizationRequirement` interface. So to create a custom authorization requirement, we need to implement `IAuthorizationRequirement` interface. This is an empty marker interface, which means there is nothing in this interface that our custom requirement class must implement.

It is in the `authorization handler` that we write our logic to allow or deny access to a resource like a `controller action` for example. An authorization handler implements `AuthorizationHandler<T>` where `T` is the type of requirement.

For example, let's say, an `admin` user can assign or remove roles of other admin users but not his own roles. To achieve this, we need to know the logged-in `UserID` and the `UserId` of the `Admin` being edited. If they are the same we do not want to allow access. The `admin UserID` being edited is passed in the URL as a query string parameter. From the authorization handler we will have access to the route data and URL query string parameters. Dependency injection is also supported. This means, we can even inject and use other services if required.

101 - Custom authorization requirement and handler example

Към момента, вписаният потребител може сам да си редактира профила и правата. Не искаме да допускаме това. Искаме Админ да може да редактира правомощията на останалите потребители, но не и своите. Това да може да направи само друг Админ. За целта трябва да знаем ID на потребителя, който се е вписал и на този, който се редактира. Ако съвпадат, трябва да се отказва достъп. Към момента, ID на потребителя, който се редактира, се подава в URL адреса като query string parameter.

Custom Requirement & Handler Example

An **Admin** user can manage **other Admin** user roles and claims but not their own claims and roles

Func delegate and **RequireAssertion()**

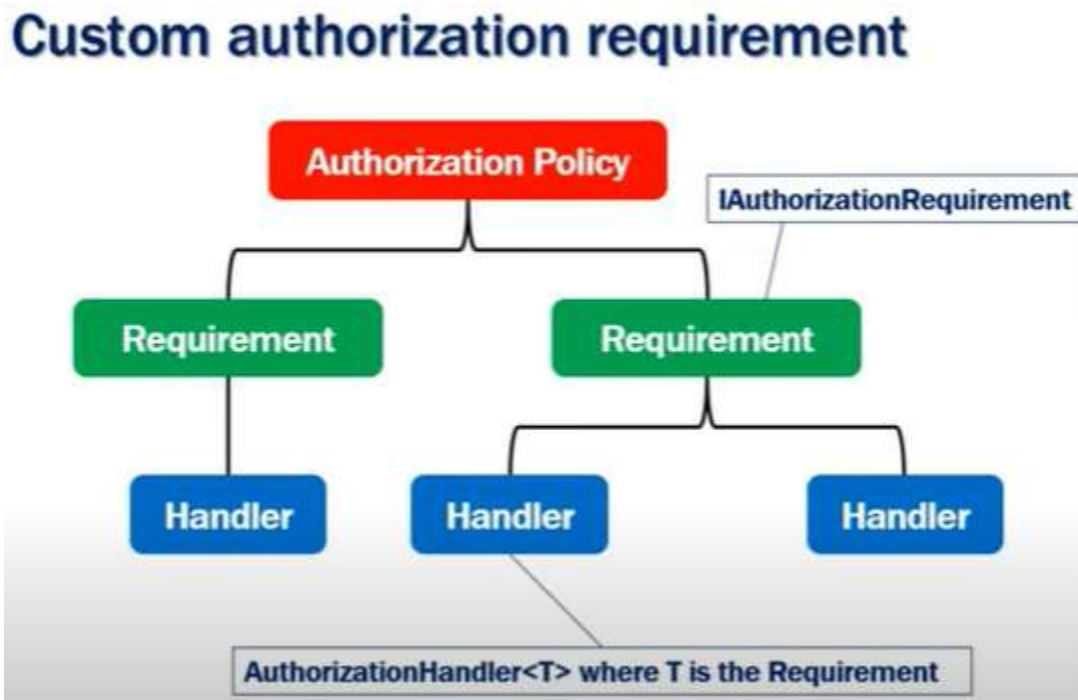
➤ Part 99 - ASP.NET Core Tutorial

Func cannot be used when

- We need to access **route data** or **query string parameter**
- We need access to other services via **dependency injection**

Трябва да създадем клас и имплементираме интерфейс `IAuthorizationRequirement`.

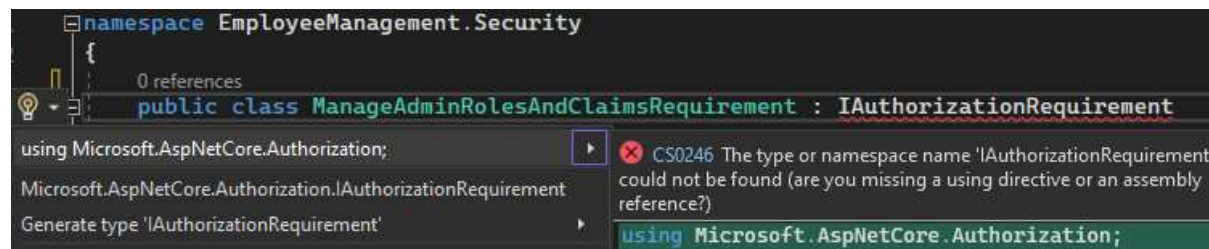
За целта в основната директория на проекта създаваме отделна папка `Security` -> Add -> class -> `ManageAdminRolesAndClaimsRequirement.cs`



To create a custom authorization requirement, create a class that implements the `IAuthorizationRequirement` interface. This is an empty marker interface, which means there is nothing in this interface that our custom requirement class must implement.

```
public class ManageAdminRolesAndClaimsRequirement : IAuthorizationRequirement
{
}
```

`IAuthorizationRequirement` interface is in `Microsoft.AspNetCore.Authorization` namespace.



`IAuthorizationRequirement` не е вграден, че да предоставя наготово някакви системни функционалности, затова трябва да го създадем и персонализираме.

В папка `Security` -> Add -> Class -> `CanEditOnlyOtherAdminRolesAndClaimsHandler.cs`

Този клас ще унаследява `AuthorizationHandler` от тип `T`, където `T` е изискване от нашия тип `ManageAdminRolesAndClaimsRequirement`, т.е. този клас ще бъде аргумент като наш generic parameter.

Creating a custom authorization handler

It is in the authorization handler that we write our logic to allow or deny access to a resource like a controller action for example. To implement a handler you inherit from [AuthorizationHandler<T>](#), and implement the [HandleRequirementAsync\(\)](#) method. The generic parameter <T> on the [AuthorizationHandler<T>](#) is the type of requirement.

```
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    0 references
    public class CanEditOnlyOtherAdminRolesAndClaimsHandler : inherit
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        0 references
        override protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            throw new System.NotImplementedException();
        }
    }
}
```

Записваме override + автоматично попълване.

Метод `HandleRequirementAsync` получава 2 входни параметъра (`context`, `requirement` = който обработваме). Обект `context` има `.Resource` property, който връща тип `object`, но ние го прехвърляме като `AuthorizationFilterContext`.

```
context = context.Resource as AuthorizationFilterContext;
if (context == null)
{
    return Task.CompletedTask;
}
```

object? AuthorizationHandlerContext.Resource { get; }
The optional resource to evaluate the AuthorizationHandlerContext.

Използваме `.Resource` за да предпазим `ManageUserRoles()` action + `ManageUserClaims()` action в `AdministrationController`, така че този `.Resource` property връща тези action methods които защитаваме като `AuthorizationFilterContext`. Съхраняваме този резултат в променливата `var authFilterContext` и ако тя е null, тогава не можем да процедираме с проверката за ауторизация, така че връщаме `.CompletedTask` и забранен достъп.

```
AuthorizationFilterContext is using Microsoft.AspNetCore.Mvc.Filters;
```

The following line returns the controller action being protected as the [AuthorizationFilterContext](#) and provides access to `HttpContext`, `RouteData`, and everything else provided by MVC and Razor Pages.

```
var authFilterContext = context.Resource as AuthorizationFilterContext;
```

If [AuthorizationFilterContext](#) is `NULL`, we cannot check if the requirement is met or not, so we return `Task.CompletedTask` and the **access is not authorised**.

```

if (authFilterContext == null)
{
    return Task.CompletedTask;
}

protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
    ManageAdminRolesAndClaimsRequirement requirement)
{
    // .prop returns controller action being protected and provides access
    var authFilterContext = context.Resource as AuthorizationFilterContext;
    if (authFilterContext == null)
    {
        return Task.CompletedTask; // Access Denied
    }
}

```

The incoming context object has list of user claims, from this list of user claims, we search for the NameOfIdentifier Claim, because it contains the ID of the logged in user. Next we want the ID of the admin being edited. It is passed in the URL as a query string parameter. To retrieve it we are using this authFilterContext object. Through this object we have access to HTTP context, route data, query string parameters and everything else provided by MVC and Razor Pages. After we have successfully retrieved the loggedInAdminId and adminIdBeingEdited, our requirement is met and the authorization succeeds.

We check:

If the user is in the Admin role

AND

has Edit Role claim type with a claim value of true

AND

the logged-in user Id is NOT EQUAL TO the Id of the Admin user being edited

```

if (context.User.IsInRole("Admin") &&
    context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
    adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
{
    context.Succeed(requirement);
}

```

`Succeed()` method specifies that the requirement is successfully evaluated.

```

public class CanEditOnlyOtherAdminRolesAndClaimsHandler : inherit
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
{
    0 references
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
        ManageAdminRolesAndClaimsRequirement requirement)
    {
        .prop returns controller action being protected and provides access
        var authFilterContext = context.Resource as AuthorizationFilterContext;
        if (authFilterContext == null)
        {
            true
            {
                return Task.CompletedTask;    Access Denied
            }
        }

        else string loggedInAdminId =
            context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;
            incoming context object has list of user claims, in which we search for .NameIdentifier to provide ID
            string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];

            if (context.User.IsInRole("Admin") &&
                context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
                true adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
            {
                context.Succeed(requirement);
            }

            else false
            return Task.CompletedTask;    Access Denied
        }
    }
}

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc.Filters;
using System.Linq;
using System.Security.Claims;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    public class CanEditOnlyOtherAdminRolesAndClaimsHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext
context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            var authFilterContext = context.Resource as AuthorizationFilterContext;
            if (authFilterContext == null)
            {
                return Task.CompletedTask;
            }

            string loggedInAdminId =
                context.User.Claims.FirstOrDefault(c => c.Type ==
ClaimTypes.NameIdentifier).Value;

            string adminIdBeingEdited =
authFilterContext.HttpContext.Request.Query["userId"];

            if (context.User.IsInRole("Admin") &&
                context.User.HasClaim(claim => claim.Type == "Edit Role" &&
claim.Value == "true") &&
                adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())

```



```

        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}

```

Authorization handler registration

We register custom authorization handler in `ConfigureServices()` method of the `Startup` class

```

options.AddPolicy("DeleteRolePolicy",
    policy => policy.RequireClaim("Delete Role"));

options.AddPolicy("EditRolePolicy",
    policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" &&
            claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));

options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin"));
});

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();

```

```

options.AddPolicy("DeleteRolePolicy",
    policy => policy.RequireClaim("Delete Role"));

options.AddPolicy("EditRolePolicy",
    policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement());

options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin"));
});

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();

```

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthorization(options =>
    {
        options.AddPolicy("EditRolePolicy", policy =>

```



```

    policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement());
});

services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();
}

```

Finally use the custom policy to protect the resources like controller action methods

```

EmployeeManagement.Controllers.AdministrationCon EditRole
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> EditRole(string id)

```

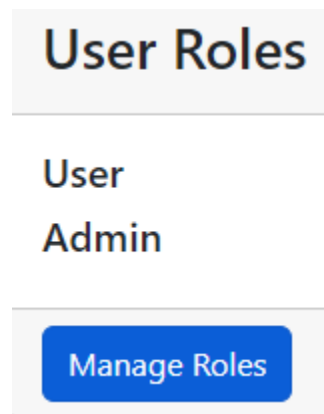
Вече `[Authorize(Policy = "EditRolePolicy")]` не ни е необходим в Get + Post на EditRole, така че го изтриваме от тях.
Необходим ни е в Get + Post на ManageUserRoles action.

```

[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)

```

При тестване, Админ може да редактира чужди права, но не и свои собствени.



Към момента сме защитили само ManageUserRoles action, но не и ManageUserClaims action, така че при Manage Claims бутоната правилата все още не важат, така че прилагаме същият custom policy и при него.

Get + Post
`[Authorize(Policy = "EditRolePolicy")]`
 ManageUserClaims

Към момента, за обработката на нашия Custom Requirement имаме само един Custom Handler, но също е възможно 1 requirement да има multiple handlers.

Step - 1 : Create the Custom Requirement

```
public class ManageAdminRolesAndClaimsRequirement : IAuthorizationRequirement
{ }
```

Step - 2 : Create the Custom Handler

```
public class CanEditOnlyOtherAdminRolesAndClaimsHandler :
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
        ManageAdminRolesAndClaimsRequirement requirement)
    {
        var authFilterContext = context.Resource as AuthorizationFilterContext;
        if (authFilterContext == null)
            return Task.CompletedTask;

        string loggedInAdminId =
            context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;

        string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];

        if (context.User.IsInRole("Admin") &&
            context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
            adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

Step - 3 : Register the Custom Handler

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthorization(options =>
    {
        options.AddPolicy("EditRolePolicy", policy =>
            policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));
    });

    services.AddSingleton<IAuthorizationHandler,
        CanEditOnlyOtherAdminRolesAndClaimsHandler>();
}
```

Step - 4 : Use the Custom Policy

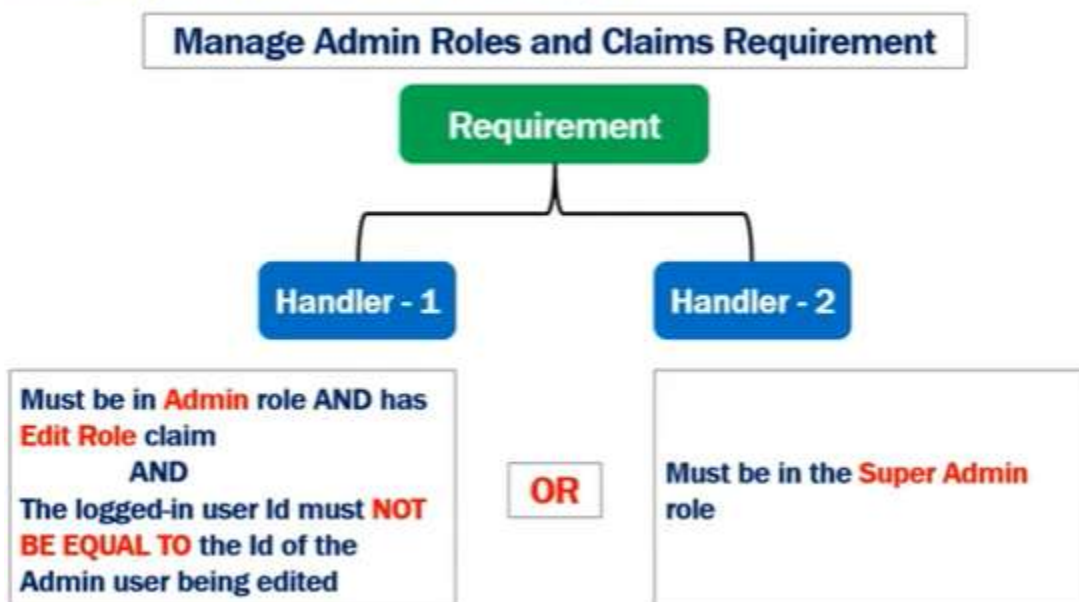
```
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
public async Task<IActionResult> ManageUserRoles(string userId)
{
    // Code
    return View();
}
```

102 - Multiple custom authorization handlers

Why Multiple Handlers

We implement **multiple handlers** for a single requirement, when want the requirement evaluation to be on an **OR** basis

Multiple Handlers Example



Към момента имаме само Handler 1, нека направим и 2-я:

Security folder -> new class SuperAdminHandler.cs

Той трябва да унаследява от <T>, където T е типът на изискването по същия начин както в предходния случай:

```
public class CanEditOnlyOtherAdminRolesAndClaimsHandler :  
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
```

```

using Microsoft.AspNetCore.Authorization;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    0 references
    public class SuperAdminHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        0 references
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            throw new System.NotImplementedException();
        }
    }
}

```

Всичко, което искаме да изпълни, е само да провери дали потребителят е Super Admin.

```

0 references
protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
    ManageAdminRolesAndClaimsRequirement requirement)
{
    if(context.User.IsInRole("Super Admin"))
    {
        context.Succeed(requirement);
    }
    else return Task.CompletedTask; Access Denied
}

```

```

using Microsoft.AspNetCore.Authorization;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    public class SuperAdminHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext
context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            if(context.User.IsInRole("Super Admin"))
            {
                context.Succeed(requirement);
            }
            return Task.CompletedTask;
        }
    }
}

```

Когато тестваме в Debug mode обаче, Super Admin не получава достъп до ManageUserRoles, затова поставяме BreakPoint в класовете на 2-та Handler класа:

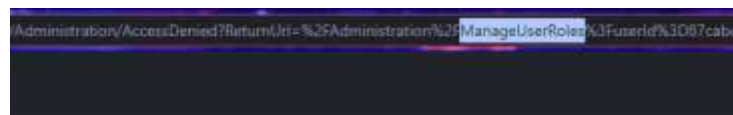
```
CanEditOnlyOt...imsHandler.cs | SuperAdminHandler.cs
EmployeeManagement | EmployeeManagement.Security.SuperA
11 | {
12 |     if(context.User.IsInRole("Super Admin"))
```

```
CanEditOnlyOt...imsHandler.cs | SuperAdminHandler.cs
EmployeeManagement | EmployeeManagement.Security.CanEditOnlyOtherAdm | HandleRequirementAsyn
14 | {
15 |     var authFilterContext = context.Resource as AuthorizationFilterContext;
```

При това положение, при повторно натискане на Manage Roles бутона, BreakPoint of the first Handler is hit.

```
CanEditOnlyOt...imsHandler.cs | SuperAdminHandler.cs
EmployeeManagement | EmployeeManagement.Security.CanEditOnlyOtherAdm | HandleRequirementAsyn
14 | {
15 |     var authFilterContext = context.Resource as AuthorizationFilterContext;
```

We are logged in as Super Admin, so Handler-1 is not valid evaluated, because the logged in user is not Admin + Edit Role claim. Continue -> the second BreakPoint is hit:



But the second BreakPoint is not hit at all. Why? Because we have forgot one of the very important steps: registering the Handler. Stop Debugging -> Startup


```

services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy",
        policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();
services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();

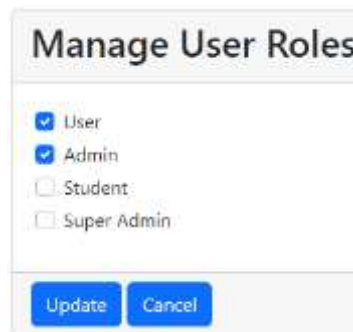
```

```

services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();

```

Даваме отново Debug -> Manage Roles -> BreakPoint Handler-1, BreakpointHandler-2 -> Access.



При проверка потвърждаваме, че Admin + Edit получава достъп, Super Admin също, но останалите потребители не.

103 - Custom authorization handler success vs failure

What can Authorization Handler Return

Success - `context.Succeed()`

Failure - `context.Fail()`

Nothing - `Task.CompletedTask`

When you have multiple handlers

- **Failure takes precedence over success**
- **When one of the handlers return failure, the policy fails even if the other handlers return success**

Към момента имаме

Handler-1: [CanEditOnlyOtherAdminRolesAndClaimsHandler](#)

Handler-2: [SuperAdminHandler](#)

И двата унаследяват изискване за авторизация от един `ManageAdminRolesAndClaimsRequirement`

В кодовете им при успех се извиква

```
context.Succeed(requirement);
```

Никъде нямаме код как да се процедира при конкретен failure.

В Handler-1 в if-блока поставяме нов BreakPoint + else, който да прихване failure:

```
else
{
    context.Fail();
}
```

```
Startup.cs  CanEditOnlyOtherAdminHandler.cs*  SuperAdminHandler.cs
EmployeeManagement  EmployeeManagement.Security.CanEditOnlyOtherAdminRoles  HandleRequirementAsync(AuthorizationHa

14
15
16 var authFilterContext = context.Resource as AuthorizationFilterContext;
17 if (authFilterContext == null)
18 {
19     return Task.CompletedTask;
20 }
21
22 string loggedInAdminId =
23     context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;
24
25 string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];
26
27 if (context.User.IsInRole("Admin") &&
28     context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
29     adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
30 {
31     context.Succeed(requirement);
32 }
33 else
34 {
35     context.Fail();
36 }
```

При Debug като SuperAdmin -> ListUsers ->Edit->Manage Roles ->

Handler-1 BreakPoint-1, BreakPoint-2, но понеже потребителят не отговаря на нито едно от условията, отива в else -> .Fail();

После прескача на Handler-2, BreakPoint3, където удовлетворява условието за Super Admin, но при Continue -> Access Denied. Това е, защото вече е извикан един Fail. Когато това се случи, независимо дали по-нататък ще се удовлетвори някакво условие, Fail е с предимство и ще се изпълни той.

В моята версия прескача направо на BreakPoint-3, дава ми достъп, но и интересен вид грешка:

```
26
27 if (context.User.IsInRole("Admin") &&
28     context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
29     adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
30 {
31     context.Succeed(requirement);
32 }
33 else
34 {
35     context.Fail();
36 }
```

110 % No issues found

Error List

Code	Description
CS2012	Cannot open 'D:\Exercises\EmployeeManagement\obj\Debug\net6.0\EmployeeManagement.pdb' for writing -- 'The process cannot access the file 'D:\Exercises\EmployeeManagement\obj\Debug\net6.0\EmployeeManagement.pdb' because it is being used by another process.'
CS8974	Converting method group 'AccessDenied' to non-delegate type 'object'. Did you intend to invoke the method?

Като правило .Fail не се използва в Handler, за да не се провали алгоритъма, ако друг Handler е успешен. Използваме Fail за Handler, само ако искаме да гарантираме отказване на достъпа, дори някой от другите Handler-и е успешен.

Трябва да извикаме .Succeed и да не използваме .Fail

When you have multiple handlers

- **Failure takes precedence over success**
- When one of the handlers return failure, the policy fails even if the other handlers return success
- **In general, do not return failure from a handler, as other handlers for the same requirement may succeed**
- Only return an explicit failure, to guarantee the failure of the policy even when the other handlers succeed
- **If none of the handlers return an explicit success the policy will not succeed**
- For a policy to succeed
 - An explicit **success** must be returned from one of the handlers, AND
 - No other handler must return an explicit **failure**

Ако искаме подредбата по подразбиране да се промени, задаваме

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false;
})
```

Set **InvokeHandlersAfterFailure** to false

- By default, all the handlers are called, irrespective of what a handler returns
 - Success
 - **Failure**
 - Nothing
- If you do not want the rest of the handlers to be called, when a failure is returned, set **InvokeHandlersAfterFailure** property to **false**. The default is true

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false; ←
});
```

В този случай (който НЕ е по подразбиране в моята версия), ако даден Handler се провали и се превключи на следващ, който изпълни изискванията, операцията се изпълнява.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy",
        policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false;

    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});
```

Сера в Handler-1 оставяме само 1-я генерален BreakPoint, а втория пред if-блока го отстраняваме. В Handler-2 имаме един BreakPoint-2. Даваме Debut и BP1 не е удовлетворен, защото сме Super Admin -> Continue не стига дори до BP2, който отговаря на условията, а прехвърля директно на Access Denied. Това е заради реда код, който току-що поставихме

options.InvokeHandlersAfterFailure = false;

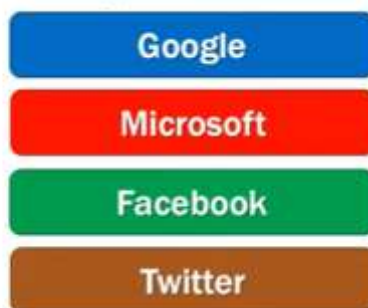
затова го премахваме, както

```
else
{
    context.Fail();
}
```


104 - External identity providers

Когато искаме да предоставим възможност на потребителя да се впише чрез външна платформа:

External Identity Providers



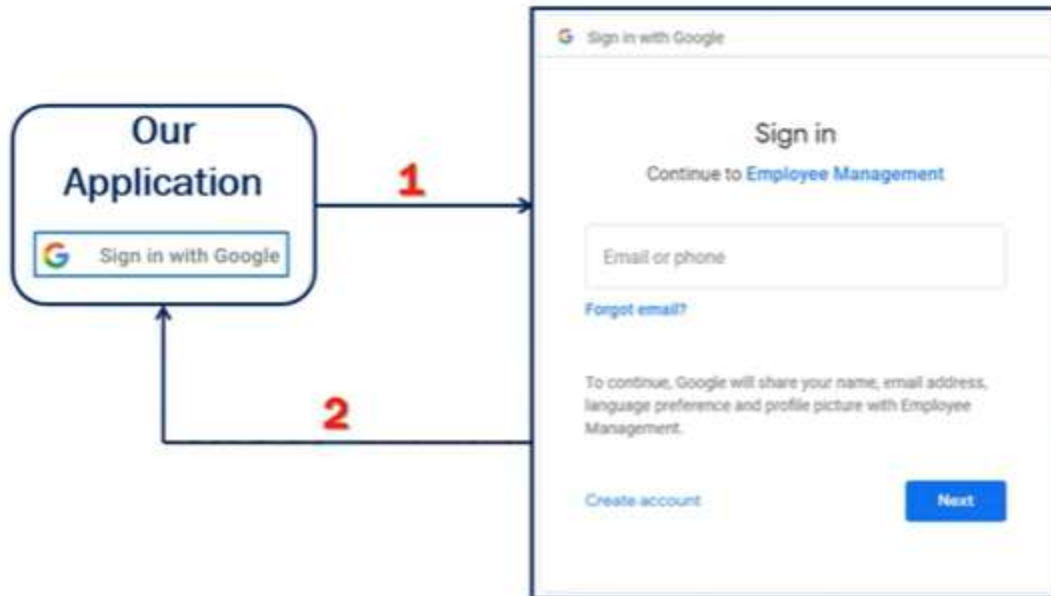
Най-важното е да се предостави възможност само за Trusted external identity providers + 1 потребител да няма по много различни акаунти от различни платформи, а той да бъде асоцииран с неговите различни към същия потребителски акаунт в нашето приложение.

How External Identity Providers Work

A login form titled "Log in" with two sections: "Use a local account to log in." and "Use another service to log in." The local section has fields for Email and Password, a "Remember me?" checkbox, and a "Log in" button. The external services section has buttons for Google, Microsoft, and Facebook.

Our application then redirects the user to Google signin page. Here the user provides his/her google login credentials. Upon a successful login, google will then send the user back to our application and a pre-configured callback function is executed. The code in this callback function checks the identity received from the external identity provider and sign-in that user into our application.

How External Identity Providers Work



To use an external identity provider like **Google**, we have to first register our application with Google. Upon successful registration we will be provided with **Client Id** and **Client Secret** which we need to use Google authentication.

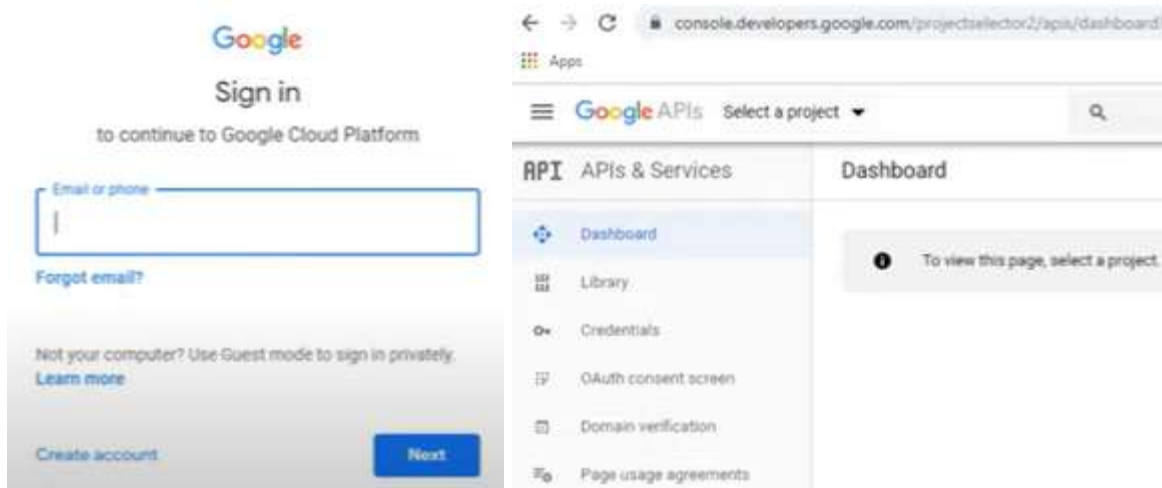
105 - Create google oauth credentials Client Id and Client Secret

Create Google OAuth Credentials

Register Application

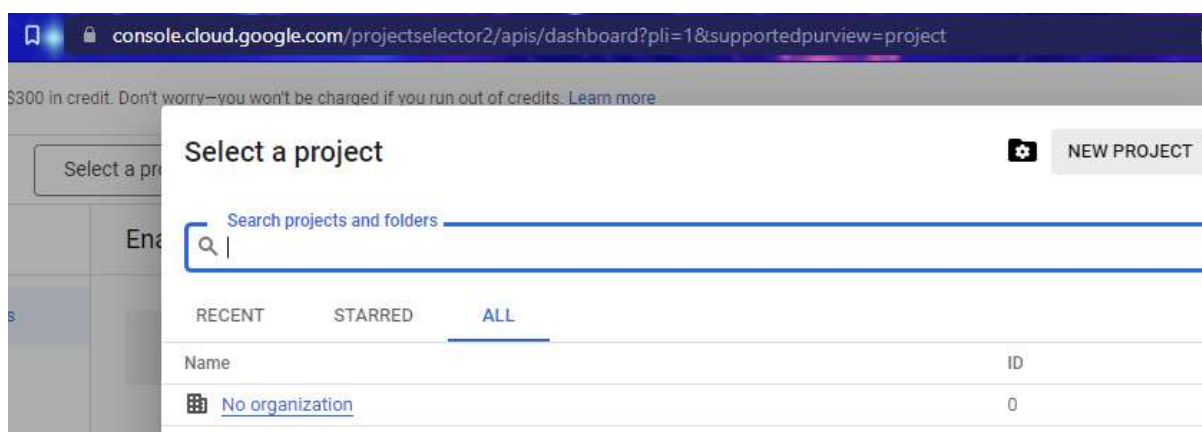
<https://console.developers.google.com>

Login with your google credentials <https://console.developers.google.com>



Create new project.

Google APIs Select a project ▼





Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)



Google Cloud

Search for resources, docs, products, and more

New Project



You have 12 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[MANAGE QUOTAS](#)

Project name *

My Project 47026



Project ID: enduring-badge-369614. It cannot be changed later. [EDIT](#)

Location *



No organization

[BROWSE](#)

Parent organization or folder

CREATE

CANCEL

Наричае проекта „Employee Mgmt STS” (Security token service), защото ще предоставим услуга за идентификация

Project name *

Employee Mgmt STS



Project ID: employee-mgmt-sts-369614. It cannot be changed later. [EDIT](#)

-> Create.



Google Cloud

Employee Mgmt STS

Search for resources, docs, products, and more (/)



APIs & Services

APIs & Services

[+ ENABLE APIS AND SERVICES](#)



Enabled APIs & services

1 hour 6 hours



Library



Credentials



OAuth consent screen



Page usage agreements

Traffic



Errors

1.0/s

0.8/s

След това е необходим Enable APIS and services, като за целта -> Librari.

Ако дясното меню не е видимо, го включваме от Google Cloud.

Пренасочва ни към страницата за търсене. Търсим "Google+api"

← API Library

Welcome to the API Library

The API Library has documentation, links, and a smart search experience.

Q Google+ api

google+ api

Employee Mgmt STS ▾

Q google+ api

X

3 results

^

(3)



Google+ API

Google

The Google+ API enables developers to build on top of the Google+ platform.

Enable



Google+ API

[Google](#)

The Google+ API enables developers to build on top of the Google+ platform.

ENABLE

Click to enable this API

OVERVIEW

DOCUMENTATION

←

API/Service Details

■ DISABLE API



Google+ API

The Google+ API enables developers to build on top of the Google+ platform.

By Google

Service name	Type	Status
plus.googleapis.com	Public API	Enabled

QUOTAS

API APIs & Services

- Enabled APIs & services
- Library
- Credentials
- OAuth consent screen**
- Page usage agreements

OAuth consent screen

Choose how you want to configure and register your app, including your target users. You can only associate one app with your project.

User Type

☐ Internal ?

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☐ External ?

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

CREATE

При мен позволява само External:

App information

This shows in the consent screen, and helps end users know who you are and contact you

App name *

Employee Management

The name of the app asking for consent

User support email *

For users to contact you with questions about their consent

App logo

BROWSE

Upload an image, not larger than 1MB on the consent screen that will help users recognize your app. Allowed image formats are JPG, PNG, and BMP. Logos should be square and 120px by 120px for the best results.

Задължителното поле само е име “Employee Management” – важно е да запомним, че това е, което потребителят ще вижда и не е необходимо да съвпада с името на проекта.

Надолу има изискване за имейл на developer – оставих моя.

Edit app registration

Your non-sensitive scopes

API ↑	Scope	User-facing description
No rows to display		

Your sensitive scopes

Sensitive scopes are scopes that request access to private user data.

API ↑	Scope	User-facing description
No rows to display		

Your restricted scopes

Restricted scopes are scopes that request access to highly sensitive user data.

OAuth consent screen

pragim@gmail.com

Scopes for Google APIs

Scopes allow your application to access your user's private data. [Learn more.](#)

If you add a sensitive scope, such as scopes that give you full access to Gmail or Drive, Google will verify your consent screen before it's published.

email

profile

openid

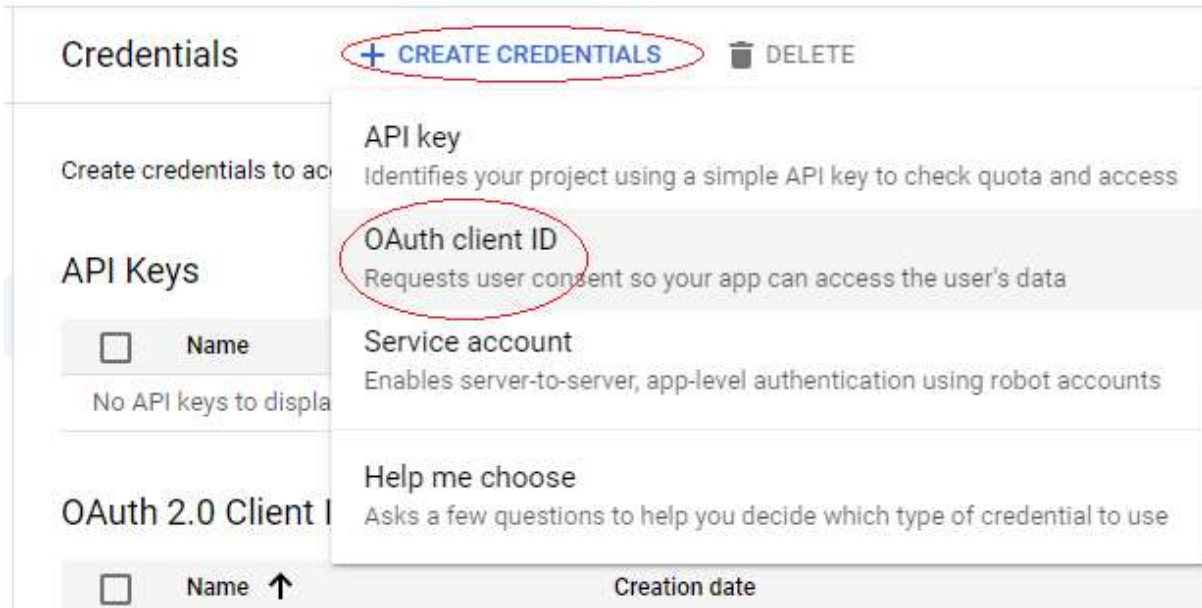
Scopes are a type or level of authenticated user data that we want to allow Google to share with the client application.

В старата версия се виждат предложения за 3 Scope: email, profile, openID.

После при мен има Test Users, където е вписан моят имейл.

Може да оставим празни -> Save

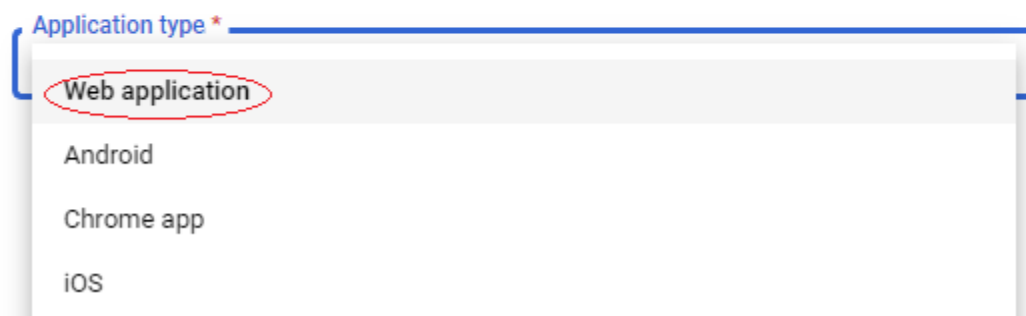
Последна стъпка е Credentials:



Create credentials -> Oath client ID

← Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.



Name: "Employee Mgmt Client"

Application type *
Web application

Name *
Employee Mgmt Client

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.



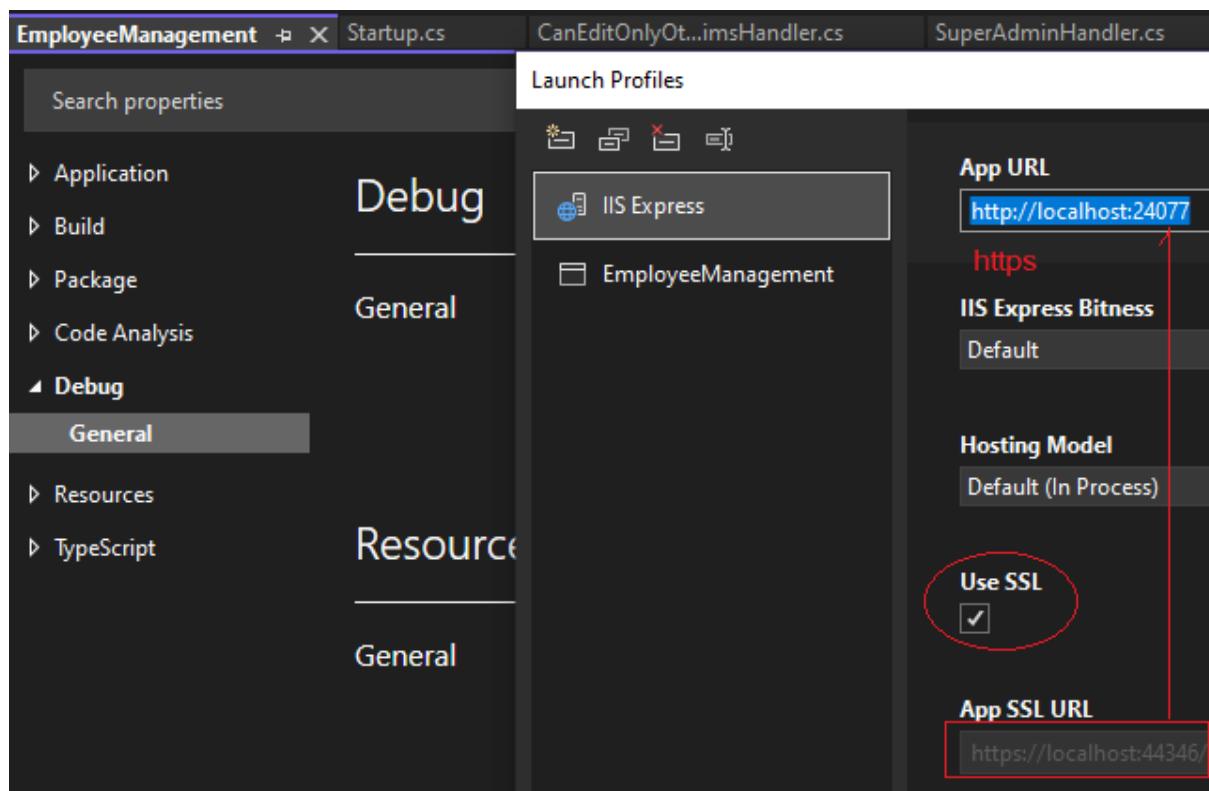
The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins ?

For use with requests from a browser

+ ADD URI

Към момента, приложението е хостнато на локалната машина. За да вземем URL: д.б. върху Employee Management проекта -> Properties -> Debug ->



Много е важно да се маркира Use SSL, защото към момента без него използваме само http, а https е много по-сигурен. Копираме адреса от App SSL URL, който е httpS -> Paste App URL.

Поставяме го също в JavaScript origins: <https://localhost:44346>

Authorized JavaScript origins

For use with requests from a browser

URIs 1 *

[+ ADD URI](#)

Authorized redirect URIs

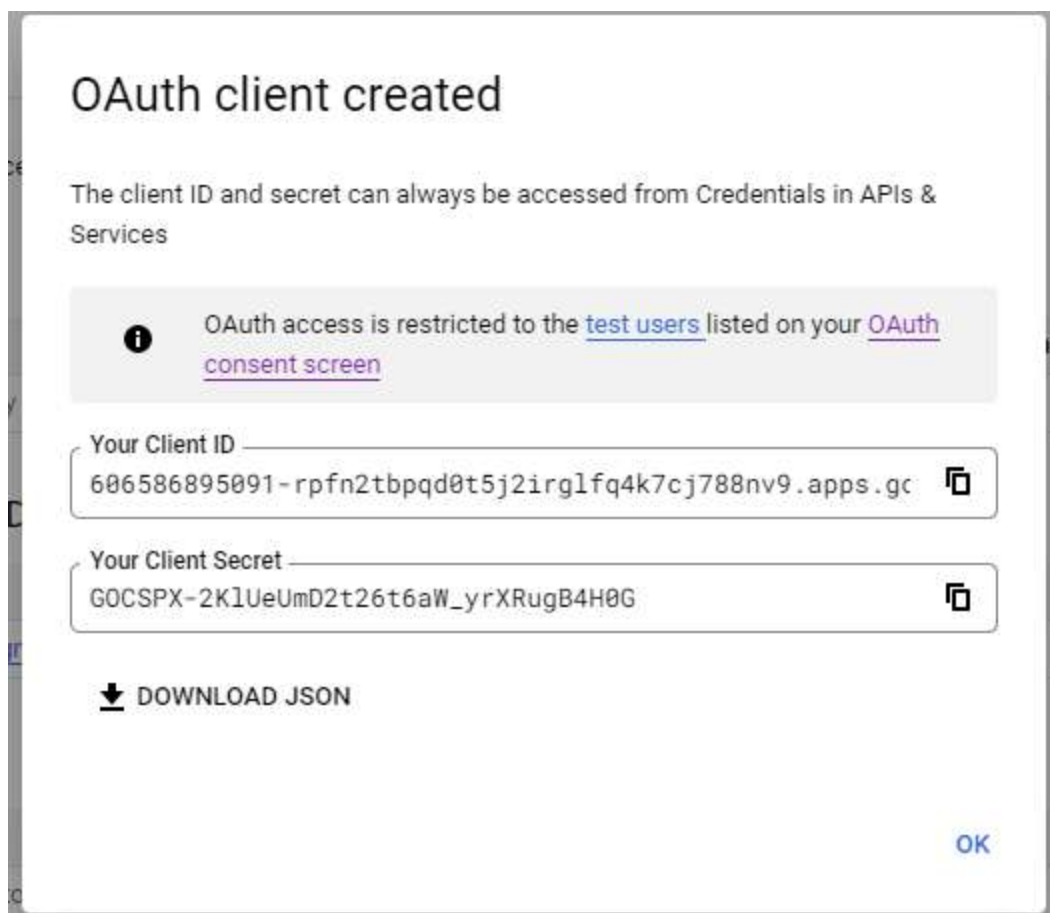
For use with requests from a web server

URIs 1 *

[+ ADD URI](#)

Последно, задаваме адрес в Authorized redirect URIs, което е пренасочването към сайта при успешно вписване в Google. По подразбиране това е адреса на хостване [/signin-google](#).

Може да се смени директорията по избор -> Create.



Готово – вече разполагаме с клиентски ID + Secret, които са необходими за регистрация на нашето приложение в Google. Как ще го направим – в следващия урок.

Create Google OAuth Credentials

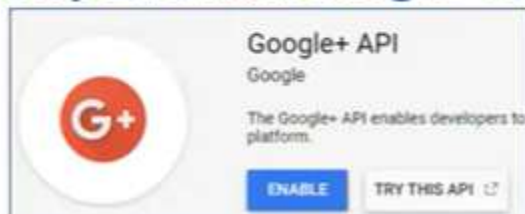
Register Application

<https://console.developers.google.com>

Step 1 : Create a Project

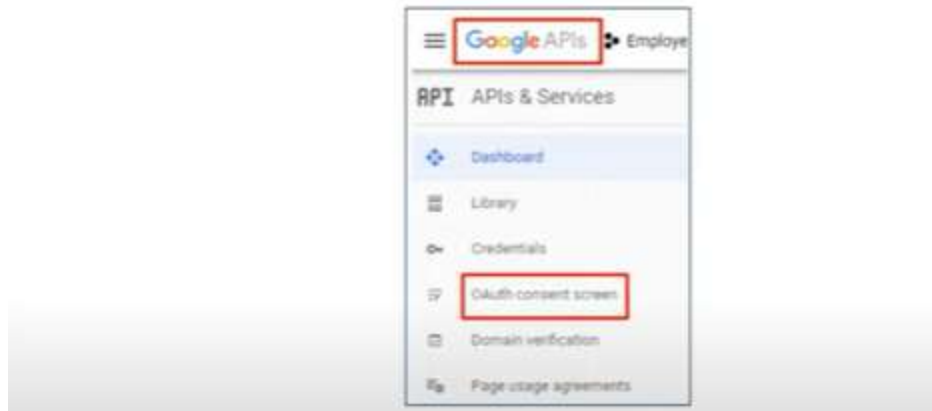


Step 2 : Enable Google+ API



Create Google OAuth Credentials

Step 3 : Configure OAuth consent screen



Create Google OAuth Credentials

Step 4 : Create OAuth client credentials

For applications that use the OAuth 2.0 protocol to call Google APIs, you can use an OAuth 2.0 client ID to generate an access token. The token contains a unique identifier. See [Setting up OAuth 2.0](#) for more information.

Application type

- ☒ Web application **1**
- ☐ Android [Learn more](#)
- ☐ Chrome App [Learn more](#)
- ☐ iOS [Learn more](#)
- ☐ Other

Name **2**

Restrictions

Enter JavaScript origins, redirect URIs, or both [Learn More](#)

Origins and redirect domains must be added to the list of Authorized Domains in the [OAuth consent settings](#).

Authorized JavaScript origins

For use with requests from a browser. This is the origin URI of the client application. It can't contain a wildcard ([https://*](#) example.com) or a path ([https://example.com/subdir](#)). If you're using a nonstandard port, you must include it in the origin URI.

3

Type in the domain and press [Add](#) to add it.

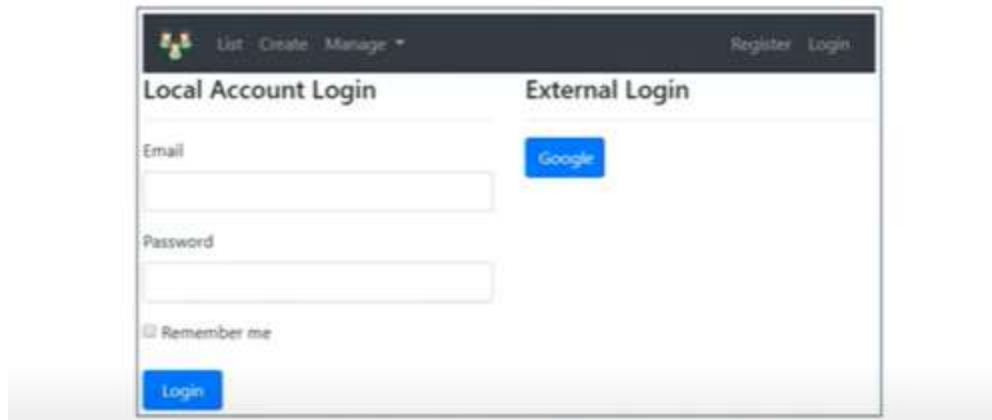
Authorized redirect URIs

For use with requests from a web server. This is the path in your application that users are redirected to after they have authenticated with Google. The path will be appended with the authorization code for access. Must have a protocol. Cannot contain URI fragments or relative paths. Cannot be a public IP address.

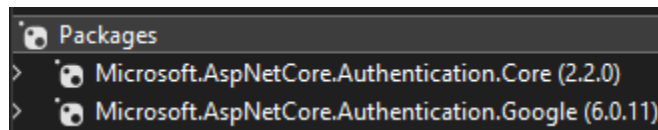
4

106 - ASP NET Core google authentication setting up the UI

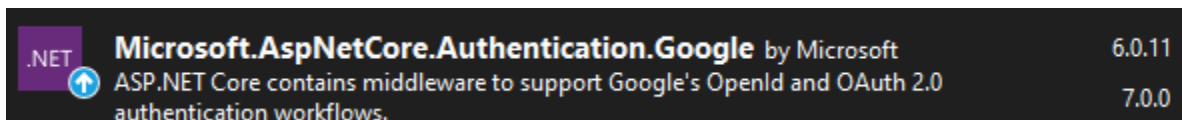
Google Authentication - Setting up the UI



Трябва да инсталираме [Microsoft.AspNetCore.Authentication.Google](#) NuGet package, за да бъде достъпен метод [AddGoogle\(\)](#).



Съответно аналогичен е пакетът за Facebook, Twitter, Microsoft. Най-добре да се инсталира направо [Microsoft.AspNetCore.Authentication.Core](#), която след версия 2.2 би трябвало да ги съдържа, но практически не ми разпозна метод `.AddGoogle` докато не инсталирах по-старата версия `.Google 6.0`, защото последната 7.0 не ми разрешава.



<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/social/?view=aspnetcore-7.0&tabs=visual-studio>

```
services.AddAuthentication()  
    .AddGoogle(options =>  
    {  
        options.ClientId = "606586895091-rpfnt2btpqd0t5j2irglfq4k7cj788nv9.apps.googleusercontent.com";  
        options.ClientSecret = "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";  
    });  
services.ConfigureApplicationCookie(options =>
```

```
    services.AddAuthentication()  
        .AddGoogle(options =>  
        {  
            options.ClientId = "606586895091-  
rpfnt2btpqd0t5j2irglfq4k7cj788nv9.apps.googleusercontent.com";  
            options.ClientSecret = "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";  
        });
```

Credentials-> Edit ->

Name	Creation date	Type	Client ID	Actions
☐ Employee Mgmt Client	Nov 24, 2022	Web application	606586895091-rpf...	

Name *
Employee Mgmt Client
The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Client ID
606586895091-rpfm2tbpqd0t5j2rplfq4k7cj798nv9.apps.googleusercontent.com
Client secret
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
Creation date
November 24, 2022 at 6:13:17 PM GMT+2

Следваща стъпка е да добавим възможност за външно вписване в изгледа и за целта трябва да добавим нови свойства в модела:

```

LoginViewModel.cs | Startup.cs | SuperAdminHandler.cs
- EmployeeManagement.ViewModels.LoginViewModel | ExternalLogins
4 references
public bool RememberMe { get; set; }

0 references
public string ReturnUrl { get; set; }

0 references
public IList<AuthenticationScheme> ExternalLogins { get; set; }

```

```
public string ReturnUrl { get; set; }
```

```
public IList<AuthenticationScheme> ExternalLogins { get; set; }
```

ReturnUrl съхранява адреса, за да може да се върне потребителят на него.

ReturnUrl is the URL the user was trying to access before authentication. We preserve and pass it between requests using **ReturnUrl** property, so the user can be redirected to that URL upon successful authentication.

ExternalLogins property stores the list of external logins (like Facebook, Google etc) that are enabled in our application.

AuthenticationScheme is in Microsoft.AspNetCore.Authentication namespace

Следва да попълним тези 2 нови properties преди да предоставим този LoginViewModel.

AccountController -> Login action.

```
AccountController.cs* CanEditOnlyOt...imsHa
agement Employee
[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Login()
{
    return View();
}
```

Подаваме променлива returnUrl:

```
EmployeeManagement.Controllers.AccountControle Login(string returnUrl)
[HttpGet]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(string returnUrl)
{
    LoginViewModel model = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
    };

    return View(model);
}
```

Google Authentication - Setting up the UI



Искаме да разделим страницата на 2, съответно намаляваме `<div class="col-md-12">` на 6.

Демонстративно:

```
AccountController.cs Login.cshtml* CanEditOnlyOt...imsHandler.cs
@model LoginViewModel

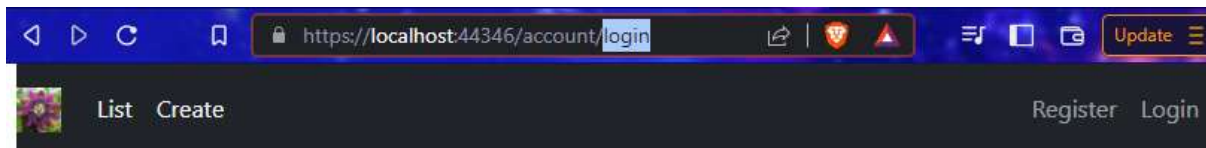
@{
    ViewBag.Title = "User Login";
}

<div class="row">
    <div class="col-md-6">_</div>

    <div class="col-md-6">
        <h1>External Login</h1>
        <hr />
        @{
            if (Model.ExternalLogins.Count == 0)
            {
                <div>No external login configured</div>
            }
        }
    </div>
</div>
```

Закоментираме:

```
EmployeeManagement.Startup Conf
//services.AddAuthentication()
//    .AddGoogle(options =>
//    {
//        options.ClientId = "606586895091-rpfn2tbppqd0t5j2irg
//        options.ClientSecret = "GOCSPX-2K1UeUmD2t26t6aW_yrX
//    });
```



Local Account Login

Email

Password

☐ Remember me

Login

External Login

No external login configured

Разкоментираме

```
Login.cshtml* Startup.cs* LoginViewModel.cs
<div class="col-md-6">
  <h1>External Login</h1>
  <hr />
  @{
    if (Model.ExternalLogins.Count == 0)
    {
      <div>No external login configured</div>
    }
    else
    {
      to do in AccountController to pass it as incoming var
      <form method="post" asp-action="ExternalLogin" asp-route-returnUrl="@Model.ReturnUrl">
        <div>
          @foreach (var provider in Model.ExternalLogins) we have only one provider yet Google
          {
            <button type="submit" class="btn btn-primary"
              name="provider" value="@provider.Name"
              title="Login using your @provider.DisplayName account">
              @provider.DisplayName
            </button>
          }
        </div>
      </form>
    }
  }
</div>
```

```
<div class="col-md-6">
  <h1>External Login</h1>
  <hr />
  @{
    if (Model.ExternalLogins.Count == 0)
    {
      <div>No external login configured</div>
    }
    else
    {
```

```

        <form method="post" asp-action="ExternalLogin" asp-route-
returnUrl="@Model.ReturnUrl">
        <div>
            @foreach(var provider in Model.ExternalLogins)
            {
                <button type="submit" class="btn btn-primary"
                    name="provider" value="@provider.Name"
                    title="Login using your @provider.DisplayName
account">
                    @provider.DisplayName
                </button>
            }
        </div>
    </form>
}
}
</div>

```

External Login

Google

Login using your Google account

+ Tool Tip -> <https://localhost:44346/account/ExternalLogin>

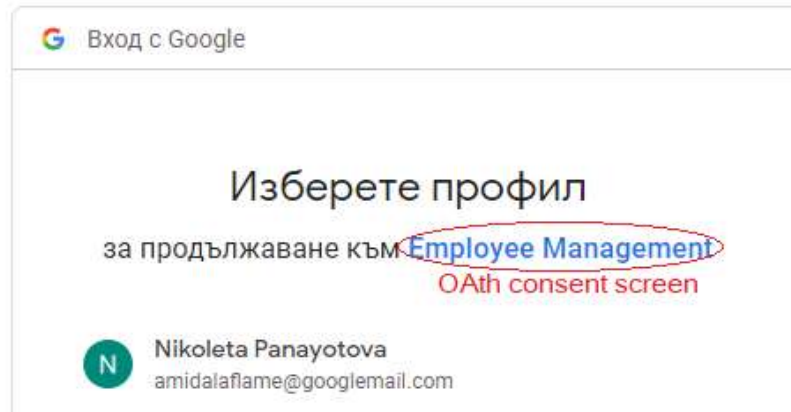
Ocrava ExternalLogin action -> AccountController

```

AccountController.cs  LoginViewModel.cs
EmployeeManagement.Controllers.AccountController ExternalLogin(string provider)
[AllowAnonymous]
[HttpPost]
0 references
public IActionResult ExternalLogin(string provider, string returnUrl)
{
    <button name="provider" MVC auto binding
    to do
    var redirectUrl = Url.Action("ExternalLoginCallback", "Account",
        new { ReturnUrl = returnUrl });
    var properties = signInManager
        .ConfigureExternalAuthenticationProperties(provider, redirectUrl);
    return new ChallengeResult(provider, properties);
}

```

accounts.google.com/o/oauth2/v2/auth/oauthchooseaccount?response_type=code&client_id=60658689



<https://localhost:44346/Account/ExternalLoginCallback> -> 404

В следващия урок ще видим имплементирането му.

Configure Google Authentication

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication().AddGoogle(options =>
    {
        options.ClientId = "XXXXX";
        options.ClientSecret = "YYYYY";
    });
}
```

LoginViewModel Changes

```
public class LoginViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    public bool RememberMe { get; set; }

    public string returnUrl { get; set; }

    public IList<AuthenticationScheme> ExternalLogins { get; set; }
}
```

Login Action in AccountController

```
[HttpGet]
[AllowAnonymous]
public async Task<IActionResult> Login(string returnUrl)
{
    LoginViewModel model = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
    };

    return View(model);
}
```

Login View

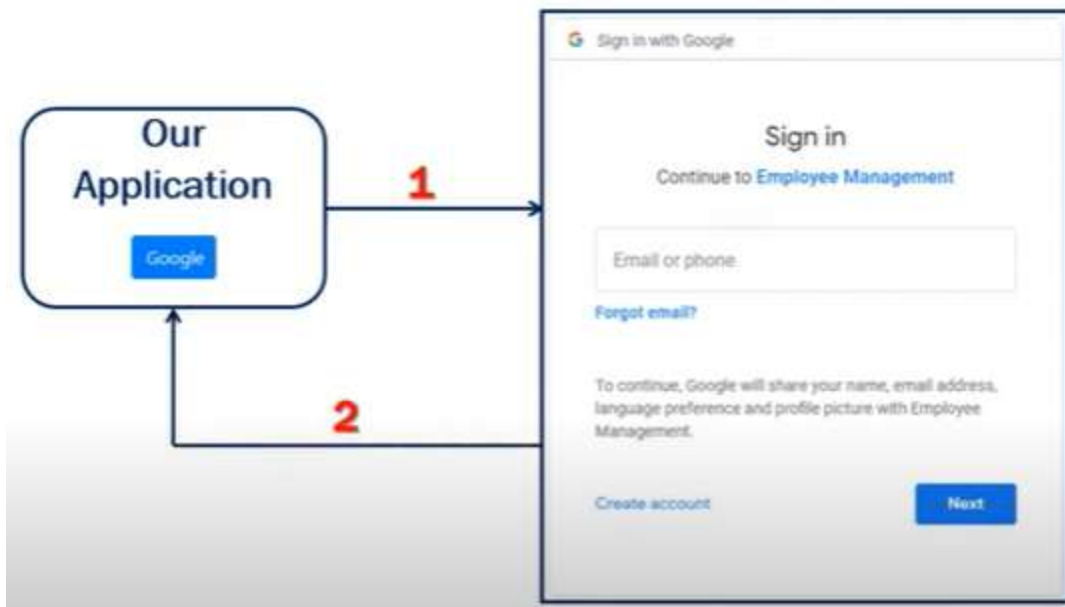
```
<div class="col-md-6">
    <h1>External Login</h1><hr />
    @{
        if (Model.ExternalLogins.Count == 0)
        {
            <div>No external logins configured</div>
        }
        else
        {
            <form method="post" asp-action="ExternalLogin" asp-route-returnUrl="@Model.ReturnUrl">
                <div>
                    @foreach (var provider in Model.ExternalLogins)
                    {
                        <button type="submit" class="btn btn-primary"
                            name="provider" value="@provider.Name"
                            title="Log in using your @provider.DisplayName account">
                            @provider.DisplayName
                        </button>
                    }
                </div>
            </form>
        }
    }
</div>
```

ExternalLogin action - AccountController

```
[AllowAnonymous]
[HttpPost]
public IActionResult ExternalLogin(string provider, string returnUrl)
{
    var redirectUrl = Url.Action("ExternalLoginCallback", "Account",
        new { ReturnUrl = returnUrl });
    var properties = signInManager
        .ConfigureExternalAuthenticationProperties(provider, redirectUrl);
    return new ChallengeResult(provider, properties);
}
```

107 - ExternalLoginCallback action

How External Identity Providers Work



```
AccountController.cs  LoginViewModel.cs
EmployeeManagement.Controllers.AccountControll... ExternalLoginCallback(string returnUrl = null, string

[AllowAnonymous]
public async Task<IActionResult>
{
    0 references
    ExternalLoginCallback(string returnUrl = null, string remoteError = null)
    {
        if returnUrl is null, we initialize it to root URL
        returnUrl = returnUrl ?? Url.Content("~/");

        LoginViewModel loginViewModel = new LoginViewModel
        {
            otherwise it will have the value of the incoming returnUrl
            ReturnUrl = returnUrl,
            ExternalLogins =
                (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
        };

        we check the second incoming parameter:
        if (remoteError != null) we received some error
        {
            ModelState
                .AddModelError(string.Empty, $"Error from external provider: {remoteError}");
            redirect back to + user can see the error received from the external provider
            return View("Login", loginViewModel);
        }
    }
}
```

```
// Get the login information about the user from the external login provider
var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null) we don't have info
{
    ModelState add this error message to the ModelState
    .AddModelError(string.Empty, "Error loading external login information.");

    return View("Login", loginViewModel); redirect to Login + error message to user
}
```

```
signInManager.GetExternalLoginInfoAsync();
```

(awaitable) Task<ExternalLoginInfo> SignInManager<ApplicationUser>.GetExternalLoginInfoAsync([string expectedXsrf = null])
Gets the external login information for the current login, as an asynchronous operation.

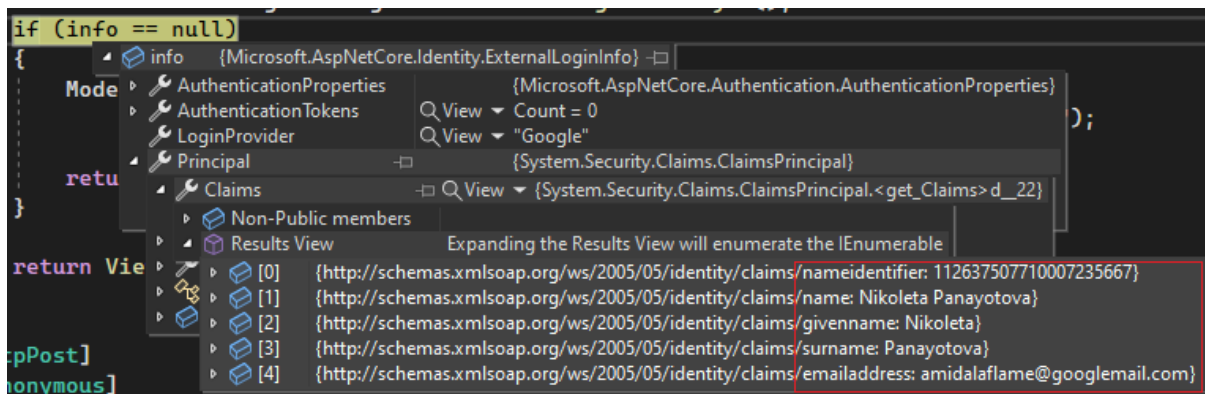
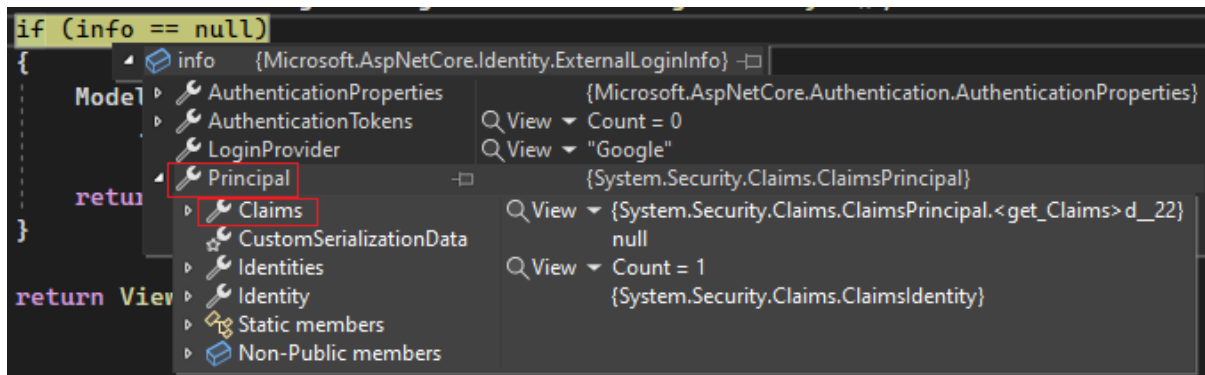
Обръщаме внимание, че от Intellisense, метод `.GetExternalLoginInfoAsync()` връща обект от тип `<ExternalLoginInfo>`. За да разберем какво означава резултат от такъв тип, нека поставим BreakPoint.

```
110 [AllowAnonymous]
111 public async Task<IActionResult>
112     ExternalLoginCallback(string returnUrl = null, string remoteError = null)
113     {
114         returnUrl ??= returnUrl ?? Url.Content("~/");
115
116         LoginViewModel loginViewModel = new LoginViewModel();
117
118         if (remoteError != null)
119         {
120             // Get the login information about the user from the external login provider
121             var info = await signInManager.GetExternalLoginInfoAsync();
122             if (info == null)
123             {
124                 ModelState
125                     .AddModelError(string.Empty, "Error loading external login information.");
126
127                 return View("Login", loginViewModel);
128             }
129             to fix this 1 x error add this:
130             return View("Login", loginViewModel); in general section temporarily
131         }
132     }
```

При успешно вписване, имаме пълен набор информация за потребителя:

```
133 if (info == null)
134 {
135     info {Microsoft.AspNetCore.Identity.ExternalLoginInfo}
136     Mod
137     AuthenticationProperties {Microsoft.AspNetCore.Authentication.AuthenticationProperties}
138     AuthenticationTokens Q View Count = 0
139     LoginProvider Q View "Google"
140     Principal {System.Security.Claims.ClaimsPrincipal}
141     ProviderDisplayName Q View "Google"
142     ProviderKey Q View "112637507710007235667"
```

ProviderKey е уникална стойност за дадения потребител от External login provider.



Logout amidalaflame@gmail.com



Ако нямаме успешно вписване, таблицата ще е празна, но аз направих едно предварително.

dbo.AspNetUserLogins [Data] dbo.AspNetUse...gins [Design] Log				
	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Google	1126375077100...	Google	84eb9e06-25b7...
	NULL	NULL	NULL	NULL

UserId from AspNetUserLogins таблицата ще кореспондира автоматично с Id на AspNetUsers и ще я попълни, така че независимо колко локални или външни вписвания има един потребител, той да е уникален.

dbo.AspNetUsers [Data] dbo.AspNetUsers [Design] dbo.AspNetUserLogins [Data]							
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	Passwor
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAA
	84eb9e06-25b7-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	NULL
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False	AQAAAA
	a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV....	george@abv.bg	GEORGE@ABV....	False	AQAAAA
	b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@ABV.BG	False	AQAAAA
	b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False	AQAAAA
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False	AQAAAA
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False	AQAAAA
	f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON@ABV.BG	False	AQAAAA
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Ако този External Login info object is not null, that means we have received login information from the external login provider (Google) we want to sign the user in using this provider so for that we are using this .ExternalSignInAsync method of the signInManager service. And to this method we pass the LoginProvider. Remember: on the external login for objective log and provider property returns our current external login provider and we have seen that value is Google at the moment. And then the ProviderKey, which is the unique identifier for the user, provided by the external login provider (in this case Google). And then we want to create a persistent cookie and if we want to bypassTwoFactor authentication. For this ExternalLoginSignInAsync method to succeed, we have to have a corresponding record in aspNetUserLogins table.

```
// If the user already has a login (i.e if there is a record in AspNetUserLogins
// table) then sign-in the user with this external login provider pass to
var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
    userId      cookie      authentication
if (signInResult.Succeeded) {
    // Existing record in db
    return LocalRedirect(returnUrl);
}
```

For a given LoginProvider, with a given ProviderKey, if we do not have a corresponding row in this AspNetLogins table, then most likely this external user might not have a corresponding local user account and local user accounts are in AspNetUserLogins table. So the row in aspNetUserLogins table is linked to a row in aspNetUsers table using this UserId column.

Ако нямаме запис на външен акаунт в AspNetLogins, по всяка вероятност липсва и кореспондиращ акаунт на локален потребител в AspNetUsers. Тогава няма да знаем кой е външният потребител и метод ExternalLoginSignInAsync няма да е Succeeded и ще търсим друго решение.


```

UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
Email = info.Principal.FindFirstValue(ClaimTypes.Email)

```

```

// If there is no record in AspNetUserLogins table, the user may not have
// a local account
else check is there a local account?
{
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);

    if (email != null) there is existing email
    {
        the external user has a local account
        var user = await userManager.FindByEmailAsync(email);
        add a row in UserLogins.dbo
        await userManager.AddLoginAsync(user, info);
        await signInManager.SignInAsync(user, isPersistent: false);
        then sign in the user
        return LocalRedirect(returnUrl);
        and redirect him
    }
}
return View("Error");

```

have

```

// If there is no record in AspNetUserLogins table, the user may not
// a local account
else
{
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);

    if (email != null)
    {
        var user = await userManager.FindByEmailAsync(email);

        await userManager.AddLoginAsync(user, info);
        await signInManager.SignInAsync(user, isPersistent: false);

        return LocalRedirect(returnUrl);
    }
}
return View("Error");

```



```

else
{
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);

    if (email != null)
    {
        var user = await userManager.FindByEmailAsync(email);
        // insert

        if (user == null) // create a new local account for the external user
        {
            user = new ApplicationUser // new instance of ApplicationUser class
            {
                // populate properties:
                UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            }; // new record in AspNetUsers table
            await userManager.CreateAsync(user);
        }

        await userManager.AddLoginAsync(user, info);
        await signInManager.SignInAsync(user, isPersistent: false);

        return LocalRedirect(returnUrl);
    }
}

```

```

        if (user == null)
        {
            user = new ApplicationUser
            {
                UserName =
info.Principal.FindFirstValue(ClaimTypes.Email),
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            };
            await userManager.CreateAsync(user);
        }

```

```

else
{
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);

    if (email != null) ...

    // If we cannot find the user email we cannot continue
    ViewBag.ErrorTitle = $"Email claim not received from: {info.LoginProvider}";
    ViewBag.ErrorMessage = "Please contact support on support@abv.bg";

    return View("Error");
}
}

```

// If we cannot find the user email we cannot continue

```

        ViewBag.ErrorTitle = $"Email claim not received from:
{info.LoginProvider}";
        ViewBag.ErrorMessage = "Please contact support on support@abv.bg";

```

Full code:

```

[AllowAnonymous]
public async Task<IActionResult>
ExternalLoginCallback(string returnUrl = null, string remoteError =
null)
{
    returnUrl = returnUrl ?? Url.Content("~/");

    LoginViewModel loginViewModel = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await
signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };

    if (remoteError != null)
    {
        ModelState
            .AddModelError(string.Empty, $"Error from external provider:
{remoteError}");

        return View("Login", loginViewModel);
    }

    // Get the login information about the user from the external login
provider
    var info = await signInManager.GetExternalLoginInfoAsync();
    if (info == null)
    {
        ModelState
            .AddModelError(string.Empty, "Error loading external login
information.");

        return View("Login", loginViewModel);
    }

    // If the user already has a login (i.e if there is a record in
AspNetUserLogins
// table) then sign-in the user with this external login provider
    var signInResult = await
signInManager.ExternalLoginSignInAsync(info.LoginProvider,
info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

    if (signInResult.Succeeded)
    {
        return LocalRedirect(returnUrl);
    }

    // If there is no record in AspNetUserLogins table, the user may not
have
    // a local account
    else
    {
        var email = info.Principal.FindFirstValue(ClaimTypes.Email);

```

```

        if (email != null)
        {
            var user = await userManager.FindByEmailAsync(email);

            if (user == null)
            {
                user = new ApplicationUser
                {
                    UserName =
info.Principal.FindFirstValue(ClaimTypes.Email),
                    Email = info.Principal.FindFirstValue(ClaimTypes.Email)
                };
                await userManager.CreateAsync(user);
            }

            await userManager.AddLoginAsync(user, info);
            await signInManager.SignInAsync(user, isPersistent: false);

            return LocalRedirect(returnUrl);
        }

        // If we cannot find the user email we cannot continue
        ViewBag.ErrorTitle = $"Email claim not received from:
{info.LoginProvider}";
        ViewBag.ErrorMessage = "Please contact support on support@abv.bg";

        return View("Error");
    }
}

```

Debug -> Google ->

```

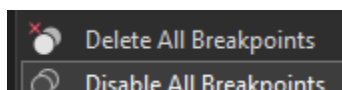
EmployeeManagement
EmployeeManagement.Controllers.AccountController
ExternalLoginCallback(string returnUrl = null,
132 var info = await signInManager.GetExternalLoginInfoAsync();
133 if (info == null)
134 {
135     ModelState
136     .AddModelError(string.Empty, "Error loading external login information.");
137
138     return View("Login", loginViewModel);
139 }
140 // If the user already has a login (i.e if there is a record in AspNetUserLogins
141 // table) then sign-in the user with this external login provider
142 var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
143 info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
144 null, because signing in with Google Provider + unique ProviderKey failed = we don't have a corresponding row in
145 Logins table if (signInResult.Succeeded)
146 {
147     return LocalRedirect(returnUrl);
148 }
149 // If there is no record in AspNetUserLogins table, the user may not have
150 // a local account
151 else
152 {
153     var email = info.Principal.FindFirstValue(ClaimTypes.Email);
154
155     if (email != null)
156     {
157         var user = await userManager.FindByEmailAsync(email);

```

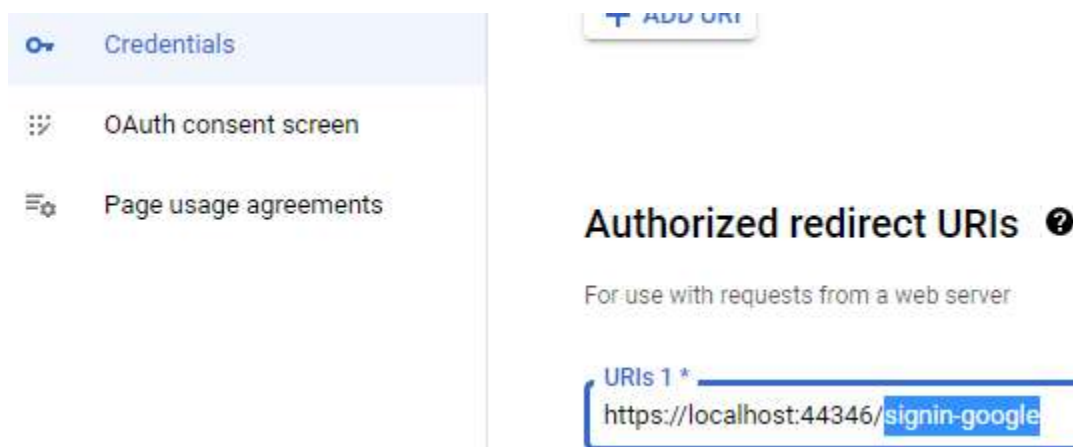
Обръщаме внимание, че паролата не се запазва при нас – тя е от външен потребител:

dbo.AspNetUsers [Data]						
	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencySt...
	aurora@abv.bg	AURORA@ABV...	False	AQAAAAEAAC...	FBRWUISYMBN...	f3a1d546-ac1d-...
▶	amidalafame@...	AMIDALAFLAM...	False	NULL	YKUHOK2WDO...	7baf363a-9021-...
	sara@abv.bg	SARA@ABV.BG	False	AQAAAAEAAC...	73VPVG65F55LE...	84ff0168-c281-...

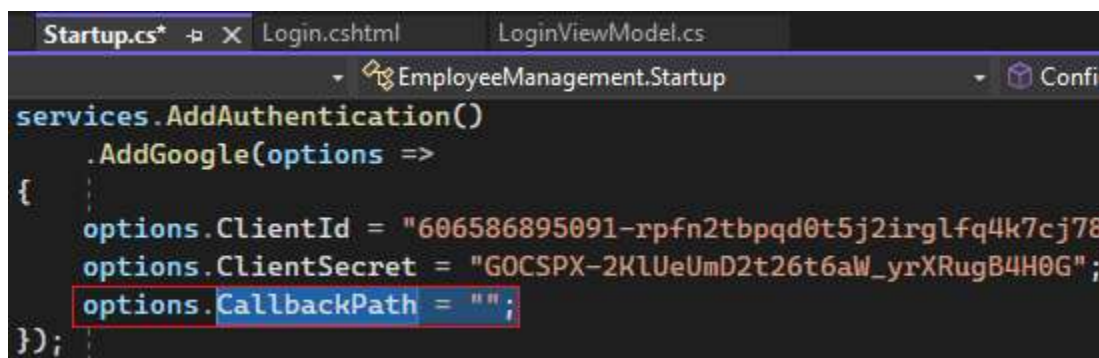
Имаме възможност за Debug -> Disable All BreakPoints:



По подразбиране, Callback URL: <https://localhost:44346/signin-google>



Ако искаме да го сменим, това става от Startup:



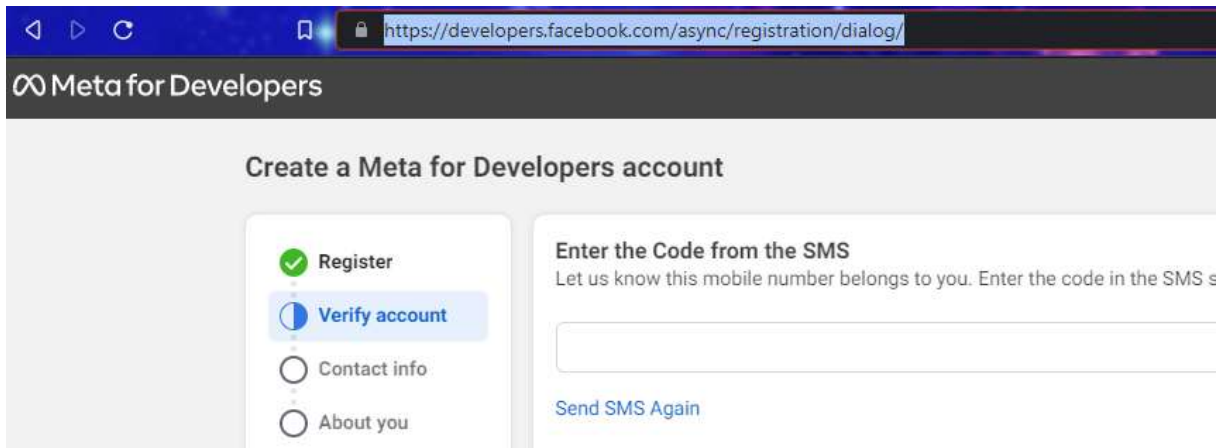
`options.CallbackPath = "";`

108 - Register application with facebook

Register Application with Facebook

<https://developers.facebook.com>

<https://developers.facebook.com/>



The screenshot shows a web browser window with the address bar displaying `https://developers.facebook.com/async/registration/dialog/`. The page header is "Meta for Developers". The main heading is "Create a Meta for Developers account". On the left, there is a vertical list of steps: "Register" (selected with a green checkmark), "Verify account" (highlighted in blue), "Contact info", and "About you". On the right, under the heading "Enter the Code from the SMS", there is a text input field and a "Send SMS Again" link.

Meta for Developers

Create a Meta for Developers account

- ☒ Register
- ☐ Verify account
- ☐ Contact info
- ☐ About you

Enter the Code from the SMS

Let us know this mobile number belongs to you. Enter the code in the SMS s

[Send SMS Again](#)



No apps yet

To get started, create your first app.

[Create App](#)

https://developers.facebook.com/apps/create/

DocsToolsSupportMy AppsSearch developer documentation

TypeDetails

Select an app type

The app type can't be changed after your app is created. [Learn more](#)

Business

Create or manage business assets like Pages, Events, Groups, Ads, Messenger, WhatsApp, and Instagram Graph API using the available business permissions, features and products.

Consumer

Connect consumer products and permissions, like Facebook Login and Instagram Basic Display to your app.

Instant Games

Create an HTML5 game hosted on Facebook.

Gaming

Connect an off-platform game to Facebook Login.

Workplace

Create enterprise tools for Workplace from Meta.

Academic research

Connect to Facebook data and tooling to perform research on Facebook.

None

Create an app with combinations of consumer and business permissions and products.

Next

Add an app name

This is the app name that will show on your My Apps page and associated with your app ID. You can change the name later in Settings.

Employee Mgmt Client

20/30

Add an app name

This is the app name that will show on your My Apps page and associated with your app ID. You can change the name later in Settings.

Employee Mgmt Client

20/30

App contact email

This is the email address we'll use to contact you about your app. Make sure it is an address you check regularly. We may contact you about policies, app restrictions or recovery if your app is deleted or compromised.

nikoleta.panayotova@protonmail.com

Business Account - Optional

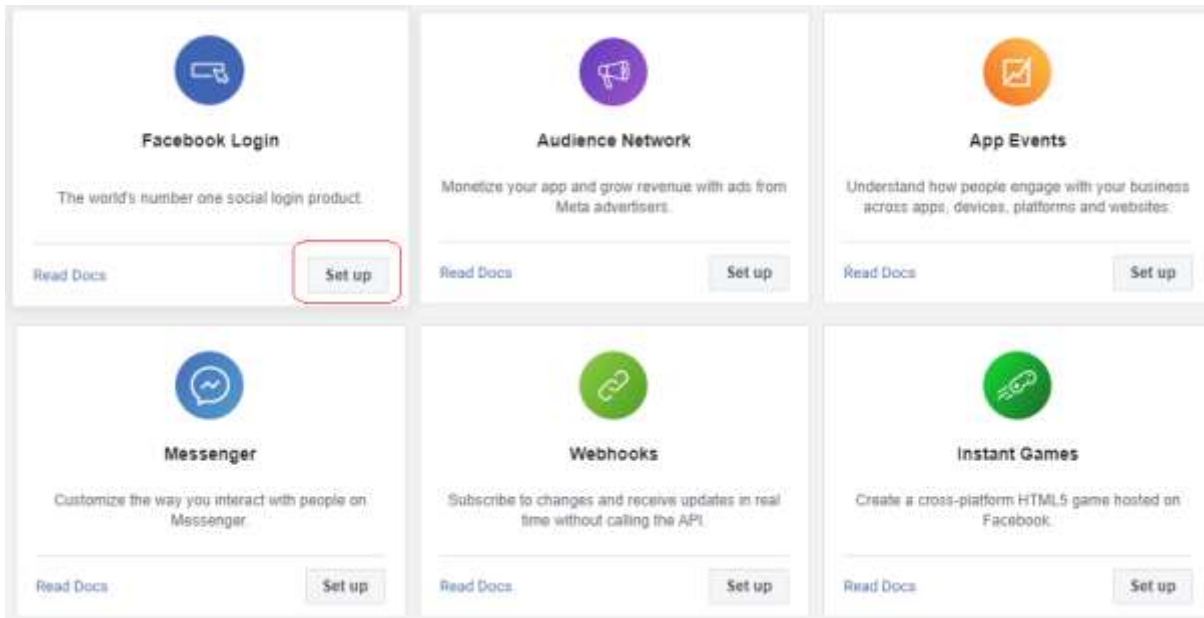
Connecting a Business Account to your app is only required for certain products and permissions. You'll be asked to connect a Business Account when you request access to those products and permissions.

No Business Manager account selected

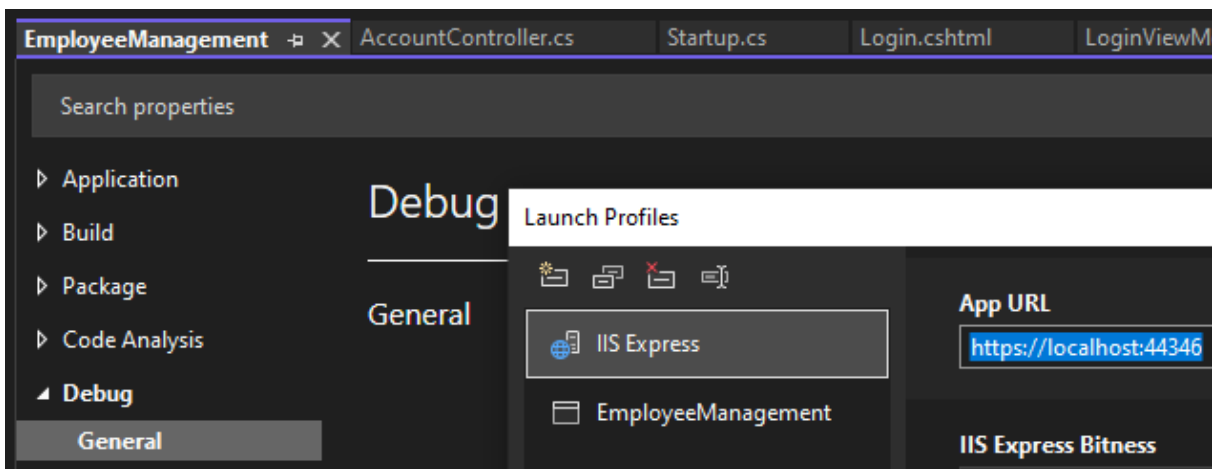
By proceeding, you agree to the [Meta Platform Terms](#) and [Developer Policies](#).

Previous

Create app



Employee Management project -> properties -> Debug ->



<https://localhost:44346/signin-facebook>

Meta for Developers Docs Tools Support My Apps Search developer documentation

Employee Mgmt Client App ID: 3085022948456094 App Mode: Development Live

Dashboard Settings App Roles Alerts App Review Products Add Product

Facebook Login Settings Quickstart Activity Log Activity Log

Client OAuth settings

☒ Client OAuth login
Enables the standard OAuth client token flow. Secure your application and prevent abuse by locking down which known redirect URIs are allowed with the options below. Scorable globally if not used. [?]

☒ Web OAuth login
Enables web-based Client OAuth Login. [?]

☒ Enforce HTTPS
Enforce the use of HTTPS for Redirect URIs and the JavaScript SDK. Strongly recommended. [?]

☐ Force Web OAuth reauthentication
When on, prompts people to enter their Facebook password in order to log in on the web. [?]

☐ Embedded Browser OAuth Login
Enables webview Redirect URIs for Client OAuth Login. [?]

☒ Use Strict Mode for redirect URIs
Only allow redirects that exactly match the valid OAuth Redirect URIs. Strongly recommended. [?]

Valid OAuth Redirect URIs
A manually specified redirect_uri used with Login on the web must exactly match one of the URIs listed here. This URI is also used by the JavaScript SDK for in-app browsers that support popups. [?]

<https://localhost:42384/sign/facebook>
Copy to clipboard

☐ I post from Facebook ☐ I login with the JavaScript SDK

Discard Save Changes

App ID: 3085022948456094 App secret: 13230e5d81901e12d6f38401cf125c35

Display name: Employee Mgmt Client Namespace:

App ID: 3085022948456094

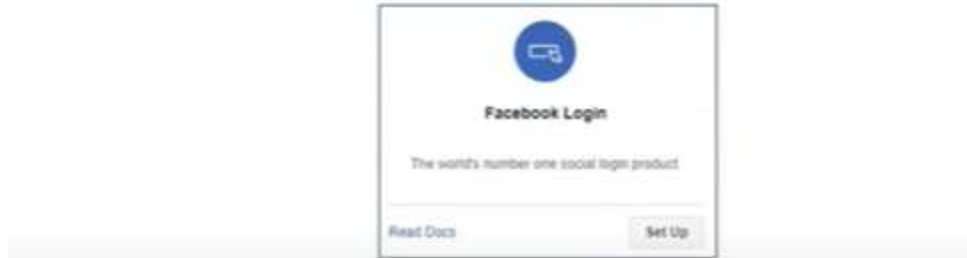
App secret: 13230e5d81901e12d6f38401cf125c35

Register Application with Facebook

<https://developers.facebook.com>



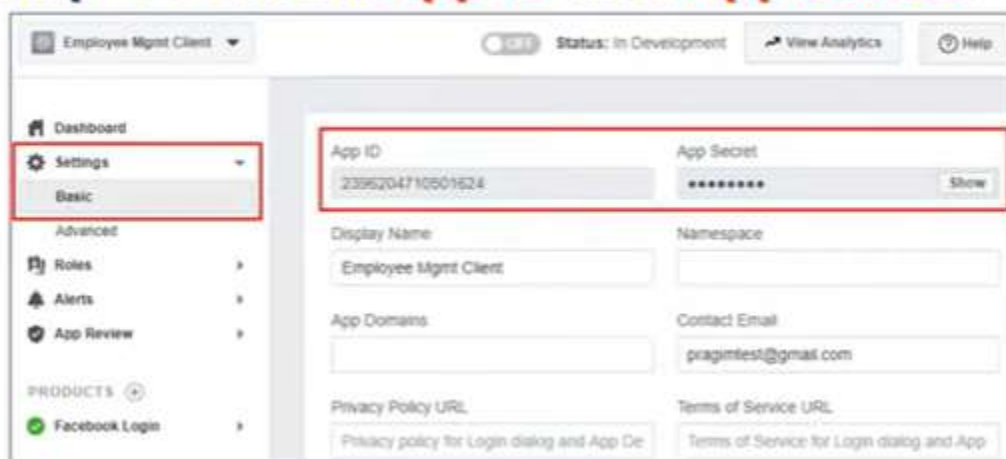
Step 2 : Add Facebook Login Product



Step 3 : Enable Client OAuth Login and include Valid OAuth Redirect URI



Step 4 : Obtain App ID and App Secret



Configure Google Authentication

Part 105 - Registering with Google to obtain **Client Id** and **Secret**

Part 106 - Set up the UI to enable **Google Authentication**

Part 107 - Handling authenticated user information received from **Google**

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication()
        .AddGoogle(options =>
        {
            options.ClientId = "XXXXX";
            options.ClientSecret = "XXXXX";
        });
}
```

Configure Facebook Authentication

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication()
        .AddGoogle(options =>
        {
            options.ClientId = "XXXXX";
            options.ClientSecret = "XXXXX";
        })
        .AddFacebook(options =>
        {
            options.AppId = "XXXXX";
            options.AppSecret = "XXXXX";
        });
}
```

<https://developers.facebook.com/apps/3085022948456094/settings/basic/>

.NET



Microsoft.AspNetCore.Authentication.Facebook by Microsoft, 54M d

6.0.11

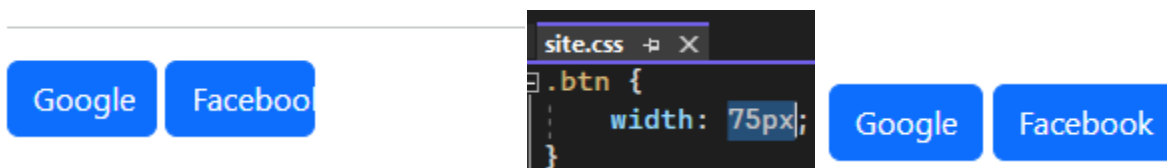
ASP.NET Core middleware that enables an application to support Facebook's OAuth 2.0 authentication workflow.

7.0.0

```
EmployeeManagement.Startup | ConfigureServices(ISe
services.AddAuthentication()
    .AddGoogle(options =>
    {
        options.ClientId = "606586895091-rpfn2tbpqd0t5j2irglfq4k7cj788
        options.ClientSecret = "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";
        //options.CallbackPath = "";
    })
    .AddFacebook(options =>
    {
        options.AppId = "3085022948456094";
        options.AppSecret = "13230e5d81901e12d6f38401cf125c35";
    });
```

```
.AddFacebook(options =>
{
    options.AppId = "3085022948456094";
    options.AppSecret = "13230e5d81901e12d6f38401cf125c35";
});
```

External Login



Бутон Facebook е изрязан, това е заради site.css фиксираната дължина 75px; По-добрият вариант е директно в страницата `style="width:auto"`

```
Startup.cs | Login.cshtml*
@foreach (var provider in Model.ExternalLogins)
{
    <button type="submit" class="btn btn-primary"
        style="width:auto"
        name="provider" value="@provider.Name"
        title="Login using your @provider.DisplayName account">
        @provider.DisplayName
    </button>
}
```

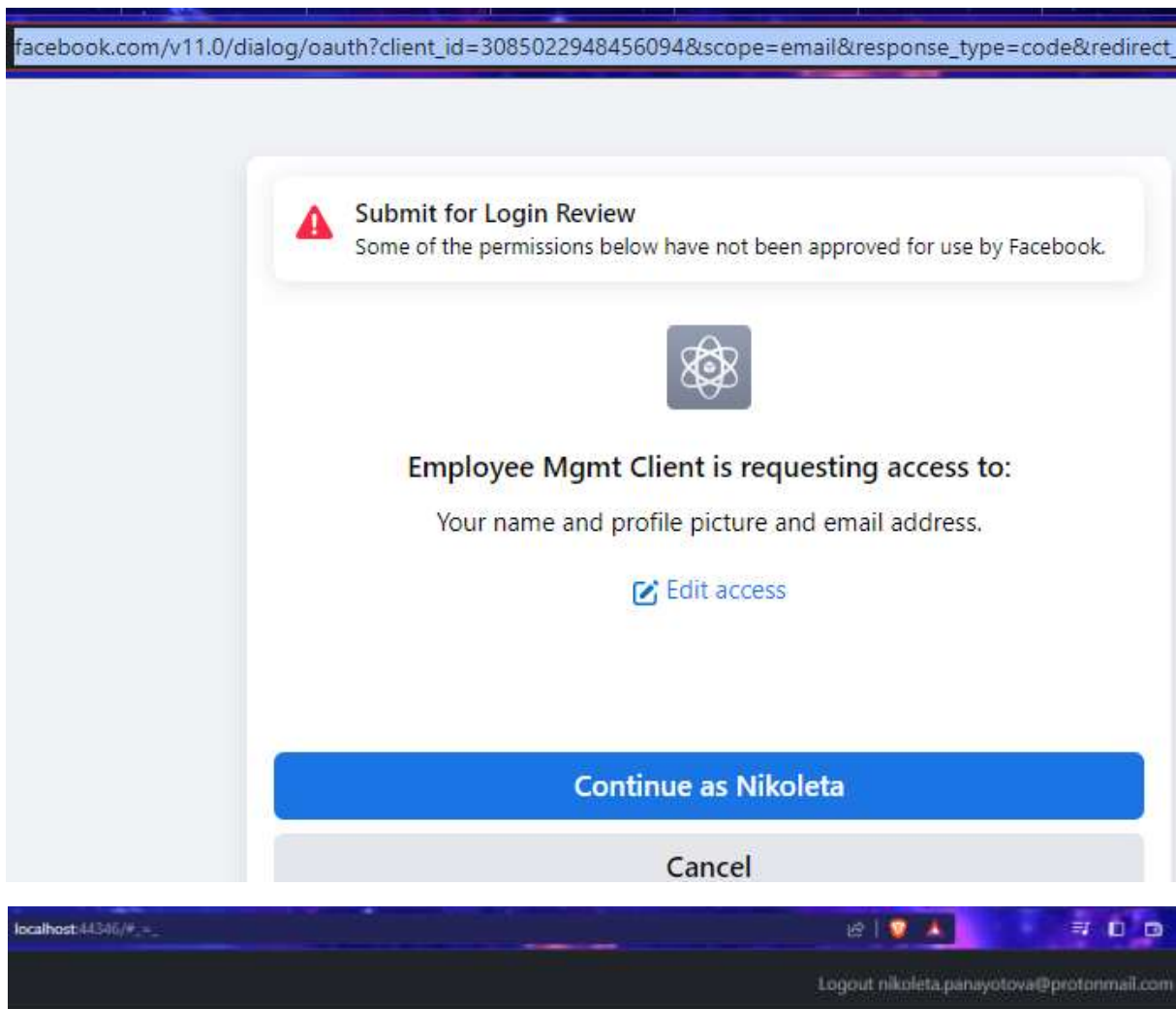
Тестваме AccountController->ExternalLogin + BreakPoint -> Facebook btn

```
EmployeeManagement | EmployeeManagement.Controllers.AccountController | ExternalLogin(string
0 references
101 public IActionResult ExternalLogin(string provider, string returnUrl)
102 {
    provider | View | "Facebook"
```

+ new BreakPoint

```
110 [AllowAnonymous]  
111 public async Task<IActionResult>  
112 ExternalLoginCallback(string returnUrl = null, string remoteError = null)  
113 {
```

Continue – при мен дава генерална грешка.



Успешно вписване с друг имейл и странен URL, тогава вече ме допуска при нов Debug до следващите BreakPoint:

```

133     if (info == null)
134     {
135         ModelState
136             .AddModelError(string.Empty, "Error loading
137
138         return View("Login", loginViewModel);
139     }
140     // If the user already has a login (i.e if there is
141     // table) then sign-in the user with this external
142     var signInResult = await signInManager.ExternalLog
143         info.ProviderKey, isPersistent: false, bypassT
144
145     if (signInResult.Succeeded)
146     {

```

	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Facebook	688753976215688	Facebook	1f032ac4-bc55-...
	Google	1126375077100...	Google	84eb9e06-25b7...
	NULL	NULL	NULL	NULL

Дали ще бъде успешно или не, зависи от UserLogins таблицата.

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash
	519-3a32950ab516	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	False	NULL
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV...	aurora@abv.bg	AURORA@ABV...	False	AQAAAAEAAC...
	84eb9e06-25b7...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	NULL

Външните акаунти са без записана парола

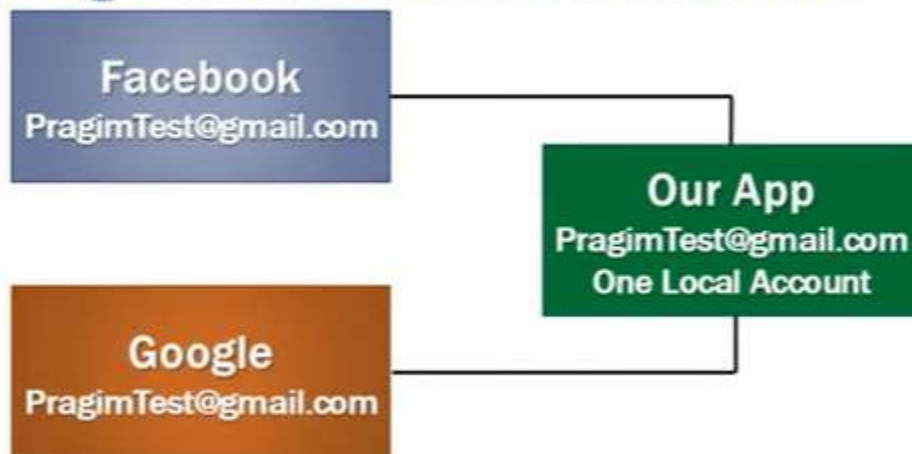
```

152     {
153         var email = info.Principal.FindFirstValue(ClaimTypes.Email);
154
155         if (email != null)
156         {
157             var user = await userManager.FindByEmailAsync(email);
158
159             if (user == null)
160             {
161                 user = new ApplicationUser
162                 {
163                     UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
164                     Email = info.Principal.FindFirstValue(ClaimTypes.Email)
165                 };
166                 await userManager.CreateAsync(user);
167             }
168
169             await userManager.AddLoginAsync(user, info);

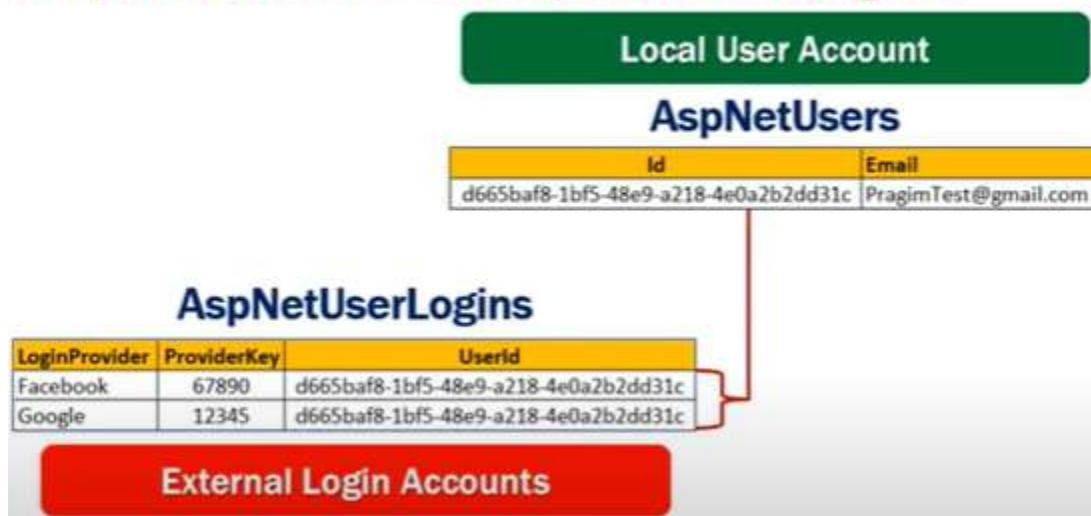
```

Ако имейлът не е намерен, се създава нов профил, което виждаме от таблиците.

Linking External and Local Accounts



AspNetUsers and AspNetUserLogins



Понеже с един и същи имейл е регистриран Google + Facebook акаунт, потребителят е един, с едно ID, съответно ако се впише с Google и се изследват елементите при Debug за Provider, email и т.н. те ще показват данните за всяко вписване, актуални към съответния BreakPoint.

110 - Secret manager

ASP.NET Core Secret Manager

Keeps production secrets like database connection strings, API and encryption keys out of source control

The main use of Secret Manager is to keep production secrets like database connection strings, API and encryption keys out of source control.

Why you should not store secrets in configuration files

We usually store database connection strings, third party service credentials, API and encryption keys in configuration files like **web.config** in class asp.net and **appSettings.json** in asp.net core.

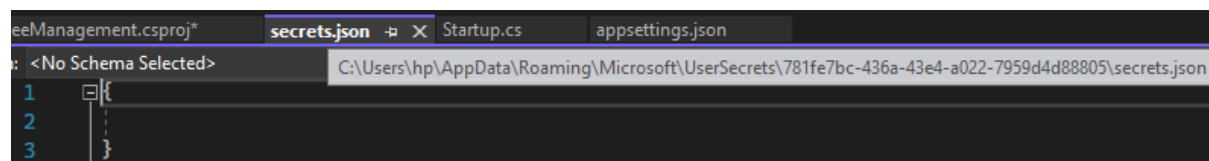
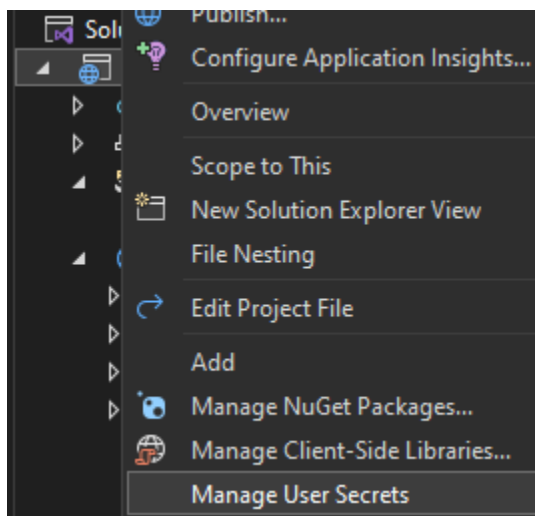
```
https://json.schemastore.org/appsettings.json
{
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json",
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;database=EmployeeDB;Tr"
  }
}
```

These configuration files are part of the project. So when they are committed to the source control repository, everyone who has access to the repository will have access to the sensitive data in these files and could be misused.

From security standpoint, it is not a good idea, to store passwords or other sensitive data in configuration files or source code.

Secret Manager allows developers to store and retrieve sensitive data during the development of an ASP.NET Core application. It stores sensitive data i.e user secrets in a file with name **secrets.json**.

To add this file to your project, right click on the project name in Solution Explorer in Visual Studio and select **Manage User Secrets** from the context menu. This adds **secrets.json** file.



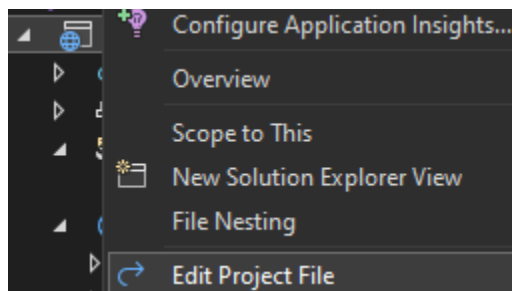
Трябва да се обърне внимание, че файлът не е в папката с проекта. Местоположението му се вижда при посочване с мишката, а структурата му много наподобява appsettings.json.

The structure of this file is similar to **appSettings.json**. The important point to keep in mind is, **this file is not part of the project folder**. It is located outside of the project folder at the following path.

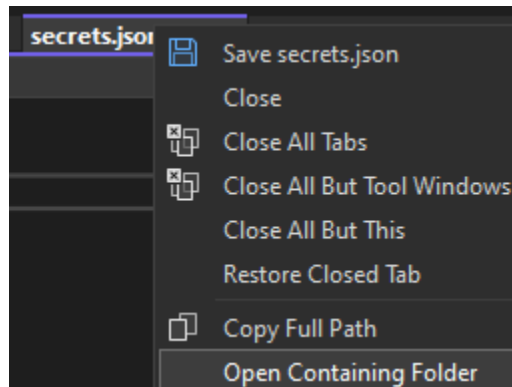
C:\Users\{UserName}\AppData\Roaming\Microsoft\UserSecrets\{ID}

- **{UserName}** is the windows user name that you use to log into the computer.
- **{ID}** is a GUID (Globally Unique Identifier)

On a single computer you may have multiple asp.net core projects and a **secrets.json** file for each project. It is this GUID, that links a given **secrets.json** file to a given asp.net core project. To establish this link, **UserSecretsId** node is included in the **.csproj** file.



```
EmployeeMan...ement.csproj*  secrets.json  appsettings.json
<TargetFramework>net6.0</TargetFramework>
<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
<UseAppHost>False</UseAppHost>
<UserSecretsId>781fe7bc-436a-43e4-a022-7959d4d88805</UserSecretsId>
</PropertyGroup>
```



C:\Users\hp\AppData\Roaming\Microsoft\UserSecrets\781fe7bc-436a-43e4-a022-7959d4d88805

Name	Date modified	Type	Size
secrets	7.12.2022 r. 15:49	JSON File	1 KB

Уникалният номер на UserSecerts съпада в директорията и файла на проекта.

ASP.NET Core Secret Manager

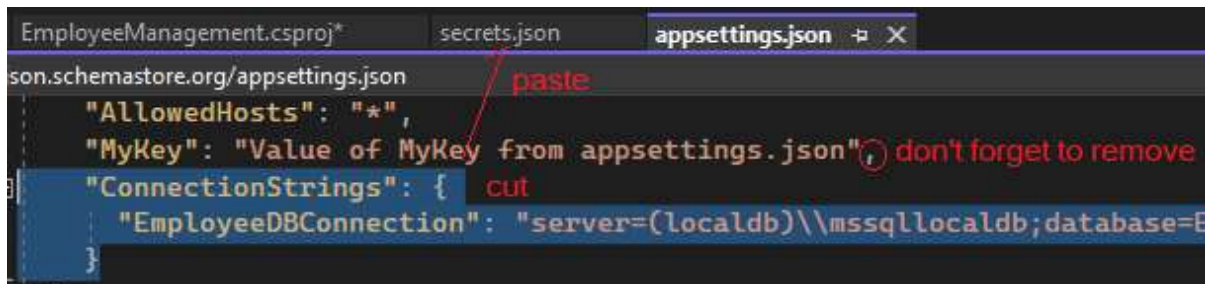
- User secrets are stored in **secrets.json**
- Structure is similar to **appSettings.json**
- Located **outside** of the project folder

C:\Users\[UserName]\AppData\Roaming\Microsoft\UserSecrets\[ID]

↓
Windows Login Name

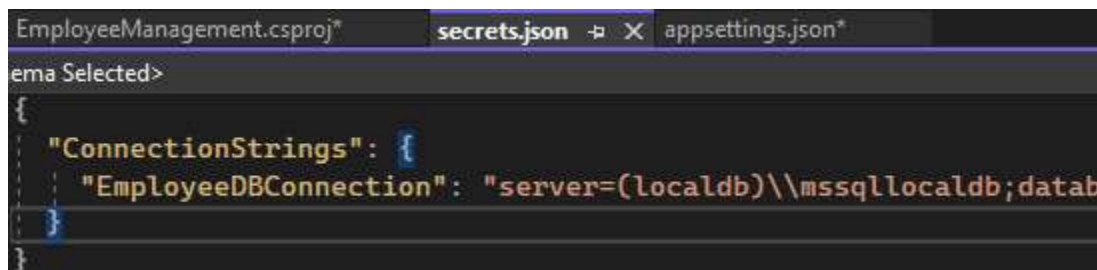


A given **secrets.json** file can be shared by multiple projects.



```
EmployeeManagement.csproj* secrets.json appsettings.json X
son.schemastore.org/appsettings.json
"AllowedHosts": "*",
"MyKey": "Value of MyKey from appsettings.json" don't forget to remove
"ConnectionStrings": { cut
  "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;database=EmployeeManagement"
}
```

```
"ConnectionStrings": {
  "EmployeeDBConnection":
"server=(localdb)\\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;Multiple
ActiveResultSets=true"
}
```



```
EmployeeManagement.csproj* secrets.json X appsettings.json*
ema Selected>
{
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;database=EmployeeManagement"
  }
}
```

При старт, приложението работи по същият начин.

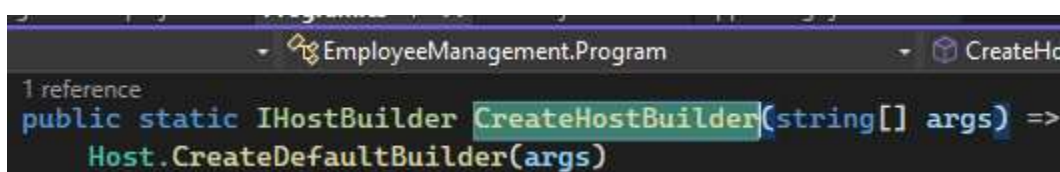
ASP.NET Core Configuration Sources

order to read from IHostBuilder:

- **appsettings.json (Part 9)**
- **User Secrets**
- **Environment variables (Part 14)**
- **Command-line argument**

IConfiguration service is setup to read configuration information from all the various configuration sources

Later configuration sources **override** the earlier configuration sources



```
EmployeeManagement.Program CreateHostBuilder
1 reference
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
```

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

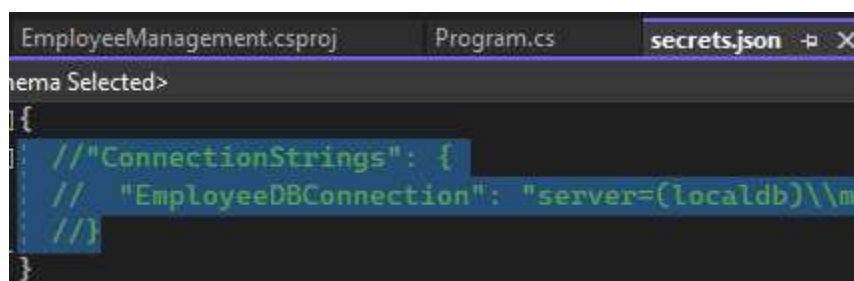
```
github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs
config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
    .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

if (env.IsDevelopment())
{
    var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
    if (appAssembly != null)
    {
        config.AddUserSecrets(appAssembly, optional: true);
    }
}

config.AddEnvironmentVariables();

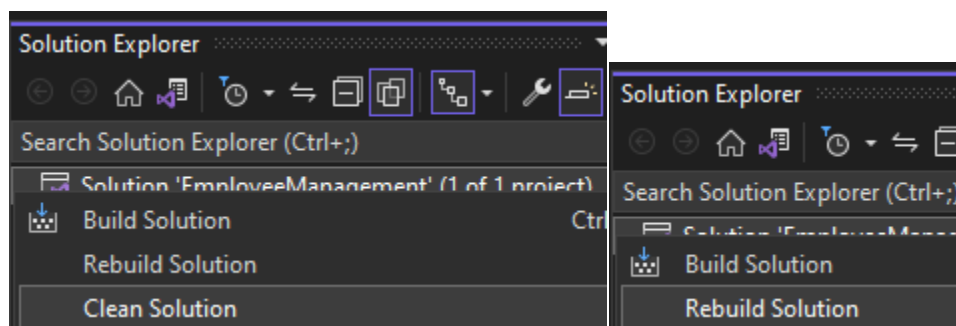
if (args != null)
{
    config.AddCommandLine(args);
}
```

Ако се закоментира препратката към DB – secrets.json



```
EmployeeManagement.csproj Program.cs secrets.json
Schema Selected>
{
  // "ConnectionStrings": {
  //   "EmployeeDBConnection": "server=(localdb)\\ms
  // }
}
```

тогава никъде няма препратка към нея в целия проект. При старт на проекта, видимо той може да функционира правилно, защото използва заредени стари данни, но ако ги почистим:



Clean Solution,
Rebuild Solution

An unhandled exception occurred while processing the request.

ArgumentNullException: Value cannot be null. (Parameter 'connectionString')

Microsoft.EntityFrameworkCore.Utilities.Check.NotEmpty(string value, string parameterName)

Stack Query Cookies Headers Routing

ArgumentNullException: Value cannot be null. (Parameter 'connectionString')

Microsoft.EntityFrameworkCore.Utilities.Check.NotEmpty(string value, string parameterName)

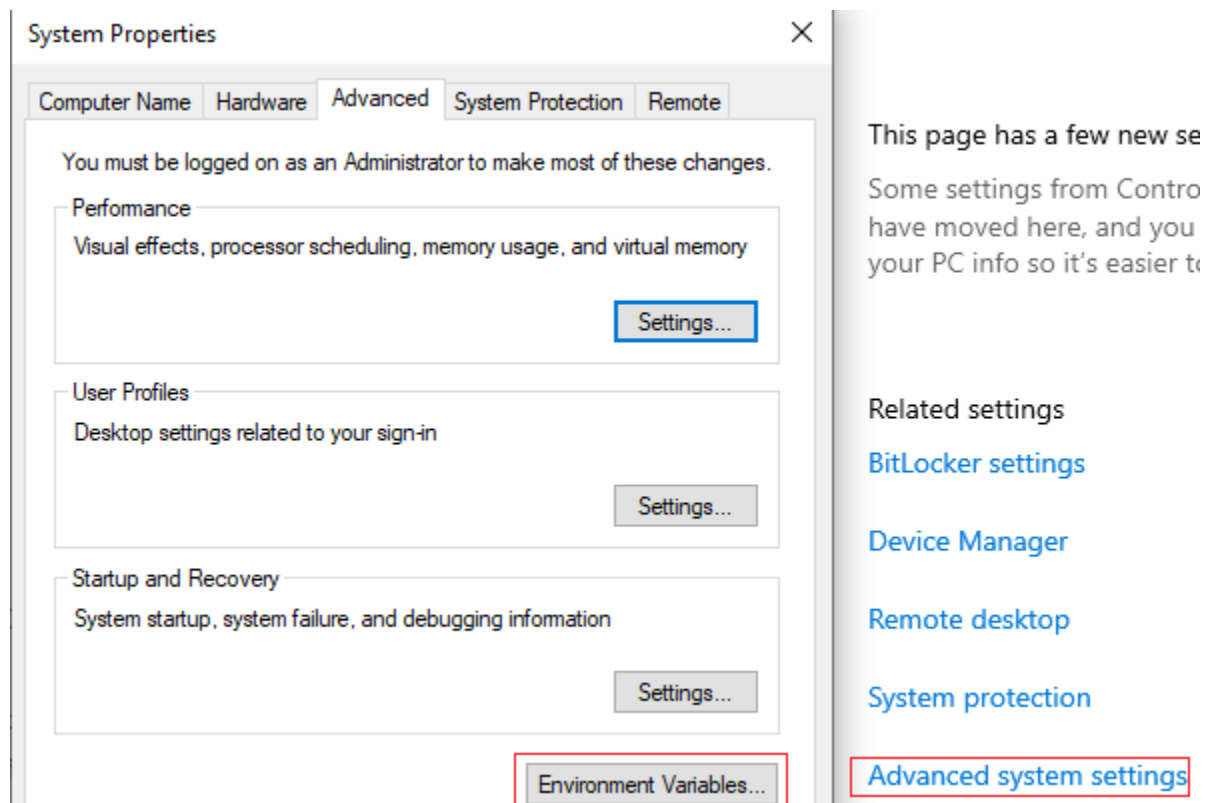
Microsoft.EntityFrameworkCore.SqlServer.DbContextOptionsExtensions.UseSqlServer(DbContextOptionsBuilder optionsBuilder, string connectionString, Action<SqlServerDbContextOptionsBuilder> sqlServerOptionsAction)

EmployeeManagement.Startup.<ConfigureServices>b__2_0(DbContextOptionsBuilder options) in Startup.cs

32. options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection"));

connectionString is null.

Run -> Control Panel -> System -> Advanced -> Environment Variables -> New:

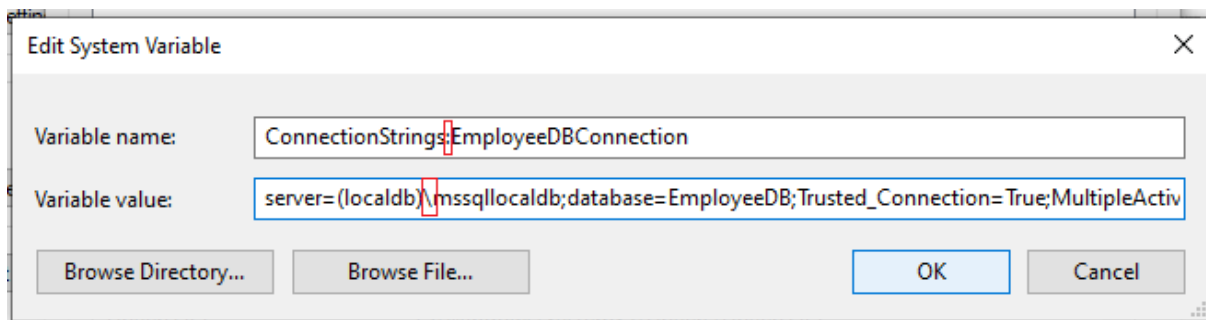


ConnectionStrings:EmployeeDBConnection

```
server=(localdb)\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true
```

```
without 1 \
```

```
server=(localdb)\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true
```



Restart VS -> works.

User secrets in production

To protect sensitive data, `secrets.json` file is deliberately kept outside of the project folder. This file is not checked into source control repository. This means `secrets.json` file is not copied onto the production server, when we actually build and deploy. So, where will the application find database connection string.

Well, on a production server store the database connection string in an environment variable. If you remember, `IConfiguration` service is setup to read configuration information from all the following configuration sources.

- appsettings.json
- User secrets
- Environment variables
- Command-line argument

This means, in spite of not having `secrets.json` file on the production server, our application should work just fine because it will find the required database connection string in the environment variable.

Secret Manager isn't for staging or production server, it should only be used on development machine. For production always use either environment variables, Azure Key Vault, or 3rd party production secret management system.

111 - Why email confirmation is important

При регистрация се изпраща потвърждение на посочения имейл. Това е важно от гледна точка на сигурността.

Why Email Confirmation is Important

Prevents Accidental Account Hijacking



Например потребител 1 се регистрира като Jon без потвърждение на имейл, допускайки печатна грешка с включване на h – john@foo.com. На следващия ден, друг потребител John, който е собственик на същия имейл, опитва да се регистрира, но връща отговор, че този имейл вече е зает. Той може да използва опция за възстановяване на паролата и така да влезе в профила с всички данни на първия потребител. Това е пробив в сигурността, освен това, първият потребител вече няма да има достъп до профила си. Затова се прави потвърждаване на регистрацията по имейл.

Why Email Confirmation is Important

Prevents Accidental Account Hijacking

Reduces Spam Registrations

Prevents Unsolicited Emails

Easy to Recover Account

Спам ботовете имат възможност да създават голям брой random – имейли, като с потвърждаване на регистрацията ще се избегнат техните набези.

112 - Block login if email is not confirmed

Block Login if Email is Not Confirmed

Local Account Login

External Login

Email not confirmed yet

Email

pragim@pragimtech.com

Password

Remember me

Login

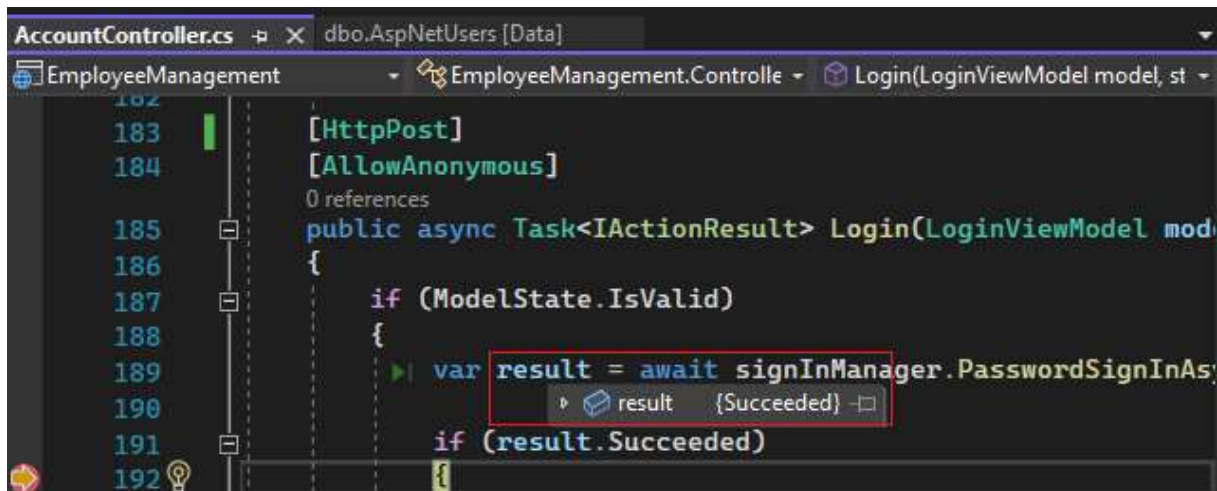
Google Facebook

Искаме да се изведе съобщение за грешка при непотвърден имейл при локално или външно вписване.

dbo.AspNetUsers [Data]							
Max Rows: 1000							
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHas
▶	519-3a32950ab516	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	False	NULL
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV...	aurora@abv.bg	AURORA@ABV...	False	AQAAAAEAA
	84eb9e06-25b7-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	NULL
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False	AQAAAAEAA
	a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV...	george@abv.bg	GEORGE@ABV...	False	AQAAAAEAA
	b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@ABV.BG	False	AQAAAAEAA
	b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False	AQAAAAEAA
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False	AQAAAAEAA
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False	AQAAAAEAA
	f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON@ABV.BG	False	AQAAAAEAA
⚙	NULL	NULL	NULL	NULL	NULL	NULL	NULL

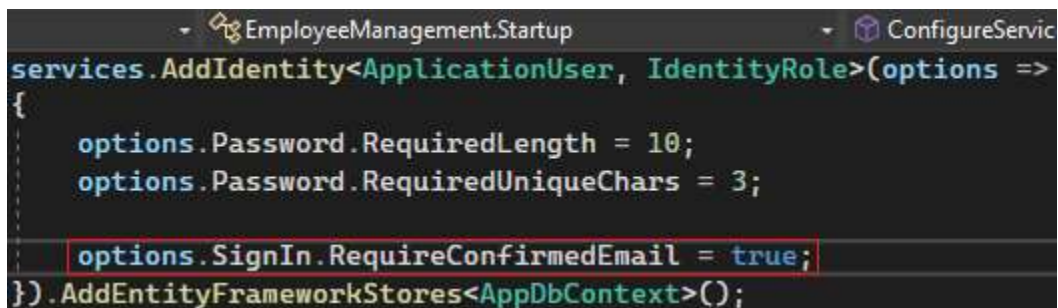
Към момента, в таблицата за локални потребители Users навсякъде EmailConfirmed = false. Това означава, че независимо кой от посочените имейли изберем, ще може да се впишем успешно без потвърждение.

Debug mode -> вписване, изхвърля ни към BreakPoint в Login, AccountController, проверяваме, че преди него променливата result = Succeeded, съответно при Continue ни вписва, като нямаме потвърждение на имейл.



```
182
183 [HttpPost]
184 [AllowAnonymous]
185 0 references
186 public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
187 {
188     if (ModelState.IsValid)
189     {
190         var result = await signInManager.PasswordSignInAsync(model.Email, model.Password, model.IsRemembered, out bool lockedOut);
191         if (result.Succeeded)
192         {
193             return RedirectToAction(nameof(Index), "Home");
194         }
195     }
196     return RedirectToAction(nameof(Login), "Home");
197 }
```

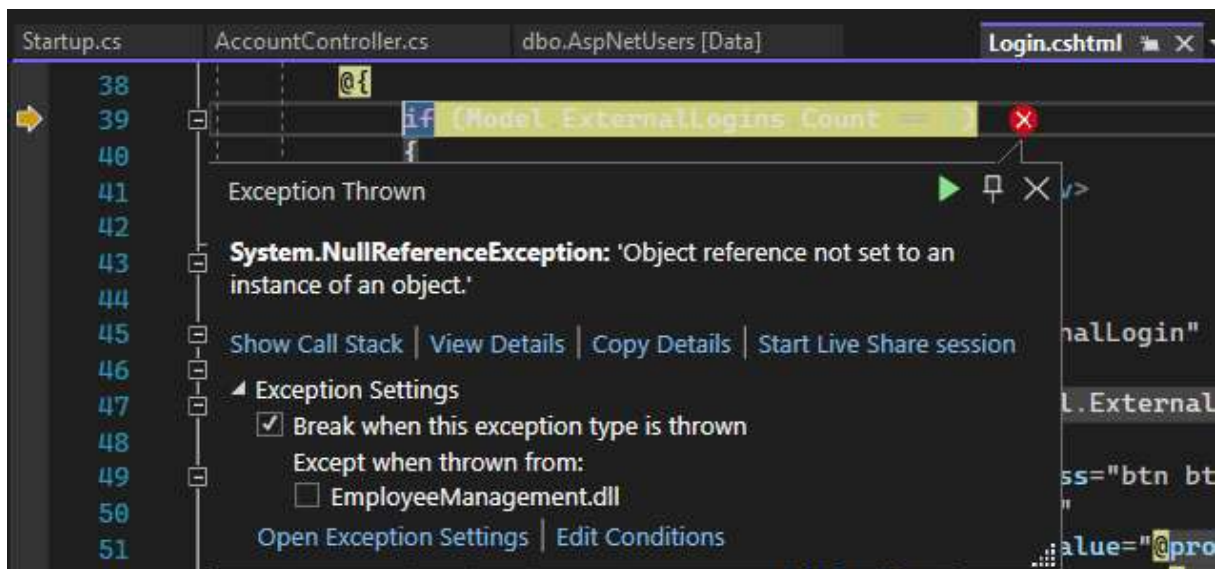
Единият начин да стане това, е да направим потвърждение на имейл задължително в Startup чрез `options.SignIn.RequireConfirmedEmail = true;`



```
services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;

    options.SignIn.RequireConfirmedEmail = true;
}).AddEntityFrameworkStores<AppDbContext>();
```

Сега при вписване с Debug ме изхвърли към



```
38
39 if (Model.ExternalLogins.Count > 0)
40 {
41     // ...
42 }
43
44
45
46
47
48
49
50
51
```

Exception Thrown

System.NullReferenceException: 'Object reference not set to an instance of an object.'

Show Call Stack | View Details | Copy Details | Start Live Share session

Exception Settings

- ☒ Break when this exception type is thrown
- Except when thrown from:
 - ☐ EmployeeManagement.dll

Open Exception Settings | Edit Conditions



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Account_Login.ExecuteAsync() in Login.cshtml, line 39

Stack

Query Cookies Headers Routing

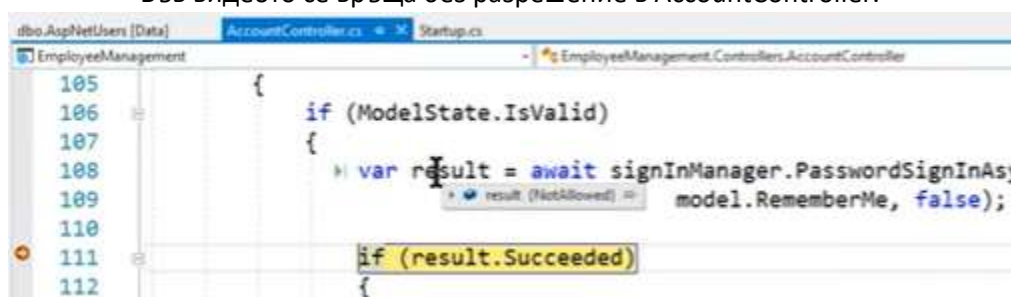
NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Account_Login.ExecuteAsync() in Login.cshtml

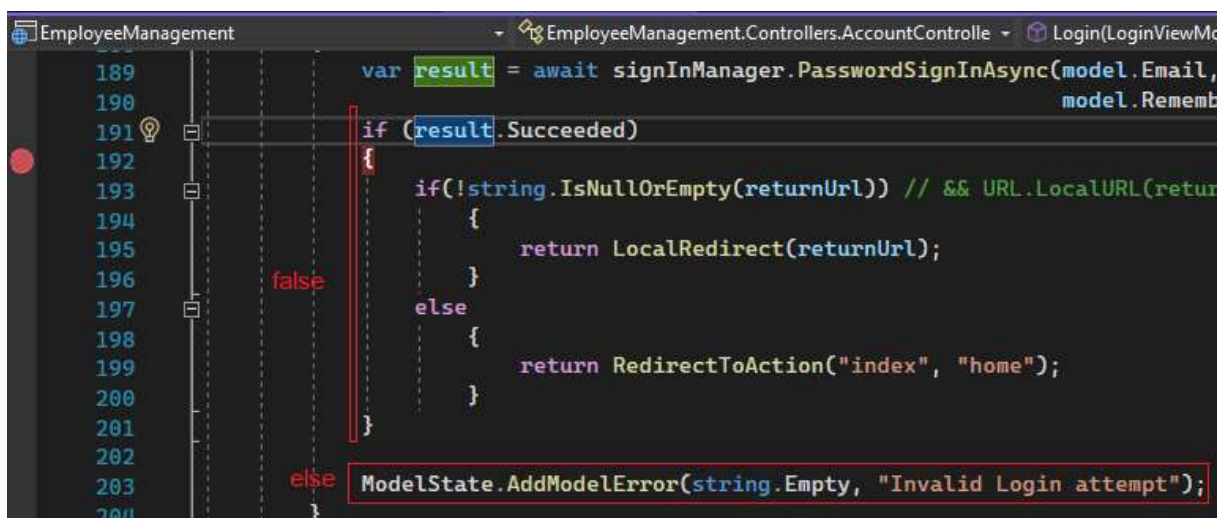
39. if (Model.ExternalLogins.Count == 0)
Microsoft.AspNetCore.Mvc.Razor.RazorView.RenderPageCoreAsync(IRazorPage page, ViewContext context)

if (Model.ExternalLogins.Count == 0)

Във видеото се връща без разрешение в AccountController.

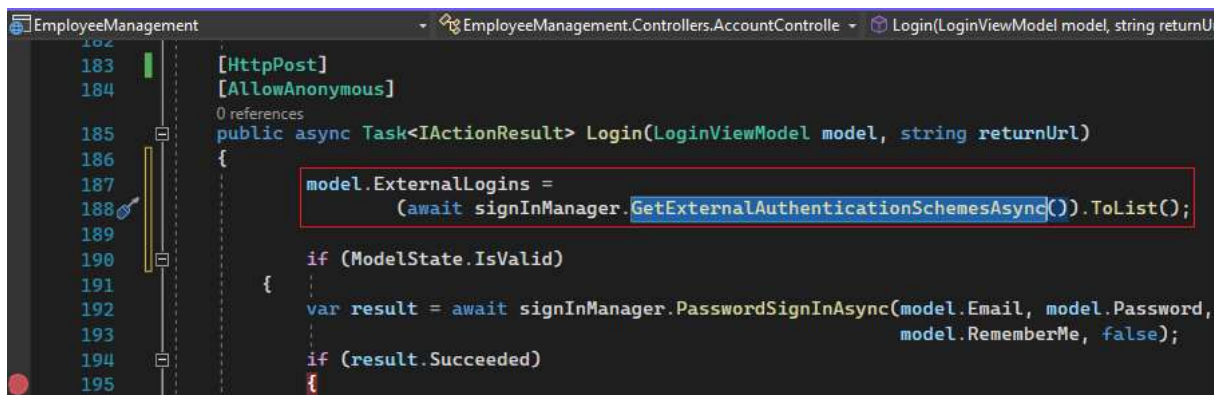


И в двата случая, въпреки че имейлът и паролата са коректни, не разрешава вписване поради непотвърждението на имейла. След Continue извежда същата грешка, въпреки, че очакваното съобщение е при неизпълнението на if-блока „Invalid Login attempt”.



Единият начин да коригираме грешката е директно в Login, за да не бъде Null, извеждаме списък с всички ExternalProviders:

```
model.ExternalLogins =  
(await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
```



```
EmployeeManagement - EmployeeManagement.Controllers.AccountController - Login(LoginViewModel model, string returnUrl)  
182 [HttpPost]  
183 [AllowAnonymous]  
184 0 references  
185 public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)  
186 {  
187     model.ExternalLogins =  
188     (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();  
189  
190     if (ModelState.IsValid)  
191     {  
192         var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,  
193             model.RememberMe, false);  
194         if (result.Succeeded)  
195         {
```

Този път при Debug + данни, валидационната грешка я няма и извежда очакваното съобщение:
Invalid Login attempt.



Local Account Login

Invalid Login attempt

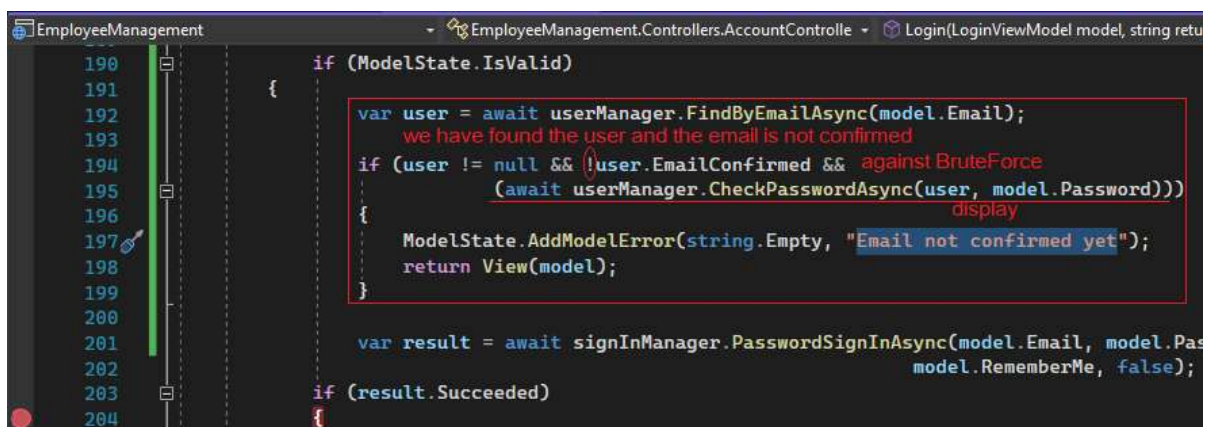
Email
aurora@abv.bg

Password

☐ Remember me

Login

Но ние не искаме него, а Email not confirmed yet.



```
EmployeeManagement - EmployeeManagement.Controllers.AccountController - Login(LoginViewModel model, string returnUrl)  
190 if (ModelState.IsValid)  
191 {  
192     var user = await userManager.FindByEmailAsync(model.Email);  
193     we have found the user and the email is not confirmed  
194     if (user != null && !user.EmailConfirmed && against BruteForce  
195         (await userManager.CheckPasswordAsync(user, model.Password)))  
196     {  
197         ModelState.AddModelError(string.Empty, "Email not confirmed yet");  
198         return View(model);  
199     }  
200  
201     var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,  
202         model.RememberMe, false);  
203     if (result.Succeeded)  
204     {
```

```
var user = await userManager.FindByEmailAsync(model.Email);  
  
if (user != null && !user.EmailConfirmed &&  
    (await userManager.CheckPasswordAsync(user, model.Password)))  
{
```

```
ModelState.AddModelError(string.Empty, "Email not confirmed yet");  
return View(model);  
}
```

Много е важно това съобщение да се изведе едва след проверка за съвпадение между имейл и парола. В противен случай, при брутфорс атака, хакерът ще въвежда рандъм имейли и без проверка за съвпадение на парола, той ще види съобщението, че имейлът не е потвърден и така ще разбере че той се използва. Той може да изчака няколко дена имейлът да бъде потвърден от истинския собственик и тогава да опита рандъм пароли със същия имейл, за да получи достъп. Ето защо е много важно това съобщение да се извежда едва след сверен имейл и парола.

Сега при тестване, при грешна парола виждаме Invalid login attempt. За хакер това съобщение няма да бъде достатъчно ориентировъчно къде е грешката, докато за истински потребител, който знае имейла си, ще насочи към грешна парола. Когато предостави правилна парола, се извежда другата грешка Email not confirmed yet.

Local Account Login Local Account Login

- Invalid Login attempt

Email

aurora@abv.bg

Password

☐ Remember me

Login

- Email not confirmed yet

Email

aurora@abv.bg

Password

☐ Remember me

Login

Искаме да направим същото из а External Login Providers в ExternalLoginCallback().

```

- EmployeeManagement.Controllers.AccountController - ExternalLoginCallback(string returnUrl = null, string
var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null)
{
    ModelState
        .AddModelError(string.Empty, "Error loading external login information.");

    return View("Login", loginViewModel);
}
paste
// If the user already has a login (i.e if there is a record in AspNetUserLogins
// table) then sign-in the user with this external login provider
var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
// If there is no record in AspNetUserLogins table, the user may not have
// a local account
else
{
    cut
    var email = info.Principal.FindFirstValue(ClaimTypes.Email);

    if (email != null)

```

Преди да преместваме най-важния ред, от който зависи имейла с Claims между GetLoginInfo - LoginSignIn, т.е. когато разполагаме с информацията и преди да впишем, проверяваме имейла.

```
var user = await userManager.FindByEmailAsync(email);
```

```

var email = info.Principal.FindFirstValue(ClaimTypes.Email);
ApplicationUser user = null;
// If the user already has a login (i.e if there is a record in AspNetUsers
// table) then sign-in the user with this external login provider
var signInResult = await signInManager.ExternalLoginSignInAsync(info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
// If there is no record in AspNetUserLogins table, the user may not have
// a local account
else
{
    if (email != null)
    {
        user = await userManager.FindByEmailAsync(email);
    }
}

```

remove var [?] (local variable) ApplicationUser user type

Създаваме нова променлива тип ApplicationUser и я зануляваме.

Тогава проверяваме дали съществува такъв имейл и ако да, намираме потребителя, който му съответства.

Ако външният провайдер потвърди потребител с такъв имейл (ние не проверяваме съответствието му с паролата, т.к. те са във външна БД, а получаваме само отговор за потвърждение), връща валидационна грешка за потвърждаване на имейла.


```

ApplicationUser user = null;

if (email != null)
{
    // Find the user
    user = await userManager.FindByEmailAsync(email);

    // If email is not confirmed, display login view with validation error
    if (user != null && !user.EmailConfirmed)
    {
        // In the external DB, so we don't need to check them
        ModelState.AddModelError(string.Empty, "Email not confirmed yet");
        return View("Login", loginViewModel);
    }
}

var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
else
{
    if (email != null)
    {
        // no need to be repeated
        user = await userManager.FindByEmailAsync(email);
        if (user == null)
    }
}

```

При тестване ми позволява вписване през Google, Facebook,
а после изписва валидационната грешка, че имейлът не е потвърден.

[AllowAnonymous]

```

public async Task<IActionResult>
ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");

    LoginViewModel loginViewModel = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };

    if (remoteError != null)
    {
        ModelState.AddModelError(string.Empty,
            $"Error from external provider: {remoteError}");

        return View("Login", loginViewModel);
    }
}

```

```

var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null)
{
    ModelState.AddModelError(string.Empty,
        "Error loading external login information.");

    return View("Login", loginViewModel);
}

```

```

// Get the email claim from external login provider (Google, Facebook etc)
var email = info.Principal.FindFirstValue(ClaimTypes.Email);
ApplicationUser user = null;

```

```

if (email != null)
{
    // Find the user
    user = await userManager.FindByEmailAsync(email);
}

```

```

// If email is not confirmed, display login view with validation error
if (user != null && !user.EmailConfirmed)
{
    ModelState.AddModelError(string.Empty, "Email not confirmed yet");
    return View("Login", loginViewModel);
}
}

```

```

var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

```

```

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
else
{
    if (email != null)
    {
        if (user == null)
        {
            user = new ApplicationUser
            {
                UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            };

            await userManager.CreateAsync(user);
        }
    }
}

```

```

await userManager.AddLoginAsync(user, info);
await signInManager.SignInAsync(user, isPersistent: false);

```

```

        return LocalRedirect(returnUrl);
    }

    ViewBag.ErrorTitle = $"Email claim not received from: {info.LoginProvider}";
    ViewBag.ErrorMessage = "Please contact support on Pragim@PragimTech.com";

    return View("Error");
}
}

```

Login Action in AccountController

```

[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl)
{
    model.ExternalLogins =
        (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();

    if (ModelState.IsValid)
    {
        var user = await userManager.FindByEmailAsync(model.Email);

        if (user != null && !user.EmailConfirmed &&
            (await userManager.CheckPasswordAsync(user, model.Password)))
        {
            ModelState.AddModelError(string.Empty, "Email not confirmed yet");
            return View(model);
        }

        // Rest of the code
    }

    return View(model);
}

```

ExternalLoginCallback Action

```
[AllowAnonymous]
public async Task<IActionResult>
    ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{
    LoginViewModel loginViewModel = new LoginViewModel...;

    var info = await signInManager.GetExternalLoginInfoAsync();
    if (info == null) ...

    var email = info.Principal.FindFirstValue(ClaimTypes.Email);
    ApplicationUser user = null;

    if (email != null)
    {
        user = await userManager.FindByEmailAsync(email);

        if (user != null && !user.EmailConfirmed)
        {
            ModelState.AddModelError(string.Empty, "Email not confirmed yet");
            return View("Login", loginViewModel);
        }
    }
    // Rest of the code
    return View();
}
```

113 - Email confirmation

ASP.NET Core Email Confirmation

Example Confirmation Link

<https://localhost:44304/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-4400ba886550&token=CfDJ8Hpirs>

action (userId;token)
query string parameters

Generate Email Confirmation Token

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Generate Email Confirmation Link

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",
    new { userId = user.Id, token = token }, Request.Scheme);
```

action controller
http, https

Use `UserManager` service `GenerateEmailConfirmationTokenAsync()` method. This method takes one parameter. The `user` for whom we want to generate the email confirmation token.

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Build the email confirmation link

Once we have the `token` generated, build the `email confirmation link`. The user simply clicks this link to confirm his email. This link executes, `ConfirmEmail` action in `Account` controller. The `user ID` and the `email confirmation token` are passed in the query string. Model binding in ASP.NET core maps the values from the query string parameters to the respective parameters on the `ConfirmEmail` action.

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",  
    new { userId = user.Id, token = token }, Request.Scheme);
```

The generated `confirmation link` would look like the following

```
https://localhost:44304/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-  
4400ba886550&token=CfDJ8Hpirs
```

The last parameter `Request.Scheme` returns the request protocol such as `Http` or `Https`. This parameter is required to generate the full absolute URL. If this parameter is not specified, a relative URL like the following will be generated.

```
/Account/ConfirmEmail?userId=987009e3-7f78-445e-8bb8-4400ba886550&token=CfDJ8Hpirs
```



```
EmployeeManagement.Controllers.AccountController.Register(RegisterViewModel model)

[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid) after registration of a new user, confirm email before login
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City
        };
        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            generate token
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
            generate link
            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                { userId = user.Id, token = token }, Request.Scheme);

            if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
        }
    }
}
```

Once we get this confirmation link, we want to email it to the user for confirmation, so he can login to his email service provider and click on it to confirm his email. We will discuss sending emails in our upcoming videos. What we will do instead in this video is log this confirmation link to a file. For that we need to inject BuildInLogger service.

```
EmployeeManagement.Controllers.AccountController.AccountController(UserManager<ApplicationUser> userManager,
SignInManager<ApplicationUser> signInManager,
ILogger<AccountController> logger)

0 references
public AccountController(UserManager<ApplicationUser> userManager,
    SignInManager<ApplicationUser> signInManager,
    ILogger<AccountController> logger)
{
    this.userManager = userManager;
    this.signInManager = signInManager;
    this.logger = logger;
}
```

log the confirmation link:

```
logger.Log(LogLevel.Warning, confirmationLink);
```

```

EmployeeManagement.Controllers.AccountController Register(RegisterViewModel model)

if (result.Succeeded)
{
    var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

    var confirmationLink = Url.Action("ConfirmEmail", "Account",
        { userId = user.Id, token = token }, Request.Scheme);

    logger.Log(LogLevel.Warning, confirmationLink);

    if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
    {
        return RedirectToAction("ListUsers", "Administration");
    }
    confirm_before_login
    await signInManager.SignInAsync(user, isPersistent: false);
    return RedirectToAction("index", "home");

    ViewBag.ErrorTitle = "Registration successful";
    ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
        "email, by clicking on the confirmation link we have emailed you";
    return View("Error");
}

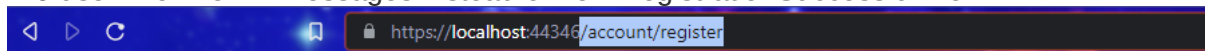
```

```

ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");

```

We use Error view + messages instead of new RegistrationSuccessful view.



An unhandled exception occurred while processing the request.

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered.

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)

Stack Query Cookies Headers Routing

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered.

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)

Microsoft.AspNetCore.Identity.UserManager<TUser>.GenerateEmailConfirmationTokenAsync(TUser user)

EmployeeManagement.Controllers.AccountController.Register(RegisterViewModel model) in AccountController.cs

73. var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

It can't find any token provider, that can generate token. To fix this, we need to provide a token provider - default from Asp.net core or our own custom.

Add Default Token Providers

NotSupportedException: No IUserTwoFactorTokenProvider<TUser> named 'Default' is registered

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddIdentity<ApplicationUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>()
        .AddDefaultTokenProviders();
}
```

.AddDefaultTokenProviders();

```
EmployeeManagement.Startup
services.AddIdentity<ApplicationUser, IdentityRole>()
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;

    options.SignIn.RequireConfirmedEmail = true;
}).AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders();
```

```
var user = new ApplicationUser
{
    UserName = model.Email,
    Email = model.Email,
    City = model.City,
};

var result = await userManager.CreateAsync(user, model.Password);

if (result.Succeeded)
{
    var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

    var confirmationLink = Url.Action("ConfirmEmail", "Account",
        new { userId = user.Id, token = token }, Request.Scheme);

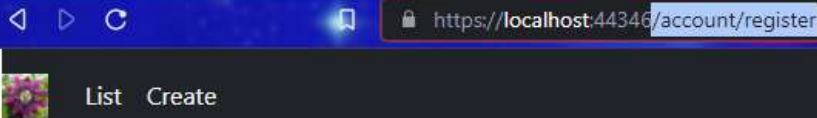
    logger.Log(LogLevel.Warning, confirmationLink);

    if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
    {
        return RedirectToAction("ListUsers", "Administration");
    }
}
```



```
}

ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");
```



Registration successful

Before you can Login, please confirm your email, by clicking on the confirmation link we have emailed you

dbo.AspNetUsers [Data] AccountController.cs Startup.cs

Max Rows: 1000

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	Password
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAA

We can find the confirmation link from nlog.config file -> *.log

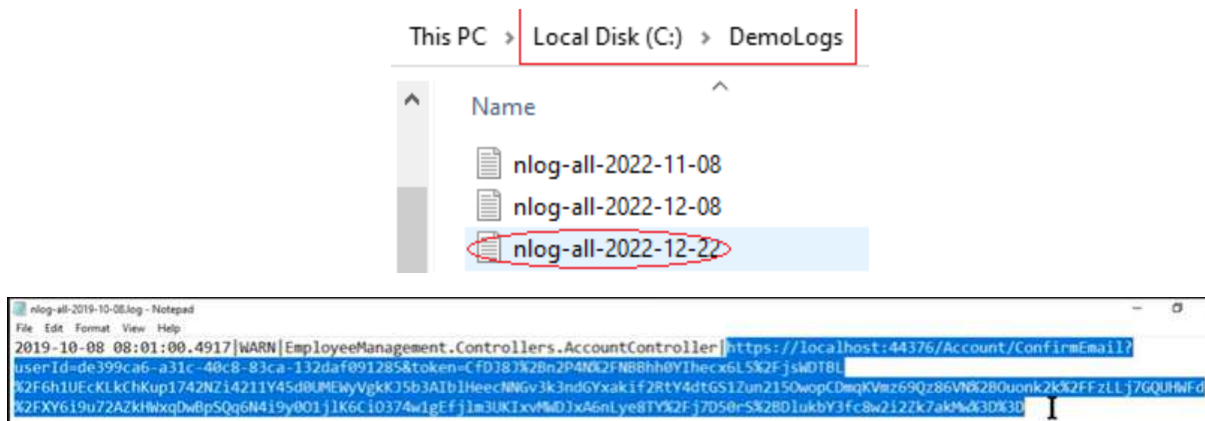
```

nlog.config X AccountController.cs Startup.cs
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <!-- the targets to write to -->
  <targets>
    <!-- write logs to file -->
    <target name="allfile" xsi:type="File"
            fileName="c:\DemoLogs\nlog-all-${shortdate}.log"/>
  </targets>

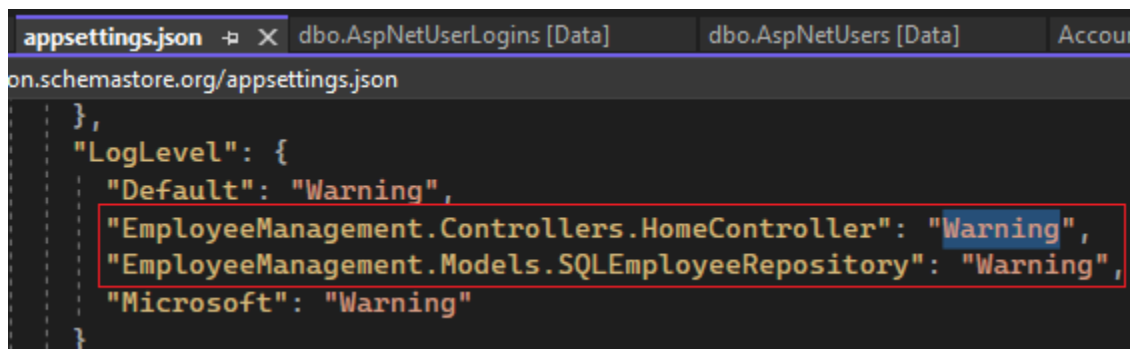
  <!-- rules to map from logger name to target -->
  <rules>
    <!--All logs, including from Microsoft-->
    <logger name="*" minlevel="Trace" writeTo="allfile" />
  </rules>
</nlog>

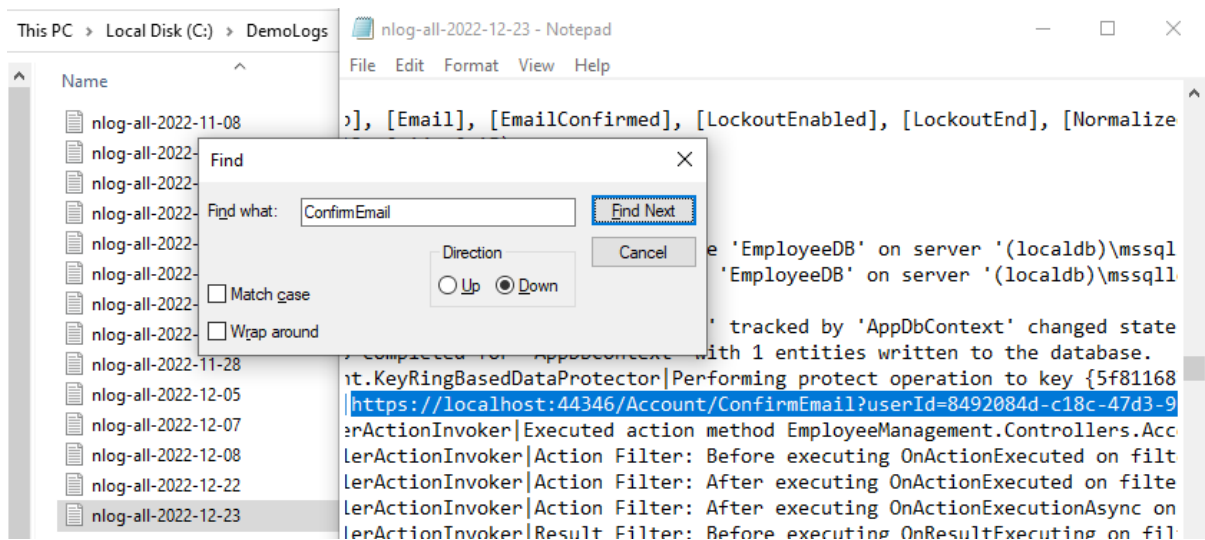
```



I can't find the confirmation link in my version, even searching "ConfirmEmail" action.

```
POST http://localhost:44346/account/register application/x-www-form-urlencoded 279 - 200 - text/
GET http://localhost:44346/_framework/aspnetcore-browser-refresh.js - -
M45D3V32MG" completed keep alive response.
GET http://localhost:44346/_framework/aspnetcore-browser-refresh.js - - - 200 12006 application/
GET http://localhost:44346/_vs/browserLink - -
M45D3V32MH" completed keep alive response.
GET http://localhost:44346/_vs/browserLink - - - 200 - text/javascript;+charset=UTF-8 17.4961ms
M45D3V32MG" disconnecting.
M45D3V32MH" disconnecting.
M45D3V32MG" stopped.
M45D3V32MH" stopped.
id "0HMN45D3V32MH" sending FIN because: "The Socket transport's send loop completed gracefully."
id "0HMN45D3V32MG" sending FIN because: "The Socket transport's send loop completed gracefully."
```





<https://localhost:44346/Account/ConfirmEmail?userId=8492084d-c18c-47d3-9bcf-cdbe57a6e1f2&token=CfDJ8IcWgV9y4PJks6bemgk2FP4WyyeS2qx2651rbQBgdvqgs4Hk1VJ8tWW%2BqVxIvNMSpsJPqVX5I3qggQ0qnS8CbJCqGHRr3I%2Bt8tqiwfUy%2BxwD6p9I9OSzFwvGfyTXbCrZr2qyjJwlbRzhmXm0khLtwmrqu2Q2oV2%2FyfvtaccXZAQMvyZh%2BXTPg4gBFfeJz0yYYNk3qnczcwJGXgQoLYmUIH1McszdbGBaZXw4TqkKvrDUGsTTi0SBtjq1E3plhvnkQA%3D%3D>

-> 404 because we don't have ConfirmEmail action, so we implement it.

```

EmployeeManagement.Controllers.AccountController ConfirmEmail(string userI
[AllowAnonymous]
0 references
public async Task<IActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    store
    var user = await userManager.FindByIdAsync(userId);
    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);
    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}

```

View/Account -> new razor view ConfirmEmail.cshtml:

<h3>Thank you for confirming your email**</h3>**

When we reload the ConfirmLink,
we are logged in and the value in Users DB table is set to true.

Without this link, the other users are not confirmed yet.

Register Action in Account Controller

```
[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> Register(RegisterViewModel model, string returnUrl)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
            UserName = model.Email,
            Email = model.Email,
            City = model.City,
        };

        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                new { userId = user.Id, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, confirmationLink);

            // Rest of the code
        }
    }
}
```

ConfirmEmail Action - Account Controller

```
[AllowAnonymous]
public async Task<IActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    var user = await userManager.FindByIdAsync(userId);
    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);
    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}
```

114 - External login email confirmation

In this video we will discuss **how to confirm the email received from external login providers such as Google, Facebook etc.**

When we use external providers like Google or Facebook to login, we receive the user email from these external login provider. We then use this email to create a local user account.

Upon creating a local user account, send email confirmation link. If the email address is not confirmed, do not allow the user to login and display the error - **Email not confirmed yet.**

On the other hand, if the email address is confirmed, create an external login (**AspNetUserLogins**) and sign-in the user.

External Login Email Confirmation



- 1) Create a new user
- 2) Generate token
- 3) Generate email confirmation link

The password hash is NULL for external login users.

Let's delete our own external account from both tables: UserLogins + Users so we can start from scratch:

dbo.AspNetUserLogins [Data]				
dbo.AspNetUsers [Data]				
ConfirmEmail.cshtml				
Max Rows: 1000				
	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Facebook	688753976215688	Facebook	1f032ac4-bc55-4dde-8519-3a32950ab516
	Google	1126375077100...	Google	84eb9e06-25b7-40da-bb92-94ac00b231e2
	NULL	NULL	NULL	NULL

dbo.AspNetUsers [Data] X ConfirmEmail.cshtml AccountController.cs Startup.cs							
Max Rows: 1000							
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAAEAAC...
▶	1f032ac4-bc55-...	@protonmail.com	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	False	NULL
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAAEAAC...
	3d9edac1-67d3-...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False	AQAAAAEAAC...
	8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False	AQAAAAEAAC...
	84eb9e06-25b7-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	NULL

ASP.NET Core Email Confirmation

Generate Email Confirmation Token

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Generate Email Confirmation Link

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",
    new { userId = user.Id, token = token }, Request.Scheme);
```

Generate Email Confirmation Link

```
logger.Log(LogLevel.Warning, confirmationLink);

ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");
```



```
AccountController.cs
EmployeeManagement.Controllers.AccountController
ExternalLoginCallback(stri

if (user == null)
{
    user = new ApplicationUser
    {
        UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
        Email = info.Principal.FindFirstValue(ClaimTypes.Email)
    };

    await userManager.CreateAsync(user); insert a new row in Users DBtable

    // After a local user account is created, generate and log the
    // email confirmation link
    var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

    var confirmationLink = Url.Action("ConfirmEmail", "Account",
        new { userId = user.Id, token = token }, Request.Scheme);

    logger.Log(LogLevel.Warning, confirmationLink);

    ViewBag.ErrorTitle = "Registration successful";
    ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
        "email, by clicking on the confirmation link we have emailed you";
    return View("Error");
}
```

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",
    new { userId = user.Id, token = token }, Request.Scheme);
```

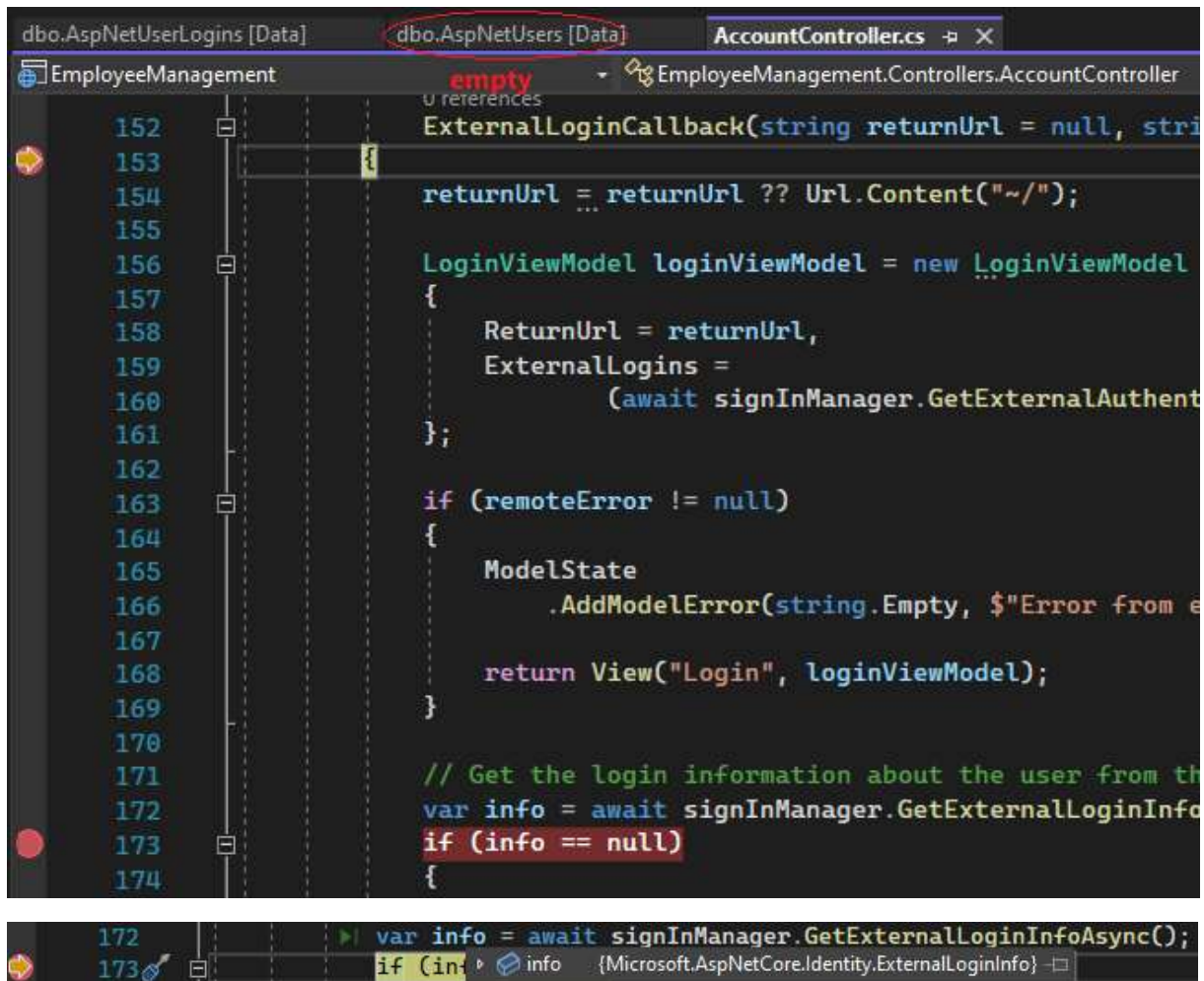
```
logger.Log(LogLevel.Warning, confirmationLink);
```

```
ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");
```

Debug -> Facebook -

> ExternalCallBack action = BreakPoint1, Users table has no info (the user don't exist) -

> Continue = Breakpoint2

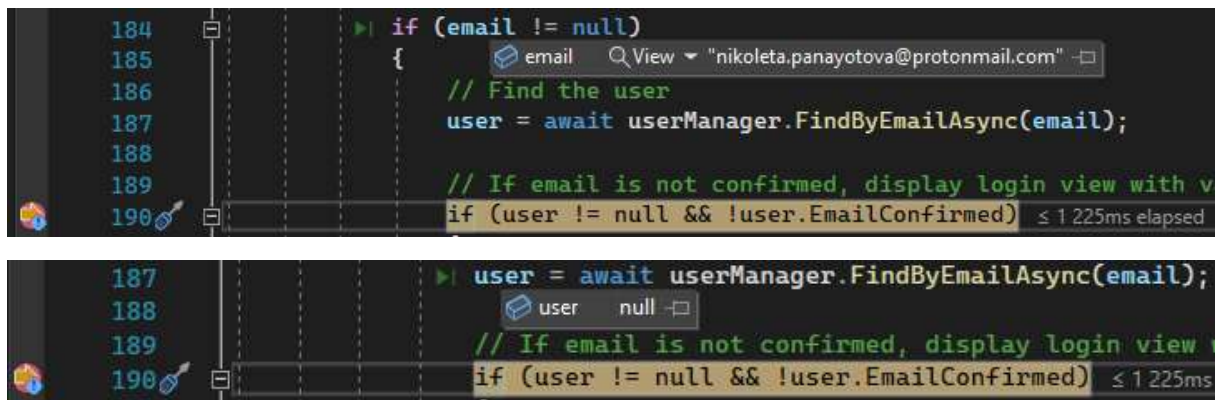


```
152 ExternalLoginCallback(string returnUrl = null, string remoteError = null)
153 {
154     returnUrl = returnUrl ?? Url.Content("~/");
155
156     LoginViewModel loginViewModel = new LoginViewModel
157     {
158         ReturnUrl = returnUrl,
159         ExternalLogins =
160             (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
161     };
162
163     if (remoteError != null)
164     {
165         ModelState
166             .AddModelError(string.Empty, $"Error from external provider: {remoteError}");
167         return View("Login", loginViewModel);
168     }
169
170     // Get the login information about the user from the database
171     var info = await signInManager.GetExternalLoginInfoAsync();
172     if (info == null)
173     {
174         return View("Login", loginViewModel);
175     }
176 }
```

172 ▶ var info = await signInManager.GetExternalLoginInfoAsync();

173 if (info == null) { Microsoft.AspNetCore.Identity.ExternalLoginInfo }

Continue->BP3



```
184 if (email != null)
185 {
186     // Find the user
187     user = await userManager.FindByEmailAsync(email);
188
189     // If email is not confirmed, display login view with validation message
190     if (user != null && !user.EmailConfirmed)
191     {
192         ModelState.AddModelError(string.Empty, "Email not confirmed. Please check your email and click the confirmation link.");
193         return View("Login", loginViewModel);
194     }
195 }
```

187 ▶ user = await userManager.FindByEmailAsync(email);

188 user null

189 // If email is not confirmed, display login view with validation message

190 if (user != null && !user.EmailConfirmed) ≤ 1 225ms elapsed

FindByEmail->null, there is no user with this email in Users.dbo, so this if {} will not be executed.

Continue->BP4

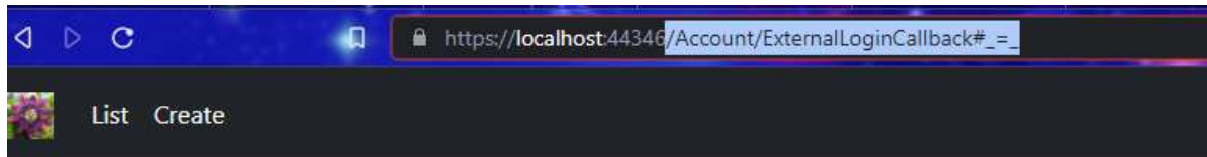
```
205 | if (email != null)
206 | {
207 |     if (user == null) ≤ 112ms elapsed
```

Users.dbo = empty -> UserLogins.dbo = empty

```
190 | if (user != null && !user.EmailConfirmed)
191 | {
192 |     ModelState.AddModelError(string.Empty, "Email not confirmed yet");
193 |     return View("Login", loginViewModel);
194 | }
195 | } Users.dbo=empty->UserLogins=empty method
196 | var signInResult = await signInManager.ExternalLoginSignInAsync(info.Login
197 |     info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
198 |
199 | if (signInResult.Succeeded)
200 | {
201 |     return LocalRedirect(returnUrl);
202 | }
203 | else control
204 | {
205 |     if (email != null)
206 |     {
207 |         if (user == null) ≤ 112ms elapsed
```

```
207 | if (user == null) ≤ 112ms elapsed
208 | {
209 |     user = new ApplicationUser
210 |     {
211 |         UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
212 |         Email = info.Principal.FindFirstValue(ClaimTypes.Email)
213 |     };
214 |
215 |     await userManager.CreateAsync(user); insert_new_row_in_Users.dbo
216 |
217 |     // After a local user account is created, generate and log the
218 |     // email confirmation link
219 |     var token = await userManager.GenerateEmailConfirmationTokenAsyn
220 |
221 |     var confirmationLink = Url.Action("ConfirmEmail", "Account",
222 |         new { userId = user.Id, token = token }, Request
223 |
224 |     logger.Log(LogLevel.Warning, confirmationLink);
225 |
226 |     ViewBag.ErrorTitle = "Registration successful";
227 |     ViewBag.ErrorMessage = "Before you can Login, please confirm yo
228 |         "email, by clicking on the confirmation link we have emailed
229 |     return View("Error");
230 | }
```

Continue->Message



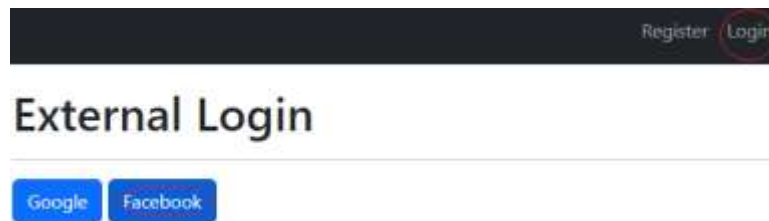
Registration successful

Before you can Login, please confirm your email, by clicking on the confirmation link we have emailed you

Users.dbo->Refresh->New user

dbo.AspNetUserLogins [Data]		dbo.AspNetUsers [Data]		AccountController.cs		
				Max Rows: 1000		
	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False
	3d9edac1-67d3-...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False
	8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False
	8492084d-c18c-...	nikoleta.panay...	ROTONMAIL.COM	nikoleta.panay...	NIKOLETA.PAN...	False

ConfirmEmails is False, so we can't login with it, but now the table is not null, user and email exist:



BP1->Continue->info is null -> Continue ->


```
152 ExternalLoginCallback(string returnUrl = null, string remoteError = null)
153 {
154     returnUrl = returnUrl ?? Url.Content("~/");
155
156     LoginViewModel loginViewModel = new LoginViewModel
157     {
158         ReturnUrl = returnUrl,
159         ExternalLogins =
160             (await signInManager.GetExternalAuthenticationSchemesAsync())
161     };
162
163     if (remoteError != null)
164     {
165         ModelState
166             .AddModelError(string.Empty, $"Error from external provider: {remoteError}");
167         return View("Login", loginViewModel);
168     }
169
170     // Get the login information about the user from the external login provider
171     var info = await signInManager.GetExternalLoginInfoAsync();
172     if (info != null)
173     {
```

info has ExternalLoginInfo -> BP2

```
172     var info = await signInManager.GetExternalLoginInfoAsync();
173     if (info == null)
174     {
175         ModelState
176             .AddModelError(string.Empty, "Error loading external login provider");
177         return View("Login", loginViewModel);
178     }
179     // Get the email claim from external login provider (Google, Facebook, etc.)
180     var email = info.Principal.FindFirstValue(ClaimTypes.Email);
181     ApplicationUser user = null;
182
183     if (email != null)
184     {
185         // Find the user
186         user = await userManager.FindByEmailAsync(email);
187         // If email is not confirmed, display login view with validation message
188         if (user != null && !user.EmailConfirmed)
189         {
```

Continue->User exists -> BP3


```

190 if (user != null && !user.EmailConfirmed) ≤ 40ms elapsed
191 {
192     ModelState.AddModelError(string.Empty, "Email not confirmed yet");
193     return View("Login", loginViewModel);
194 }
195 }
196 var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
197     info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
198
199 if (signInResult.Succeeded)
200 {
201     return LocalRedirect(returnUrl);
202 }
203 else
204 {
205     if (email != null)
206     {
207         if (user == null)

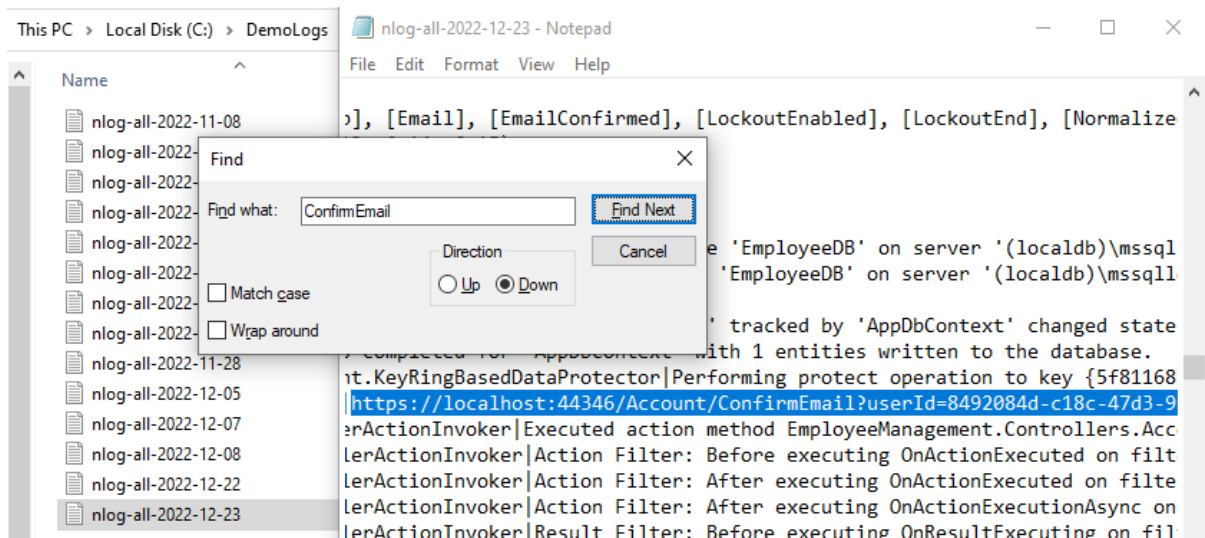
```

email is not confirmed {false} -> Continue

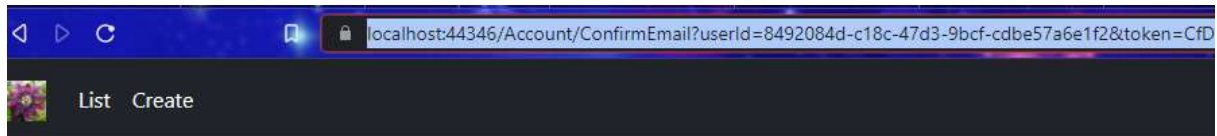
Local Account Login

• Email not confirmed yet

c:\DemoLogs\



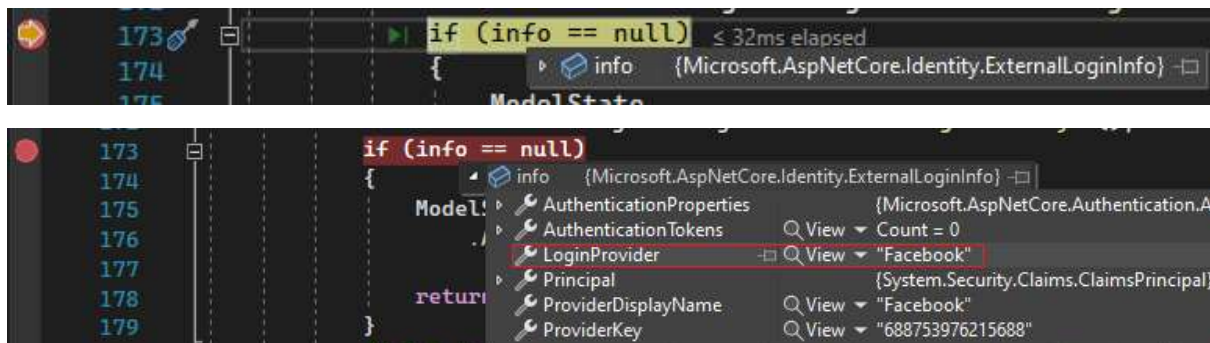
https://localhost:44346/Account/ConfirmEmail?userId=8492084d-c18c-47d3-9bcf-cdbe57a6e1f2&token=CfDJ8IcWgV9y4PJks6bemgk2FP4WyyeS2qx2651rbQBgdvqgs4Hk1VJ8tWW%2BqVxIvNMSpsJPqVX5I3qgqQ0qnS8CbJCqGHRr3I%2Bt8tqiwfUy%2BxwD6p9I9OSzFwvGfyTXbCrZr2qyjJwlbRzhmXm0kLtwmrqu2Q2oV2%2FyfvTaccXZAQMvyZh%2BXTpg4gBFfEjz0yYYNk3qnczcwJGXgQoLYmUIH1McszdbGBaZXw4TqkKvrDUgsTTi0SBtjq1E3plhvnkQA%3D%3D



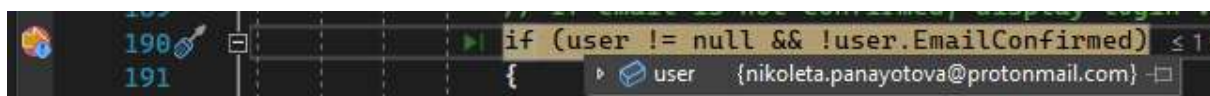
Thank you for confirming your email

Data] dbo.AspNetUsers [Data] AccountController.cs					
Max Rows: 1000					
UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash
Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAAEAAC...
aurora@abv.bg	AURORA@ABV...	aurora@abv.bg	AURORA@ABV...	False	AQAAAAEAAC...
Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False	AQAAAAEAAC...
Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False	AQAAAAEAAC...
nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	True	NULL

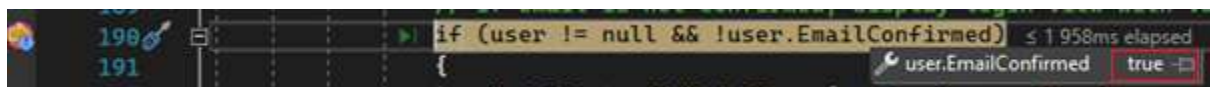
New test login in Debug mode from Facebook -> BP1->BP2->



BP3



this time EmailConfirmed = True



New BP5

signInResult = {Failed} for LoginProvider "Facebook"

dbo.AspNetUserLogins [Data]				
Max Rows: 1000				
	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Facebook	688753976215688	Facebook	8492084d-c18c-...
	NULL	NULL	NULL	NULL

Logout->Remove all breakpoints->Login with the same email in Google account.

(I use different emails)

<https://localhost:44346/Account/ConfirmEmail?userId=d5dcffab-a1ec-4aa6-aaf5-911d60422e4f&token=CfDJ8lcWgV9y4PKs6bemgk2FP7SmQKGHH7mntemmhDtInLNzU8ziXuqKIGTGOhQZxZWqbSWHxXmdhjpsse6ERKLAEz8f5pdL34UTeSbbHWAvPnOwA6N%2BlcDf9M33yWqUKNggme84ScBL%2F7KaYMay51GC%2BNB9IDsQ3TtSLYghg%2FUs7YAYncGrasvZu%2BMh191yhe5LQuOssx0amaADD9h2Qw7Q%2BI45Mec0ujOqYnSpbYehK1xBS%2BWraCwW3j4XDDuw%2FFA%3D%3D>



dbo.AspNetUserLogins [Data]				
Max Rows: 1000				
	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Facebook	688753976215688	Facebook	8492084d-c18c-...
	Google	1126375077100...	Google	d5dcffab-a1ec-...
	NULL	NULL	NULL	NULL

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False
	3d9edac1-67d3-...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False
	8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False
	8492084d-c18c-...	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	True
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False
	96944fa4-2400-...	Dima4@abv.bg	DIMA4@ABV.BG	Dima4@abv.bg	DIMA4@ABV.BG	False
	a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV....	george@abv.bg	GEORGE@ABV....	False
	b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@ABV.BG	False
	b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False
	c0ec608d-d07a-...	Dima2@abv.bg	DIMA2@ABV.BG	Dima2@abv.bg	DIMA2@ABV.BG	False
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False
▶	d5dcffab-a1ec-...	amidalafame@...	AMIDALAFLAM...	amidalafame@...	AMIDALAFLAM...	True
	f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON@ABV.BG	False
⚙	NULL	NULL	NULL	NULL	NULL	NULL

[AllowAnonymous]

public async Task<IActionResult>

ExternalLoginCallback(string returnUrl = null, string remoteError = null)

{

 returnUrl = returnUrl ?? Url.Content("~/");

 LoginViewModel loginViewModel = new LoginViewModel

 {

 ReturnUrl = returnUrl,

 ExternalLogins =

 (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()

 };

 if (remoteError != null)

 {

 ModelState.AddModelError(string.Empty,
 \$"Error from external provider: {remoteError}");

 return View("Login", loginViewModel);

 }

 var info = await signInManager.GetExternalLoginInfoAsync();

 if (info == null)

 {

 ModelState.AddModelError(string.Empty,
 "Error loading external login information.");

 return View("Login", loginViewModel);


```

}

var email = info.Principal.FindFirstValue(ClaimTypes.Email);
ApplicationUser user = null;

if (email != null)
{
    user = await userManager.FindByEmailAsync(email);

    if (user != null && !user.EmailConfirmed)
    {
        ModelState.AddModelError(string.Empty, "Email not confirmed yet");
        return View("Login", loginViewModel);
    }
}

var signInResult = await signInManager.ExternalLoginSignInAsync(
    info.LoginProvider, info.ProviderKey,
    isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
else
{
    if (email != null)
    {
        if (user == null)
        {
            user = new ApplicationUser
            {
                UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            };

            await userManager.CreateAsync(user);

            // After a local user account is created, generate and log the
            // email confirmation link
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                new { userId = user.Id, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, confirmationLink);

            ViewBag.ErrorTitle = "Registration successful";
            ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
                "email, by clicking on the confirmation link we have emailed you";
            return View("Error");
        }
    }
}

```

```

    }

    await userManager.AddLoginAsync(user, info);
    await signInManager.SignInAsync(user, isPersistent: false);

    return LocalRedirect(returnUrl);
}

ViewBag.ErrorTitle = $"Email claim not received from: {info.LoginProvider}";
ViewBag.ErrorMessage = "Please contact support on Pragim@PragimTech.com";

return View("Error");
}
}
}
*****

[HttpGet]
[AllowAnonymous]
public async Task<ActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);

    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}
}

```

115 - Forgot password

The first step is to include the **Forgot Password** link on the Login view.

 [List](#) [Create](#) [Manage](#) [Register](#) [Login](#)

Local Account Login

Email

Password

☐ Remember me

Login

[Forgot Password?](#)

External Login

[Google](#) [Facebook](#)

```
Login.cshtml  X
<button type="submit" class="btn btn-primary">Login</button>
<div>
  <a asp-action="ForgotPassword">Forgot Password?</a>
</div>
```

```
<div>
  <a asp-action="ForgotPassword">Forgot Password?</a>
</div>
```

! https://localhost:44346/account/ForgotPassword

Forgot Password in ASP.NET Core

Email


```
public class ForgotPasswordViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }
}
```

```
@model ForgotPasswordViewModel

<div class="row">
  <div class="col-md-12">
    <form method="post">
      <div asp-validation-summary="All" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <button type="submit" class="btn btn-primary">Submit</button>
    </form>
  </div>
</div>
```

We just need the user email address to send the password reset link. So the `ForgotPasswordViewModel` class contains just one property - `Email`

```
[Required]
[EmailAddress]
public string Email { get; set; }
```

Forgot Password View

Views->Account -> Razor View

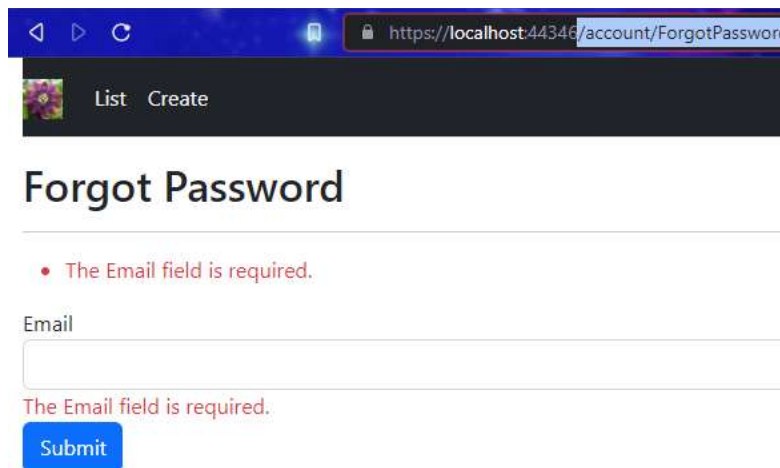
```
@model ForgotPasswordViewModel

<h2>Forgot Password</h2>
<hr />
<div class="row">
  <div class="col-md-12">
    <form method="post">
      <div asp-validation-summary="All" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <button type="submit" class="btn btn-primary">Submit</button>
    </form>
  </div>
</div>
```

Forgot Password Action Methods

Include the following [HTTP GET](#) and [POST ForgotPassword\(\)](#) actions in the [Account](#) controller. The code in the actions is commented and self-explanatory.

```
[HttpGet]
[AllowAnonymous]
public IActionResult ForgotPassword()
{
    return View();
}
```



Forgot Password in ASP.NET Core

Generate Password Reset Token

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

service method (passing object)

1

Build Password Reset Link

```
var passwordResetLink = Url.Action("ResetPassword", "Account",
    new { email = model.Email, token = token }, Request.Scheme);
```

action controller URL http, https +

2

Log Password Reset Link

```
logger.Log(LogLevel.Warning, passwordResetLink);
```

record C:\DemoLogs

3

Send User to Forgot Password Confirmation View

```
return View("ForgotPasswordConfirmation");
```

4

Forgot Password Action

```
[HttpPost][AllowAnonymous]
public async Task<IActionResult> ForgotPassword(ForgotPasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = await userManager.FindByEmailAsync(model.Email);
        if (user != null && await userManager.IsEmailConfirmedAsync(user))
        {
            var token = await userManager.GeneratePasswordResetTokenAsync(user);

            var passwordResetLink = Url.Action("ResetPassword", "Account",
                new { email = model.Email, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, passwordResetLink);

            return View("ForgotPasswordConfirmation");
        }
        return View("ForgotPasswordConfirmation");
    }
    return View(model);
}
```

```
[HttpPost]
[AllowAnonymous]
public async Task<IActionResult> ForgotPassword(ForgotPasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        // Find the user by email
        var user = await userManager.FindByEmailAsync(model.Email);
        // If the user is found AND Email is confirmed
        if (user != null && await userManager.IsEmailConfirmedAsync(user))
        {
            // Generate the reset password token
            var token = await userManager.GeneratePasswordResetTokenAsync(user);

            // Build the password reset link
            var passwordResetLink = Url.Action("ResetPassword", "Account",
                new { email = model.Email, token = token }, Request.Scheme);

            // Log the password reset link
            logger.Log(LogLevel.Warning, passwordResetLink);

            // Send the user to Forgot Password Confirmation view
            return View("ForgotPasswordConfirmation");
        }

        // To avoid account enumeration and brute force attacks, don't
        // reveal that the user does not exist or is not confirmed
        return View("ForgotPasswordConfirmation");
    }
}
```

```

    }

    return View(model);
}

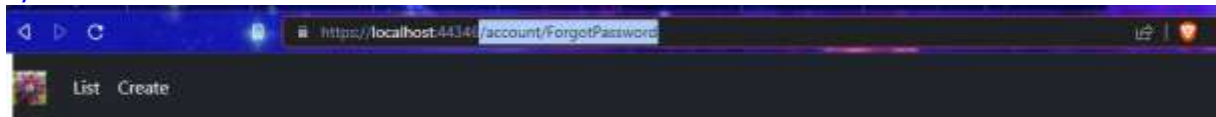
```

Forgot Password Confirmation View

```

<h4>
    If you have an account with us, we have sent an email
    with the instructions to reset your password.
</h4>

```



If you have an account with us, we have sent an email with the instructions to reset your password.

Startup.cs dbo.AspNetUsers [Data] ForgotPassword.cshtml ForgotPasswordViewModel.cs

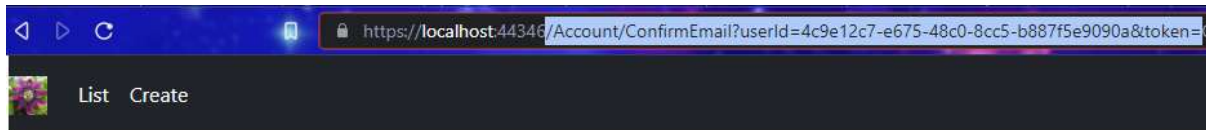
Max Rows: 1000

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False

В случая умишлено зададох имейл, който не е потвърден, за да не се изпълни едното условие.

Създадох втори имейл, който потвърдих

https://localhost:44346/Account/ConfirmEmail?userId=4c9e12c7-e675-48c0-8cc5-b887f5e9090a&token=CfDJ8NXI8GzyYcNDu0rWycevLiwfOpKm5U90O6ZNkOvK5Bfmfeyky1SI2eUh9JtRC4DsJUimmvgHaNAOb1JiUV9HOI8EKq3Uv1gzdwSXHsk7jzeLpPIYdhVwTKft5CJ3Y10zHGt5IYmO38HS35F9R28itTx9wotDBD3aG2nErdZuBQfToi1vmANaUQZn2bF%2FNme26XWGVm3B2drGwqEsnDjlGPWQ835JSxYj%2BjXGd%2Bryp0qoyBNXHr1k0f5XJk4ctaP5w%3D%3D



Thank you for confirming your email

Startup.cs appsettings.json **dbo.AspNetUsers [Data]** ForgotPassword.cshtml

Max Rows: 1000

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False
	3d9edac1-67d3...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False
	4c9e12c7-e675-...	Anthony@abv....	ANTHONY@ABV.BG	Anthony@abv....	ANTHONY@AB...	True

Сега отговаря на второто условие, съответно имаме линк за ResetPassword:

<https://localhost:44346/Account/ResetPassword?email=Anthony@abv.bg&token=CfDJ8NXI8GzyYcNDu0rWycevLizOli%2FTrx4Zy%2BjSE6qOfgkOihqquoM61F8V1VdL6mE89mof8Gx4xUfWjwAYi9Pbiq3QIM0DFsC2Wo3sfxxOHMnZltkaHM3IfTXF9ocdKxQaMM7OIZaleRWYj6BP2D9kNf8NCUcUD2c%2BFQ%2B29N1cEEulslvuLMo6ZRVE1TIpeq87gR9nWljR%2F4UPKTIAe4blJyAZ4XMCKbRb5n49jdYPgwRx>

At the moment, if we try to use the password reset link, we get a **404 error**. This is because we do not have `ResetPassword()` action in the `AccountController`. We will discuss how to implement this in our next video.

116 - Reset password

Reset Password in ASP.NET Core

To reset password

- Email
 - Password reset token
 - New Password &
 - Confirm Password
- In the password reset link
- User provides

Reset Password - View & View Model

4 properties:

- Email
- Password &
- Confirm Password
- Password reset token

```
public class ResetPasswordViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [DataType(DataType.Password)]
    [Display(Name = "Confirm password")]
    [Compare("Password", ErrorMessage = "Password and Confirm Password must match")]
    public string ConfirmPassword { get; set; }

    public string Token { get; set; }
}
```

```
using System.ComponentModel.DataAnnotations;
using System.Xml.Linq;
```

```
namespace EmployeeManagement.ViewModels
{
```

```
    public class ResetPasswordViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm password")]
        [Compare("Password", ErrorMessage = "Password and Confirm Password must match")]
        public string ConfirmPassword { get; set; }

        public string Token { get; set; }
    }
}
```

Reset Password View & View Code

Reset Password

Password

Confirm password

Reset

```
<form method="post">
  <div asp-validation-summary="All" class="text-danger"></div>
  <input asp-for="Token" type="hidden" />
  <input asp-for="Email" type="hidden" />
  <div class="form-group">
    <label asp-for="Password"></label>
    <input asp-for="Password" class="form-control" />
    <span asp-validation-for="Password" class="text-danger"></span>
  </div>
  <div class="form-group">
    <label asp-for="ConfirmPassword"></label>
    <input asp-for="ConfirmPassword" class="form-control" />
    <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
  </div>
  <button type="submit" class="btn btn-primary">Reset</button>
</form>
```

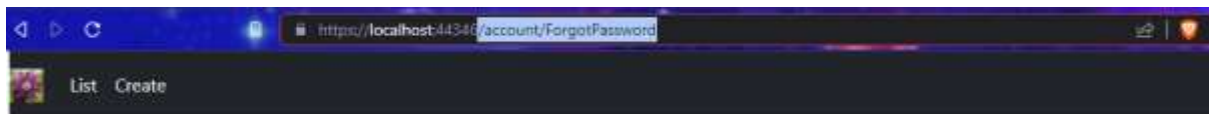
View→Account→ResetPassword.cshtml

@model ResetPasswordViewModel

```
<h2>Reset Password</h2>
<hr />
<div class="row">
  <div class="col-md-12">
    <form method="post">
      <div asp-validation-summary="All" class="text-danger"></div>
      <input asp-for="Token" type="hidden" />
      <input asp-for="Email" type="hidden" />
      <div class="form-group">
        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="ConfirmPassword"></label>
        <input asp-for="ConfirmPassword" class="form-control" />
        <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
      </div>
      <button type="submit" class="btn btn-primary">Reset</button>
    </form>
  </div>
</div>
```


AccountController:

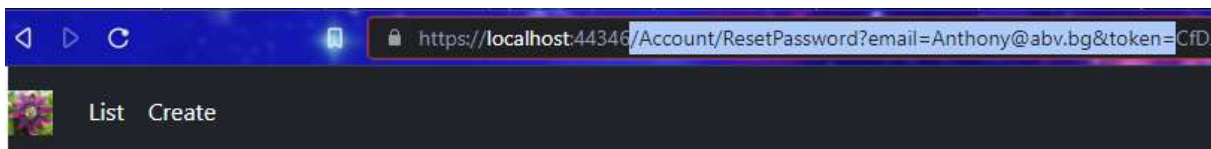
```
[HttpGet]
[AllowAnonymous]
public IActionResult ResetPassword(string token, string email)
{
    // If password reset token or email is null, most likely the
    // user tried to tamper the password reset link
    if (token == null || email == null)
    {
        ModelState.AddModelError("", "Invalid password reset token");
    }
    return View();
}
```



If you have an account with us, we have sent an email with the instructions to reset your password.

C:\DemoLogs

https://localhost:44346/Account/ResetPassword?email=Anthony@abv.bg&token=CfDJ8NXl8GzyYcNDu0rWycevLizOli%2FTrx4Zy%2BjSE6qOfgkOihqquoM61F8V1VdL6mE89mof8Gx4xUfWjwAYi9Pbiq3QIM0DFsC2Wo3sfxxOHMnZltkaHM3IfTXF9ocdKxQaMM7OlzaleRWYj6BP2D9kNf8NCUcUD2c%2BFQ%2B29N1cEEulsIvuLMo6ZRVE1TIpeq87gR9nWIjR%2F4UPKTIAe4blJyAZ4XMckbRb5n49jdYPgwRx



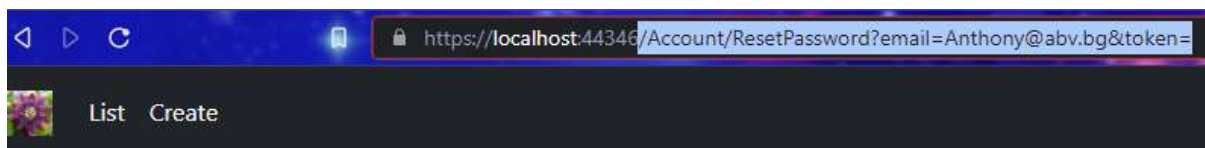
Reset Password

Password

Confirm password

Reset

Ако изтрием токена или имейла, if-а няма да бъде изпълнен:



Reset Password

- Invalid password reset token

Ако ги възстановим, валидацията изчезва.

Ако въведем парола и дадем Reset, връща 404 за post на ResetPassword() action.

Reset Password Action

```
public async Task<IActionResult> ResetPassword(ResetPasswordViewModel model)
{
    if (ModelState.IsValid) password, confirmPassword, email, token
    {
        var user = await userManager.FindByEmailAsync(model.Email);
        if (user != null) if the user is found
        {
            var result = await userManager.ResetPasswordAsync(user, model.Token, model.Password);
            if (result.Succeeded)
            {
                return View("ResetPasswordConfirmation");
            }
            foreach (var error in result.Errors)
            {
                ModelState.AddModelError("", error.Description);
            }
            return View(model); with all errors
        }
        else return View("ResetPasswordConfirmation"); without message against brute force attacks
    }
    return View(model);
}
```

Интересното тук е, че ако не се намери потребител с дадения имейл, не се извежда съобщение, умишлено, за да се намалят шансовете за brute-force атака, а просто се връща страницата за ResetPasswordConfirmation.

[HttpPost]

[AllowAnonymous]

```
public async Task<IActionResult> ResetPassword(ResetPasswordViewModel model)
{
```

```
    if (ModelState.IsValid)
    {
```

```
        // Find the user by email
```

```
        var user = await userManager.FindByEmailAsync(model.Email);
```

```
        if (user != null)
        {
```

```
            // reset the user password
```

```

        var result = await userManager.ResetPasswordAsync(user, model.Token,
model.Password);
        if (result.Succeeded)
        {
            return View("ResetPasswordConfirmation");
        }
        // Display validation errors. For example, password reset token already
        // used to change the password or password complexity rules not met
        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }
        return View(model);
    }

    // To avoid account enumeration and brute force attacks, don't
    // reveal that the user does not exist
    return View("ResetPasswordConfirmation");
}
// Display validation errors if model state is not valid
return View(model);
}

```

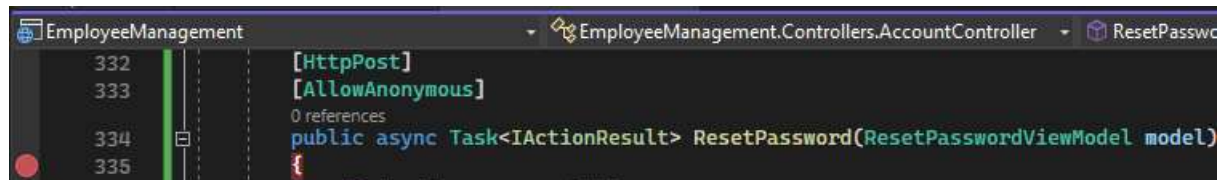
ResetPasswordConfirmation.cshtml

```

<h4>
    Your password is reset. Please click <a asp-action="Login">here to login</a>
</h4>

```

Експериментално включваме Debug mode ->



https://localhost:44346/Account/ResetPassword?email=Anthony@abv.bg&token=CfDJ8NXI8GzyYcNDu0rWycevLizOli%2FTrx4Zy%2BjSE6qOfgkOihqqoM61F8V1VdL6mE89mof8Gx4xUfWjwAYi9Pbiq3QIM0DFsC2Wo3sfxxOHMnZltkaHM3IfTXF9ocdKxQaMM7OlzaleRWYj6BP2D9kNf8NCUcUD2c%2BFQ%2B29N1cEEulsIvuLMo6ZrVE1TIpeq87gR9nWIjR%2F4UPKTIAe4blJyAZ4XMckbRb5n49jdYPgwRx

премахваме последните символи (за по-добра видимост, аз оставих само 3 символа от токена token=CfD), изгледът остава същия след refresh и въвеждаме нова парола, отговаряща на всички условия за сложност:

BP1 – извлечени данни, включително скъсения токен:

```

EmployeeManagement
EmployeeManagement.Controllers.AccountController
ResetPassword(ResetPasswordView

332 [HttpPost]
333 [AllowAnonymous]
334 public async Task<IActionResult> ResetPassword(ResetPasswordViewModel model)
335 {
336     if (ModelState.IsValid)
337     {
338         // Find the user by email
339         var user = await userManager.FindByEmailAsync(model.Email);
340
341         if (user != null)
342         {
343             // reset the user password
344             var result = await userManager.ResetPasswordAsync(user, model.Token, model.Password);
345             if (result.Succeeded)
346             {
347                 return View("ResetPasswordConfirmation");
348             }
349         }
350     }
351 }

```

model {EmployeeManagement.ViewModels.ResetPassword}
Email Q View - "Anthony@abv.bg"
Token Q View - "Cfd"

Имейлът е разпознат -> BP2

```

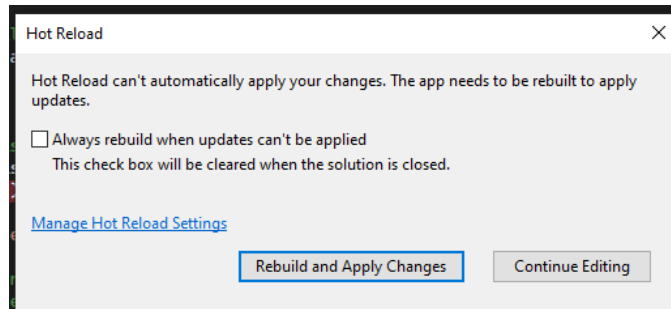
EmployeeManagement
EmployeeManagement.Controllers.AccountController
ResetPassword(ResetPasswordViewModel model)

332 [HttpPost]
333 [AllowAnonymous]
334 public async Task<IActionResult> ResetPassword(ResetPasswordViewModel model)
335 {
336     if (ModelState.IsValid)
337     {
338         // Find the user by email
339         var user = await userManager.FindByEmailAsync(model.Email);
340
341         if (user != null)
342         {
343             // reset the user password
344             var result = await userManager.ResetPasswordAsync(user, model.Token, model.Password);
345             if (result.Succeeded)
346             {
347                 return View("ResetPasswordConfirmation");
348             }
349         }
350     }
351 }

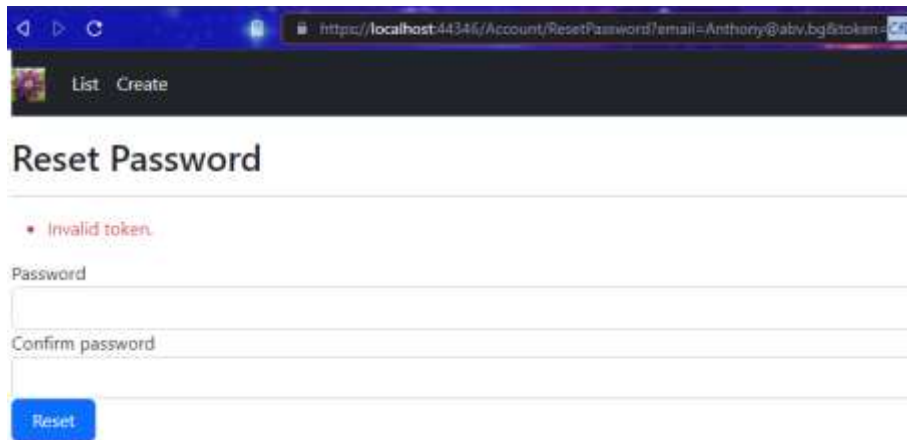
```

model.Email Q View - "Anthony@abv.bg"

При мен обаче, user остава неразпознат и извежда грешка:

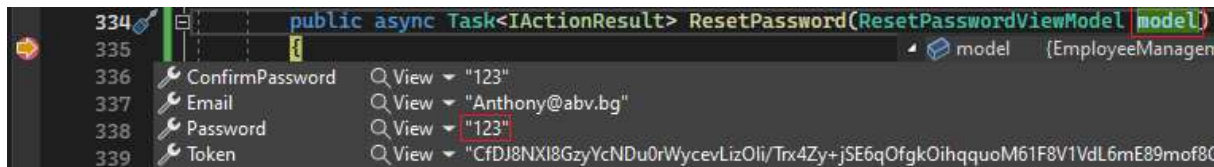


При старт без Debug ->

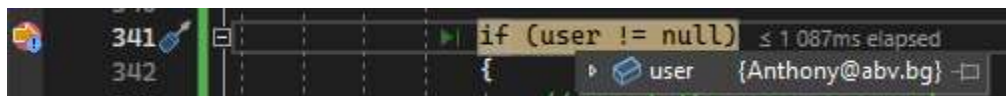


Следваща проверка в Debug mode е с истинския token и парола, неотговаряща на правилата „123“:

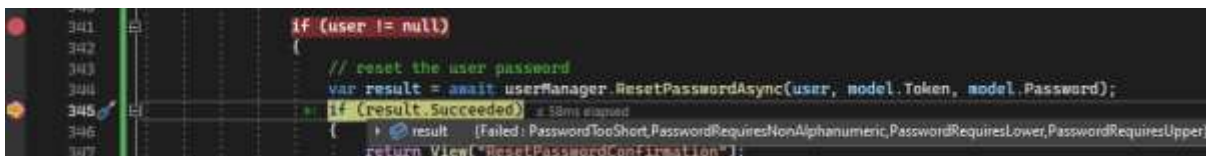
BP1:



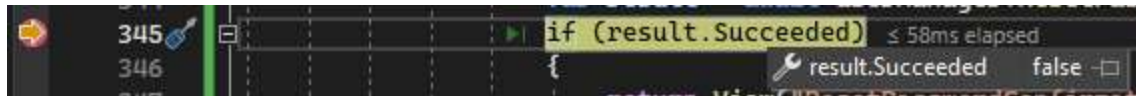
BP2:



BP3:



Имаме множество валидационни грешки, които се виждат в `.Succeeded`.



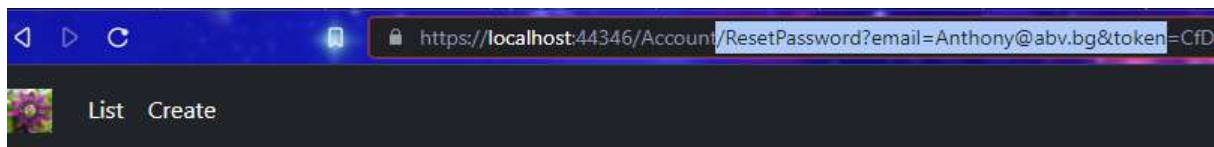
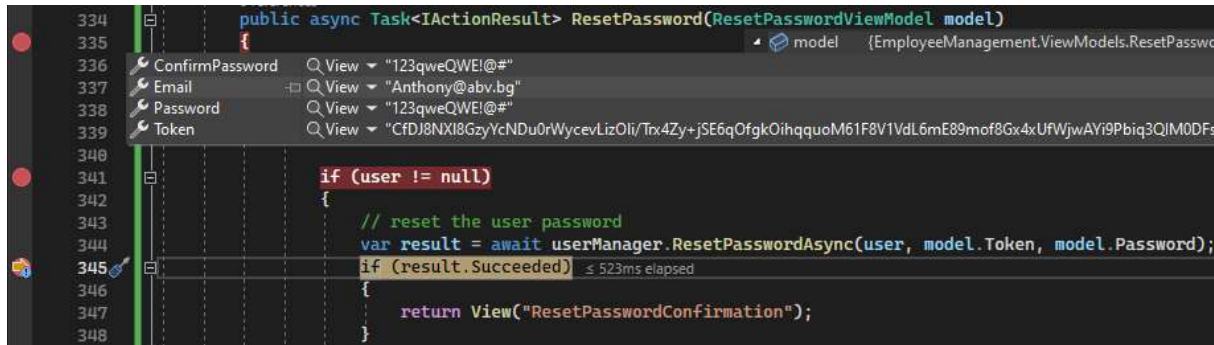
Continue:

Reset Password

- Passwords must be at least 10 characters.
- Passwords must have at least one non alphanumeric character.
- Passwords must have at least one lowercase ('a'-'z').
- Passwords must have at least one uppercase ('A'-'Z').

Password
 Confirm password

Въвеждаме нова валидна парола и отново ни изхвърля към BP1 ->



Your password is reset. Please click [here to login](#)

Въвеждаме новата парола и се вписваме (за по-голяма сигурност, това не става автоматично).

117 - How tokens are generated and validated

Token Generation and Validation

- Email Confirmation Token : **Parts 113 & 114**
- Password Reset Token : **Parts 115 & 116**

To generate Email Confirmation Token

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

To generate Password Reset Token

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

Both these methods, [GenerateEmailConfirmationTokenAsync\(\)](#) and [GeneratePasswordResetTokenAsync\(\)](#) internally calls [GenerateUserTokenAsync\(\)](#) method.

<https://github.com/dotnet/aspnetcore/blob/release/5.0/src/Identity/Extensions.Core/src/UserManager.cs>

```
namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Provides the APIs for managing user in a persistence store.
    /// </summary>
    /// <typeparam name="TUser">The type encapsulating a user.</typeparam>
    public class UserManager<TUser> : IDisposable where TUser : class
    {
        /// <summary>
        /// The data protection purpose used for the reset password related methods.
        /// </summary>
        public const string ResetPasswordTokenPurpose = "ResetPassword";

        /// <summary>
        /// The data protection purpose used for the change phone number methods.
        /// </summary>
        public const string ChangePhoneNumberTokenPurpose = "ChangePhoneNumber";

        /// <summary>
        /// The data protection purpose used for the email confirmation related methods.
        /// </summary>
        public const string ConfirmEmailTokenPurpose = "EmailConfirmation";
    }
}
```

Token Generation and Validation

```
public virtual Task<string> GenerateEmailConfirmationTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.EmailConfirmationTokenProvider, ConfirmEmailTokenPurpose);
}

public virtual Task<string> GeneratePasswordResetTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.PasswordResetTokenProvider, ResetPasswordTokenPurpose);
}
```

```
public virtual Task<string>
    GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)
{
    // Method Implementation
}
```

`GenerateUserTokenAsync()` has 3 parameters.

- The `user` the token will be for
- The `token provider` which will actually generate the token
- The token purpose - For example, password reset or email confirmation. Token generated for a given purpose must be used only for that purpose. For example, an **email confirmation token cannot be used to reset a password**.

```
public virtual Task<string>
    GenerateEmailConfirmationTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.EmailConfirmationTokenProvider, ConfirmEmailTokenPurpose);
}

public virtual Task<string>
    GeneratePasswordResetTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.PasswordResetTokenProvider, ResetPasswordTokenPurpose);
}
```

Both [Email Confirmation Tokens](#)
and [Password Reset Tokens](#)
are generated by the built-in [DataProtectorTokenProvider](#).

DataProtectorTokenProvider Class

- **GenerateAsync()** : Generates Token
- **ValidateAsync()** : Validates Token

GenerateAsync() method generates the token and

ValidateAsync() method validates the token

The source code of this class is on the following github page

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>

DataProtectorTokenProvider GenerateAsync()

As you can see from the code, the generated token contains

- Token Creation Time
- User ID
- Token Purpose
- Security Stamp

All this data is then encrypted and then base 64 encoded so it can be sent over the network.

ASP.NET Core uses **Data Protection API** (in short **DP API**) for encryption. Later in this video series, we will discuss how to use DP API and encrypt our application data like query strings for example.

```

github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs
public virtual async Task<string> GenerateAsync(string purpose, UserManager<TUser> manager, TUser user)
{
    if (user == null)
    {
        throw new ArgumentNullException(nameof(user));
    }
    var ms = new MemoryStream();
    var userId = await manager.GetUserIdAsync(user);
    using (var writer = ms.CreateWriter())
    {
        writer.Write(DateTimeOffset.UtcNow); creation time
        writer.Write(userId);
        writer.Write(purpose ?? "");
        string stamp = null;
        if (manager.SupportsUserSecurityStamp)
        {
            stamp = await manager.GetSecurityStampAsync(user);
        }
        writer.Write(stamp ?? ""); using DP API - data protection API
                                   encryption and decryption
    }
    var protectedBytes = Protector.Protect(ms.ToArray());
    return Convert.ToBase64String(protectedBytes);
} base 64 encoded to be safely send over the wire

```

```

public virtual async Task<string> GenerateAsync(string purpose,
    UserManager<TUser> manager, TUser user)
{
    if (user == null)
    {
        throw new ArgumentNullException(nameof(user));
    }
    var ms = new MemoryStream();
    var userId = await manager.GetUserIdAsync(user);
    using (var writer = ms.CreateWriter())
    {
        writer.Write(DateTimeOffset.UtcNow);
        writer.Write(userId);
        writer.Write(purpose ?? "");
        string stamp = null;
        if (manager.SupportsUserSecurityStamp)
        {
            stamp = await manager.GetSecurityStampAsync(user);
        }
        writer.Write(stamp ?? "");
    }
    var protectedBytes = Protector.Protect(ms.ToArray());
}

```



```

    return Convert.ToBase64String(protectedBytes);
}

```

DataProtectorTokenProvider ValidateAsync()

As the name implies `ValidateAsync()` method validates the token. First it is decoded from Base 64 and then decrypted. Decryption is done by the Data Protection API (DP API).

The token `creation time` is read from the token. To this, the the token life span is added. If this computed `DateTime` is less than current UTC `DateTime`, the token has expired. So the method returns false. **Tokens cannot be used after they have expired.** The default token lifespan is one day. We can change this to meet our application requirements. We will discuss how to do this in our next video.

The `User ID` in the token is compared against the User ID the token is being used. If the **User IDs are not equal**, the method return false invalidating the token. A token generated for a given user must only be used by that user.

The purpose from the token is read to make sure it is used for the intended purpose. **Token generated for a purpose must be used only for that purpose.** If we try to use it for a different purpose, token validation fails. For example, if a token is generated for a given user for the purpose of resetting password, it can be used only by that specific user for resetting his password. If we try to use it for a different user or purpose, token validation fails. Simply put, Password Reset Token cannot be used to confirm an email address.

The security stamp in the token is compared with the user current security stamp in the database. If they are different, the token validation fails.

```

public virtual async Task<bool> ValidateAsync(string purpose,
                                             string token, UserManager<TUser> manager, TUser user)
{
    try
    {
        var unprotectedData = Protector.Unprotect(Convert.FromBase64String(token));
        var ms = new MemoryStream(unprotectedData);
        using (var reader = ms.CreateReader())
        {
            var creationTime = reader.ReadDateTimeOffset();
            var expirationTime = creationTime + Options.TokenLifespan;
            if (expirationTime < DateTimeOffset.UtcNow)
            {
                return false;
            }

            var userId = reader.ReadString();
            var actualUserId = await manager.GetUserIdAsync(user);
            if (userId != actualUserId)
            {
                return false;
            }
            var purp = reader.ReadString();

```

```

        if (!string.Equals(purp, purpose))
        {
            return false;
        }
        var stamp = reader.ReadString();
        if (reader.PeekChar() != -1)
        {
            return false;
        }

        if (manager.SupportsUserSecurityStamp)
        {
            return stamp == await manager.GetSecurityStampAsync(user);
        }
        return stamp == "";
    }
}
// ReSharper disable once EmptyGeneralCatchClause
catch
{
    // Do not leak exception
}
return false;
}

```

DataProtectorTokenProvider Class

- **GenerateAsync()** : Generates Token
- **ValidateAsync()** : Validates Token

Generated Token Contains



Token Validation Fails

Tampered and cannot be decrypted

Expired

Used against a different User ID

Used for a different purpose

Security stamp is different

118 - Password reset token lifetime

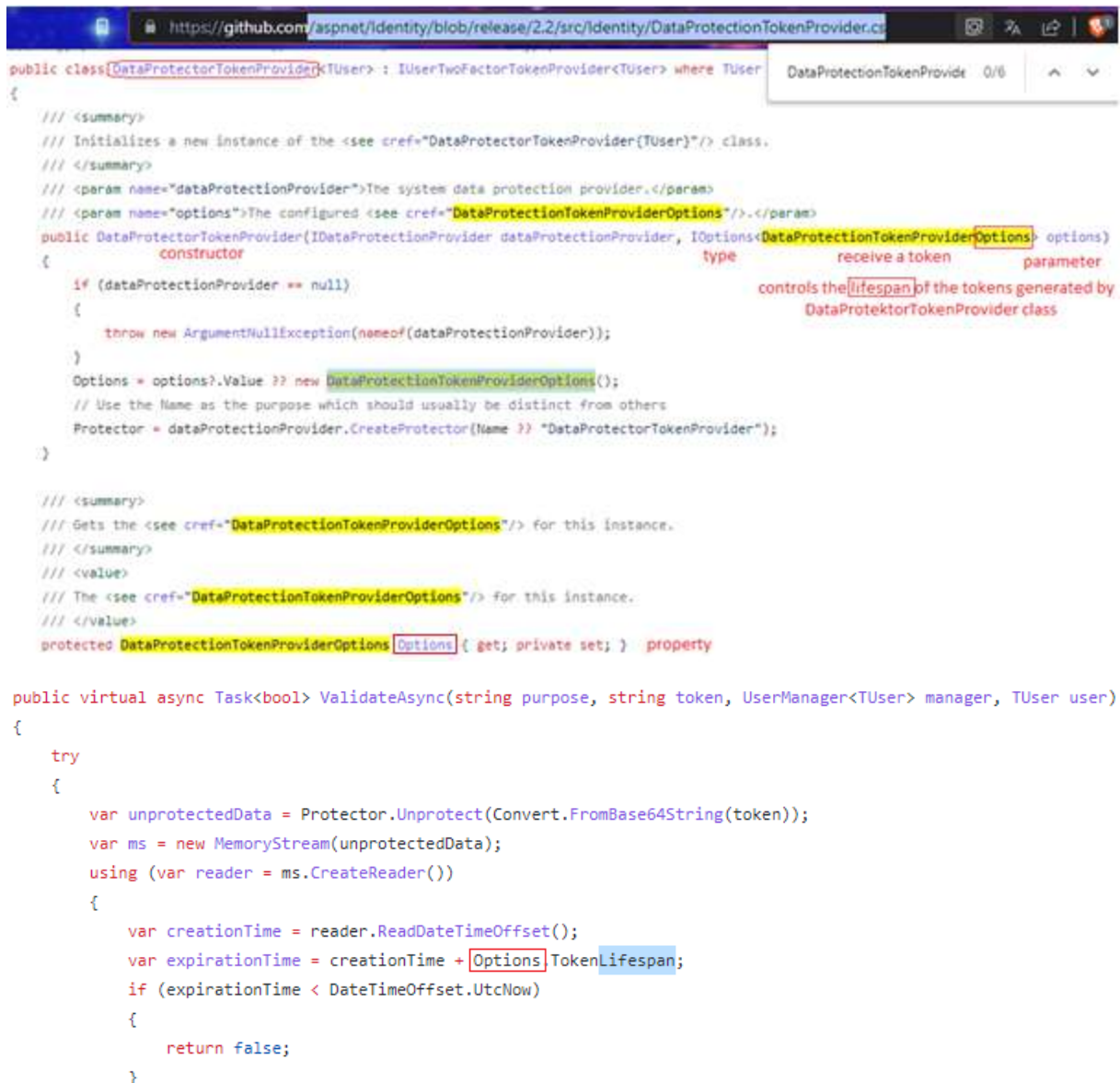
Password Reset Token Lifetime

- **DataProtectorTokenProvider** class generates both Password Reset and Email Confirmation tokens
- Token life span is controlled by **DataProtectionTokenProviderOptions** class
- The default token life span is **1 day**

Both, **password reset token** and **email confirmation token** are generated by the built-in **DataProtectorTokenProvider** class. In our **previous video**, we discussed in detail, how this class generates and validates these tokens.

So, it is the **DataProtectorTokenProvider** class that generates the password reset token. The token life span is controlled by **DataProtectionTokenProviderOptions** class. You can see this yourself, if you look at the source code of **DataProtectorTokenProvider** class at the following link.

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>



```

public class DataProtectionTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where TUser : IUser
{
    /// <summary>
    /// Initializes a new instance of the <see cref="DataProtectionTokenProvider{TUser}"> class.
    /// </summary>
    /// <param name="dataProtectionProvider">The system data protection provider.</param>
    /// <param name="options">The configured <see cref="DataProtectionTokenProviderOptions">.</param>
    public DataProtectionTokenProvider(IDataProtectionProvider dataProtectionProvider, IOptions<DataProtectionTokenProviderOptions> options)
    {
        constructor
        {
            if (dataProtectionProvider == null)
            {
                throw new ArgumentNullException(nameof(dataProtectionProvider));
            }
            Options = options?.Value ?? new DataProtectionTokenProviderOptions();
            // Use the Name as the purpose which should usually be distinct from others
            Protector = dataProtectionProvider.CreateProtector(Name ?? "DataProtectionTokenProvider");
        }
    }

    /// <summary>
    /// Gets the <see cref="DataProtectionTokenProviderOptions"> for this instance.
    /// </summary>
    /// <value>
    /// The <see cref="DataProtectionTokenProviderOptions"> for this instance.
    /// </value>
    protected DataProtectionTokenProviderOptions Options { get; private set; } property

    public virtual async Task<bool> ValidateAsync(string purpose, string token, UserManager<TUser> manager, TUser user)
    {
        try
        {
            var unprotectedData = Protector.Unprotect(Convert.FromBase64String(token));
            var ms = new MemoryStream(unprotectedData);
            using (var reader = ms.CreateReader())
            {
                var creationTime = reader.ReadDateTimeOffset();
                var expirationTime = creationTime + Options.TokenLifespan;
                if (expirationTime < DateTimeOffset.UtcNow)
                {
                    return false;
                }
            }
        }
    }
}

```

The default token life span is 1 day. You can see this yourself if you look at the source code of `DataProtectionTokenProviderOptions` class at the following link.

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProviderOptions.cs>

```

namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Contains options for the <see cref="DataProtectorTokenProvider{TUser}"/>.
    /// </summary>
    public class DataProtectionTokenProviderOptions
    {
        /// <summary>
        /// Gets or sets the name of the <see cref="DataProtectorTokenProvider{TUser}"/>. Default
        /// </summary>
        /// <value>
        /// The name of the <see cref="DataProtectorTokenProvider{TUser}"/>.
        /// </value>
        public string Name { get; set; } = "DataProtectorTokenProvider";

        /// <summary>
        /// Gets or sets the amount of time a generated token remains valid. Defaults to 1 day.
        /// </summary>
        /// <value>
        /// The amount of time a generated token remains valid.
        /// </value>
        public TimeSpan TokenLifespan { get; set; } = TimeSpan.FromDays(1);
    }
}

```

```

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));

services.AddMvc(options =>

```

```

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));

```

Това означава, че всички токени имат продължителност 5 часа живот оттук нататък, не само за пароли, но и за имейли. Принципно токени на имейлите имат по-продължителна годност от токени на паролите.

Password Reset Token Lifetime

Changes token lifespan of all token types to 5 Hours

```
public void ConfigureServices(IServiceCollection services)
{
    // rest of the code

    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5));

    // rest of the code
}
```

To change lifetime of just one type of token, we have to create a custom

- **DataProtectorTokenProvider**
- **DataProtectionTokenProviderOptions**

Може да персонализираме само токени на имейлите да имат продължителност 3 дена, което ще дискутираме в следващото видео.

119 - Custom token provider

ASP.NET Core Custom Token Provider

- **DataProtectorTokenProvider** class can generate different types of tokens
- Token life span is controlled by **DataProtectionTokenProviderOptions** class
- The default token life span is **1 day**

```
public void ConfigureServices(IServiceCollection services)
{
    // Changes token life span of all token types
    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5));
}
```

DataProtectorTokenProvider прави промени генерално върху всички видове токени – за пароли, имейли и прочие. Ако искаме да персонализираме конкретно за даден специфичен вид, например

валидността на токена на имейлите да бъде 3 дена, трябва да персонализираме двата класа: DataProtectorTokenProvider + DataProtectionTokenProviderOptions, Като помним че във втория Options се намира продължителността за TimeSpan. Така че може да използваме вградените му възможности и унаследявания наготово.

```
services.Configure<DataProtectionTokenProviderOptions>() Go To Definition

using System;

namespace Microsoft.AspNetCore.Identity
{
    public class DataProtectionTokenProviderOptions
    {
        public DataProtectionTokenProviderOptions();

        public string Name { get; set; }
        public TimeSpan TokenLifespan { get; set; }
    }
}
```

Security folder -> Add -> Class ->

```
namespace EmployeeManagement.Security
{
    0 references
    public class CustomEmailConfirmationTokenProviderOptions : DataProtectionTokenProviderOptions
    {
    }
}
```

```
using Microsoft.AspNetCore.Identity;

namespace EmployeeManagement.Security
{
    public class CustomEmailConfirmationTokenProviderOptions :
DataProtectionTokenProviderOptions
    {
    }
}
```

Step 1 : Create Custom Options & Provider

```
public class CustomEmailConfirmationTokenProviderOptions
    : DataProtectionTokenProviderOptions
{
}
```

We pass our custom parameter instead of `DataProtectionTokenProviderOptions` because of inheritance relationship :

```
public class CustomEmailConfirmationTokenProvider<TUser>
    : DataProtectorTokenProvider<TUser> where TUser : class
{
    public CustomEmailConfirmationTokenProvider(IDataProtectionProvider dataProtectionProvider,
        IOption<CustomEmailConfirmationTokenProviderOptions> options)
        : base(dataProtectionProvider, options)
    {
    }
}
```

We pass <TUser>

github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs

```
namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Provides protection and validation of identity tokens.
    /// </summary>
    /// <typeparam name="TUser">The type used to represent a user.</typeparam> where TUser is just a class
    public class DataProtectorTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where TUser : class
    {
```

```
public DataProtectorTokenProvider(IDataProtectionProvider dataProtectionProvider, IOption<DataProtectionTokenProviderOptions>
{
    constructor ( parameter 1 ; our own custom parameter 2 )
}
```

Let's add our custom class also to the security folder (without Options):

```
CustomEmailC...nProvider.cs*  CustomEmailC...derOptions.cs  DataProtecti...om meta...
agement  EmployeeManagement.Security.CustomEmailConfigur  CustomEmailConfigurationTokenProvid
using Microsoft.AspNetCore.DataProtection;
using Microsoft.Extensions.Options;

namespace EmployeeManagement.Security
{
    1 reference
    public class CustomEmailConfigurationTokenProvider<TUser>
        where TUser : class
    {
        0 references
        public CustomEmailConfigurationTokenProvider(IDataProtectionProvider dataProtectionProvider,
            IOption<CustomEmailConfirmationTokenProviderOptions> options)
            : base(dataProtectionProvider, options)
        { }
    }
}
```

base class `DataProtectorTokenProvider` in core 3.0 has addition parameter `ILogger`, the following code is working:

```

public CustomEmailConfirmationTokenProvider(IDataProtectionProvider dataProtectionProvider,
    IOptions<CustomEmailConfirmationTokenProviderOptions> options,
    ILogger<CustomEmailConfirmationTokenProvider<TUser>> logger)
    : base(dataProtectionProvider, options, logger)
{ }

```

```

using Microsoft.AspNetCore.DataProtection;
using Microsoft.AspNetCore.Identity;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using NLog;

namespace EmployeeManagement.Security
{
    public class CustomEmailConfirmationTokenProvider<TUser>
        : DataProtectorTokenProvider<TUser> where TUser : class
    {
        public CustomEmailConfirmationTokenProvider(IDataProtectionProvider
dataProtectionProvider,
            IOptions<CustomEmailConfirmationTokenProviderOptions> options,
            ILogger<CustomEmailConfirmationTokenProvider<TUser>> logger)
            : base(dataProtectionProvider, options, logger)
        { }
    }
}

```

Step 2 : Register Custom Token Provider

```

public void ConfigureServices(IServiceCollection services)
{
    // Rest of the code

    services.AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
    })
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()
    .AddTokenProvider<CustomEmailConfirmationTokenProvider>
        (<ApplicationUser>("CustomEmailConfirmation"));

    // Rest of the code
}

```



```

public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Password.RequiredLength = 10;
        options.Password.RequiredUniqueChars = 3;

        options.SignIn.RequireConfirmedEmail = true;
        options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
    }).AddEntityFrameworkStores<AppDbContext>()
        .AddDefaultTokenProviders()
        .AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");
}

```

```

options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
}).AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()

```

```

.AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");

```

```

    options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
}).AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()
    .AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");

//services.AddAuthorization(options =>
//options.AddPolicy("Role",
//policy => policy.RequireRole("Admin")));

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));
services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromDays(3));

services.AddMvc(options =>

```

```

services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromDays(3));

```

Step 3 : Change email conf. token lifespan

```

public void ConfigureServices(IServiceCollection services)
{
    // Changes token lifespan of all token types
    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5)); all tokens

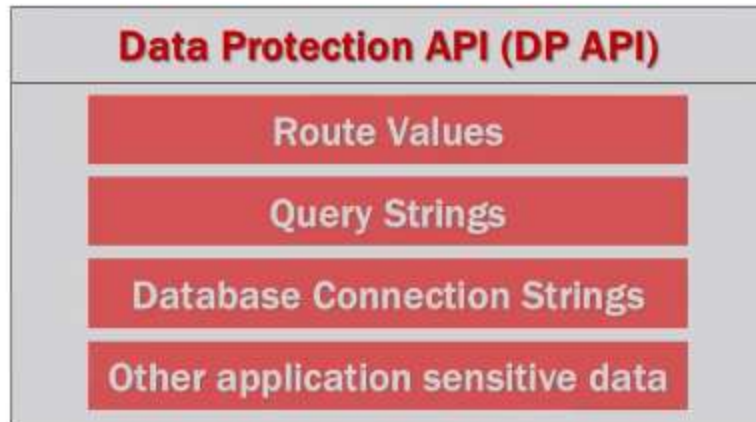
    // Changes token lifespan of just the Email Confirmation Token type
    services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromDays(3)); only email tokens
}

```

120 - Encryption and Decryption Example

Ще използваме DP API за криптиране и декриптиране на Route value

ASP.NET Core Encryption & Decryption



Като пример, когато натиснем Edit-бутонa, ни прехвърля към /home/edit/2. Ние ще искаме да криптираме ID да не се вижда като integer value 2, за да не може да се прочете.

Security folder -> add -> class DataProtectionPurposeStrings

На този етап той ще съдържа само един purpose string с един property, който наричаме EmployeeIdRouteValue

```
namespace EmployeeManagement.Security
{
    0 references
    public class DataProtectionPurposeStrings
    {
        public readonly string EmployeeIdRouteValue = "EmployeeIdRouteValue";
    }
}
```

Register purpose string class with DI (dependency injection) container:

Това ни позволява да инжектираме instance на този клас във всеки контролер в приложението.

Прелистваме до последно в Startup.cs

```
Startup.cs  X
EmployeeManagement.Startup  Conf
services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();
services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();
services.AddSingleton<DataProtectionPurposeStrings>();
}
```

```
services.AddSingleton<DataProtectionPurposeStrings>();
```

От 117 урок

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>

A screenshot of a web browser displaying the GitHub repository for the DataProtectorTokenProvider class. The browser's address bar shows the URL: https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs. The code is shown in a light blue editor with syntax highlighting. The class is named DataProtectorTokenProvider and implements IUserTwoFactorTokenProvider. It has two parameters: dataProtectionProvider and options. The code includes a summary, parameter descriptions, and a constructor that initializes the provider and protector. A tooltip for the dataProtectionProvider parameter is visible on the right side of the code.

```
public class DataProtectorTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where T
{
    /// <summary>
    /// Initializes a new instance of the <see cref="DataProtectorTokenProvider{TUser}" /> class.
    /// </summary>
    /// <param name="dataProtectionProvider">The system data protection provider.</param>
    /// <param name="options">The configured <see cref="DataProtectionTokenProviderOptions" />.</param>
    public DataProtectorTokenProvider(IDataProtectionProvider dataProtectionProvider, IOptions<DataProt
    {
        if (dataProtectionProvider == null)
        {
            throw new ArgumentNullException(nameof(dataProtectionProvider));
        }
        Options = options?.Value ?? new DataProtectionTokenProviderOptions();
        // Use the Name as the purpose which should usually be distinct from others
        Protector = dataProtectionProvider.CreateProtector(Name ?? "DataProtectorTokenProvider");
    }
}
```

Class DataProtectorTokenProvider отговаря за генериране на email confirmation token, password reset token, encryption and decryption tokens, затова и самият клас се нарича така DataProtectorTokenProvider. Като разлистим надолу, той съдържа property Protector:

```
protected IDataProtector Protector { get; private set; }
type prop
```

Който съдържа 2 ценни метода: protect и unprotect, които ще използваме.

Надолу виждаме методът, който генерира token,

```
public virtual async Task<string> GenerateAsync(string purpose, UserManager<TUser> manager, TUser user)
{
```

А генерираният token после се подава на Protect метод на IDataProtector, така че tokenът може да бъде криптиран. По-надолу имаме метод Validate, и Unprotect за декриптиране.

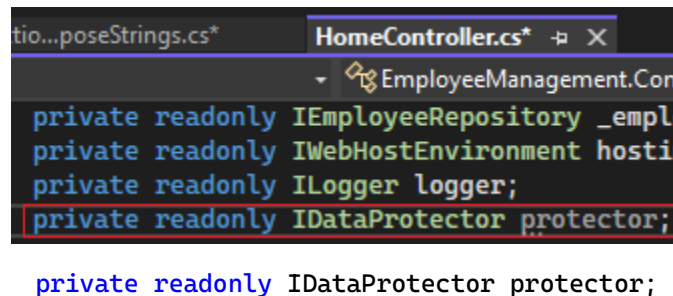
```

        var protectedBytes = Protector.Protect(ms.ToArray());
        return Convert.ToBase64String(protectedBytes);
    }

    /// <summary>
    /// Validates the protected <paramref name="token"/> for the specified <paramref nam
    /// </summary>
    /// <param name="purpose">The purpose the token was be used for.</param>
    /// <param name="token">The token to validate.</param>
    /// <param name="manager">The <see cref="userManager{TUser}"/> to retrieve user prop
    /// <param name="user">The <typeparamref name="TUser"/> the token was generated for.
    /// <returns>
    /// A <see cref="Task{TResult}"/> that represents the result of the asynchronous val
    /// containing true if the token is valid, otherwise false.
    /// </returns>
    public virtual async Task<bool> ValidateAsync(string purpose, string token, UserMana
    {
        try
        {
            var unprotectedData = Protector.Unprotect(Convert.FromBase64String(token));

```

Ще използваме всичко това, за да криптираме и декриптираме ID в URL-адреса.



```

private readonly IEmployeeRepository _employeeRepository;
private readonly IWebHostEnvironment hostingEnvironment;
private readonly ILogger logger;
private readonly IDataProtector protector;

```

За да го използваме, нека разберем какъв е принципът.

How to get an instance of the class that implements IDataProtector. How the build-in DataProtector provide a class? Notice, to get an instance of IDataProtector, we are using DataProtectionProvider and this variable is of type IDataProtectionProvider. This interface is injected into the class, using constructor and then on this interface we have CreateProtector method, and this method is passing the purpose string. They are using the Name of the DataProtectorTokenProvider to use as a purpose string. In our case, we have a separate class, that contains our purpose string. So in addition to injecting this interface IDataProtectionTokenProvider, we also need to inject our class that contains our purpose string. Let's first inject IDataProtectionProvider into our HomeController.

```

io...poseStrings.cs* HomeController.cs*
EmployeeManagement.Controllers.HomeController HomeController(IEm
private readonly IWebHostEnvironment hostingEnvironment, //auto private field
private readonly ILogger logger;
private readonly IDataProtector protector;

0 references
public HomeController(IEmployeeRepository employeeRepository,
    IWebHostEnvironment hostingEnvironment,
    ILogger<HomeController> logger,
    IDataProtectionProvider dataProtectionProvider,
    DataProtectionPurposeStrings dataProtectionPurposeStrings)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
    this.logger = logger;
    protector = dataProtectionProvider
        .CreateProtector(dataProtectionPurposeStrings.EmployeeIdRouteValue);
}

```

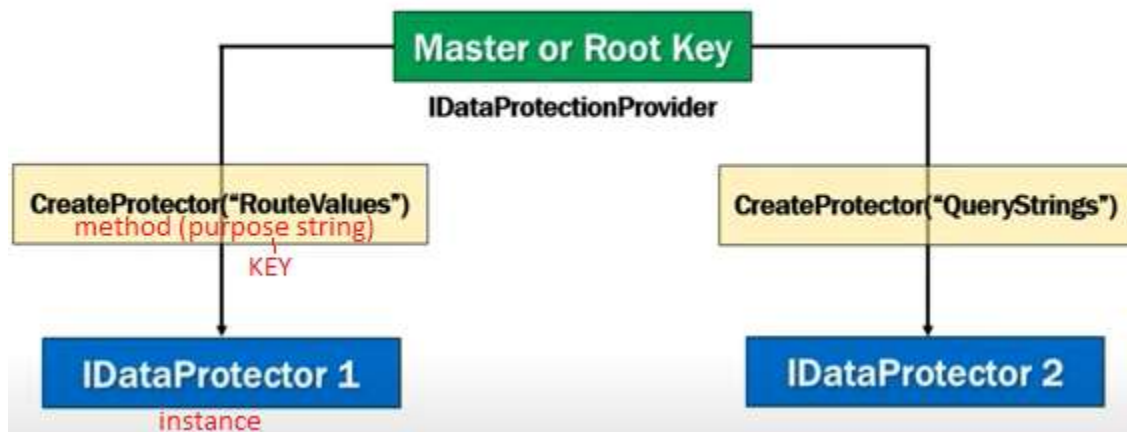
```

private readonly IDataProtector protector;

public HomeController(IEmployeeRepository employeeRepository,
    IWebHostEnvironment hostingEnvironment,
    ILogger<HomeController> logger,
    IDataProtectionProvider dataProtectionProvider,
    DataProtectionPurposeStrings
dataProtectionPurposeStrings)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
    this.logger = logger;
    protector = dataProtectionProvider
        .CreateProtector(dataProtectionPurposeStrings.EmployeeIdRouteValue);
}

```

ASP.NET Core Encryption & Decryption



You can think of purpose string as an encryption key. This key is then combined with the master or root key to generate a unique key. The data that is encrypted by a given combination of purpose string and root key can only be decrypted by that same combination of keys.

The purpose string is inherent to the security of the data protection system, as it provides isolation between cryptographic consumers, even if the root keys are the same.

Ако погледнем Employee модела, виждаме, че има property, което съхранява четимия текст. Необходимо ни е още едно property за криптирания текст и за да не бъде асоцииран с нито една колона от БД, слагаме атрибут [NotMapped]

```
namespace EmployeeManagement.Models
{
    45 references
    public class Employee
    {
        17 references
        public int Id { get; set; }
        [NotMapped]
        0 references
        public string EncryptedId { get; set; }
        [Required]
        [MaxLength(60, ErrorMessage = "Name cannot exceed 60 characters")]
    }
}
```

```
[NotMapped]
public string EncryptedId { get; set; }
```

Това криптирано ID трябва да важи и в index страницата, където се намира списъкът с Employees.

```
to...poseStrings.cs* Employee.cs HomeController.cs* X
- EmployeeManagement.Controllers.HomeCo
[AllowAnonymous]
0 references
public ActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}
```

We use the link Select statement,

e.EncryptedId присвоява e.ID plaintext и го конвертира ToString.

За да се осъществи самото криптиране, използваме protector, който създадохме в конструктора и неговият .Protect метод.


```

[AllowAnonymous]
0 references
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee()
        .Select(e =>
        {
            e.EncryptedId = protector.Protect(e.Id.ToString());
            return e;
        });
    return View(model);
}

```

```

[AllowAnonymous]
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee()
        .Select(e =>
        {
            // Encrypt the ID value and store in EncryptedId property
            e.EncryptedId = protector.Protect(e.Id.ToString());
            return e;
        });
    return View(model);
}

```

Имаме 1 грешка, Нека погледнем Index view – маркира Index action, защото EncryptedID все още не се използва.

```

Index.cshtml X Employee.cs HomeController.cs
<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
class="btn btn-primary m-1">View</a>
<a asp-controller="home" asp-action="edit" asp-route-id="@employee.Id"
class="btn btn-primary m-1">Edit</a>

```

Вместо видимото Id, нека сложим EncryptedID и видим резултата.

```

Index.cshtml X Employee.cs HomeController.cs
<a asp-controller="home" asp-action="details" asp-route-id="@employee.EncryptedId"
class="btn btn-primary m-1">View</a>

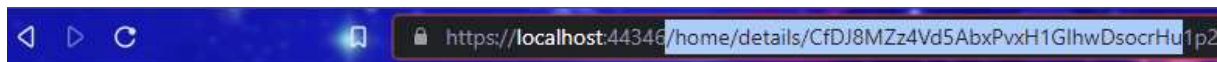
```

```

<a asp-controller="home" asp-action="details" asp-route-id="@employee.EncryptedId"

```

Когато изберем бутон view от списъка, получаваме грешка.



An unhandled exception occurred while processing the request.

InvalidOperationException: Nullable object must have a value.

System.Nullable<T>.get_Value()

Stack

Query Cookies Headers Routing

InvalidOperationException: Nullable object must have a value.

System.Nullable<T>.get_Value()

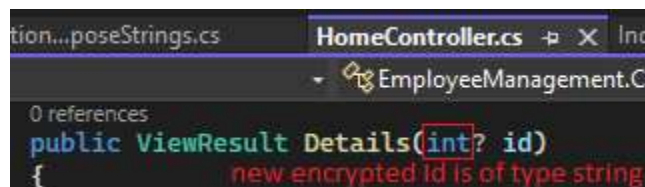
EmployeeManagement.Controllers.HomeController.Details(Nullable<int> id) in HomeController.cs

59. Employee employee = _employeeRepository.GetEmployee(id.Value);

lambda_method210(Closure, object, object[])

Линкът, вместо откритото ID, съдържа криптираното.

HomeController.Details(Nullable<int> id



Сменяме типа на Id да съответства на string, както е при криптираното Id,

След това създаваме нова променлива, тип стринг

```
string decryptedId = protector.Unprotect(id);  
  
Employee employee = _employeeRepository.GetEmployee(id.Value);  
  
if (employee == null)  
{  
    Response.StatusCode = 404;  
    return View("EmployeeNotFound", id.Value);  
}
```

но понеже не я използваме никъде другаде, можем да сменим името ѝ, за да е по-ясно, също типът да бъде int, за да може да се използва долу, като имаме предвид, че метод Unprotect връща резултат string, значи използваме .ToInt32 за да го конвертираме от string -> int и използваме по този начин.

```
//string decryptedId = protector.Unprotect(id);
int employeeId = Convert.ToInt32(protector.Unprotect(id));

Employee employee = _employeeRepository.GetEmployee(employeeId);

if (employee == null)
{
    Response.StatusCode = 404;
    return View("EmployeeNotFound", employeeId);
}
```

/home/details/CfDJ8MZz4Vd5AbxPvxH1GlhwDsqx0Jy4jRhB1KSR8-GIAvH95

encrypted ID

John

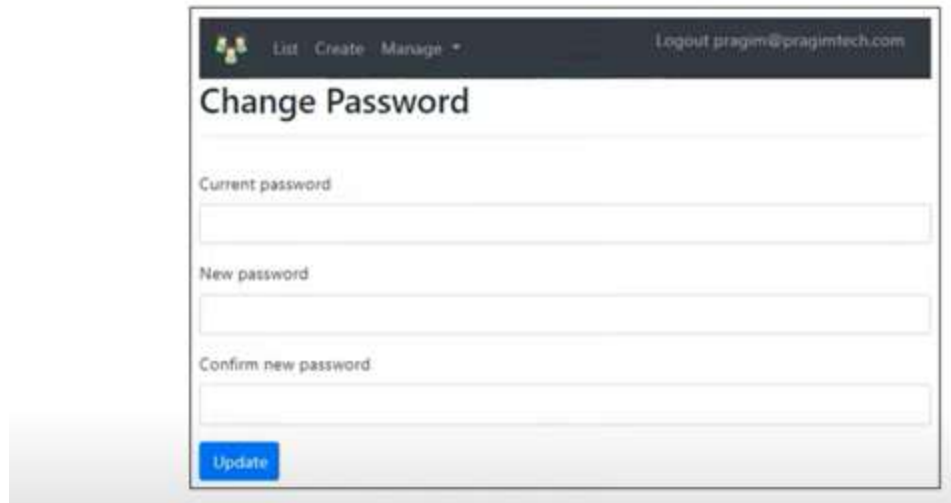


Employee ID: 2

ID в адреса на брауъра е криптирано, но вътре като информация в страницата е видимо.

121 - Change Password

Change Password in ASP.NET Core



Change Password

Current password

New password

Confirm new password

Update

ViewModels folder -> ChangePasswordViewModel

С всички данни, които са ни необходими, в случая 3 проп:

```
ChangePassw...ViewModel.cs
namespace EmployeeManagement.ViewModels
{
    0 references
    public class ChangePasswordViewModel
    {
        [Required]
        [DataType(DataType.Password)]
        [Display(Name = "Current password")]
        0 references
        public string CurrentPassword { get; set; }

        [Required]
        [DataType(DataType.Password)]
        [Display(Name = "New password")]
        0 references
        public string NewPassword { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm new password")]
        [Compare("NewPassword", ErrorMessage =
            "The new password and confirmation password do not match.")]
        0 references
        public string ConfirmPassword { get; set; }
    }
}
```



```

public class ChangePasswordViewModel
{
    [Required]
    [DataType(DataType.Password)]
    [Display(Name = "Current password")]
    public string CurrentPassword { get; set; }

    [Required]
    [DataType(DataType.Password)]
    [Display(Name = "New password")]
    public string NewPassword { get; set; }

    [DataType(DataType.Password)]
    [Display(Name = "Confirm new password")]
    [Compare("NewPassword", ErrorMessage =
        "The new password and confirmation password do not match.")]
    public string ConfirmPassword { get; set; }
}

```

Views folder -> Accounts subfolder -> Add -> New item -> Razor View - Empty:

```

@model ChangePasswordViewModel

<h2>Change Password</h2>
<hr />
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="CurrentPassword"></label>
                <input asp-for="CurrentPassword" class="form-control" />
                <span asp-validation-for="CurrentPassword" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="NewPassword"></label>
                <input asp-for="NewPassword" class="form-control" />
                <span asp-validation-for="NewPassword" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="ConfirmPassword"></label>
                <input asp-for="ConfirmPassword" class="form-control" />
                <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Update</button>
        </form>
    </div>
</div>

```

```
@model ChangePasswordViewModel
```

```

<h2>Change Password</h2>
<hr />
<div class="row">
    <div class="col-md-12">
        <form method="post">

```

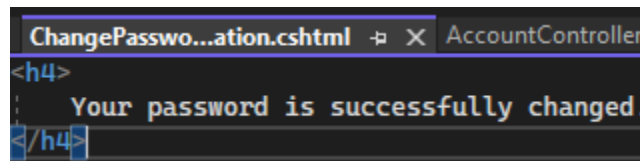


```

        <div asp-validation-summary="All" class="text-danger"></div>
        <div class="form-group">
            <label asp-for="CurrentPassword"></label>
            <input asp-for="CurrentPassword" class="form-control" />
            <span asp-validation-for="CurrentPassword" class="text-
danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="NewPassword"></label>
            <input asp-for="NewPassword" class="form-control" />
            <span asp-validation-for="NewPassword" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="ConfirmPassword"></label>
            <input asp-for="ConfirmPassword" class="form-control" />
            <span asp-validation-for="ConfirmPassword" class="text-
danger"></span>
        </div>
        <button type="submit" class="btn btn-primary">Update</button>
    </form>
</div>
</div>

```

След като промяната на паролата е осъществена, трябва да ни прехвърли към страница ChangePasswordConfirmation, съответно я добавяме в същата папка като Razor View:



```

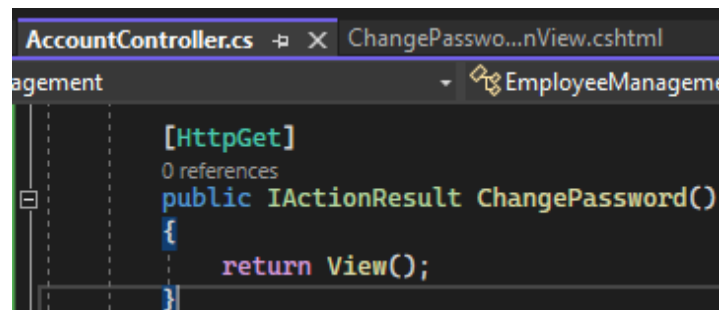
<h4>
    Your password is successfully changed.
</h4>

```

```

<h4>
    Your password is successfully changed.
</h4>

```



```

[HttpGet]
0 references
public IActionResult ChangePassword()
{
    return View();
}

```

```

[HttpGet]
public IActionResult ChangePassword()
{
    return View();
}

```

Само логнати потребители имат възможност да сменят паролата и видят този ChangePasswordView, защото имаме глобален филтър в Startup.cs

```
EmployeeManagement.Startup
options.Filters.Add(new AuthorizeFilter(policy));
```

```
options.Filters.Add(new AuthorizeFilter(policy));
```

който се отразява върху всички actions в рамките на контролерите, освен ако не са предварително декорирани с [AllowAnonymous] атрибут.

След това ни е необходим Post, който да прихване резултата, подаден от Get, като първо проверява дали съществува такъв потребител, ако не – препраща го към login view. Ако е намерен, сравнява старата и нова пароли чрез .ChangePasswordAsync. Ако няма съвпадение и новата парола не покрива правилата за сложност, връща описание на грешките. Ако всичко е наред, обновява бисквитките за логването чрез .RefreshSignInAsync.

```
EmployeeManagement.Controllers.AccountController - ChangePassword([HttpPost]
0 references
public async Task<IActionResult> ChangePassword(ChangePasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = await userManager.GetUserAsync(User);
        if (user == null)
        {
            return RedirectToAction("Login");
        }

        var result = await userManager.ChangePasswordAsync(user,
            model.CurrentPassword, model.NewPassword);

        if (!result.Succeeded)
        {
            foreach (var error in result.Errors)
            {
                ModelState.AddModelError(string.Empty, error.Description);
            }
            return View();
        }

        await signInManager.RefreshSignInAsync(user);
        return View("ChangePasswordConfirmation");
    }
}
```

```
[HttpPost]
public async Task<IActionResult> ChangePassword(ChangePasswordViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = await userManager.GetUserAsync(User);
        if (user == null)
        {
```

```

    return RedirectToAction("Login");
}

// ChangePasswordAsync changes the user password
var result = await userManager.ChangePasswordAsync(user,
    model.CurrentPassword, model.NewPassword);

// The new password did not meet the complexity rules or
// the current password is incorrect. Add these errors to
// the ModelState and rerender ChangePassword view
if (!result.Succeeded)
{
    foreach (var error in result.Errors)
    {
        ModelState.AddModelError(string.Empty, error.Description);
    }
    return View();
}

// Upon successfully changing the password refresh sign-in cookie
await signInManager.RefreshSignInAsync(user);
return View("ChangePasswordConfirmation");
}

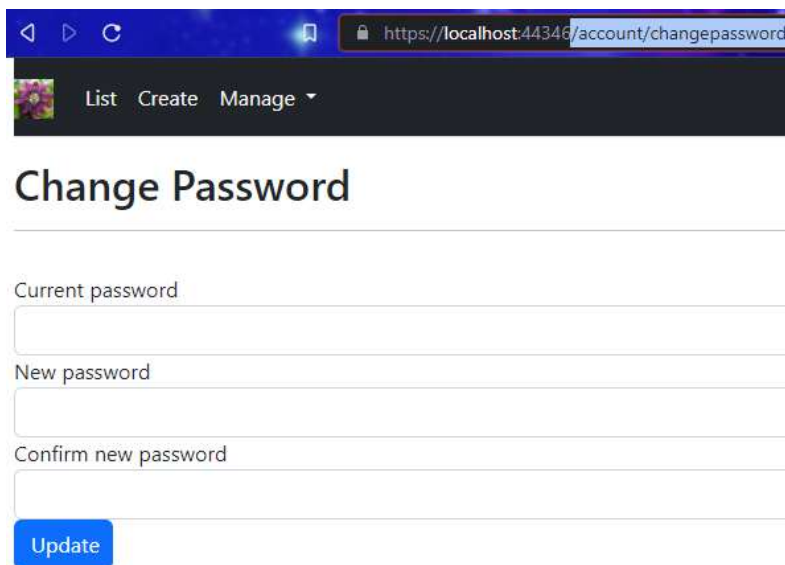
return View(model);
}

```

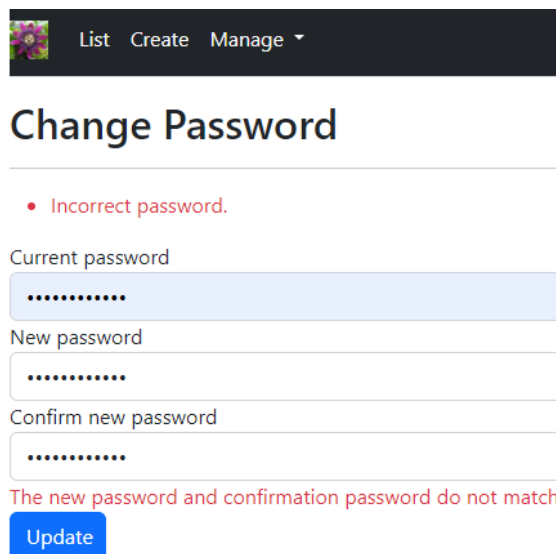
Последна стъпка е да обновим падащото меню, добавяйки Change Password.

```
html  _Layout.cshtml  AccountController.cs*  ChangePassword.cshtml
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle text-light"
            href="#"
            id="navbarDropdownMenuLink"
            role="button"
            data-toggle="dropdown"
            @*data-hover="dropdown"*@
            aria-haspopup="true"
            aria-expanded="false">
            Manage
        </a>
        <ul class="dropdown-menu" aria-labelledby="navbarDropdownMenuLink">
            <li>
                <a class="dropdown-item" asp-controller="Administration"
                    asp-action="ListUsers">Users</a>
            </li>
            <li>
                <a class="dropdown-item" asp-controller="Administration"
                    asp-action="ListRoles">Roles</a>
            </li>
            <li>
                <a class="dropdown-item" asp-controller="Account"
                    asp-action="ChangePassword">Password</a>
            </li>
        </ul>
    </li>
}
```

```
<li>
    <a class="dropdown-item" asp-controller="Account"
        asp-action="ChangePassword">Password</a>
</li>
```



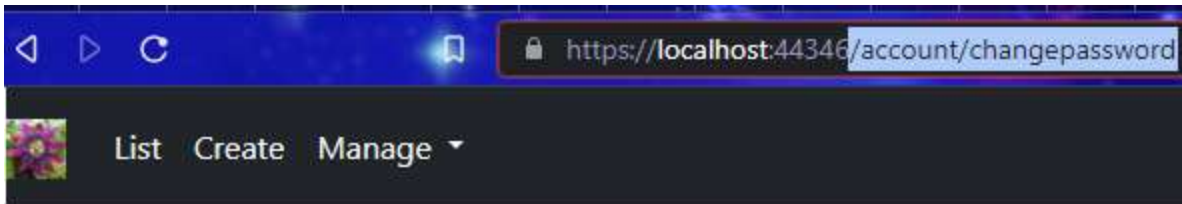
При проверка за грешки, първоначално проверява за съвпадения между новата парола и повторението ѝ, директно от браузъра, без да влиза в БД. Едва след това всичко останало като сложност.



Хубав момент е, че букви на български, не се разпознават, ако се забрави смяната на езиците.

- Passwords must have at least one lowercase ('a'-'z').
- Passwords must have at least one uppercase ('A'-'Z').

Друго интересно нещо е, че БД се обновява, независимо дали разполагаме с видимата страница за потребителя ChangePasswordConfirmation.



Your password is successfully changed.

Change Password in ASP.NET Core

Use **ChangePasswordAsync()** method of the **UserManager** service to change a logged-in user password

```
var result = await userManager.ChangePasswordAsync(user,
    model.CurrentPassword, model.NewPassword);
```

Use **RefreshSignInAsync()** method of **SignInManager** service to refresh the logged-in user sign-in cookie

```
await signInManager.RefreshSignInAsync(user);
```

122 - Add password to local account linked to external login

Ако разгледаме БД, виждаме, че за потребителите, които използват външно вписване като Google, Facebook, няма съхранена парола.

dbo.AspNetUsers [Data]						
dbo.AspNetUserLogins [Data]						
_Layout.cshtml						
Max Rows: 1000						
Id	UserName	NormalizedUs...	Email	Norma...	EmailConfirmed	Passwor
14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1...	False	AQAAAA
1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AUROR...	True	AQAAAA
3d9edac1-67d3-...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@...	False	AQAAAA
8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3...	False	AQAAAA
8492084d-c18c-...	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLE...	True	NULL
87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@...	True	AQAAAA
96944fa4-2400-...	Dima4@abv.bg	DIMA4@ABV.BG	Dima4@abv.bg	DIMA4...	True	AQAAAA
974b660c-e34d-...	anthony@abv.bg	ANTHONY@AB...	anthony@abv.bg	ANTHO...	True	AQAAAA
a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV....	george@abv.bg	GEORG...	True	AQAAAA
b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@...	True	AQAAAA
b2056455-c2b4...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@A...	True	AQAAAA
c0ec608d-d07a...	Dima2@abv.bg	DIMA2@ABV.BG	Dima2@abv.bg	DIMA2...	False	AQAAAA
cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID...	False	AQAAAA
d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@A...	False	AQAAAA
d5dcffab-a1ec-...	@googlemail.com	AMIDALAFLAM...	amidalafame@...	AMIDA...	True	NULL

Ще искаме да оставим възможност потребителят да въведе локална парола, така че да използва по свое желание или локален, или външен акаунт към 1 и същи потребител.

Add Password to Local Account


List · Create · Manage ▾
Logout pragimtest@gmail.com

Add Password

You have used an external account to login and do not have a local username/password. Simply set a new password if you want to login using a local account. Use your email as the username.

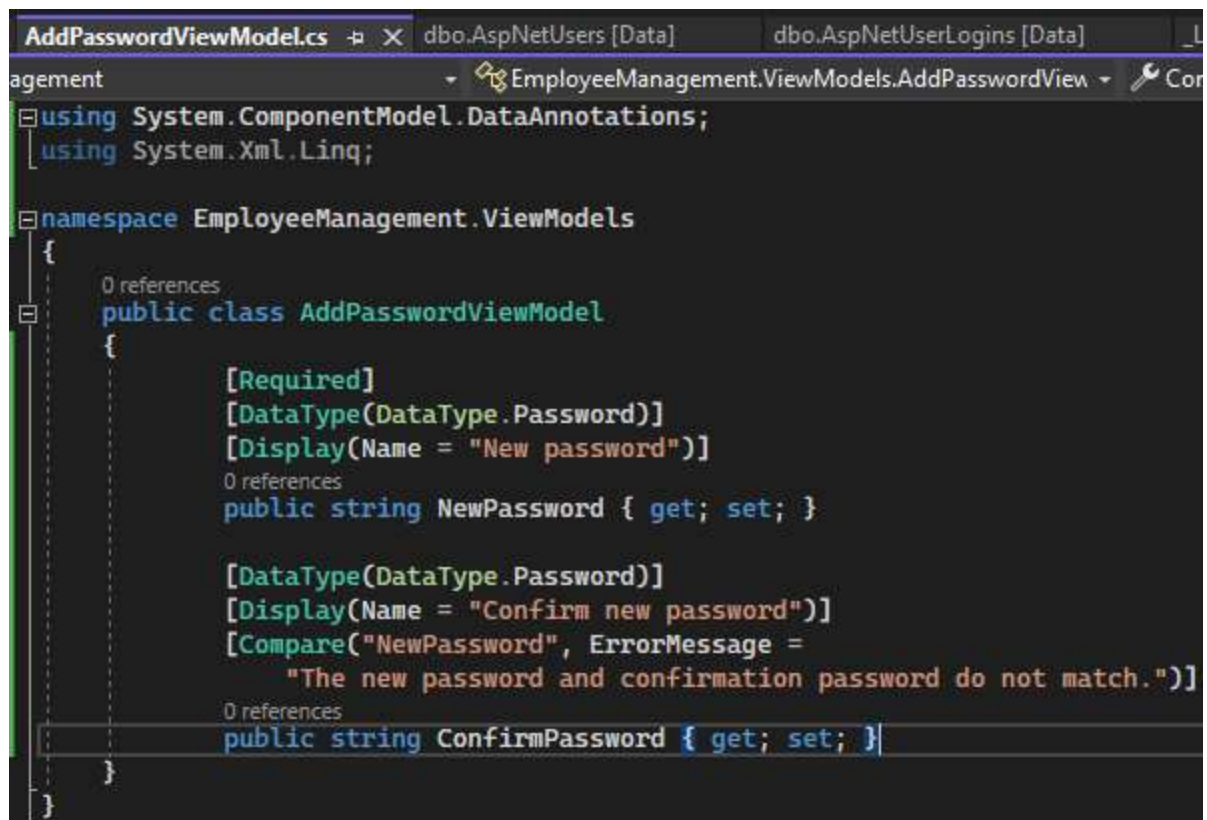
New password

Confirm new password

[Set Password](#)

Забележете, че това специализирано view не изисква парола, защото потребителят вече е вписан като външен. Неговата текуща парола при нас е NULL, затова само добавяме новата, която той избере.

AddPasswordViewModel.cs – той ще съдържа само 2 Properties: New Password, Confirm Password.



```
using System.ComponentModel.DataAnnotations;
using System.Xml.Linq;

namespace EmployeeManagement.ViewModels
{
    0 references
    public class AddPasswordViewModel
    {
        [Required]
        [DataType(DataType.Password)]
        [Display(Name = "New password")]
        0 references
        public string NewPassword { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm new password")]
        [Compare("NewPassword", ErrorMessage =
            "The new password and confirmation password do not match.")]
        0 references
        public string ConfirmPassword { get; set; }
    }
}
```

```
public class AddPasswordViewModel
{
    [Required]
    [DataType(DataType.Password)]
    [Display(Name = "New password")]
    public string NewPassword { get; set; }

    [DataType(DataType.Password)]
    [Display(Name = "Confirm new password")]
    [Compare("NewPassword", ErrorMessage =
        "The new password and confirmation password do not match.")]
    public string ConfirmPassword { get; set; }
}
```

Остава да направим съответното View -> Account folder:

```
AddPassword.cshtml X AddPasswordViewModel.cs dbo.AspNetUsers [Data] dbo.AspNetUserLo
@model AddPasswordViewModel

<h2>Add Password</h2>
<hr />
<p class="text-info">
    You have used an external account to login and do not have a local username and
    password. Simply set a new password if you want to login using a local account.
    Use your email as the username.
</p>
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="NewPassword"></label>
                <input asp-for="NewPassword" class="form-control" />
                <span asp-validation-for="NewPassword" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="ConfirmPassword"></label>
                <input asp-for="ConfirmPassword" class="form-control" />
                <span asp-validation-for="ConfirmPassword" class="text-danger">
            </span>
            </div>
            <button type="submit" class="btn btn-primary"
                style="width:auto">
                Set Password
            </button>
        </form>
    </div>
</div>
```

@model AddPasswordViewModel

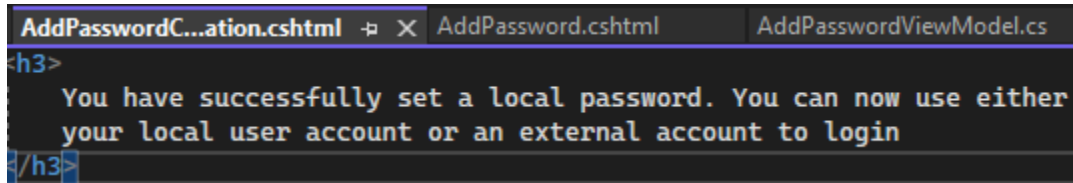
```
<h2>Add Password</h2>
<hr />
<p class="text-info">
    You have used an external account to login and do not have a local username and
    password. Simply set a new password if you want to login using a local account.
    Use your email as the username.
</p>
<div class="row">
    <div class="col-md-12">
        <form method="post">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="NewPassword"></label>
                <input asp-for="NewPassword" class="form-control" />
                <span asp-validation-for="NewPassword" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="ConfirmPassword"></label>
                <input asp-for="ConfirmPassword" class="form-control" />
                <span asp-validation-for="ConfirmPassword" class="text-danger">
            </span>
            </div>
        </form>
    </div>
</div>
```

```

        </div>
        <button type="submit" class="btn btn-primary"
            style="width:auto">
            Set Password
        </button>
    </form>
</div>
</div>

```

Account folder -> AddPasswordConfirmation View



```

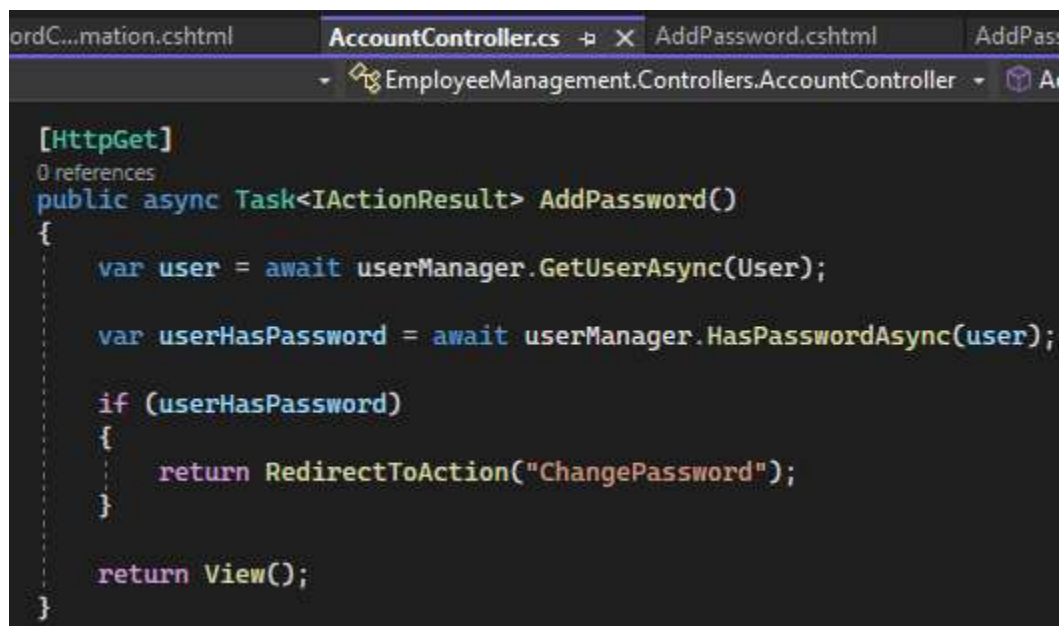
AddPasswordC...ation.cshtml  X AddPassword.cshtml AddPasswordViewModel.cs
<h3>
    You have successfully set a local password. You can now use either
    your local user account or an external account to login
</h3>

```

```

<h3>
    You have successfully set a local password. You can now use either
    your local user account or an external account to login
</h3>

```



```

ordC...ation.cshtml AccountController.cs  X AddPassword.cshtml AddPas
EmployeeManagement.Controllers.AccountController  A

[HttpGet]
0 references
public async Task<IActionResult> AddPassword()
{
    var user = await userManager.GetUserAsync(User);

    var userHasPassword = await userManager.HasPasswordAsync(user);

    if (userHasPassword)
    {
        return RedirectToAction("ChangePassword");
    }

    return View();
}

```

```

[HttpGet]
public async Task<IActionResult> AddPassword()
{
    var user = await userManager.GetUserAsync(User);

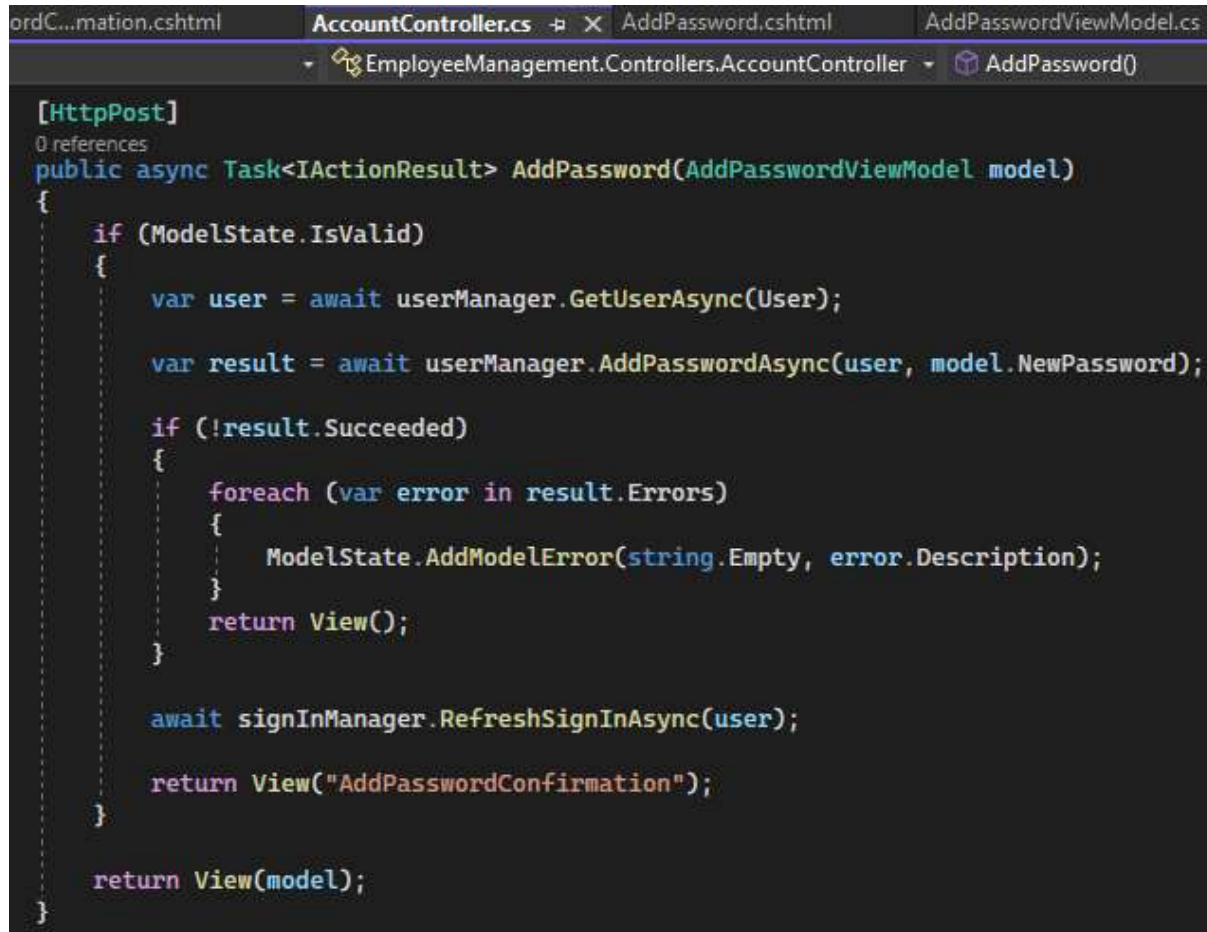
    var userHasPassword = await userManager.HasPasswordAsync(user);

    if (userHasPassword)
    {
        return RedirectToAction("ChangePassword");
    }
}

```



```
    }  
    return View();  
}
```



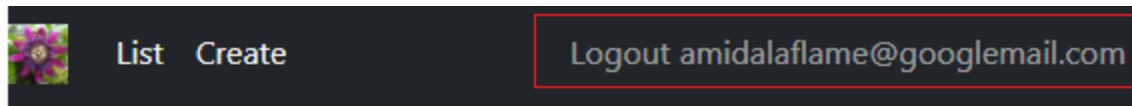
The screenshot shows the Visual Studio IDE with the following tabs: 'ordC...mation.cshhtml', 'AccountController.cs', 'AddPassword.cshhtml', and 'AddPasswordViewModel.cs'. The 'AccountController.cs' tab is active, and the 'AddPassword()' method is selected. The code is as follows:

```
[HttpPost]  
0 references  
public async Task<IActionResult> AddPassword(AddPasswordViewModel model)  
{  
    if (ModelState.IsValid)  
    {  
        var user = await userManager.GetUserAsync(User);  
  
        var result = await userManager.AddPasswordAsync(user, model.NewPassword);  
  
        if (!result.Succeeded)  
        {  
            foreach (var error in result.Errors)  
            {  
                ModelState.AddModelError(string.Empty, error.Description);  
            }  
            return View();  
        }  
  
        await signInManager.RefreshSignInAsync(user);  
  
        return View("AddPasswordConfirmation");  
    }  
  
    return View(model);  
}
```

```
[HttpPost]  
public async Task<IActionResult> AddPassword(AddPasswordViewModel model)  
{  
    if (ModelState.IsValid)  
    {  
        var user = await userManager.GetUserAsync(User);  
  
        var result = await userManager.AddPasswordAsync(user,  
model.NewPassword);  
  
        if (!result.Succeeded)  
        {  
            foreach (var error in result.Errors)  
            {  
                ModelState.AddModelError(string.Empty, error.Description);  
            }  
            return View();  
        }  
  
        await signInManager.RefreshSignInAsync(user);  
  
        return View("AddPasswordConfirmation");  
    }  
}
```

```
}    return View(model);
```

Когато отворим /account/changepassword



Change Password

Current password

blank

New password

Confirm new password

Update

Текущата парола е NULL, но все още кодът не ни позволява да оставим празно поле на Current password, въпреки, че сме вписани с външен акаунт, затова изменяме кода на Change Password action:

```
rdC...ation.cshtml | AccountController.cs | AddPassword.cshtml | AddPassw
EmployeeManagement.Controllers.AccountController | Cha

[HttpGet]
0 references
public async Task<IActionResult> ChangePassword()
{
    var user = await userManager.GetUserAsync(User);

    var userHasPassword = await userManager.HasPasswordAsync(user);

    if (!userHasPassword)
    {
        return RedirectToAction("AddPassword");
    }

    return View();
}
```

```
var user = await userManager.GetUserAsync(User);

var userHasPassword = await userManager.HasPasswordAsync(user);

if (!userHasPassword)
{
    return RedirectToAction("AddPassword");
}
```



List Create

Logout amidalaflame@googlemail.com

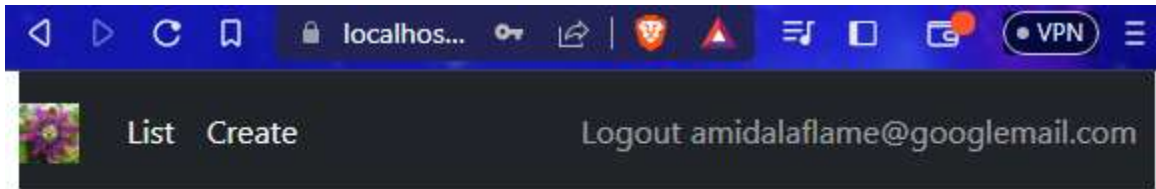
Add Password

You have used an external account to login and do not have a local username and password. Simply set a new password if you want to login using a local account. Use your email as the username.

New password

Confirm new password

Set Password



You have successfully set a local password. You can now use either your local user account or an external account to login

След Refresh на БД, полето за парола вече е криптирано и не е NULL

dbo.AspNetUsers [Data]									
Id	UserName	NormalizedUs...	Email	Norma...	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencySt...	Phone
14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1...	False	AQAAAAEAAAC...	ZMDYDH7OR6...	9ddc3369-433c...	NULL
1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV...	aurora@abv.bg	AUROR...	True	AQAAAAEAAAC...	FBRWUISYMBN...	f3a1d546-ac1d-...	NULL
3d9edac1-67d3-...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@...	False	AQAAAAEAAAC...	3DPD54CCIFYX...	3c5c7a46-8a91-...	NULL
8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3...	False	AQAAAAEAAAC...	BSH62K544MBL...	79c61fd7-a2fb-...	NULL
8492084d-c18c-...	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLE...	True	NULL	UN3TDA6BLYEP...	d3fe7137-8245-...	NULL
87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@...	True	AQAAAAEAAAC...	73VPVG65F55LE...	84ff0168-c281-...	NULL
96944fa4-2400-...	Dima4@abv.bg	DIMA4@ABV.BG	Dima4@abv.bg	DIMA4...	True	AQAAAAEAAAC...	BSK7F3BGKNZ...	35124ffd-1882-...	NULL
974b660c-e34d-...	anthony@abv.bg	ANTHONY@AB...	anthony@abv.bg	ANTHO...	True	AQAAAAEAAAC...	K34RFCCGCPY...	d3bbb4c5-bc6b...	NULL
a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV...	george@abv.bg	GEORG...	True	AQAAAAEAAAC...	DUIAJEO6OZQ...	30f229f9-4bfa-4...	NULL
b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@...	True	AQAAAAEAAAC...	YGX2FAZLF6C5...	2ffb4fe8-ae72-4...	NULL
b2056455-c2b4-...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@A...	True	AQAAAAEAAAC...	KAICVICD7UY5...	75657981-e670-...	NULL
c0ec608d-d07a-...	Dima2@abv.bg	DIMA2@ABV.BG	Dima2@abv.bg	DIMA2...	False	AQAAAAEAAAC...	SE2KPDYDGI6...	94e780b9-49a6-...	NULL
cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID...	False	AQAAAAEAAAC...	BI3N2KVBNLNV...	fe5c9479-99a8-...	NULL
d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@A...	False	AQAAAAEAAAC...	YMAO6U5Z2QT...	4b1dfa7-6e33-...	NULL
d5dcffab-a1ec-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDA...	True	AQAAAAEAAACcQAAAAEOgkBMZiz2xSIKuacZOUqa9kpOC3m1x7			
f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON...	False	AQAAAAEAAAC...	CEMUROF6TNT...	6b458222-882f-...	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Съответно имаме възможност за вписване през локален акаунт със същия имейл и нашата нововъведена парола, а при избор на account/changepassword вече не се вижда стария view за add password, а стандартния за change password



List Create

Logout amidalaflame@googlemail.com

Change Password

Current password

New password

Confirm new password

Update

Add Password to Local Account

Use **ChangePasswordAsync()** method of the **UserManager** service to change a logged-in user password

```
var result = await userManager.AddPasswordAsync(user, model.NewPassword);
```

Use **RefreshSignInAsync()** method of **SignInManager** service to refresh the logged-in user sign-in cookie

```
await signInManager.RefreshSignInAsync(user);
```

123 - Account lockout

Заклучването на акаунт се прави със защитна цел, за да се намали вероятността от brute-force атаки.

ASP.NET Core Account Lockout



Политиката за ограничаване включва например до 5 неправилни опита за въвеждане на парола, след което 15 минути забрана за достъп, след което още право на 5 опита и ако те са грешни, може би още 15 минути блок. Това би забавило една брут-форс атака с години. Това може да се комбинира с други ограничения като 1 месец да нямаш право да сменяш парола и т.н.

Configure Account Lockout Options

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    });

    // Rest of the code
}
```

```
EmployeeManagement.Startup  ConfigureServices(IServiceCollection services)
services.AddDbContextPool<AppDbContext>(
    options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBCon

services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequiredLength = 10;
    options.Password.RequiredUniqueChars = 3;

    options.SignIn.RequireConfirmedEmail = true;
    options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";

    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
}).AddEntityFrameworkStores<AppDbContext>()
```

```
options.Lockout.MaxFailedAccessAttempts = 5;
```

```
options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
```

Enable Account Lockout in ASP.NET Core

```
public class SignInManager<TUser> where TUser : class
{
    public virtual Task<SignInResult> PasswordSignInAsync(string userName, string
        password, bool isPersistent, bool lockoutOnFailure)
    }
}
```

Pass **true** to enable account lockout

За да имаме тази функционалност, всичко, което трябва да направим, е резултат True за bool lockOnFailure. Това се изпълнява от SignInManager service в Account controller, Login action, Post:

```
AccountController.cs | AddPasswordConfirmation.cshtml | AddPasswordViewModel.cs
EmployeeManagement.Controllers.AccountController | Login(LoginViewModel model, string returnUrl)
var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
    model.RememberMe, false);
```

От IntelliSense виждаме, че последният параметър на .PasswordSignInAsync – bool lockOutFailure

```
.PasswordSignInAsync(model.Email, model.Password,
Search Sol

(awaitable) Task<Microsoft.AspNetCore.Identity.SignInResult>
SignInManager<ApplicationUser>.PasswordSignInAsync(string userName, string
password, bool isPersistent, bool lockoutOnFailure) (+ 1 overload)
Attempts to sign in the specified userName and password combination as an
asynchronous operation.

Returns:
The task object representing the asynchronous operation containing the
SignInResult for the sign-in attempt.
```

е бил зададен при нас като false, т.е. няма защитно заключване, затова го активираме с true.

```
.PasswordSignInAsync(model.Email, model.Password,
    model.RememberMe, true); Lock
```

С тази промяна, всеки грешен опит за въвеждане на парола ще се регистрира, като стойността в колоната на БД Users ще се увеличава +1 при всяка грешка.

	PhoneNumber...	TwoFactorEna...	LockoutEnd	LockoutEnabled	AccessFailedC...	City
▶	False	False	NULL	True	0	Sofia
	False	False	NULL	True	0	NULL
	False	False	NULL	True	0	London

Така че, когато стойността достигне зададената (в нашия случай за 5 грешни опита), акаунтът ще се заключи. От IntelliSense също забелязваме, че стойността, която връща PasswordSignInAsync –

SignInResult. Това означава, че докато трае заключването на акаунта, той няма да се отключи дори да предоставим верни данни за име и парола на потребителя.

Returns:

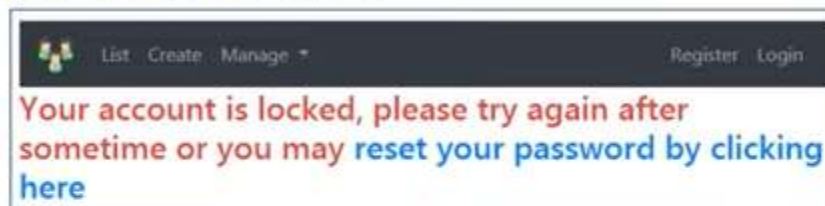
The task object representing the asynchronous operation containing the SignInResult for the sign-in attempt.

```
AccountController.cs
var result = await signInManager.PasswordSignInAsync(model.Email, model.Password,
                                                    model.RememberMe, true);
if (result.Succeeded)
{
    if(!string.IsNullOrEmpty(returnUrl)) // && URL.LocalURL(returnURL));
    {
        return LocalRedirect(returnUrl);
    }
    else
    {
        return RedirectToAction("index", "home");
    }
}
if (result.IsLockedOut)
{
    return View("AccountLocked");
}
ModelState.AddModelError(string.Empty, "Invalid Login attempt");
```

```
if (result.IsLockedOut)
{
    return View("AccountLocked");
}
```

Все още не разполагаме с този View, така че го създаваме:

Account Locked View



```
<h3 class="text-danger">
    Your account is locked, please try again after sometime or you may
    <a asp-action="ForgotPassword" asp-controller="Account">
        reset your password by clicking here
    </a>
</h3>
```

```
AccountLocked.cshtml | X | dbo.AspNetUsers [Data] | AddPassword.cshtml | Acc...
<h3 class="text-danger">
  Your account is locked, please try again after sometime or you may
  <a asp-action="ForgotPassword" asp-controller="Account">
    reset your password by clicking here
  </a>
</h3>
```

```
<h3 class="text-danger">
  Your account is locked, please try again after sometime or you may
  <a asp-action="ForgotPassword" asp-controller="Account">
    reset your password by clicking here
  </a>
</h3>
```

При 3 неуспешни опита виждаме промяната в БД, но все още нямаме заключване:

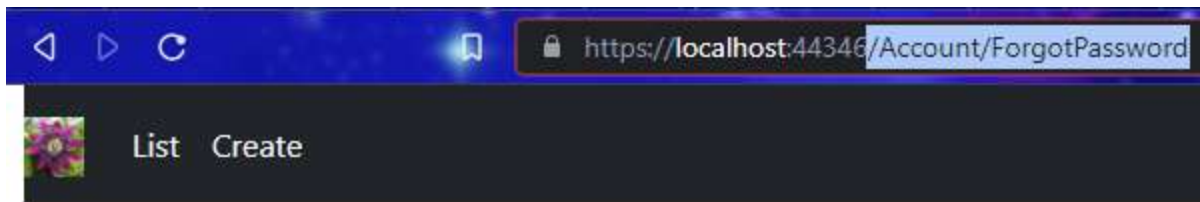
TwoFactorEna...	LockoutEnd	LockoutEnabled	AccessFailedC...	City
False	NULL	True	0	Sofia
False	NULL	True	0	NULL
False	NULL	True	0	London
False	NULL	True	0	Plovdiv
False	NULL	True	0	NULL
False	NULL	True	0	NULL
False	NULL	True	3	Sofia
False	NULL	True	0	Sofia



Your account is locked, please try again after
sometime or you may [reset your password by
clicking here](#)

Сега имаме 2 възможности: или да изчакаме 15 мин да се отключи акаунта, или да използваме линка за възстановяване на паролата, чрез който да бъде изпратена нова парола на имейла.

Ще използваме линка за възстановяване на парола, който имаме готов от урок 115



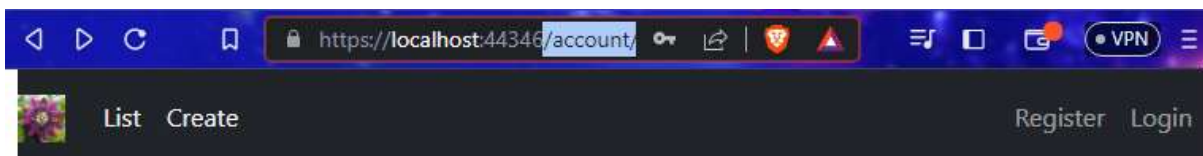
Forgot Password

Email

dima4@abv.bg

Submit

Тази функционалност е същата като от линка за регистрация:



Local Account Login

Email

Password

☐ Remember me

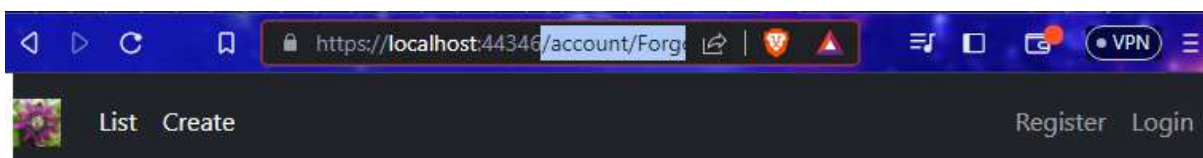
Login

[Forgot Password?](#)

External Login

Google

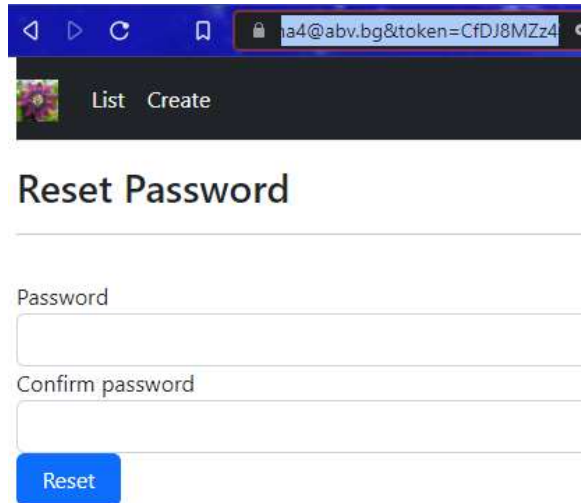
Facebook



If you have an account with us, we have sent an email with the instructions to reset your password.

ps://localhost:44346/Account/ResetPassword?email=dima4@abv.bg&token=CfDJ8MZz4Vd5AbxPvxH1GIhwD:

https://localhost:44346/Account/ResetPassword?email=dima4@abv.bg&token=CfDJ8MZz4Vd5AbxPvxH1GIhwDsoTPyD%2BQ6DvABZwbu3tbXdRLcRzDARIuexAqGRCy4qUn%2B1Z5S XVABBZ8xY7n%2BGjksUrwTIX4cflK2dQivU8GgAXQ%2B4SjaHJFyhlvv%2Fo5Pcnka9%2 FSVvePOsxRSOP8ZpE1BKqtHFw%2FcNAipbE3Q4hiaOiu8s7CIbrwgjzdvzoasZIH2fAwxencO UI%2Bka0%2FXywhK5zm1mdulmK%2BHaU2ZwzuL



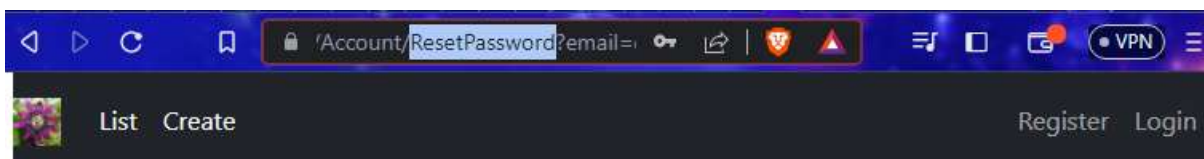
Reset Password

Password

Confirm password

Reset

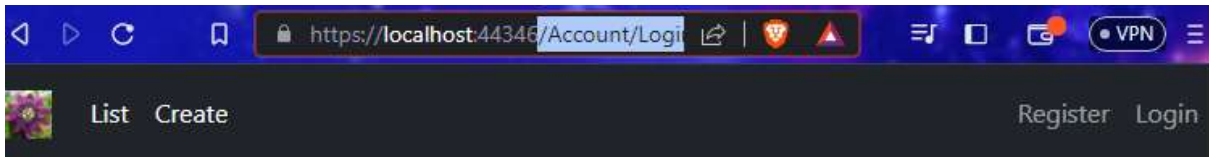
LockoutEnd	LockoutEnabled	AccessFailedC...	City
NULL	True	0	Sofia
NULL	True	0	NULL
NULL	True	0	London
NULL	True	0	Plovdiv
NULL	True	0	NULL
NULL	True	0	NULL
3/24/2023 7:57:16 AM +00:00	True	0	Sofia
NULL	True	0	Sofia



Register Login

Your password is reset. Please click [here to login](#)

Когато се впишем с новата парола, връща същата грешка, че акаунтът ни е блокиран.



Your account is locked, please try again after sometime or you may [reset your password by clicking here](#)

Един от начините да поправим това, е да въведем актуалното време в колоната за LockoutEnd.

```
Set Lockout End Date on Password Reset

var result = await userManager.ResetPasswordAsync(user, model.Token, model.Password);
if (result.Succeeded)
{
    if (await userManager.IsLockedOutAsync(user))
    {
        await userManager.SetLockoutEndDateAsync(user, DateTimeOffset.UtcNow);
    }

    return View("ResetPasswordConfirmation");
}
```

С това, достъпът със смяна на парола е възстановен.

124 – Wrap up

ASP.NET Core MVC Course Wrap Up

20 Hours

124 Videos

Partial
Views

View
Components

Razor
Pages

Blazor

Other
Concepts

Download source code: https://drive.google.com/drive/folders/1z49q-8xkKu8N8VjdemYKTs_4IbzBeLWM

ASP.NET Core MVC Course Wrap Up

Source Code

https://drive.google.com/drive/folders/1z49q-8xkKu8N8VjdemYKTs_4lbzBeLWM?usp=sharing

Software Required

Visual Studio 2017

.NET Core SDK 2.2

Applying Database Migrations

Update-Database

Slides, Text Version and all the Videos

<https://www.pragimtech.com/courses/asp-net-core-mvc-tutorial-for-beginners/>

<https://www.pragimtech.com/courses/asp-net-core-mvc-tutorial-for-beginners/>

ASP.NET Core MVC Course Wrap Up

Thank You